

Machine Learning: Nozioni e Appunti

Contents

1	Perceiver	6
1.1	Il problema della complessità quadratica	6
1.2	Differenze con il Transformer	7
1.2.1	Step (1): Calcolo delle matrici Q, K, V	8
1.2.2	Step (2): Calcolo della mappa di attenzione	9
1.2.3	Step (3): Calcolo delle feature	10
1.3	Architettura	11
1.4	Forward Pass	12
1.4.1	Immagine di Input (Byte Array)	13
1.4.2	Positional Encoding (Fourier Features)	14
1.4.3	Proiezione Lineare Input	16
1.4.4	Latent Array	18
1.4.5	Cross-Attention Block	19
1.4.6	Scaled Dot-Product Attention	24
1.4.7	Caso Multi-Head	27
1.4.8	Proiezione Lineare e Residual	28
1.4.9	Feed-Forward MLP	29
1.4.10	Riassunto: blocco Cross-Attention + MLP	31
1.4.11	Latent Transformer Block	33
1.4.12	Weight Sharing e Iterazioni	36
1.4.13	Output (Pooling + Classificazione)	39
1.5	Backward Pass	44
1.6	Dettagli di Training (ImageNet)	78
1.7	Risultati Sperimentali del Paper Originale	79
1.8	Ablation Studies	84
2	Perceiver IO	89
2.1	Introduzione e Architettura	90
2.2	Input Array X	92
2.3	Encode Module	102
2.4	Process Module	106
2.5	Decode Module e Output Queries	109
2.5.1	Output Query Array $Y^{(0)}$	110
2.5.2	Blocco Decode	114
3	Implementazione e Risultati Sperimentali del Nostro Progetto	124

3.1	Overview del progetto	124
3.2	Divergenze tra la nostra implementazione e il paper	125
3.3	CIFAR-10 — Ablation study sul Perceiver	126
3.4	CIFAR-10 — Perceiver IO	128
3.5	ModelNet40 — Classificazione point cloud	129
3.6	Perceiver IO su linguaggio — MLM pretraining + GLUE	130
3.7	Analisi delle attention maps	132
3.8	Conclusioni e lessons learned	133
A	Softmax	136
A.1	Proprietà fondamentali	137
A.2	Parametro di temperatura	137
A.3	Stabilità numerica: il trucco log-sum-exp	137
A.4	Esempio numerico	138
A.5	Collegamento al Perceiver	138
B	Fourier Features e Positional Encoding	139
B.1	Il problema: l'attenzione è cieca alla posizione	139
B.2	Tre approcci al positional encoding	139
B.3	Fourier Features nel Perceiver	140
B.4	Perché frequenze multiple?	140
B.5	Perché concatenare la coordinata grezza x_d ?	140
B.6	Esempio numerico	141
B.7	Collegamento al Perceiver	141
C	Cross-Entropy Loss	142
C.1	Perché il logaritmo?	142
C.2	Connessione con la softmax: gradiente pulito	143
C.3	Esempio numerico	143
C.4	Cross-entropy vs MSE per la classificazione	143
C.5	Collegamento al Perceiver	144
D	Layer Normalization	144
D.1	Layer Norm vs Batch Norm	144
D.2	Pattern pre-norm (usato nel Perceiver)	145
D.3	Parametri γ e β	145
D.4	Esempio numerico	146
D.5	Collegamento al Perceiver	146
E	Funzioni di Attivazione	147
E.1	ReLU (Rectified Linear Unit)	147
E.2	GELU (Gaussian Error Linear Unit)	147
E.3	Sigmoid	148
E.4	Tanh	149
E.5	Tabella comparativa	149
E.6	Perché GELU invece di ReLU nel Perceiver?	149
F	Residual Connections	150
F.1	Il problema: degradazione nelle reti profonde	150

F.2	Perché funziona: flusso del gradiente	151
F.3	Pattern pre-norm nel Perceiver	151
F.4	Esempio numerico: confronto flusso del gradiente	151
F.5	Collegamento al Perceiver	152
G	Ottimizzatori	152
G.1	SGD (Stochastic Gradient Descent)	152
G.2	SGD con Momentum	153
G.3	Adam (Adaptive Moment Estimation)	154
G.4	AdamW (Adam con Weight Decay disaccoppiato)	156
G.5	LAMB (Layer-wise Adaptive Moments for Batch training)	157
G.6	Learning Rate Scheduling	159
G.7	Tabella comparativa degli ottimizzatori	160
H	Perceptrone	161
H.1	Struttura	161
H.2	Esempio pratico	162
H.3	Training	162
H.4	Limiti: il problema XOR	163
I	Reti Neurali Feed-Forward	164
I.1	Architettura	164
I.2	Forward Propagation	165
I.3	Loss Function: MSE	165
I.4	Backpropagation	165
J	Reti Neurali Ricorrenti (RNN)	167
J.1	Il loop nello stato nascosto	167
J.2	Srotolamento nel tempo	168
J.3	Vanishing e Exploding Gradient	168
K	LSTM (Long Short-Term Memory)	169
K.1	Architettura: le 4 equazioni	169
L	GRU (Gated Recurrent Unit)	172
L.1	Confronto GRU vs LSTM	173
M	Reti Neurali Convoluzionali (CNN)	173
M.1	Architettura overview	174
M.2	Convolution Layer	174
M.3	Pooling	177
M.4	Fully Connected e Output	178
M.5	Training	179
M.6	ResNet e Residual Connections	180
N	Transformer	181
N.1	Architettura overview	181
N.2	Input: Tokenizzazione ed Embedding	183
N.3	Encoder	183

N.4	Scaled Dot-Product Attention	184
N.5	Multi-Head Attention	186
N.6	Feed-Forward Network (FFN)	186
N.7	Decoder	187
N.8	Positional Encoding	188
N.9	Riassunto e confronto con il Perceiver	189
O	Vision Transformer (ViT)	189
O.1	Pipeline completa	190
O.1.1	1. Patch Splitting	190
O.1.2	2. Flattening	190
O.1.3	3. Linear Projection (Patch Embedding)	190
O.1.4	4. CLS Token	190
O.1.5	5. Positional Encoding	191
O.1.6	6. Transformer Encoder ($L = 12$ layer)	191
O.1.7	7. Classificazione (MLP Head)	191
O.2	ViT vs CNN: differenze fondamentali	192
P	Complementi: Self-Attention vs Cross-Attention	193
P.0.1	Cosa cambia sostanzialmente con tipi di input diversi nel Perceiver?	196
P.0.2	Come cambiano le matrici e gli step successivi	196
P.0.3	Tabella Riassuntiva	198
Q	Dropout	199
Q.1	Meccanismo: inverted dropout	199
Q.2	Train vs. inference	199
Q.3	Interpretazione: ensemble implicito	199
Q.4	Dove e quando si applica	199
R	Inizializzazione dei Pesi: Xavier e He	200
R.1	Il problema: propagazione della varianza	200
R.2	Xavier/Glorot Initialization	201
R.3	He/Kaiming Initialization	202
R.4	Tabella comparativa	202
S	Regolarizzazione L1 e L2 (Weight Decay)	202
S.1	L2 Regularization (Ridge / Weight Decay)	202
S.2	L1 Regularization (Lasso)	203
S.3	L2 nella loss vs. weight decay in Adam	203
T	Data Augmentation	204
T.1	Tecniche standard (pipeline ImageNet)	204
T.2	Tecniche avanzate	204
T.3	Perché è più critica per il Perceiver	205
U	Perceiver IO — Risultati Sperimentali	205
U.1	Optical Flow (Sintel)	206
U.2	Language Modeling a livello byte	206
U.3	Multimodal Autoencoding (Kinetics-700)	207

U.4	Classificazione ImageNet (mantenuta)	207
U.5	Messaggio chiave: generalità dimostrata	207
V	Domande Probabili per l'Esame	208
V.1	Sul Perceiver (domande specifiche sul progetto)	209
V.2	Sulla teoria del corso (domande generali)	211
V.3	Sul codice e gli esperimenti	212

1 Perceiver

Convenzione sulle citazioni e mappa delle appendici del paper

In questi appunti si distingue sempre tra:

- **App. X** (senza articolo, abbreviato) = appendice del *paper originale*.
- **Appendice X** (nome esteso, o tramite `\ref{app:...}`) = appendice di *questi appunti*.

Appendici del paper “Perceiver” (Jaegle et al., 2021), riferimenti chiave:

- **App. A** — Extended related work: Set Transformer, Linformer, altre architetture di attenzione efficiente.
- **App. B** — Ablations: include le Tabelle 5–7 (weight sharing, cross-attends) e le Figure 5–6 (effetto di N, D, L, T). Citata in §1.8.
- **App. C** — Architectural details: LayerNorm pre-norm, single-head cross-attention, $d_{QKV} = \min(D, C) = 261$ per ImageNet, no bias in Q/K/V, no dropout, inizializzazione latenti $\mathcal{N}(0, 0.02^2)$ troncata. Citata più volte in §1.4.5.
- **App. D** — Position encodings and Fourier features: frequenze linearly spaced tra 1 e $\mu/2$ (Nyquist), confronto esplicito con NeRF ($f_k = 2^k$). Citata in §1.4.2.
- **App. E** — Audiovisual attention maps: visualizzazioni delle attention maps sui task AudioSet.
- **App. F** — Notes on changes from the original version: fix principali rispetto alla prima versione arXiv (rimozione dropout, pooling prima della proiezione). Citata in §1.4.11.

Il **Perceiver** (Jaegle et al., 2021) è un’architettura basata su Transformer progettata per elaborare **modalità di input molto diverse** (immagini, audio, video, point cloud) con un’**unica architettura**, senza componenti domain-specific. L’idea centrale è proiettare un input di dimensione arbitraria M in un **latent array** L di dimensione fissa $N \ll M$ tramite cross-attention, rendendo il costo *lineare* in M (invece che quadratico) e soprattutto *disaccoppiando la profondità della rete dalla dimensione dell’input*: si possono impilare molti layer di self-attention nello spazio latente senza che il costo cresca con M . Il testo viene affrontato nell’estensione Perceiver IO (§2).

1.1 Il problema della complessità quadratica

Le **CNN** sono scalabili ($O(MKL)$, lineare in M) ma richiedono architetture specializzate per ogni dominio. I **Transformer** sono generali ma soffrono di complessità quadratica $O(M^2)$ nella *self-attention*, che li rende impraticabili su input ad alta dimensionalità (es. $M = 50,176$ pixel per ImageNet 224×224).

Il Perceiver unisce il meglio di entrambi: la **generalità** dei Transformer e la **scalabilità** delle CNN, introducendo un *latent bottleneck* che riduce la complessità.

Riduzione della complessità

$$\underbrace{O(M^2)}_{\text{Self-attention Transformer}} \longrightarrow \underbrace{O(MN)}_{\text{Cross-attention}} + \underbrace{O(N^2)}_{\text{Latent self-attention}}$$

Con $N = 512$ e $M = 50,176$: il fattore di riduzione è $\approx 100\times$.

1.2 Differenze con il Transformer

Nel Transformer standard, la **self-attention** calcola Q, K e V tutti dallo stesso input X : ogni token confronta se stesso con tutti gli altri, producendo una matrice di attenzione quadrata $M \times M$. Il Perceiver sostituisce questo meccanismo con la **cross-attention**, dove Q proviene da un array diverso (i latenti L) rispetto a K e V (che provengono dall'input X).

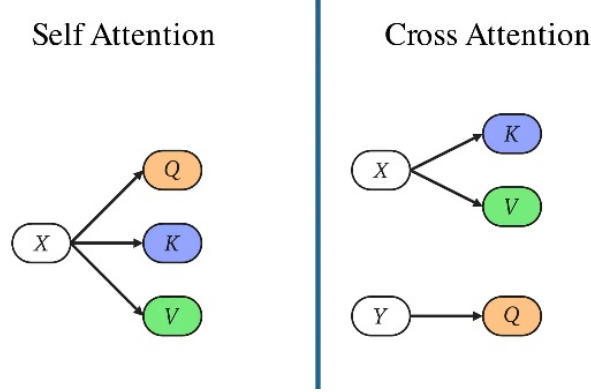


Figure 1: Self-Attention vs Cross-Attention. Nella self-attention, Q, K, V provengono tutti dallo stesso input X . Nella cross-attention, K e V provengono dall'input X , mentre Q proviene da un altro array (i latenti L nel Perceiver).

Aspetto	Transformer (Self-Att)	Perceiver (Cross-Att)
Sorgente Q	Input X (M token)	Latenti L (N latenti)
Sorgente K, V	Input X (M token)	Input X (M token)
Matrice attenzione	$M \times M$ (quadrata)	$N \times M$ (rettangolare)
Complessità	$O(M^2)$	$O(MN)$, con $N \ll M$

I **latenti** sono *parametri appresi* (inizializzati come learned position encodings, cfr. Appendice C del paper), non uno stato che dipende dall'esempio di input come accade in un RNN. Diventano però analoghi a uno stato ricorrente a causa delle T iterazioni con *weight sharing*: lo stesso modulo cross-attention + latent transformer viene riapplicato più volte ai latenti, in modo simile a un RNN che srotola il proprio passo nel tempo. A differenza di un RNN, però, a ogni iterazione i latenti accedono all'intero input simultaneamente, non a un elemento alla volta.

Rispetto al concetto tradizionale dei Transformer:

- **Input:** $M \times C$; come nei Transformer con M numero di unità di input e C dimensione dell'embedding.
- **Latenti:** $N \times D$; introdotti dal Perceiver, dove N è il numero di latenti e D la dimensione di ciascun latente.
- **Query (Q)** $N \times D$: le query provengono dai latenti, di numero fisso N , indipendentemente dalla dimensione dell'input M . Avere N diverso da M consente di mantenere la complessità computazionale gestibile, poiché $N \ll M$.
- **Key (K) e Value (V)** $M \times D$: derivano direttamente dall'input, che ha M elementi, ognuno con una rappresentazione di dimensione D .

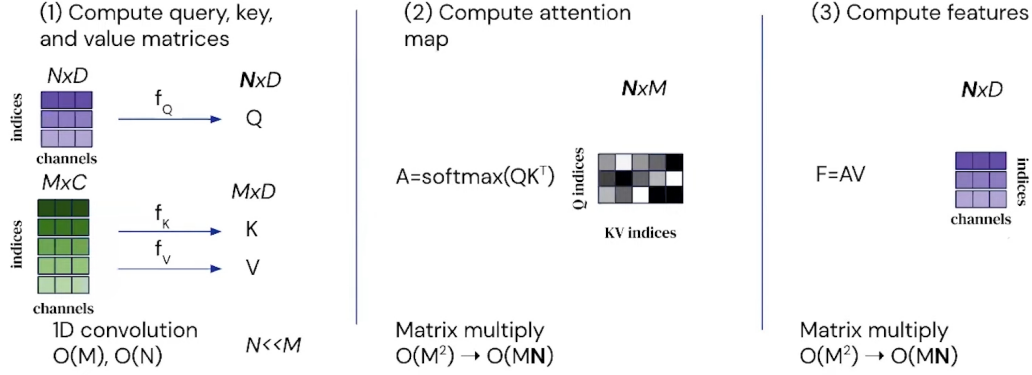


Figure 2: Schema della cross-attention nel Perceiver (vista d'insieme dei 3 step). Le singole fasi vengono analizzate in dettaglio di seguito.

La complessità dell'attenzione passa da $O(M^2)$ a $O(MN)$, poiché le Query hanno N righe (latenti) e le Key hanno M righe (input), con $N \ll M$.

1.2.1 Step (1): Calcolo delle matrici Q , K , V

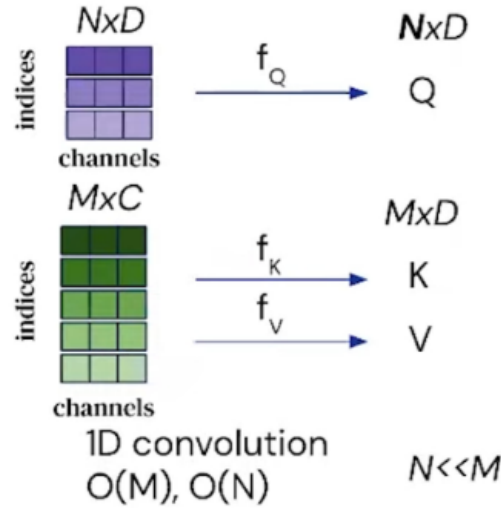


Figure 3: Step (1) — Il latent array ($N \times D$) viene proiettato in Q tramite f_Q , mentre l'input array ($M \times C$) viene proiettato in K e V tramite f_K e f_V . Le proiezioni hanno costo $O(N)$ e $O(M)$. $N \ll M$.

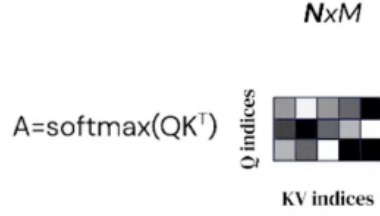
Il primo step calcola le matrici Q , K , V da due sorgenti diverse:

- Le **Query** vengono dal **latent array** (viola, $N \times D$): ogni latente “chiede” quale parte dell'input gli serve.
- Le **Key e Value** vengono dall'**input array** (verde, $M \times C$): ogni token dell'input “descrive” il proprio contenuto.

Questo è il punto chiave che distingue il Perceiver dal Transformer standard: Q e K/V *non* provengono dallo stesso array. La matrice di attenzione risultante avrà dimensione $N \times M$ (non $M \times M$).

1.2.2 Step (2): Calcolo della mappa di attenzione

(2) Compute attention map



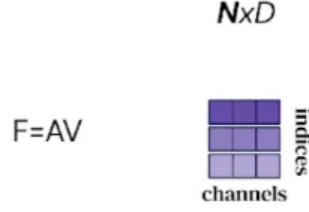
Matrix multiply
 $O(M^2) \rightarrow O(MN)$

Figure 4: Step (2) — $A = \text{softmax}(QK^T / \sqrt{d_{QKV}}) \in \mathbb{R}^{N \times M}$: la matrice di attenzione è *rettangolare* (N righe per M colonne), non quadrata ($M \times M$). La complessità scende da $O(M^2)$ a $O(MN)$.

Quando viene calcolata la mappa di attenzione, invece di essere all'interno di un unico insieme di dati (come i token di testo in un [Transformer](#) standard), il Perceiver calcola l'attenzione tra due insiemi distinti: **i latenti** e **l'input**. La [softmax](#) (Appendice A) trasforma i punteggi $QK^T / \sqrt{d_{QKV}}$ in pesi non-negativi che sommano a 1 lungo ogni riga. Il risultato: una matrice rettangolare $N \times M$ invece di quadratica $M \times M$, riducendo la complessità da $O(M^2)$ a $O(MN)$ dove N è solitamente molto più piccolo di M .

1.2.3 Step (3): Calcolo delle feature

(3) Compute features



Matrix multiply
 $O(M^2) \rightarrow O(MN)$

Figure 5: Step (3) — $F = AV \in \mathbb{R}^{N \times d_{QKV}}$: ogni latente ora contiene una media pesata dei Value dell’input, secondo i pesi di attenzione calcolati nello step (2). Complessità: $O(M^2) \rightarrow O(MN)$. La dimensione D dei latenti viene raggiunta solo *dopo* la proiezione di output W_o (cfr. §1.4.5), non qui.

Il calcolo delle feature segue lo stesso comportamento: la moltiplicazione $F = AV$ produce un output di dimensione $N \times d_{QKV}$ (es. 512×261 per ImageNet), molto più piccolo rispetto all’input originale $M \times C$ (50176×261). L’output è indipendente dalla dimensione dell’input, il che rende più facile costruire reti profonde partendo dai latenti.

Queste modifiche portano a due vantaggi fondamentali:

- **Complessità lineare rispetto all’input:** il calcolo dell’attenzione diventa $O(MN)$ grazie alla trasformazione della matrice da quadrata ($M \times M$) a rettangolare ($N \times M$).
- **Mappatura indipendente dalla dimensione dell’input:** il Perceiver può trattare input molto grandi trasformandoli in una rappresentazione compatta attraverso i latenti, condensando informazioni complesse in una forma gestibile.

1.3 Architettura

L'idea centrale del Perceiver è separare la dimensione dei dati in ingresso dalla capacità espressiva della rete. Un input arbitrariamente grande — un'immagine, un segnale audio, un point cloud, un testo byte-level — viene prima proiettato in un **latent array** di dimensione fissa $N \ll M$, e solo su questo array la rete svolge la sua elaborazione profonda. Ne conseguono due proprietà che definiscono l'architettura: lo stesso modello gestisce modalità molto diverse senza modifiche, e la profondità della rete non è più vincolata dalla lunghezza dell'input.

L'elaborazione si snoda in tre stadi. Un **encoder a cross-attention** legge l'input $X \in \mathbb{R}^{M \times C_{\text{tot}}}$ e lo comprime nel latent array $L \in \mathbb{R}^{N \times D}$. Un **latent transformer** raffina L applicando self-attention profonda, interamente nello spazio compresso. Una **testa di pooling e classificazione** aggrega infine i latenti e produce i logit. In questa sezione ne vediamo la struttura d'insieme; i dettagli algoritmici di ciascun blocco sono sviluppati nel Forward Pass (§1.4).

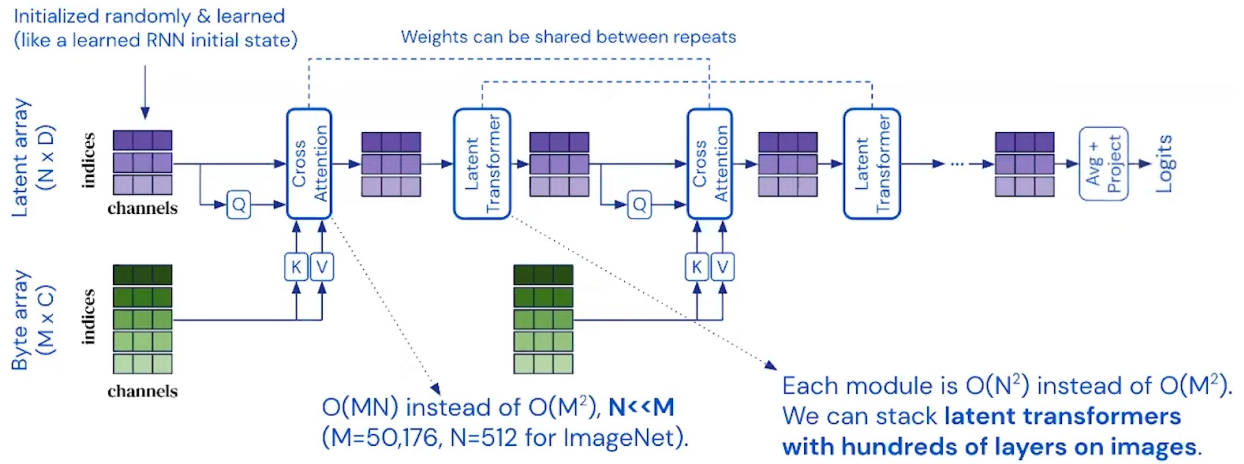


Figure 6: Architettura del Perceiver: il Byte Array ($M \times C$) entra da sinistra, viene letto dal Latent Array ($N \times D$) tramite Cross-Attention, poi i latenti vengono raffinati dal Latent Transformer, e infine mediati (Average + Project) per produrre i Logits. I pesi possono essere condivisi tra le iterazioni T (linea tratteggiata in alto).

La Figura 6 illustra il flusso dei dati da sinistra verso destra. L'input grezzo — ad esempio un'immagine 224×224 — viene srotolato in un *byte array* di forma $M \times C$ e arricchito con Fourier features posizionali, ottenendo una matrice $M \times C_{\text{tot}}$. Questa matrice entra nel blocco di **cross-attention**, che agisce in modo asimmetrico: le Query sono prodotte dai latenti (sono solo N , poche), mentre Key e Value provengono dall'input (sono M , molte). La matrice di attenzione risulta rettangolare $N \times M$ e il costo scende da $O(M^2)$ a $O(MN)$, rendendo trattabile l'elaborazione di input ad altissima dimensionalità.

Il latent array così popolato passa al **latent transformer**, una pila di L blocchi di self-attention che operano interamente nello spazio compresso e raffinano iterativamente la rappresentazione. L'intero modulo cross-attention + latent transformer viene poi ripetuto T volte: come indicato dalla linea tratteggiata in alto nella figura, i pesi possono essere condivisi tra le iterazioni, e in quel caso il modello si comporta come un RNN che affina lo stesso stato latente ad ogni passo. Al termine, un **global average pooling** sui latenti produce un vettore di dimensione D , che un classificatore lineare proietta nei logit delle classi.

La tabella seguente riassume i valori di questi iperparametri (N , D , M , T , L , H , d_k) nella configurazione ImageNet del paper originale (Jaegle et al., 2021).

Table 1: Iperparametri del Perceiver per ImageNet (Jaegle et al., 2021, §4.1 + App. C).

Simbolo	Valore	Descrizione
<i>Dimensioni globali</i>		
N	512	Numero di latenti
D	1024	Dimensione di ciascun latente
M	50 176	Numero di pixel (224×224)
C	3	Canali RGB
C_{tot}	261	$C + d(2K + 1) = 3 + 258$ (canali + Fourier features)
K	64	Bande di frequenza per asse
<i>Iterazioni e profondità</i>		
T	8	Iterazioni (cross-att + latent transformer)
ℓ	6	Layer di self-attention per blocco del latent transformer
<i>Cross-attention (encoder: input $\rightarrow$ latenti)</i>		
H_{cross}	1	Numero di teste (single-head, App. C)
d_{QKV}	261	Dim. di Q, K, V = $\min(D, C_{\text{tot}})$
<i>Self-attention (latent transformer)</i>		
H_{self}	8	Numero di teste
d_{head}	128	Dim. per testa = D/H_{self}

Input $\rightarrow$ Fourier $\rightarrow$ Latenti $\rightarrow$ Cross-Att $\rightarrow$ Latent Transf. $\rightarrow \times T \rightarrow$ Pooling

1.4 Forward Pass

Il **forward pass** traccia in dettaglio come i dati attraversano l'architettura descritta in §1.3 (Figura 6), dalla preparazione dell'input fino ai logit. Il **backward pass** (§1.5) ripercorre lo stesso cammino in senso inverso per calcolare i gradienti e aggiornare i pesi. In questa sezione seguiamo la pipeline step-by-step, evidenziando nel riquadro sottostante lo step corrente.

Il forward pass trasforma un input di dimensione arbitraria (M elementi) in una classificazione, passando attraverso uno spazio latente compatto (N elementi, con $N \ll M$).

Il processo si articola in due fasi principali:

1. **Preparazione dell'input:** l'immagine grezza viene trasformata in un byte array arricchito con informazioni posizionali (Fourier features), producendo una matrice $M \times C_{\text{tot}}$.
2. **Elaborazione nel latent space:** un latent array appreso ($N \times D$) interroga l'input tramite cross-attention, poi viene raffinato iterativamente dal latent transformer. Infine, i latenti vengono mediati (pooling) e classificati.

Di seguito analizziamo ogni step della pipeline, evidenziando nel riquadro sottostante lo step corrente:

1.4.1 Immagine di Input (Byte Array)



L'input, rappresentato come un byte array $M \times C$, rappresenta un modo uniforme per gestire input eterogenei, indipendentemente dalla loro origine o modalità (immagini, testo, audio, ecc.).

Nel caso studiato per **ImageNet** abbiamo un'immagine RGB di dimensione 224×224 questa verrà processata:

- Ogni pixel dell'immagine è rappresentato da tre valori (canali RGB), quindi l'immagine può essere vista come una matrice di dimensioni $[224 \times 224] (M) \times [3] (C)$
- Il numero totale di pixel sarà $[224 \times 224] = 50176$
- Il numero totale di valori (byte) sarà $[224 \times 224] \times [3] = 150528$

L'immagine viene convertita in un array lineare o "appiattito" (semplice **reshape** della griglia di pixel in una sequenza).

Ogni pixel (con i suoi 3 canali RGB) dell'immagine viene linearizzato e questa viene trasformata in un array monodimensionale di lunghezza 150528

$$\text{Byte Array} = [r_1, g_1, b_1, r_2, g_2, b_2, \dots, r_{224 \times 224}, g_{224 \times 224}, b_{224 \times 224}]$$

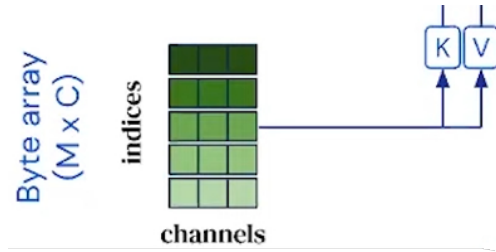


Figure 7: Il byte array ($M \times C$): ogni riga è un pixel con C canali. Da questo array vengono generate le matrici Key (K) e Value (V) per la cross-attention.

Checkpoint dimensioni — fine §1.4.1

A questo punto abbiamo l'immagine cruda organizzata come matrice:

Oggetto	Forma simbolica	ImageNet	Contenuto di una riga
Byte array (grezzo)	$M \times C$	$50\,176 \times 3$	R, G, B del pixel

Dove $M = 224 \times 224 = 50\,176$ è il numero totale di pixel e $C = 3$ sono i canali RGB.

È solo il contenuto "cromatico" dei pixel. **Manca l'informazione posizionale:** questo byte array da solo, dato al Perceiver, non distinguerebbe due immagini che differiscono solo per una permutazione dei pixel. La prossima sezione aggiunge la posizione tramite Fourier features.

1.4.2 Positional Encoding (Fourier Features)



Nota bene: Il Perceiver **non lavora direttamente solo con i 3 canali RGB**, ma arricchisce **ciascun pixel con informazioni di posizione (x,y)**. Infatti, un pixel con valore [R,G,B] contiene solo informazione di colore, ma **non contiene informazione di posizione** (es: “questo rosso è in alto a sinistra o in basso a destra?”). Senza posizione, il modello non potrebbe distinguere due immagini identiche ma traslate.

Per fare ciò utilizziamo le **Fourier Features**. Queste sono una tecnica di mappatura che trasforma un input scalare o vettoriale $x \in \mathbb{R}^d$ in uno spazio a dimensionalità più alta, applicando funzioni seno e coseno a frequenze multiple.

Formalmente, data una matrice di frequenze $B \in \mathbb{R}^{m \times d}$, la mappatura delle Fourier features è definita come:

$$\gamma(x) = [\cos(\pi Bx), \sin(\pi Bx)] \in \mathbb{R}^{2m}$$

dove:

- x è il vettore di input (d-dimensionale),
- B contiene i vettori di frequenze (scelti in modo deterministico o campionati da una distribuzione, ad esempio gaussiana),
- il risultato è un embedding di dimensione $2m$, in cui ogni componente dell’input viene rappresentato tramite combinazioni sinusoidali a diverse scale.

Sia (x, y) la posizione normalizzata di un pixel (tipicamente in $[-1, 1]$ o $[0, 1]$).

Per una serie di frequenze $f_1, f_2, \dots, f_L$, calcoliamo:

$$\phi(x) = [\sin(f_1\pi x), \cos(f_1\pi x), \dots, \sin(f_L\pi x), \cos(f_L\pi x)]$$

$$\phi(y) = [\sin(f_1\pi y), \cos(f_1\pi y), \dots, \sin(f_L\pi y), \cos(f_L\pi y)]$$

Le frequenze f_k sono **linearly spaced** tra 1 e $\mu/2$, dove $\mu/2$ è la **frequenza di Nyquist** corrispondente al sampling rate di riferimento μ (Jaegle et al., 2021, §3.2 e App. D). Per ImageNet 224×224 : $\mu = 224$ e $\mu/2 = 112$. Le basse frequenze catturano posizioni globali (“metà sinistra vs. destra”), le alte frequenze catturano posizioni precise fino al limite di Nyquist.

Calcolo di C : perché 261 e non 3

Formula dal paper (Jaegle et al., 2021, §3.2 + App. D):

Per ogni dimensione spaziale d , il Perceiver costruisce un vettore posizionale concatenando le Fourier features con la **coordinata grezza** x_d :

$$\text{PE}(x_d) = \left[\underbrace{\sin(f_1\pi x_d), \cos(f_1\pi x_d), \dots, \sin(f_K\pi x_d), \cos(f_K\pi x_d)}_{2K \text{ valori sinusoidali}}, \underbrace{x_d}_{1 \text{ coordinata grezza}} \right]$$

dove:

- $x_d \in [-1, 1]$ è il **valore grezzo della coordinata** lungo la dimensione d , normalizzato nell'intervallo $[-1, 1]$. Per un'immagine 2D: $x_1 = x$ (posizione orizzontale), $x_2 = y$ (posizione verticale).
- $f_k, k = 1, \dots, K$, sono le frequenze, **linearly spaced tra 1 e $\mu/2$** (Nyquist del sampling rate di riferimento). Per ImageNet: $\mu/2 = 112$.
- K è il numero di bande di frequenza per asse (nel paper: $K = 64$).

Perché concatenare la coordinata grezza x_d ?

Le Fourier features (sin/cos) sono **periodiche**: frequenze alte possono mappare posizioni diverse nello stesso valore. Concatenando il valore grezzo x_d (che è *unico* per ogni posizione), il modello ha sempre accesso alla posizione esatta, senza ambiguità. Le sinusoidi forniscono sensibilità multi-scala, il valore grezzo fornisce l'identità posizionale.

Calcolo per un'immagine 2D con $d = 2$ assi e $K = 64$ bande:

Per ogni asse: $2K + 1 = 2 \times 64 + 1 = 129$ valori (128 sinusoidi + 1 coordinata grezza).

Con $d = 2$ assi (x e y): $d \times (2K + 1) = 2 \times 129 = 258$ valori posizionali.

Sommati ai 3 canali RGB:

$$C_{\text{tot}} = \underbrace{3}_{\text{RGB}} + \underbrace{d \times (2K + 1)}_{2 \times 129 = 258 \text{ Fourier} + \text{coord. grezze}} = \mathbf{261}$$

Ogni pixel diventa un vettore di 261 valori:

$$\left[R, G, B, \underbrace{x_1, \sin(f_1 \pi x_1), \cos(f_1 \pi x_1), \dots, \cos(f_{64} \pi x_1)}_{129 \text{ per asse } x}, \underbrace{x_2, \sin(f_1 \pi x_2), \dots, \cos(f_{64} \pi x_2)}_{129 \text{ per asse } y} \right]$$

Esempio concreto: un pixel in posizione (0.3, 0.7) con colore rosso

- Canali RGB: $[255, 0, 0]$ (normalizzati: $[1.0, 0, 0]$)
- Posizione normalizzata: $(x, y) = (0.3, 0.7)$
- Fourier per $x = 0.3$ con 3 frequenze illustrative ($f_1 = 1, f_2 = 2, f_3 = 4$, scelte per comodità di calcolo; il paper ne usa 64 linearly spaced tra 1 e 112):

$$\begin{aligned} & [0.3, \sin(0.3\pi), \cos(0.3\pi), \sin(0.6\pi), \cos(0.6\pi), \sin(1.2\pi), \cos(1.2\pi), \dots] \\ & = [0.3, 0.809, 0.588, 0.951, -0.309, -0.588, 0.809, \dots] \end{aligned}$$

- Analogamente per $y = 0.7$
- Vettore finale: $[1.0, 0, 0, \underbrace{0.3, 0.809, 0.588, \dots}_{129}, \underbrace{0.7, \dots}_{129}] \in \mathbb{R}^{261}$

Perché **concatenare** e non sommare? Perché la concatenazione preserva l'informazione di colore e posizione separatamente, mentre la somma la mescolerebbe in modo ambiguo. Il Perceiver usa la concatenazione (a differenza dei Transformer per il linguaggio che sommano il positional encoding).

Perché non le frequenze di NeRF ($f_k = 2^k$)?

NeRF (Mildenhall et al., 2020) usa frequenze **esponenziali** $f_k = 2^k$, mentre il Perceiver le prende linearly spaced tra 1 e $\mu/2$. Il paper giustifica esplicitamente la scelta (App. D):

“la parametrizzazione NeRF produce frequenze molto alte anche con un numero moderato di bande — ad esempio la 64-esima banda avrebbe frequenza $2^{64} \approx 1.8 \times 10^{19}$ ”.

Con $K = 64$ bande, NeRF porterebbe a valori numerici del tutto incompatibili con la precisione in virgola mobile. Il paper riporta inoltre **instabilità di training** già oltre $k \approx 15$ bande con la parametrizzazione esponenziale. Lo spacing lineare evita il problema perché, aggiungendo bande, si campiona più densamente lo spettro nel range $[1, \mu/2]$ anziché salire indefinitamente. Nel codice di progetto (`src/utils/positional_encoding.py`) le frequenze sono ottenute con `torch.linspace`, coerentemente col paper.

Quindi il Byte Array finale ha dimensione:

$$\text{Byte Array} \in \mathbb{R}^{M \times C_{\text{tot}}} = \mathbb{R}^{50176 \times 261}$$

Checkpoint dimensioni — fine §1.4.2

Dopo l’aggiunta delle Fourier features, ogni pixel è descritto da $C_{\text{tot}} = 261$ numeri invece di $C = 3$:

Oggetto	Forma simbolica	ImageNet	Contenuto di una riga
Byte array (grezzo)	$M \times C$	$50\,176 \times 3$	RGB
Byte array (con Fourier PE) X	$M \times C_{\text{tot}}$	$50\,176 \times 261$	3 RGB + 258 Fourier

Le costanti simboliche:

- M = numero di elementi di input (50 176 per ImageNet)
- C = canali originali (3 per RGB)
- $C_{\text{tot}} = C + d(2K + 1)$: canali totali dopo Fourier, con d = numero di dimensioni spaziali (2 per immagini) e K = numero di bande di frequenza (64 per ImageNet, paper §4.1)
- Per ImageNet: $C_{\text{tot}} = 3 + 2 \cdot (2 \cdot 64 + 1) = 3 + 258 = 261$

Da qui in avanti, quando si scrive “byte array” si intende la versione arricchita $X \in \mathbb{R}^{M \times C_{\text{tot}}}$. I valori posizionali rendono il Perceiver capace di distinguere tra due immagini che differiscono per una permutazione dei pixel (cfr. Tab. 2 del paper).

1.4.3 Proiezione Lineare Input



Proiezione Lineare $C_{\text{tot}} = 261 \rightarrow D = 1024$

Nel paper originale, le proiezioni K e V nella cross-attention vengono applicate **direttamente** dal byte array ($C = 261$) allo spazio delle chiavi (d_k). Tuttavia, in alcune implementazioni si usa una proiezione lineare intermedia $C_{\text{tot}} \rightarrow D$ per uniformare le dimensioni:

Proiezione lineare dell’input (non nel paper originale)

Nell’**implementazione del paper** (Jaegle et al., 2021, App. C) *non* esiste una proiezione intermedia $C_{\text{tot}} \rightarrow D$: le matrici W_K, W_V proiettano direttamente dal byte array $\mathbb{R}^{50176 \times 261}$ a dimensione $d_{QKV} = \min(D, C_{\text{tot}}) = 261$.

Alcune implementazioni (e.g., il nostro codice di progetto) scelgono invece di proiettare l’input a $H \cdot d_{\text{head}}$, per uniformare il flusso multi-head. Nel nostro codice $H = 8$ e $d_{\text{head}} = 64$, quindi:

$$W_{\text{embed}} \in \mathbb{R}^{C_{\text{tot}} \times (H \cdot d_{\text{head}})}, \quad x \in \mathbb{R}^{261} \longrightarrow x_{\text{embed}} \in \mathbb{R}^{512}$$

La dimensione finale del latente $D = 1024$ viene raggiunta *dopo* l'attention, tramite la proiezione di output $W_o \in \mathbb{R}^{512 \times 1024}$ che riporta lo spazio a D per la **residual connection** (Appendice F) con i latenti.

Riepilogo delle dimensioni intermedie per ImageNet:

- **Paper:** $d_{QKV} = 261$, poi $W_o : 261 \rightarrow 1024$.
- **Nostro codice:** $d_{QKV} = 512$, poi $W_o : 512 \rightarrow 1024$.
- In *entrambi i casi*, la dimensione 1024 non compare *dentro* l'attention: è solo la dim dei latenti (input Q) e la dim di output dopo W_o .

Checkpoint dimensioni — fine §1.4.3

A questo punto del forward pass abbiamo una sola matrice, l'input pronto per la cross-attention:

Oggetto	Forma simbolica	ImageNet	Fonte
Byte array X	$M \times C_{\text{tot}}$	50 176 × 261	RGB (3) + Fourier PE (258)

Perché restiamo a 261 e non saliamo a 1024?

La tentazione sarebbe di “uniformare” subito il byte array alla dimensione dei latenti ($D = 1024$), ma *il paper non lo fa*. Il motivo è che la cross-attention che segue userà come dimensione interna il minimo tra le due sorgenti, $d_{QKV} = \min(D, C_{\text{tot}}) = \min(1024, 261) = 261$ (cfr. App. C del paper). Espandere l'input a 1024 solo per poi ricomprimerlo a 261 dentro la cross-attention sarebbe uno spreco di parametri e di FLOPs: i 261 valori per pixel contengono già tutta l'informazione utile (colore + posizione), aumentare la dimensione non aggiunge nulla. Formalmente, una proiezione lineare $261 \rightarrow 1024$ ha *rango* ≤ 261 : produrrebbe 1024 feature ma solo 261 linearmente indipendenti (le altre ridondanti). Espandere non può *creare* informazione che non c'è.

Il nostro codice fa invece una proiezione a $H \cdot d_{\text{head}} = 512$ perché adotta multi-head ($H = 8, d_{\text{head}} = 64$) anche nella cross-attention: in quel caso d_{QKV} è forzato dalla scelta delle teste, non dal minimo degli input. Ma è una divergenza implementativa, non una necessità del paper.

Cosa arriva adesso. La prossima sezione (§1.4.4) introduce il *latent array* L , la seconda matrice che serve alla cross-attention; poi §1.4.5 combina X e L producendo Q, K, V — aggiungeremo quelle righe al checkpoint.

1.4.4 Latent Array



È una **matrice di vettori latenti** $L \in \mathbb{R}^{N \times D}$ **interamente appresa** (parametri del modello).

- **N** = numero di latenti (slot/ “unità di memoria”).
- **D** = dimensione del canale/embedding dei latenti

Questi vettori **non derivano** direttamente dall’input e **non hanno un legame spaziale** con token/pixel specifici: **sono una base compatta che il modello usa per riassumere l’input**.

Nel caso specifico del Perceiver per ImageNet, l’array latente è $N = 512$ ($N \ll M(50176)$) e $D = 1024$.

L’array latente $L \in \mathbb{R}^{512 \times 1024}$ (per ImageNet) è inizializzato campionando ciascun elemento da una distribuzione:

$$L_{ij} \sim \mathcal{N}(0, 0.02^2), \text{ troncata a } [-2, 2].$$

$$L = \begin{bmatrix} 0.012 & -0.004 & 0.021 & \cdots & -0.017 \\ -0.009 & 0.015 & 0.003 & \cdots & 0.008 \\ 0.020 & 0.001 & -0.014 & \cdots & 0.027 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -0.006 & 0.022 & -0.010 & \cdots & 0.004 \end{bmatrix}_{512 \times 1024}$$

In pratica i numeri dentro L sono quasi tutti piccolissimi, tipicamente compresi fra -0.05 e +0.05 (perché la gaussiana è molto stretta attorno a 0).

Il troncamento a $[-2, 2]$ esiste per sicurezza, ma *con* $\sigma = 0.02$ è rarissimo che si producano valori oltre 0.1 - quindi quasi mai entra davvero in gioco.

Questi numeri sono i vettori di partenza dei 512 slot latenti: ognuno è un vettore di 1024 valori inizializzati così.

Perché?

- **Evitare gradienti instabili:** se i latenti partissero con valori grandi, i prodotti QK^T sarebbero enormi, la softmax saturerebbe, e i gradienti diventerebbero instabili.
- **Garantire neutralità:** media zero significa che nessun latente ha un “vantaggio” iniziale su un altro.
- **Sicurezza numerica:** il troncamento assicura che non ci siano valori anomali che potrebbero falsare il training nelle prime epoche.
- **Permettere specializzazione graduale:** partendo con valori piccoli e neutri, i latenti vengono “riempiti” progressivamente dall’informazione dell’input, e col tempo ciascuno si specializza (bordo, colore, texture, pattern globali...).

Checkpoint dimensioni — fine §1.4.4

Adesso abbiamo i due array che la cross-attention metterà in comunicazione:

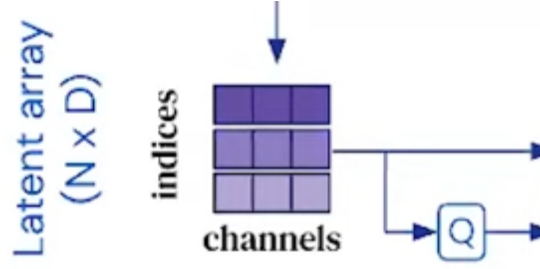


Figure 8: Latent array ($N \times D$): matrice appresa che genera le Query (Q) per la cross-attention.

Oggetto	Forma simbolica	ImageNet	Natura
Byte array X	$M \times C_{\text{tot}}$	$50\,176 \times 261$	input (3 RGB + 258 Fourier)
Latent array L	$N \times D$	512×1024	parametri appresi, init $\mathcal{N}(0, 0.02^2)$ troncata

Le nuove costanti simboliche:

- N = numero di latenti (512 per ImageNet, paper §4.1)
- D = dimensione di ciascun latente (1024 per ImageNet, paper §4.1)
- Vincolo fondamentale: $N \ll M$ (qui $N/M = 512/50\,176 \approx 1\%$), è ciò che permette la riduzione da $O(M^2)$ a $O(MN)$

Osservazioni:

- L non deriva dall'immagine: è un parametro del modello, uguale per tutti gli esempi del batch. Cambia solo durante il training (backward) — non durante l'inferenza.
- Le due matrici hanno dimensione-feature *diversa* ($C_{\text{tot}} = 261$ vs $D = 1024$). Sarà la cross-attention a farle incontrare in uno spazio comune: $d_{QKV} = \min(D, C_{\text{tot}}) = 261$.
- Il numero di righe è molto diverso ($M = 50\,176$ vs $N = 512$): è questa asimmetria che rende la complessità $O(MN)$ invece di $O(M^2)$.

Prossimo passo (§1.4.5): da X e L otterremo Q, K, V .

1.4.5 Cross-Attention Block

Input → Fourier → Latenti → **Cross-Att** → Latent Transf. → $\times T$ → Pooling

Cross-attention: single-head nel paper, multi-head nel nostro codice

Per ImageNet il paper originale (App. C) usa una cross-attention a **singola testa** ($H_{\text{cross}} = 1$). Q, K, V hanno tutte dimensione $d_{QKV} = 261$, pari al *minimo* tra i canali dei latenti ($D = 1024$) e quelli dell'input ($C_{\text{tot}} = 261$). L'output dell'attention viene riportato a $D = 1024$ tramite una proiezione finale $W_o \in \mathbb{R}^{261 \times 1024}$, così da poterlo sommare residualmente ai latenti.

Il nostro codice di progetto (`src/perceiver/attention.py`, classe `CrossAttention`) adotta invece una convenzione più standard, con cross-attention **multi-head** ($H = 8$, $d_{\text{head}} = 64$, totale 512). È una scelta legittima e più simile alla pratica corrente, ma *diverge dal paper*: quando si descrive l'architettura originale occorre tenere presente la distinzione.

Nel seguito di questa sezione descriviamo la formulazione della cross-attention usando d_k come dimensione generica di proiezione; il valore esatto (261 nel paper, 64 per testa nel nostro codice) dipende dal setting.

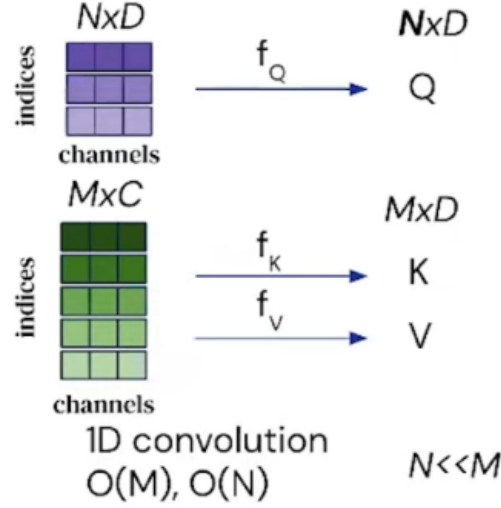


Figure 9: Step (1) del blocco: calcolo delle matrici Q , K , V . Le Query provengono dal latent array ($N \times D$), le Key e Value dall’input array ($M \times C_{\text{tot}}$). $N \ll M$.

Cosa deve fare il blocco. La cross-attention deve permettere agli N latenti di “leggere” informazione dai M elementi dell’input byte array. Concretamente: ciascun latente emette una *Query* (una domanda: “cosa sto cercando?”), ogni elemento dell’input produce una *Key* (una descrizione: “io rappresento questo”) e un *Value* (un contenuto informativo effettivo). Il blocco calcola la compatibilità tra ogni query e ogni key, la trasforma in una distribuzione di probabilità tramite softmax, e la usa per aggregare i value in un nuovo latente.

Punto di partenza: due array da armonizzare. All’ingresso del blocco abbiamo due matrici di forme molto diverse:

- Il **byte array** $X \in \mathbb{R}^{50176 \times 261}$ (un’immagine ImageNet dopo aver concatenato RGB + Fourier features: 50 176 pixel, 261 valori ciascuno).
- Il **latent array** $L \in \mathbb{R}^{512 \times 1024}$ (parametri appresi del modello: 512 latenti di dimensione 1024).

Le due matrici vivono in spazi di dimensione differente (261 vs 1024): *prima* di calcolare Q , K , V dobbiamo decidere in quale spazio far incontrare queste due sorgenti.

Passaggio 1: LayerNorm pre-attention. Il paper (App. C) specifica che entrambi gli array, prima delle proiezioni lineari che producono Q , K , V , attraversano una **LayerNorm** (pattern pre-norm). La LayerNorm è un’operazione *riga per riga*: per ogni pixel del byte array (o per ogni latente) si calcolano media e deviazione standard delle sue feature e si riporta la distribuzione a media 0 e varianza 1, poi si applicano due parametri appresi γ e β per-feature.

Cosa fa in dettaglio, per ogni riga. Consideriamo una generica riga $x = (x_1, \dots, x_F)$ (con $F = 261$ nel byte array, $F = 1024$ nel latente). LayerNorm esegue questi 4 passaggi:

1. Calcola la **media** sulla riga:

$$\mu = \frac{1}{F} \sum_{j=1}^F x_j$$

2. Calcola la **deviazione standard** sulla riga:

$$\sigma = \sqrt{\frac{1}{F} \sum_{j=1}^F (x_j - \mu)^2}$$

3. **Normalizza** ogni elemento: $\hat{x}_j = (x_j - \mu)/(\sigma + \epsilon)$, con $\epsilon \approx 10^{-5}$ per stabilità numerica. Dopo questo step la riga ha media 0 e varianza 1 *per costruzione matematica*.

4. Applica **scale e shift** appresi: $\tilde{x}_j = \gamma_j \cdot \hat{x}_j + \beta_j$, dove $\gamma, \beta \in \mathbb{R}^F$ sono vettori di parametri.

I parametri γ, β sono **diversi per ciascuna LayerNorm** del Perceiver (due blocchi distinti: uno sul byte array, uno sul latent array), e sono gli stessi per tutte le righe dentro la stessa LayerNorm (una LN su 50,176 pixel ha un solo vettore γ di dim 261, non 50,176 vettori diversi).

Dimensioni dei parametri di ciascuna LayerNorm:

LayerNorm	Input	γ, β	Output
LN _{input} (sul byte array)	$\mathbb{R}^{M \times C_{\text{tot}}} = \mathbb{R}^{50176 \times 261}$	$\mathbb{R}^{261}$ ciascuno	$\mathbb{R}^{50176 \times 261}$
LN _{latent} (sul latent array)	$\mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 1024}$	$\mathbb{R}^{1024}$ ciascuno	$\mathbb{R}^{512 \times 1024}$

Perché LayerNorm non cambia le dimensioni: ogni passaggio sopra (media, standardizzazione, scale, shift) è un'operazione *element-wise* (o per riga) che produce un output di forma identica all'input. La LayerNorm riceve $N \times F$ e restituisce $N \times F$, sempre.

Risultato finale:

$$\tilde{X} = \text{LN}_{\text{input}}(X) \in \mathbb{R}^{M \times C_{\text{tot}}} = \mathbb{R}^{50176 \times 261}, \quad \tilde{L} = \text{LN}_{\text{latent}}(L) \in \mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 1024}$$

Da qui in avanti, ogni volta che compaiono X e L nelle proiezioni, sono sempre queste versioni normalizzate.

Perché LayerNorm *prima* dell'attention (pre-norm)?

Il pattern *pre-norm* applica LN *prima* della sotto-funzione (attention o MLP) e somma il risultato al residuo non normalizzato: $y = x + f(\text{LN}(x))$. L'alternativa *post-norm* (del Transformer originale) è $y = \text{LN}(x + f(x))$.

Il pre-norm (usato dal Perceiver e da GPT-2) rende il training più stabile su reti profonde: il residuo x è un percorso "pulito" sul quale il gradiente può fluire senza attraversare la LayerNorm, evitando vanishing gradients nei modelli molto deep. È lo stesso motivo per cui il Perceiver preferisce questo schema.

Dettagli e derivazione completa della LayerNorm (forward + backward con la formula a 3 termini $1/(\sigma + \epsilon)$) si trovano in Appendice D di questi appunti.

Passaggio 2: scelta della dimensione interna dell'attention. Le matrici Q, K, V devono avere tutte la **stessa dimensione-feature** (chiamiamola d_{QKV}), altrimenti il prodotto scalare $QK^\top$ non è definito. Il paper (App. C, citazione letterale):

"The queries, keys, and values have the same number of channels as the minimum of the input channels, which is typically the key/value input (i.e. 261 for ImageNet)."

Cioè si sceglie:

$$d_{QKV} = \min(D, C_{\text{tot}}) = \min(1024, 261) = 261$$

Perché proprio il minimo? L'argomento è di efficienza. Se il byte array ha già solo 261 canali, proiettare K e V in una dimensione più grande (es. 1024) non aggiungerebbe informazione: il

contenuto informativo per pixel è già “tutto lì” in quei 261 numeri. Espandere sarebbe spreco di parametri e FLOPs. Scegliere il minimo tiene Q, K, V in una dimensione compatta (261) e usa 1024 solo dove serve davvero: come spazio dei latenti e come target di output per poter fare la residual connection (lo vedremo alla fine).

Passaggio 3: calcolo delle Query Q (dai latenti). Le query partono dai latenti, che vivono in dimensione $D = 1024$. Servono due cose da W_Q : accettare un vettore in input di dimensione 1024, e produrre un output in dimensione $d_{QKV} = 261$. Quindi:

$$W_Q \in \mathbb{R}^{D \times d_{QKV}} = \mathbb{R}^{1024 \times 261}$$

Il prodotto riga-per-riga è:

$$Q = \tilde{L} W_Q$$

dimensionalmente: $(512 \times 1024)(1024 \times 261) = (512 \times 261)$. Ogni latente ha prodotto una query di 261 valori:

$$Q \in \mathbb{R}^{512 \times 261}$$

I 512 restano (W_Q non cambia il numero di latenti), la dimensione-feature scende da 1024 a 261.

Passaggio 4: calcolo delle Key K (dall’input). Le key partono dal byte array, che vive già in dimensione $C_{\text{tot}} = 261$. Servono a W_K : accettare un vettore di 261 valori e produrre un output pure in $d_{QKV} = 261$. Quindi:

$$W_K \in \mathbb{R}^{C_{\text{tot}} \times d_{QKV}} = \mathbb{R}^{261 \times 261}$$

Il calcolo:

$$K = \tilde{X} W_K$$

dimensioni: $(50176 \times 261)(261 \times 261) = (50176 \times 261)$:

$$K \in \mathbb{R}^{50176 \times 261}$$

Passaggio 5: calcolo dei Value V (dall’input). Stesso procedimento, diversa matrice di pesi:

$$W_V \in \mathbb{R}^{261 \times 261}, \quad V = \tilde{X} W_V \in \mathbb{R}^{50176 \times 261}$$

Perché W_K e W_V sono "quadrati" 261×261 ? Può sembrare inutile: se input e output hanno la stessa dimensione, perché non usare la matrice identità? Perché la trasformazione è **appresa**: W_K impara a produrre key “ottimali” per il matching con le query (es. riduce il peso di feature rumorose, mescola canali correlati), e W_V impara a produrre value “ottimali” per essere poi pesati dall’attention. La dimensione è la stessa, ma lo *spazio* in cui vivono i 261 numeri è completamente diverso da quello di partenza.

Niente bias nelle proiezioni Q/K/V. Il paper (App. C) descrive le tre proiezioni come “linear layers” senza menzionare termini di bias. Il nostro codice conferma (`nn.Linear(..., bias=False)` in `src/perceiver/attention.py`, L84–85). Quindi non compare un $+b_Q, +b_K, +b_V$: la trasformazione è esattamente Wx , non $Wx + b$. I bias compaiono invece nei layer successivi (W_o , MLP, classificatore finale).

Riepilogo delle forme dopo Passaggio 5.

Matrice	Forma simbolica	ImageNet	Viene da	Significato riga
$\tilde{L}$ (input Q)	$N \times D$	512×1024	LN del latent array	un latente
$\tilde{X}$ (input K/V)	$M \times C_{\text{tot}}$	$50\,176 \times 261$	LN del byte array	un pixel
W_Q	$D \times d_{QKV}$	1024×261	pesi appresi	—
W_K	$C_{\text{tot}} \times d_{QKV}$	261×261	pesi appresi	—
W_V	$C_{\text{tot}} \times d_{QKV}$	261×261	pesi appresi	—
$Q = \tilde{L}W_Q$	$N \times d_{QKV}$	512×261	proiezione latenti	query di un latente
$K = \tilde{X}W_K$	$M \times d_{QKV}$	$50\,176 \times 261$	proiezione input	key di un pixel
$V = \tilde{X}W_V$	$M \times d_{QKV}$	$50\,176 \times 261$	proiezione input	value di un pixel

L'asimmetria chiave: Q ha *poche* righe ($N = 512$) e K, V ne hanno *tante* ($M = 50\,176$). Le colonne sono tutte uguali ($d_{QKV} = 261$). Questa asimmetria è ciò che permette al prodotto $QK^\top$ di essere rettangolare $N \times M = 512 \times 50\,176$ invece di quadrato $M \times M$: è la riduzione di complessità da $O(M^2)$ a $O(MN)$ descritta nella §1.1 (il problema della complessità quadratica).

Inizializzazione di W_Q, W_K, W_V : Xavier Uniform

Le matrici di peso vengono inizializzate con Xavier Uniform (Glorot & Bengio, 2010). Formalmente: $W_{ij} \sim U(-a, a)$ con $a = \sqrt{6/(fan_{in} + fan_{out})}$. Per $W_K : 261 \times 261$: $a = \sqrt{6/522} \approx 0.107$, quindi i pesi iniziali stanno in $[-0.107, +0.107]$. Xavier mantiene la varianza delle attivazioni costante attraverso i layer, evitando che i segnali esplodano o collapsino a zero. Dopo l'inizializzazione, i pesi evolvono tramite backpropagation — solo allora la rete “impara” qualcosa.

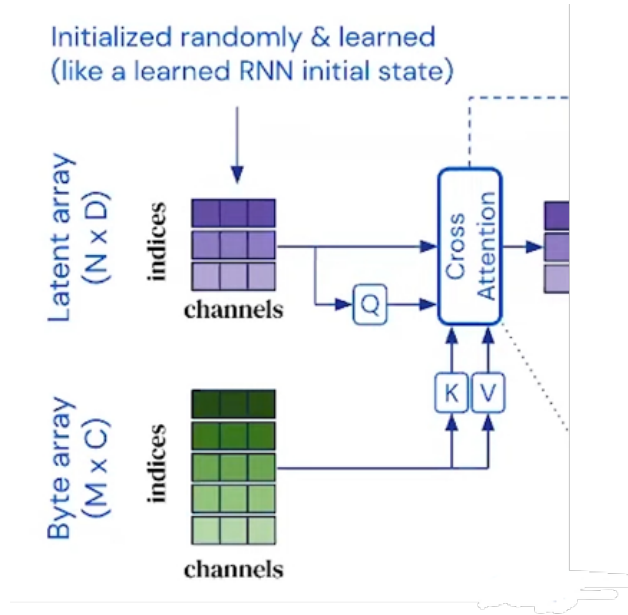


Figure 10: Schema complessivo della cross-attention: il latent array (512×1024) proiettato a Q (512×261) interroga il byte array ($50\,176 \times 261$) proiettato a K, V ($50\,176 \times 261$). Il prodotto $QK^\top$ produce una matrice di attenzione rettangolare $512 \times 50\,176$.

1.4.6 Scaled Dot-Product Attention

A questo punto disponiamo di $Q \in \mathbb{R}^{512 \times 261}$ dai latenti e di $K, V \in \mathbb{R}^{50176 \times 261}$ dall'input. Il blocco di *scaled dot-product attention* li combina in quattro operazioni principali (prodotto $QK^\top$, scaling, softmax, aggregazione con V), illustrate nello schema che segue.

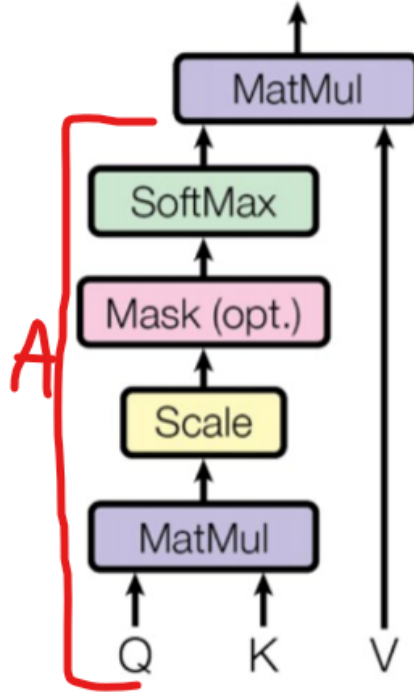


Figure 11: Pipeline della scaled dot-product attention nel Perceiver: $QK^\top$, scaling per $\sqrt{d_{QKV}}$, softmax per riga, prodotto finale con V . Il passo “Mask” non viene usato (il Perceiver è non-causale).

Passaggio 1: prodotto scalare $QK^\top$. Misuriamo la compatibilità tra ogni query e ogni key calcolandone il prodotto scalare: due vettori allineati producono un numero alto, due vettori in direzioni diverse un numero basso o negativo. In forma matriciale,

$$\text{Scores} = Q K^\top \in \mathbb{R}^{512 \times 50176},$$

dove ogni riga corrisponde a un latente, ogni colonna a un pixel, e l'entry (i, j) dice “quanto il latente i trova interessante il pixel j ”. L'intuizione tipica è quella di un latente specializzato, ad esempio, nel cercare il colore rosso: produrrà score alti sui pixel rossi e score bassi sugli altri, e la softmax successiva tradurrà questa graduatoria in una distribuzione di attenzione concentrata lì.

Passaggio 2: scaling per $\sqrt{d_{QKV}}$. Si divide ogni score per $\sqrt{d_{QKV}}$ prima della softmax:

Scaled Scores

$$\text{Scaled Scores} = \frac{QK^\top}{\sqrt{d_{QKV}}} \approx \frac{QK^\top}{16} \quad (\text{ImageNet}, \sqrt{261} \approx 16).$$

► Dimensioni invariate: $\mathbb{R}^{512 \times 50176}$.

Serve solo per evitare che i valori enormi saturino la softmax: senza scaling gli score hanno deviazione

standard $\sqrt{d_{QKV}} \approx 16$ (dopo LayerNorm Q, K hanno varianza 1, e sommare d_{QKV} prodotti la porta a d_{QKV}), la softmax concentra tutta la massa su un solo pixel e i gradienti si annullano.

Passaggio 2bis: masking (opzionale, non nel Perceiver). Alcune architetture applicano una maschera ai punteggi prima della softmax, mettendo a $-\infty$ le posizioni “proibite” (così $e^{-\infty} = 0$ e il peso diventa zero senza alterare la normalizzazione):

Masking

$$\text{Masked Scaled Scores}_{ij} = \begin{cases} \text{Scaled Scores}_{ij}, & \text{se } \text{Mask}_{ij} = 1 \\ -\infty, & \text{se } \text{Mask}_{ij} = 0 \end{cases}$$

Dove $\text{Mask}_{ij} = 1$ se la posizione è rilevante per la query, $= 0$ se va ignorata. È tipico di:

- **GPT** (causal mask): un token vede solo il passato, non il futuro.
- **BERT** (padding mask): ignora i token di riempimento nelle sequenze di lunghezza variabile.

► Il Perceiver **non usa maschere** (paper §3: “*all attention modules in the Perceiver are non-causal: we use no masks*”): ogni latente deve poter leggere tutti i 50 176 pixel dell’immagine.

Passaggio 3: softmax per riga. Gli Scaled Scores S sono numeri reali qualsiasi. Per usarli come pesi di una media pesata sui value servono due proprietà: devono essere *non negativi* (altrimenti “sottrarremmo” i pixel invece di guardarli) e devono *sommare a 1 su ogni riga* (ogni latente ha un budget totale di attenzione fisso da spartire tra i 50 176 pixel). La softmax, applicata indipendentemente a ciascuna riga, garantisce entrambe le proprietà:

Pesi di attenzione

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^M e^{S_{ik}}}, \quad S_{ij} = \frac{(QK^\top)_{ij}}{\sqrt{d_{QKV}}}.$$

L’esponenziale a numeratore assicura la positività, la normalizzazione per la somma di riga assicura che i pesi sommino a 1, e la softmax preserva l’ordine degli score. Il risultato $A \in \mathbb{R}^{512 \times 50\,176}$ ha la stessa forma degli scores di partenza e si legge come una distribuzione di probabilità per ogni latente: la riga i dice con quale probabilità il latente i “guarda” ciascuno dei 50 176 pixel. In pratica, tutti i framework calcolano la softmax sottraendo prima il massimo della riga per evitare overflow numerico — è un trucco trasparente che non cambia il risultato algebrico (dettagli in Appendice A).

Passaggio 4: aggregazione AV . Secondo MatMul tra gli attention weights A e i value V , cioè $F = AV$. Questo passo serve a prendere i contenuti V degli input e combinarli secondo i pesi di attenzione calcolati prima: è il momento in cui i latenti “assorbono” le informazioni rilevanti dall’input.

Aggregazione

$$F = AV \in \mathbb{R}^{N \times d_{QKV}} = \mathbb{R}^{512 \times 261}.$$

► Ogni latente è ora un vettore di 261 feature: i pixel su cui aveva alta attenzione contribuiscono molto, quelli ignorati contribuiscono poco.

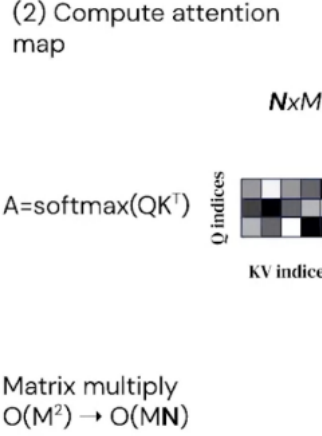


Figure 12: Passaggio 3 — *Compute attention map*: $A = \text{softmax}(QK^\top / \sqrt{d_{QKV}}) \in \mathbb{R}^{N \times M}$. La matrice è rettangolare (righe = indici Q dai latenti, colonne = indici KV dai pixel): $N \times M = 512 \times 50176$. Complessità $O(MN)$ invece di $O(M^2)$ — cfr. §1.2.

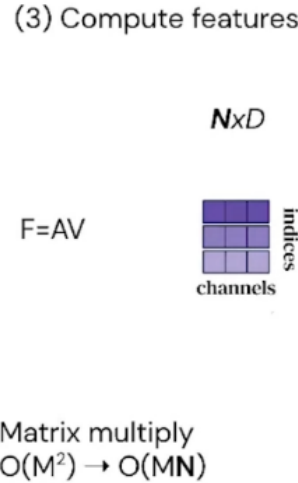


Figure 13: Passaggio 4 — *Compute features*: $F = AV \in \mathbb{R}^{N \times d_{QKV}} = 512 \times 261$ (righe = latenti, colonne = canali interni dell'attention). La dimensione $D = 1024$ verrà raggiunta solo dopo la proiezione W_o del Passaggio 5.

Passaggio 5: output projection W_o . Il risultato F vive in dimensione $d_{QKV} = 261$, ma i latenti originali sono in dimensione $D = 1024$. Per poter sommare l'output ai latenti nella residual connection serve riportarlo a 1024:

$$Z' = F W_o + b_o, \quad W_o \in \mathbb{R}^{261 \times 1024}, \quad Z' \in \mathbb{R}^{512 \times 1024}.$$

A differenza di W_Q, W_K, W_V , la proiezione di output usa anche il bias $b_o \in \mathbb{R}^{1024}$ (nel codice: `self.to_out = nn.Linear(...)`, che ha bias di default). W_o ha un doppio ruolo: riporta l'output dallo spazio interno dell'attention allo spazio dei latenti e aggiunge una trasformazione appresa che rielabora le feature prima della somma residuale.

Passaggio 6: residual connection. Infine si somma Z' al latent array *originale* L (non a quello normalizzato $\tilde{L}$, perché siamo in regime pre-norm):

$$L^{(\text{post-CA})} = L + Z' \in \mathbb{R}^{512 \times 1024}.$$

In questo modo il “canale” dei latenti viene preservato: anche se la cross-attention producesse un output quasi nullo, i latenti non verrebbero distrutti. La cross-attention si comporta così come una *correzione* incrementale dei latenti, non come una loro sostituzione.

Riepilogo completo delle forme nella cross-attention (ImageNet).

Oggetto	Forma simbolica	ImageNet	Operazione	Costo
X (byte array)	$M \times C_{\text{tot}}$	$50\,176 \times 261$	input	—
L (latenti)	$N \times D$	512×1024	parametri appresi	—
$\tilde{X}, \tilde{L}$	idem	idem	LayerNorm	$O(MC_{\text{tot}} + ND)$
$Q = \tilde{L} W_Q$	$N \times d_{QKV}$	512×261	proiezione	$O(ND d_{QKV})$
$K = \tilde{X} W_K$	$M \times d_{QKV}$	$50\,176 \times 261$	proiezione	$O(MC_{\text{tot}} d_{QKV})$
$V = \tilde{X} W_V$	$M \times d_{QKV}$	$50\,176 \times 261$	proiezione	$O(MC_{\text{tot}} d_{QKV})$
$\text{Scores} = QK^\top / \sqrt{d_{QKV}}$	$N \times M$	$512 \times 50\,176$	scaled dot-product	$O(MN d_{QKV})$
$A = \text{softmax}(\text{Scores})$	$N \times M$	$512 \times 50\,176$	softmax per riga	$O(MN)$
$F = AV$	$N \times d_{QKV}$	512×261	aggregazione (features)	$O(MN d_{QKV})$
$Z' = FW_o + b_o$	$N \times D$	512×1024	output projection	$O(N d_{QKV} D)$
$L + Z'$	$N \times D$	512×1024	residual	$O(ND)$

Osservazione finale. Il numero 1024 compare esattamente in tre posti: (1) come dimensione delle colonne dei latenti in input, (2) come colonne di W_Q in input, (3) come colonne di W_o in output. Dentro l’attention vera e propria (scores, softmax, aggregazione) tutto lavora a dimensione $d_{QKV} = 261$. Il numero 261 è la *dimensione interna* del blocco, 1024 è solo la dimensione di “interfaccia” con i latenti.

1.4.7 Caso Multi-Head

A quale modulo si riferisce questa sezione?

La formulazione multi-head qui descritta (con $H = 8$ teste e $d_{\text{head}} = 128$) corrisponde a:

- **Paper:** il multi-head del *latent transformer* (self-attention sui latenti $N \times D$). La cross-attention dell’ImageNet Perceiver è invece *single-head* con $d_{QKV} = 261$ (cfr. nota in §1.4.5).
- **Nostro codice:** multi-head è usato in entrambi i blocchi (cross e self), con $H = 8$ e $d_{\text{head}} = 64$ (totale 512) — stessa formulazione, parametri diversi.

Le formule che seguono sono identiche nei due casi; cambiano solo i valori numerici.

Ogni testa produce:

$$Z_h = A_h V_h \in \mathbb{R}^{512 \times 128}$$

Concatenando le 8 teste:

$$Z = \text{Concat}(Z_1, \dots, Z_8) \in \mathbb{R}^{512 \times 1024}$$

Proiezione lineare

Anche qui applichi:

$$Z' = ZW_o$$

con $W_o \in \mathbb{R}^{1024 \times 1024}$.

Dimensioni:

$$Z' \in \mathbb{R}^{512 \times 1024}$$

Perché è indispensabile

- **Normalizzazione della concatenazione:** senza W_o , avresti un “blocco” di feature derivanti dalle diverse teste, non rimescolate tra loro.
- **Mixing delle teste:** W_o permette di combinare l’informazione proveniente dalle varie teste in un’unica rappresentazione coerente.
- **Compatibilità:** porta l’output nello stesso spazio dei latenti L per la residual connection.

Nel multi-head, la proiezione non è solo utile: è **necessaria** per unire davvero le informazioni delle teste.

1.4.8 Proiezione Lineare e Residual



A questo punto abbiamo:

$$Z' \in \mathbb{R}^{512 \times 1024}$$

Latent array originale (prima della normalizzazione) $L \in \mathbb{R}^{512 \times 1024}$

Facciamo la **somma residua** (vedi Appendice F; pre-norm: la LayerNorm è stata applicata *prima* dell’attenzione, non qui):

$$L^{(t)'} = L + Z'$$

Con $t = 0$ essendo il primo blocco di cross-attention

Questo step serve a:

- Non perdere l’informazione originaria dei latenti.
- Previene che l’output degeneri se l’attenzione “sbaglia”: nel peggiore dei casi, il modello può imparare a far sì che $Z' \approx 0$, mantenendo intatto L .
- Rende il flusso del gradiente più stabile (evita problemi di vanishing gradient nei modelli molto profondi).

$L^{(t)'}$ è il residuo: il modello mantiene l’informazione originale di L e ci aggiunge solo una correzione/aggiornamento data dall’attenzione

1.4.9 Feed-Forward MLP



Pre-Norm:

$$\tilde{L} = \text{LayerNorm} \left(L^{(t)'} \right)$$

La **LayerNorm** (vedi Appendice D) prende ogni vettore latente (cioè ogni riga di L') e:

1. Calcola la media e la deviazione standard dei suoi valori.
2. Normalizza ogni valore sottraendo la media e dividendo per la deviazione standard (aggiungendo ϵ per stabilità numerica).
3. Applica due parametri appresi (γ, β) per scalare e traslare il risultato.

Formula

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j$$

Definizioni:

- $L'_i = i$ -esimo latente (vettore di dimensione D)
- $\mu_i = \frac{1}{D} \sum_{j=1}^D L'_{ij} = \text{media}$
- $\sigma_i = \sqrt{\frac{1}{D} \sum_{j=1}^D (L'_{ij} - \mu_i)^2} = \text{deviazione standard}$
- $\epsilon \in \mathbb{R} = \text{piccolo valore costante (es. } 10^{-5} \text{) per stabilità numerica}$
- $\gamma, \beta \in \mathbb{R}^D = \text{parametri appresi}$

Ciò che ottengo è

$$\tilde{L} \in \mathbb{R}^{512 \times 1024},$$

cioè, i latenti stabilizzati e pronti a entrare nello step successivo

Perché serve una MLP:

- Evita che i valori nei latenti crescano senza controllo o collassino a 0.
- Rende più stabile il training (la distribuzione dei valori resta bilanciata).
- Permette al modello di riutilizzare i latenti in più blocchi senza "deriva numerica".

Perché aggiungiamo una MLP?

- L'attenzione ($Z = AV$) serve a mescolare informazione tra token/latenti, pesando quanto ogni elemento deve guardare gli altri.
- Però, matematicamente, **l'attenzione è una combinazione lineare dei valori V**.
- Se ci fermassimo lì, ogni latente sarebbe solo un "mix lineare" di altri $\rightarrow$ potere espressivo limitato.

- 1. **Espansione (Linear 1):** Ogni vettore latente da 1024 dimensioni viene proiettato in uno spazio più ampio (4096).

$$H = \tilde{L}W_1 + b_1$$

- Dimensioni: $H \in \mathbb{R}^{512 \times 4096}$
- Scopo: permettere più combinazioni lineari (più capacità rappresentativa).

- 2. **Attivazione non lineare:** Si applica **GELU** (vedi Appendice E) elemento per elemento:

$$H' = \sigma(H)$$

Mantiene dimensione: 512×4096 .

- 3. **Compressione (Linear 2):** Si proietta di nuovo nello spazio originale di 1024 dimensioni:

$$F = H'W_2 + b_2$$

- Dimensioni: $F \in \mathbb{R}^{512 \times 1024}$
- Scopo: riportare l'output nella stessa dimensione dei latenti per poter fare residual connection.

Formula generale:

$$F = \text{MLP}(\tilde{L}) = \sigma(\tilde{L}W_1 + b_1)W_2 + b_2$$

- $\tilde{L} \in \mathbb{R}^{N \times D}$ — input (qui 512×1024).
- $W_1 \in \mathbb{R}^{1024 \times 4096}$, $b_1 \in \mathbb{R}^{4096}$
- $W_2 \in \mathbb{R}^{4096 \times 1024}$, $b_2 \in \mathbb{R}^{1024}$
- σ = funzione di attivazione non lineare (GELU).

Con $W_1, W_2 + \text{GELU}$, il **modello diventa capace di rappresentare funzioni non lineari complesse**.

Residual connection

In output dalla MLP: $F \in \mathbb{R}^{512 \times 1024}$. La residual si somma all'input del sotto-blocco *prima* della normalizzazione (pattern *pre-norm*), ovvero a L' (output della residual di attenzione, non a $\tilde{L}$ che è già normalizzato):

$$L'' = L' + F$$

Significa che **non buttiamo via l'input originale, ma aggiungiamo il contributo appreso dalla MLP**.

Così se la MLP non impara niente di utile, almeno il flusso dell'informazione rimane intatto.

Quindi alla fine del blocco di MLP ottengo $L'' \in \mathbb{R}^{512 \times 1024}$

Nota: nel Perceiver si utilizza il pattern **pre-norm**: la LayerNorm viene applicata *prima* di ogni sotto-blocco (attenzione o MLP), non dopo la residual connection.

1.4.10 Riassunto: blocco Cross-Attention + MLP

Il blocco completo prende in input il latent array L e il byte array X , e restituisce un latent aggiornato $\hat{L}$ di forma identica a L . Qui sotto tutti i passaggi con formule e dimensioni numeriche per ImageNet:

$$N = 512, \quad M = 50\,176, \quad D = 1024, \quad C_{\text{tot}} = 261, \quad d_{QKV} = 261.$$

Input del blocco

$$\begin{aligned} \text{Latent array:} & \quad L \in \mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 1024} \\ \text{Byte array (input + Fourier PE):} & \quad X \in \mathbb{R}^{M \times C_{\text{tot}}} = \mathbb{R}^{50\,176 \times 261} \end{aligned}$$

#	Operazione	Formula	Dimensione
Cross-Attention			
1	LayerNorm pre-att.	$\tilde{L} = \text{LN}(L), \quad \tilde{X} = \text{LN}(X)$	invariate
2	Proiezione Q	$Q = \tilde{L} W_Q, \quad W_Q \in \mathbb{R}^{1024 \times 261}$	$\mathbb{R}^{512 \times 261}$
	Proiezione K	$K = \tilde{X} W_K, \quad W_K \in \mathbb{R}^{261 \times 261}$	$\mathbb{R}^{50\,176 \times 261}$
	Proiezione V	$V = \tilde{X} W_V, \quad W_V \in \mathbb{R}^{261 \times 261}$	$\mathbb{R}^{50\,176 \times 261}$
3	Scaled scores	$S = \frac{QK^\top}{\sqrt{d_{QKV}}} = \frac{QK^\top}{\sqrt{261}}$	$\mathbb{R}^{512 \times 50\,176}$
4	Softmax per riga	$A_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^M e^{S_{ik}}}$	$\mathbb{R}^{512 \times 50\,176}$
5	Aggregazione AV	$F = AV$	$\mathbb{R}^{512 \times 261}$
6	Output projection	$Z' = F W_o + b_o, \quad W_o \in \mathbb{R}^{261 \times 1024}$	$\mathbb{R}^{512 \times 1024}$
7	Residual	$L' = L + Z'$	$\mathbb{R}^{512 \times 1024}$
MLP (pre-norm, GELU, hidden 4D)			
8	LayerNorm pre-MLP	$\tilde{L}' = \text{LN}(L')$	$\mathbb{R}^{512 \times 1024}$
9	Espansione	$H = \tilde{L}' W_1 + b_1, \quad W_1 \in \mathbb{R}^{1024 \times 4096}$	$\mathbb{R}^{512 \times 4096}$
	GELU	$H' = \text{GELU}(H)$	$\mathbb{R}^{512 \times 4096}$
	Compressione	$\phi = H' W_2 + b_2, \quad W_2 \in \mathbb{R}^{4096 \times 1024}$	$\mathbb{R}^{512 \times 1024}$
10	Residual post-MLP	$\hat{L} = L' + \phi$	$\mathbb{R}^{512 \times 1024}$

Output del blocco

$$\hat{L} \in \mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 1024}$$

(stessa forma di L : il blocco non altera la dimensione del latent array, solo il suo contenuto, ora arricchito di informazione letta dall'input tramite cross-attention e raffinata dalla MLP.)

Checkpoint dimensioni — fine blocco Cross-Attention

Abbiamo completato il primo blocco cross-attention + MLP (§1.4.5–§1.4.9). Dimensioni rispetto al punto di partenza:

Oggetto	Forma simb.	ImageNet	Status
Byte array X	$M \times C_{\text{tot}}$	$50\,176 \times 261$	invariato
Latente originale L	$N \times D$	512×1024	invariato
$\hat{L}$ (post blocco)	$N \times D$	512×1024	contenuto aggiornato

Cosa è successo in $\hat{L}$. Ogni riga è un latente che ha “letto” informazione dai $50\,176$ pixel tramite cross-attention (una media pesata dei Value), è stato riportato allo spazio $D = 1024$ tramite W_o , sommato al latente originale per la residual connection, poi raffinato da una MLP espansiva ($1024 \rightarrow 4096 \rightarrow 1024$) con GELU e un'altra residual. Forma identica all'input del blocco; contenuto semanticamente trasformato.

Prossimo step (§1.4.11): un blocco di Latent Transformer opera *solo sui latenti* (512×1024), senza più coinvolgere l'input, con self-attention multi-head in dimensione 1024 (non 261: qui non c'è più “minimo tra due sorgenti”).

1.4.11 Latent Transformer Block

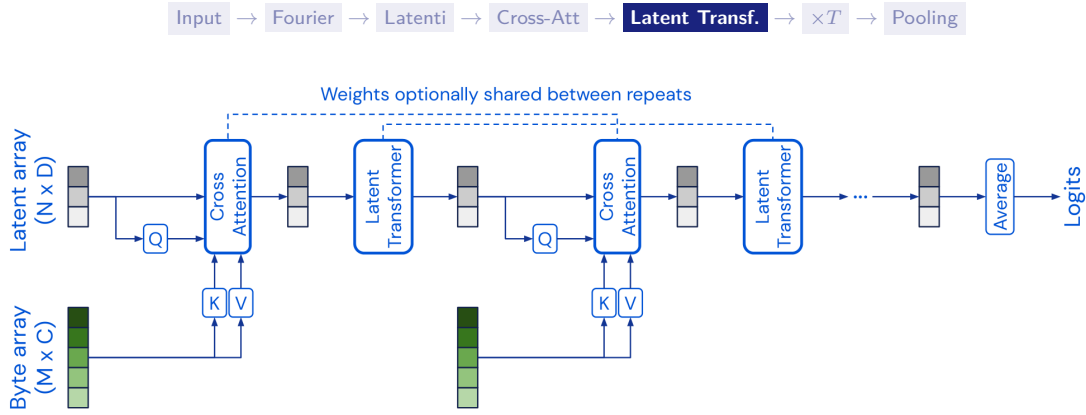


Figure 14: Il Latent Transformer nella pipeline del Perceiver originale (Jaegle et al., 2021, Fig. 1): si alterna alla cross-attention e opera *solo* sui latenti $\hat{L} \in \mathbb{R}^{N \times D}$ tramite self-attention.

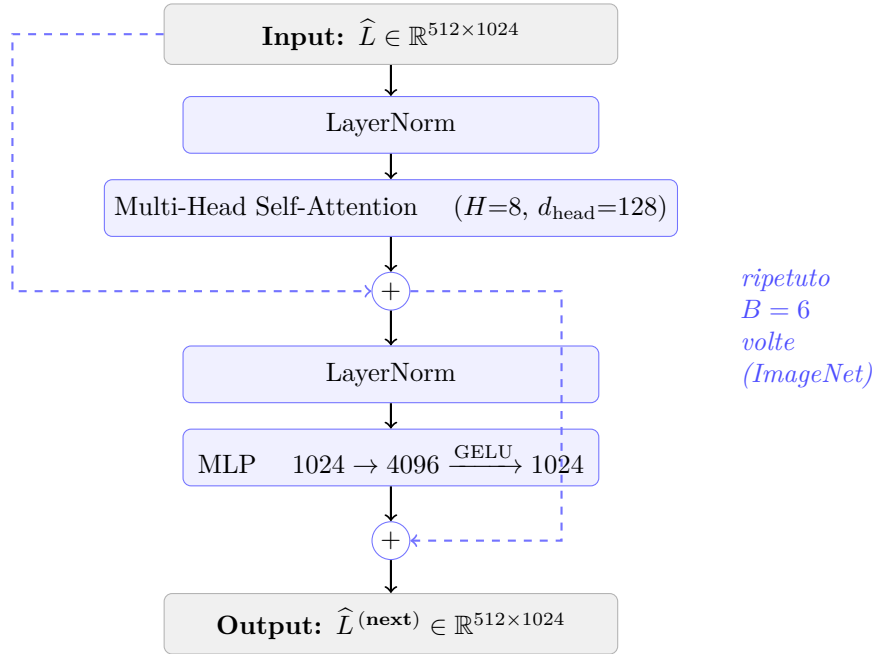


Figure 15: Struttura interna di un blocco di Latent Transformer (pre-norm, architettura GPT-2). Opera *solo* sui latenti — forma invariata $N \times D = 512 \times 1024$ — con self-attention multi-head ($H = 8$ teste da $d_{\text{head}} = 128$) seguita da una MLP a hidden $4D$. Le frecce tratteggiate sono le residual connections. Il blocco è ripetuto $B = 6$ volte tra due iterazioni di cross-attention.

Architettura GPT-2 nel latent transformer

Il latent transformer del Perceiver non è un Transformer “generico”: il paper (§3.1) adotta esplicitamente l'**architettura GPT-2** (Radford et al., 2019), che a sua volta è il *decoder-only* del Transformer originale (Vaswani et al., 2017) senza cross-attention sull’encoder. In pratica: LayerNorm pre-self-attention, self-attention multi-head, residual, LayerNorm pre-MLP, MLP con GELU, residual. Niente encoder, niente masking causale (il Perceiver è non-causale su tutti i blocchi).

Dimensioni per ImageNet: $H_{\text{self}} = 8$ teste, $d_{\text{head}} = D/H_{\text{self}} = 128$, MLP hidden dim = $4D = 4096$ (standard GPT-2).

Input

$$\hat{L} \in \mathbb{R}^{512 \times 1024}$$

(latenti usciti dal Cross Attention Block).

1. Layer Normalization (pre-self-attention)

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{\hat{L}_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j, \quad \tilde{L} \in \mathbb{R}^{512 \times 1024}$$

- Normalizziamo ogni latente.
- Dimensione invariata.

La normalizzazione viene rieseguita perché prima di ogni blocco di latent e cross si normalizza in quanto il blocco di latent non si fida del cross attention

2. Calcolo di Q,K,V (provenienti dai latenti) Ora andranno ricalcolati Q, K, V, ma essendo ora in un blocco di Latent Transformer, ci comporteremo come nel caso della self-attention dove come input abbiamo $\tilde{L}$, quindi avremo:

$$\begin{aligned} Q &= \tilde{L}W_q, & W_q &\in \mathbb{R}^{1024 \times d_k}, & Q &\in \mathbb{R}^{512 \times d_k} \\ K &= \tilde{L}W_k, & W_k &\in \mathbb{R}^{1024 \times d_k}, & K &\in \mathbb{R}^{512 \times d_k} \\ V &= \tilde{L}W_v, & W_v &\in \mathbb{R}^{1024 \times d_v}, & V &\in \mathbb{R}^{512 \times d_v} \end{aligned}$$

Con $H = 8$ teste e $D = 1024$: $d_k = d_v = D/H = 1024/8 = 128$.

3. Self-Attention sui latenti

Dopo aver calcolato Q, K, V :

$$\begin{aligned} A &= \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right), & A &\in \mathbb{R}^{512 \times 512} \\ Z &= AV, & Z &\in \mathbb{R}^{512 \times d_v} \end{aligned}$$

4. Proiezione + Residual (post-attention)

$$\begin{aligned} Z' &= ZW_O, & W_O &\in \mathbb{R}^{d_v \times D} \\ L' &= \hat{L} + Z', & L' &\in \mathbb{R}^{512 \times 1024} \end{aligned}$$

5. Feed-Forward MLP Pre-Norm:

$$\tilde{L}' = \text{LayerNorm}(L')$$

Espansione:

$$H = \tilde{L}'W_1 + b_1, \quad W_1 \in \mathbb{R}^{1024 \times 4096}, \quad H \in \mathbb{R}^{512 \times 4096}$$

Attivazione non lineare:

$$H' = \sigma(H) \text{ (GELU)}$$

Compressione:

$$F = H'W_2 + b_2, \quad W_2 \in \mathbb{R}^{4096 \times 1024}, \quad F \in \mathbb{R}^{512 \times 1024}$$

Residual finale

$$L'' = L' + F, \quad L'' \in \mathbb{R}^{512 \times 1024}$$

Output del blocco

$$L^{\text{out}} = L'' \in \mathbb{R}^{512 \times 1024}$$

Checkpoint dimensioni — fine §1.4.10 (una iterazione completa)

Abbiamo ora completato *una* passata $T = 1$ del blocco Cross-Attention + MLP + Latent Transformer. Dimensioni:

Oggetto	Forma simbolica	ImageNet	Status
Byte array X	$M \times C_{\text{tot}}$	$50\,176 \times 261$	input, immutato
Latente post cross-att + MLP	$N \times D$	512×1024	post blocco 1 (cross)
Latente post latent transf.	$N \times D$	512×1024	output di una iterazione completa

Invarianza dimensionale. Cross-attention e Latent Transformer preservano entrambi la forma 512×1024 (“è la stessa matrice, solo aggiornata”): è ciò che permette di ripetere il blocco T volte senza problemi di compatibilità.

Cosa cambia nel Latent Transformer rispetto alla cross-attention.

- Q, K, V provengono *tutti* dai latenti (512×1024), non dall’input.
- d_{QKV} non è più $\min(D, C_{\text{tot}}) = 261$; qui $\min(D, D) = D = 1024$, quindi si lavora in 1024.
- Multi-head effettivo: $H = 8$, $d_{\text{head}} = 1024/8 = 128$ (in linea con GPT-2).
- Matrice di attenzione $A \in \mathbb{R}^{512 \times 512}$ (quadrata, non rettangolare), costo $O(N^2) = O(512^2)$: piccolo grazie al bottleneck.

Prossimo step (§1.4.12): le $T = 8$ iterazioni con weight sharing.

1.4.12 Weight Sharing e Iterazioni



Questa è una delle idee più importanti del Perceiver: dopo aver descritto *un* blocco (cross-attention + latent transformer), l'intera pipeline del forward viene **riapplicata in serie** $T = 8$ volte ai medesimi latenti, *condividendo i pesi* tra le iterazioni. Questa sezione spiega il funzionamento, mostra un diagramma esplicito della ricorrenza e illustra con un esempio numerico come il weight sharing si comporti nel backward pass.

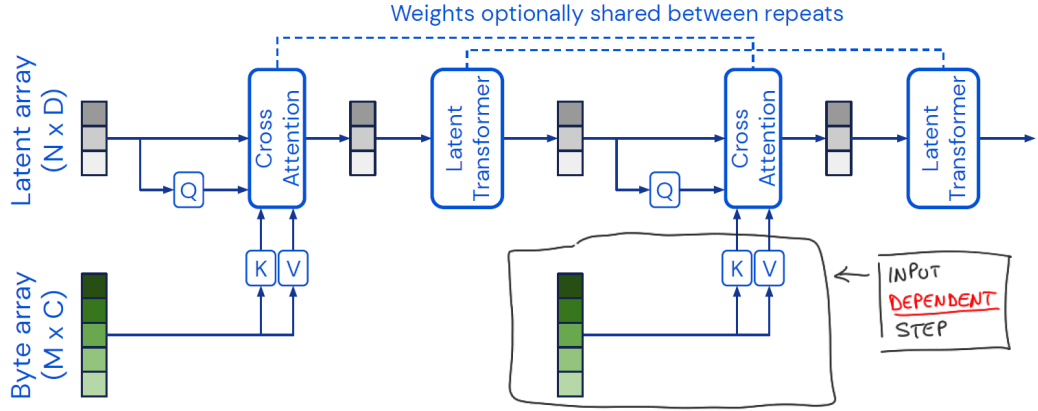
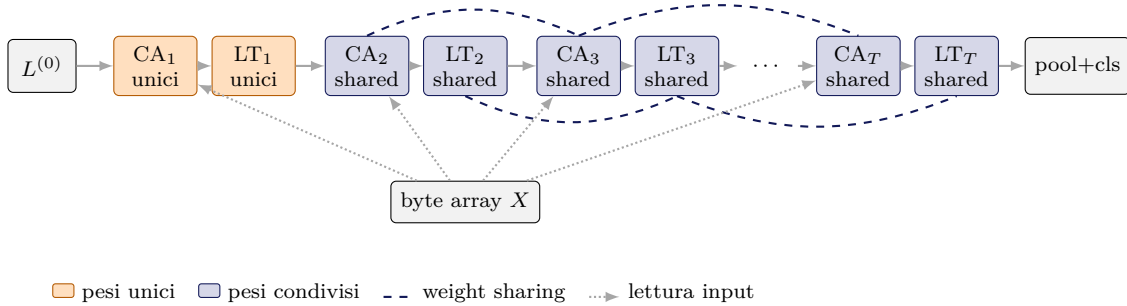


Figure 16: Architettura completa del Perceiver con weight sharing (figura dal paper Jaegle et al., 2021, Fig. 1). I blocchi Cross-Attention e Latent Transformer vengono ripetuti T volte condividendo i pesi tra le iterazioni (linee tratteggiate).

Schema della ricorrenza. Il diagramma seguente chiarisce chi condivide cosa con chi.



Regola precisa (paper, §4.1). I pesi sono condivisi tra tutti i blocchi *tranne il primo*:

- **Cross-Attention:** CA_1 ha pesi *unici*; i blocchi $CA_2, CA_3, \dots, CA_T$ condividono tutti lo stesso set di pesi ($W_Q^{sh}, W_K^{sh}, W_V^{sh}, W_o^{sh}$).
- **Latent Transformer:** analogamente, LT_1 ha pesi *unici* e $LT_2, \dots, LT_T$ condividono lo stesso blocco di $L = 6$ self-attention layer.

Perché il primo blocco è escluso? Il paper riporta che condividere i pesi già dal primo blocco porta a *instabilità di training*. L'intuizione: la prima cross-attention è qualitativamente diversa dalle successive perché riceve i latenti nella loro inizializzazione “neutra” (learned position encoding $\mathcal{N}(0, 0.02^2)$), mentre dalla seconda in poi legge latenti già “popolati” di informazione dall’input. Due

regimi così diversi non possono condividere gli stessi pesi senza causare oscillazioni del gradiente nelle prime epoche.

Effetto sui parametri e sulle prestazioni (Tab. 7 del paper):

- Senza weight sharing: $\sim 326.2\text{M}$ parametri, 72.9% validation accuracy (vs 87.7% train: gap di 14.8%, forte overfitting).
- Con weight sharing: $\sim 44.9\text{M}$ parametri, 78.0% validation accuracy (vs 79.5% train: gap 1.5%, nessun overfitting) — riduzione $\approx 7\times$ dei parametri e *miglioramento* contemporaneo delle prestazioni.

Il weight sharing agisce dunque come regolarizzatore: le iterazioni successive sono passi di *raffinamento progressivo* dei latenti, e il modello assume formalmente la struttura di un **RNN** con core ricorrente (il blocco shared CA+LT) applicato a uno stato ricorrente (i latenti) che legge ripetutamente lo stesso input.

Come si comporta il weight sharing nel backward pass

Quando un peso W viene usato in più blocchi del forward, ogni istanza del peso produce un contributo al gradiente durante il backward. Per il training corretto, questi contributi **si sommano**:

$$\frac{\partial \mathcal{L}}{\partial W^{\text{sh}}} = \sum_{t=2}^T \frac{\partial \mathcal{L}}{\partial W^{(t)}}$$

dove $W^{(t)}$ è “il peso come se fosse usato solo alla t -esima iterazione”. In PyTorch questo accade automaticamente: quando più nodi del grafo computazionale puntano allo stesso `nn.Parameter`, i gradienti calcolati in `backward()` vengono accumulati sul `.grad` del parametro — è lo stesso meccanismo di un RNN srotolato (BPTT, *back-propagation through time*).

Esempio numerico con $T = 3$ iterazioni. Supponiamo di avere un peso scalare condiviso $w = 2.0$ tra le iterazioni $t = 2$ e $t = 3$. Durante il backward si calcolano:

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = -0.3, \quad \frac{\partial \mathcal{L}}{\partial w^{(3)}} = +0.1.$$

Il gradiente effettivo che aggiorna w è la *somma*:

$$\frac{\partial \mathcal{L}}{\partial w^{\text{sh}}} = -0.3 + 0.1 = -0.2.$$

Con learning rate $\eta = 0.01$: $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w^{\text{sh}}} = 2.0 - 0.01 \cdot (-0.2) = 2.002$.

Attenzione: fare l’aggiornamento *due volte* (una con -0.3 e una con $+0.1$, come se fossero pesi distinti) produrrebbe un risultato diverso e sbagliato: l’accumulazione deve avvenire *prima* del passo dell’ottimizzatore, non dopo.

Perché il primo blocco *non* partecipa alla somma. I pesi di CA_1 e LT_1 sono parametri separati, quindi i loro gradienti stanno in un *accumulator* diverso: non si sommano con quelli dei blocchi shared, ma vengono aggiornati in modo indipendente.

Checkpoint dimensioni — fine §1.4.11 (dopo $T = 8$ iterazioni)

Dopo aver ripetuto il blocco $T = 8$ volte con weight sharing, i latenti sono nello stato finale. **Le dimensioni non cambiano:** ogni iterazione preserva $N \times D = 512 \times 1024$, perché è progettata per essere applicata ricorsivamente.

Oggetto	Forma simbolica	ImageNet	Status
Byte array X	$M \times C_{\text{tot}}$	$50\,176 \times 261$	riusato da ogni CA_t
Latente iniziale $L^{(0)}$	$N \times D$	512×1024	parametro appreso
Latente iter. 1: $L^{(1)}$	$N \times D$	512×1024	post $\text{CA}_1 + \text{LT}_1$ (pesi propri)
Latente iter. 2: $L^{(2)}$	$N \times D$	512×1024	post $\text{CA}_2 + \text{LT}_2$ (pesi shared)
...	...	...	...
Latente finale $L^{(T)}$	$N \times D$	512×1024	dopo $T = 8$ iterazioni

Osservazione chiave. Per il paper, la struttura $L^{(t)} = \text{LT}_t(\text{CA}_t(L^{(t-1)}, X))$ con $\text{CA}_t = \text{CA}_{\text{sh}}$ e $\text{LT}_t = \text{LT}_{\text{sh}}$ per $t \geq 2$ è *formalmente* un RNN con:

- “stato ricorrente” = latente $L^{(t)}$ (dimensione fissa 512×1024);
- “input (costante nel tempo)” = byte array X ;
- “funzione di transizione” = il blocco shared $(\text{CA}_{\text{sh}}, \text{LT}_{\text{sh}})$;
- “stato iniziale” = $L^{(0)}$ (ma $L^{(1)}$ è fatto con pesi propri, quindi in realtà è un RNN con “warm-up” speciale al primo passo).

Prossimo step (§1.4.13): da $L^{(T)}$ alla classificazione finale.

1.4.13 Output (Pooling + Classificazione)

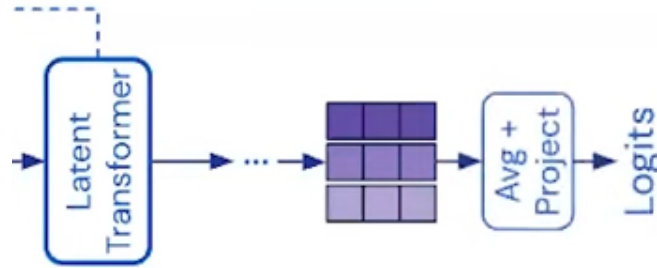


Figure 17: Output del Perceiver: i latenti finali vengono mediati (Average + Project) e proiettati in logit per la classificazione.

Classification Head

Pooling

Pooling *prima* della proiezione: un fix del paper

Il Perceiver esegue l'average pooling *prima* della proiezione lineare nei logit. L'Appendice F del paper chiarisce che questa è una modifica rispetto alla prima versione su arXiv: “*We have slightly improved the results ... by averaging before rather than after projecting when computing the output logits*”. Matematicamente le due operazioni (pooling e proiezione lineare) commutano se applicate a tutte le posizioni, ma fare pooling prima riduce il costo computazionale della proiezione (si applica a un solo vettore invece che a N) e stabilizza il training.

Il pooling è un'operazione che serve a riassumere più vettori in uno solo. Nel caso del Perceiver, i vettori sono i latenti finali:

$$L^{(\text{finale})} \in \mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 1024}$$

dove hai $N = 512$ latenti, ciascuno di dimensione $D = 1024$.

Il pooling medio li combina facendo la media elemento per elemento:

$$h = \frac{1}{N} \sum_{i=1}^N L_i^{(\text{finale})}, \quad h \in \mathbb{R}^D = \mathbb{R}^{1024}$$

Risultato: un unico vettore compatto che rappresenta l'intero input.

Perché facciamo Pooling?

1. Compressione delle informazioni

- Hai N latenti $\rightarrow$ troppi per passare direttamente al classificatore.
- Il pooling produce un solo vettore gestibile (h).

2. Uniformità

- Indipendentemente da quanti latenti usi, dopo il pooling hai sempre un vettore di dimensione fissa ($D = 1024$).
- Questo permette di collegarlo facilmente al classificatore lineare.

3. Evita parametri aggiuntivi

- La media non introduce nuovi pesi.
- È un metodo semplice ed efficiente per combinare i latenti.

4. Analogia con altri modelli

- Nei Transformer (BERT, GPT), si usa un token speciale (CLS) per riassumere.
- Nel Perceiver non c'è un token speciale: il pooling medio è la soluzione più diretta.

Input → Fourier → Latenti → Cross-Att → Latent Transf. → $\times T$ → Pooling

Classificatore Fully Connected Layer (Logits)

Il vettore h entra in un layer lineare:

$$z = hW_c + b_c$$

- $h \in \mathbb{R}^{1024} \rightarrow$ rappresentazione compatta dell'input.
- $W_c \in \mathbb{R}^{1024 \times 1000} \rightarrow$ matrice di pesi che proietta lo spazio dei latenti (1024 dimensioni) nello spazio delle classi (1000 per ImageNet).
- $b_c \in \mathbb{R}^{1000} \rightarrow$ vettore di bias.
- $z \in \mathbb{R}^{1000} \rightarrow$ vettore dei logit, cioè punteggi grezzi per ogni classe.

Concretamente: da un vettore di 1024 numeri ne ottieni uno di 1000 numeri, ognuno corrispondente a una classe.

Perché serve questo passaggio?

- Il pooling ti ha dato un riassunto dell'input.
- Ma per decidere a quale classe appartiene, serve uno strato che traduce quel riassunto in punteggi di classificazione.
- Il layer lineare è la funzione più semplice: una combinazione lineare + bias.
- Ogni colonna di W_c può essere vista come un "filtro" che misura quanto h è compatibile con una determinata classe.
- Il bias b_c sposta il punteggio di base per ciascuna classe.
- L'output z contiene i punteggi ancora non normalizzati.

Input → Fourier → Latenti → Cross-Att → Latent Transf. → $\times T$ → Pooling

Softmax sui logits

Prendi il vettore dei logits z e applichi la funzione softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

Questo trasforma i logits in probabilità normalizzate:

- Tutti i valori $p_i \geq 0$.
- La somma di tutte le probabilità = 1.
- Risultato: un vettore $p \in \mathbb{R}^{1000}$, che rappresenta la probabilità di appartenenza a ciascuna classe.

Scelta della classe predetta

Si prende l'indice della probabilità massima:

$$\hat{y} = \arg \max_i p_i$$

$\hat{y}$ è la classe predetta (ad esempio, "gatto" se la classe 281 corrisponde al gatto in ImageNet).



Loss

La **loss** è il modo in cui “misuri” quanto la predizione del modello è distante dalla verità, e serve a guidare l'aggiornamento dei pesi durante il training.

Esempio su ImageNet

- Il modello produce i logits $z \in \mathbb{R}^{1000}$.
- Dopo la softmax, ottieni le probabilità predette:

$$p_i^{pred} = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

con $i = 1, \dots, 1000$.

- L'etichetta vera y^{true} (la classe corretta) viene rappresentata come one-hot vector:

$$y^{true} = (0, 0, \dots, 1, \dots, 0)$$

dove c'è un solo 1 nella posizione della classe giusta.

- Se sei in fase di training, hai anche l'etichetta vera y .
- Usi la [cross-entropy loss](#) (vedi Appendice C):

$$\mathcal{L} = - \sum_{i=1}^K y_i^{true} \log(p_i^{pred})$$

Nel nostro caso:

$$\mathcal{L} = - \sum_{i=1}^{1000} y_i^{true} \log(p_i^{pred})$$

- Dove y_i è 1 per la classe corretta e 0 per tutte le altre (one-hot encoding).
- Questa loss viene poi usata per aggiornare i pesi tramite **backpropagation**.

Noi vogliamo che la probabilità della classe corretta sia la più alta possibile:

$$p_{corretta} \rightarrow 1$$

Quindi serve una funzione di **costo** che:

- sia **bassa** quando $p_{corretta}$ è vicina a 1,
- sia **alta** quando $p_{corretta}$ è vicina a 0.

Vediamo perché c'è il log:

1. Monotonia

- Il logaritmo è crescente: se p cresce, $\log(p)$ cresce.
- Quindi massimizzare p equivale a massimizzare $\log(p)$.
- Usiamo il -log perché stiamo minimizzando la loss (anziché massimizzare la probabilità).

2. Penalità forte per probabilità basse Se $p_{corretta} = 0.9$:

$$-\log(0.9) \approx 0.105 \rightarrow \text{loss bassa.}$$

Se $p_{corretta} = 0.01$:

$$-\log(0.01) \approx 4.6 \rightarrow \text{loss altissima.}$$

- Il log "esplode" quando $p \rightarrow 0$, così punisce molto le predizioni sbagliate e sicure.

Intuizione visiva

La funzione $-\log(p)$:

- Quando $p = 1$: $-\log(1) = 0 \rightarrow$ perfetta predizione.
- Quando $p \rightarrow 0$: $-\log(p) \rightarrow +\infty \rightarrow$ predizione pessima.

È esattamente il comportamento che vogliamo!

Checkpoint dimensioni finale — fine §1.4 (Forward Pass completo)

Ecco lo stato *completo* di tutte le dimensioni al termine del forward pass per ImageNet:

Oggetto	Forma simbolica	ImageNet	Cosa rappresenta
<i>Input</i>			
Byte array grezzo	$M \times C$	$50\,176 \times 3$	solo RGB
Byte array X (con Fourier PE)	$M \times C_{\text{tot}}$	$50\,176 \times 261$	3 RGB + 258 Fourier
<i>Latenti (parametro appreso)</i>			
Latente iniziale $L^{(0)}$	$N \times D$	512×1024	init $\mathcal{N}(0, 0.02^2)$ troncata
<i>Dentro una cross-attention (paper)</i>			
W_Q	$D \times d_{QKV}$	1024×261	proiezione dei latenti
W_K	$C_{\text{tot}} \times d_{QKV}$	261×261	proiezione input $\rightarrow K$
W_V	$C_{\text{tot}} \times d_{QKV}$	261×261	proiezione input $\rightarrow V$
$Q = L W_Q$	$N \times d_{QKV}$	512×261	query dai latenti
$K = X W_K$	$M \times d_{QKV}$	$50\,176 \times 261$	key dall'input
$V = X W_V$	$M \times d_{QKV}$	$50\,176 \times 261$	value dall'input
$QK^\top / \sqrt{d_{QKV}}$	$N \times M$	$512 \times 50\,176$	scores rettangolari
$A = \text{softmax}(\cdot)$	$N \times M$	$512 \times 50\,176$	pesi di attenzione
$F = AV$	$N \times d_{QKV}$	512×261	aggregazione (features)
W_o	$d_{QKV} \times D$	261×1024	output projection
$Z' = FW_o + b_o$	$N \times D$	512×1024	output proiettato
$L + Z'$ (post residual)	$N \times D$	512×1024	latente aggiornato
<i>Dentro un latent transformer (self-attention)</i>			
Q, K, V (tutti dai latenti)	$N \times D$	512×1024	$H = 8$, $d_{\text{head}} = D/H = 128$
A quadrata	$N \times N$	512×512	costo $O(N^2)$
$L^{(t)}$ post latent transf.	$N \times D$	512×1024	
<i>Iterazione $T = 8$ volte (weight sharing dal 2° blocco in poi)</i>			
$L^{(T)}$ (latente finale)	$N \times D$	512×1024	output iterazioni
<i>Output head</i>			
$h = \frac{1}{N} \sum_i L_i^{(T)}$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	pooling
W_c	$D \times K_{\text{cls}}$	1024×1000	classificatore
$z = hW_c + b_c$	$\mathbb{R}^{K_{\text{cls}}}$	$\mathbb{R}^{1000}$	logit (1 per classe)
$p = \text{softmax}(z)$	$\mathbb{R}^{K_{\text{cls}}}$	$\mathbb{R}^{1000}$	probabilità
$\hat{y} = \arg \max_i p_i$	scalare	$\{0, \dots, 999\}$	classe predetta
$\mathcal{L} = -\log p_{y_{\text{true}}}$	scalare	$\mathbb{R}$	cross-entropy loss

Legenda dei simboli:

- M = numero di elementi di input (pixel per ImageNet)
- N = numero di latenti
- C = canali originali dell'input, $C_{\text{tot}} = C + d(2K + 1)$ dopo Fourier
- D = dimensione dei latenti (canale del latent transformer)
- d_{QKV} = dimensione interna dell'attention, $\min(D, C_{\text{tot}})$ nel paper
- K = bande di frequenza per Fourier (da non confondere con K_{cls} = numero di classi)
- T = numero di iterazioni cross-att + latent transformer
- H = numero di teste dell'attention, $d_{\text{head}} = D/H$

Due osservazioni finali:

- La forma 512×1024 dei latenti viene preservata da tutti i blocchi successivi alla cross-attention iniziale, fino al pooling. Questo è il “canale” costante su cui l'informazione viaggia.

- L'unica compressione “hard” della pipeline è il pooling finale $512 \times 1024 \rightarrow \mathbb{R}^{1024}$ (un fattore $512 \times$ di riduzione), seguito dalla proiezione lineare a 1000 classi. Tutto il resto è “trasformazione di pari dimensione” (i blocchi latent) o “lettura asimmetrica” (la cross-attention $M \rightarrow N$).

1.5 Backward Pass

Alla fine del forward pass abbiamo la loss $\mathcal{L}$, cioè una misura numerica di quanto la predizione del modello (p) è lontana dalla verità (y). Questa loss però, da sola, non basta a migliorare i pesi: per sapere come modificare i parametri della rete dobbiamo calcolare quanto la loss dipende dai valori interni del modello (logits, latenti, pesi, ecc.). Per fare ciò utilizziamo il gradiente.

Forward & Backward Pass — Overview

Forward ($\rightarrow$):

Input $\rightarrow$ Embedding $\rightarrow$ Cross-Att $\rightarrow$ Res+LN $\rightarrow$ Self-Att ($\times T$) $\rightarrow$ Pooling $\rightarrow$ Classifier $\rightarrow$ Softmax $\rightarrow$ Loss

Backward ($\leftarrow$):

Loss $\leftarrow$ Softmax $\leftarrow$ Classifier $\leftarrow$ Pooling $\leftarrow$ Latenti $\leftarrow$ Latent Transf. ($\times T$) $\leftarrow$ MLP $\leftarrow$ Attn $\leftarrow$ Q/K/V $\leftarrow$ LN $\leftarrow$ Cross-Att $\leftarrow$ Embedding $\leftarrow$ Input

Struttura di questa sezione e priorità di studio

Il backward pass viene derivato step-by-step, risalendo dalla Loss fino all'Input. Per ogni blocco si mostra: (1) il richiamo del forward, (2) la derivata con la regola della catena, (3) le formule dei gradienti verso pesi e input. I marker `\currentstep{...}` indicano la posizione attuale nel percorso.

Per uno studio mirato all'esame, queste sono le priorità (in ordine decrescente di importanza):

1. **Classifier + Softmax + Cross-Entropy** (§1.5, prima parte): la combinazione softmax + CE produce il celebre gradiente $\partial\mathcal{L}/\partial z = p - y$. *Essenziale, chiesto quasi sempre.*
2. **Scaled Dot-Product Attention**: backward attraverso softmax e prodotti $QK^\top$, AV . È il cuore geometrico dell'attention — *chiesto frequentemente.*
3. **Proiezioni lineari Q, K, V e Output projection W_o** : applicazione diretta della chain rule a $Y = XW$. *Veloce ma istruttivo.*
4. **Latent Transformer (self-attention + MLP)**: stessa logica del Cross-Attention. *Saperlo riconoscere come “stesso blocco”.*
5. **LayerNorm + Residual + Weight Sharing**: dettagli tecnici. *Approfondimento, non sempre richiesto.*

Se hai poco tempo, leggi a fondo i punti 1–3, sfoglia il resto.

Nota sul weight sharing nel backward

Le formule che seguono derivano il gradiente *per un singolo blocco* (una cross-attention o un latent transformer). Nel Perceiver il blocco CA+LT è però applicato $T = 8$ volte con **pesi condivisi** tra le iterazioni $2, \dots, T$ (cfr. §1.4.12). Di conseguenza, il gradiente di un peso condiviso W^{sh} è la *somma* dei gradienti calcolati in ogni iterazione:

$$\frac{\partial\mathcal{L}}{\partial W^{\text{sh}}} = \sum_{t=2}^T \frac{\partial\mathcal{L}}{\partial W^{(t)}}$$

In PyTorch questa somma è automatica: ogni chiamata a `backward()` accumula il gradiente sul `.grad` del parametro condiviso. I pesi del primo blocco (CA_1, LT_1), che sono unici, ricevono un solo contributo.

Nota su b_Q, b_K, b_V : bias nelle proiezioni Q/K/V

Nel paper originale (App. C) e nel codice del progetto (`src/perceiver/attention.py: nn.Linear(..., bias=False)`) le proiezioni di Query, Key e Value nella cross-attention e nel latent self-attention **non usano bias**. Le formule che seguono includono b_Q, b_K, b_V per completezza didattica — derivare con bias è un modo pulito per praticare la chain rule — ma nel Perceiver reale i termini b_Q, b_K, b_V e i relativi gradienti $\partial\mathcal{L}/\partial b_Q, \partial\mathcal{L}/\partial b_K, \partial\mathcal{L}/\partial b_V$ *non esistono*.

I bias sono invece presenti (e le formule valgono) in:

- **Output projection** W_o della cross-attention: $Z' = ZW_o + b_o$
- **MLP feed-forward**: $H = \tilde{L}W_1 + b_1, F = H'W_2 + b_2$
- **Classificatore finale**: $z = hW_c + b_c$
- **Input embedding**: $X = X_{\text{raw}}W_E + b_E$

$$\mathcal{L} = - \sum_{i=1}^K y_i^{\text{true}} \log(p_i^{\text{pred}})$$

1 Perché serve il gradiente?

- L'obiettivo del training è ridurre la loss $\mathcal{L}$ aggiustando i pesi della rete.
- L'algoritmo che usiamo (tipicamente **discesa del gradiente**) aggiorna i parametri nella direzione che diminuisce la loss più velocemente.
- Per sapere in che direzione muovere i pesi, dobbiamo sapere quanto la loss cambia se cambiano i valori interni della rete (logits, pesi, attivazioni, ecc.).

Questa **informazione ce la dà il gradiente**.

Il backward pass funziona con la **regola della catena**: se hai una funzione composta, per aggiornare i pesi devi risalire gradualmente dall'uscita (loss) fino ai parametri interni.

Nel nostro caso:

$$h \rightarrow z \rightarrow p \rightarrow \mathcal{L}$$

- Per arrivare a W_c e h , bisogna passare per $z \rightarrow$ Quindi il primo gradiente utile da calcolare è:

$$\frac{\partial \mathcal{L}}{\partial z}$$

Con $z = hW_c + b_c$

2. Perché proprio $\frac{\partial \mathcal{L}}{\partial z}$?

La loss $\mathcal{L}$ dipende dai logits z attraverso softmax + cross-entropy.

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$$

dove:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Se sostituiamo p_k nella loss

$$\mathcal{L} = - \sum_{k=1}^K y_k \log \left(\frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \right)$$

Semplificando il logaritmo (Ricorda che $\log \frac{a}{b} = \log a - \log b$)

Quindi:

$$\mathcal{L} = - \sum_{k=1}^K y_k \left(z_k - \log \sum_{j=1}^K e^{z_j} \right)$$

Separando i due termini

$$\mathcal{L} = - \sum_{k=1}^K y_k z_k + \sum_{k=1}^K y_k \log \sum_{j=1}^K e^{z_j}$$

Ma siccome y è one-hot (cioè $\sum_k y_k = 1$):

$$\mathcal{L} = - \sum_{k=1}^K y_k z_k + \log \sum_{j=1}^K e^{z_j}$$

Derivando rispetto a z_i calcoliamo $\frac{\partial \mathcal{L}}{\partial z_i}$.

Primo termine:

$$\frac{\partial}{\partial z_i} \left(- \sum_{k=1}^K y_k z_k \right) = -y_i$$

Secondo termine:

$$\frac{\partial}{\partial z_i} \left(\log \sum_{j=1}^K e^{z_j} \right) = \frac{1}{\sum_{j=1}^K e^{z_j}} \cdot e^{z_i} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = p_i$$

Risultato finale

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - y_i$$

oppure, in forma vettoriale:

$$\frac{\partial \mathcal{L}}{\partial z} = p - y$$

Per aggiornare $\mathbf{W}_c, \mathbf{b}_c$ e $\mathbf{h}$ devo sapere quanto cambia la loss se cambiano loro.
Con la **regola della catena**:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial h} &= \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial h} \\ \frac{\partial \mathcal{L}}{\partial W_c} &= \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial W_c} \\ \frac{\partial \mathcal{L}}{\partial b_c} &= \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial b_c} \end{aligned}$$

Gradiente rispetto a h Formula

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial h}$$

Dove:

- $z = hW_c + b_c$
- La derivata di z rispetto a h è:

$$\frac{\partial z}{\partial h} = W_c$$

Quindi:

$$\frac{\partial \mathcal{L}}{\partial h} = (p - y)W_c^\top$$

Esplicitando indici:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \sum_{k=1}^K (p_k - y_k) W_{c,jk} \quad j = 1, \dots, D$$

Significato: ogni componente di h riceve un contributo da tutte le classi, pesato dai corrispondenti elementi della matrice W_c .

Gradiente rispetto a W_c Formula

$$\frac{\partial \mathcal{L}}{\partial W_c} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial W_c}$$

Dove:

- $z = hW_c + b_c$.
- Ogni logit è:

$$z_k = \sum_{j=1}^D h_j W_{c,jk} + b_{c,k}$$

Derivata di z_k rispetto a $W_{c,jk}$:

$$\frac{\partial z_k}{\partial W_{c,jk}} = h_j$$

Quindi:

$$\frac{\partial \mathcal{L}}{\partial W_{c,jk}} = (p_k - y_k) h_j$$

Forma compatta matriciale:

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^\top (p - y)$$

Significato: ogni peso $W_{c,jk}$ viene aggiornato in base al valore della componente h_j e all'errore della classe k .

Gradiente rispetto a b_c Formula

$$\frac{\partial \mathcal{L}}{\partial b_c} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial b_c}$$

Dove:

- $z = hW_c + b_c$.
- Derivata di z_k rispetto a $b_{c,k}$:

$$\frac{\partial z_k}{\partial b_{c,k}} = 1$$

Quindi:

$$\frac{\partial \mathcal{L}}{\partial b_{c,k}} = (p_k - y_k)$$

Forma compatta:

$$\frac{\partial \mathcal{L}}{\partial b_c} = p - y$$

Significato: ogni bias riceve direttamente l'errore associato alla propria classe.

Quindi:

Verso h :

$$\frac{\partial \mathcal{L}}{\partial h} = (p - y)W_c^\top$$

→ distribuisce l'errore nello spazio dei latenti.

Verso W_c :

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^\top (p - y)$$

→ aggiorna la matrice dei pesi del classificatore.

Verso b_c :

$$\frac{\partial \mathcal{L}}{\partial b_c} = p - y$$

→ ogni bias prende l'errore della sua classe.

Checkpoint gradienti — fine Loss + Softmax + Classifier

Partendo dalla loss, abbiamo calcolato i primi gradienti risalendo fino all'input del classifier (h , il vettore post-pooling). Regola chiave: *ogni gradiente $\partial \mathcal{L} / \partial X$ ha la stessa forma di X .*

Gradiente	Forma simbolica	ImageNet	Uso
$\partial \mathcal{L} / \partial z = p - y$	$\mathbb{R}^{K_{\text{cls}}}$	$\mathbb{R}^{1000}$	intermedio
$\partial \mathcal{L} / \partial W_c = h^\top (p - y)$	$\mathbb{R}^{D \times K_{\text{cls}}}$	$\mathbb{R}^{1024 \times 1000}$	aggiorna W_c
$\partial \mathcal{L} / \partial b_c = p - y$	$\mathbb{R}^{K_{\text{cls}}}$	$\mathbb{R}^{1000}$	aggiorna b_c
$\partial \mathcal{L} / \partial h = (p - y)W_c^\top$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	propaga verso latenti post-pooling

Stato: due parametri già “pronti per l'update” (W_c, b_c). L'errore è un vettore di dim $D = 1024$ da ridistribuire sui $N = 512$ latenti originari tramite il backward del pooling (prossimo step).

Ricordiamo cosa succede nel forward

- Hai N latenti finali $L^{(\text{finale})} \in \mathbb{R}^{N \times D}$.
- Applichi average pooling per ottenere un unico vettore compatto $h \in \mathbb{R}^D$:

$$h = \frac{1}{N} \sum_{i=1}^N L_i$$

Quindi ogni decisione di classificazione dipende da tutti i latenti, mediati insieme.

Nel backward pooling

Il backward serve a propagare l'errore (il gradiente della loss) non solo a h , ma a ciascun latente.

Perché?

- Se ci fermassimo a h , sapremmo solo "quanto l'errore dipende dal vettore medio", ma non sapremmo come aggiornare i singoli latenti L_i .
- La rete, invece, ha bisogno di sapere quanto ciascuna latente ha contribuito all'errore, per poterne correggere i valori attraverso i pesi che l'hanno generato.

Il backward pooling è quindi il meccanismo che distribuisce l'errore dai logit $\rightarrow$ al vettore medio $h \rightarrow$ ai latenti L_i .

Sappiamo che:

$$\frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^D$$

Ora, per ogni latente L_i :

$$h = \frac{1}{N} \sum_{i=1}^N L_i$$

Quando scrivi

$$\frac{\partial h}{\partial L_i}$$

non stai derivando un numero rispetto a un numero, ma un vettore rispetto a un altro vettore. Il risultato non è uno scalare, ma una **matrice jacobiana**.

Dato che

$$h = \frac{1}{N} (L_1 + L_2 + \dots + L_N)$$

prendiamo un singolo L_i .

Il contributo di L_i a h è:

$$h = \dots + \frac{1}{N} L_i + \dots$$

cioè, semplicemente L_i moltiplicato per $\frac{1}{N}$

Se guardo una componente h_j :

$$h_j = \frac{1}{N} L_{i,j} + (\text{altre parti})$$

quindi:

$$\frac{\partial h_j}{\partial L_{i,k}} = \begin{cases} \frac{1}{N} & \text{se } j = k \\ 0 & \text{se } j \neq k \end{cases}$$

Derivata di h rispetto a un singolo latente L_i :

$$\frac{\partial h}{\partial L_i} = \frac{1}{N} I_D$$

Dovendo propagare l'errore da h ad ogni latente L_i , applicando la catena:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial L_i} &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial L_i} \\ \frac{\partial \mathcal{L}}{\partial L_i} &= \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \end{aligned}$$

Cioè, l'errore della Loss viene ridistribuita ad ogni latente in modo uniforme (ovvero riceve la stessa quota $\frac{1}{N}$ del gradiente)

Facciamo la media perché i latenti, dopo molti strati di self-attention, sono già stati “mescolati” e contengono informazioni simili → trattarli uguali non è un grosso problema.

Quindi: con la media “accusi tutti allo stesso modo”. Se vuoi dare più colpa a chi pesa di più, devi introdurre **pooling pesato** (ad esempio attention pooling).

Checkpoint gradienti — fine Pooling backward

Il pooling non ha parametri (è una media): non c'è nulla da aggiornare. Il suo backward è solo *propagare*: l'errore su h viene ridistribuito uniformemente sui N latenti.

Gradiente	Forma simbolica	ImageNet	Uso
$\partial \mathcal{L} / \partial h$ (da classifier)	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	in ingresso al pooling
$\partial \mathcal{L} / \partial L_i^{(\text{finale})} = \frac{1}{N} \partial \mathcal{L} / \partial h$	$\mathbb{R}^D (\times N)$	$\mathbb{R}^{1024} (\times 512)$	propaga a ogni latente
$\partial \mathcal{L} / \partial L^{(\text{finale})}$ (matrice completa)	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	input al backward latent transformer

Parametri pronti finora: W_c, b_c (del classifier). Da qui in avanti entriamo nella pipeline “profonda”: T blocchi Cross-Attention + Latent Transformer da attraversare in reverse, ciascuno con molti parametri.

Backward nel Latent Transformer

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← **Latent Transformer**

Attualmente abbiamo fatto loss → softmax → classificatore lineare → pooling → latenti i-esimi

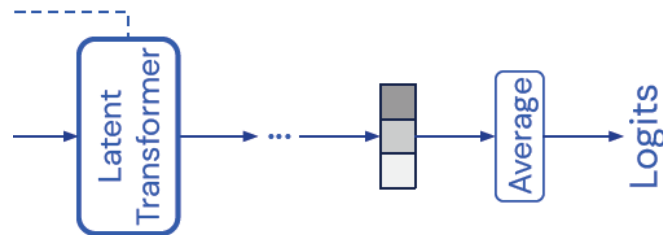


Figure 18: Schema del classification head: Latent Transformer → Average → Logits, attraverso cui risale il gradiente durante il backpropagation.

Ed abbiamo un gradiente su ciascun latente finale:

$$\frac{\partial \mathcal{L}}{\partial L_i^{(\text{finale})}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^D, \quad i = 1, \dots, N$$

Ora dobbiamo risalire dentro al Latent Transformer Module (cioè, i blocchi self-attention + MLP).

L'ultimo step del Latent Transformer era il residual quindi faccio il gradiente sul Residual post-MLP

$$L^{(\text{finale})} = \tilde{L} + F$$

dove:

- $\tilde{L}$ = latenti normalizzati,
- F = output dell'MLP.

Quindi una somma elemento per elemento.

Dovendo fare una derivata rispetto a una somma, la regola generale:

$$y = a + b$$

allora:

$$\frac{\partial y}{\partial a} = 1 \qquad \frac{\partial y}{\partial b} = 1$$

Applicazione al caso nostro

$$L^{(\text{finale})} = \tilde{L} + F$$

- Ogni elemento di $L^{(\text{finale})}$ dipende linearmente sia dal corrispondente elemento di $\tilde{L}$ che da quello di F .
- Non c'è nessun coefficiente moltiplicativo diverso da 1.

Quindi:

$$\frac{\partial L^{(\text{finale})}}{\partial \tilde{L}} = I \qquad \frac{\partial L^{(\text{finale})}}{\partial F} = I$$

dove I è la matrice identità.

Applicando la regola della catena:

$$\frac{\partial \mathcal{L}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \cdot \frac{\partial L^{(\text{finale})}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \cdot I = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

stessa cosa per F :

$$\frac{\partial \mathcal{L}}{\partial F} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Risultato: il gradiente in ingresso si copia uguale nei due rami perché la residual connection è una semplice somma

Backward su MLP

Nel Forward il blocco MLP era:

1. Espansione

$$H = \tilde{L}W_1 + b_1 \quad (W_1 \in \mathbb{R}^{1024 \times 4096}, H \in \mathbb{R}^{N \times 4096})$$

2. Attivazione (GELU)

$$H' = \sigma(H) \quad (\text{stessa forma } N \times 4096)$$

3. Compressione

$$F = H'W_2 + b_2 \quad (W_2 \in \mathbb{R}^{4096 \times 1024}, F \in \mathbb{R}^{N \times 1024})$$

4. Residual connection

$$L^{(\text{finale})} = \tilde{L} + F$$

Nel Backward Partiamo dal gradiente che arriva da

$$\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Come hai visto:

$$\frac{\partial \mathcal{L}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \qquad \frac{\partial \mathcal{L}}{\partial F} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Ora concentriamoci sul ramo F

1. Compressione:

$$F = H'W_2 + b_2$$

Gradiente rispetto a W_2 :

$$\frac{\partial \mathcal{L}}{\partial W_2} = H'^{\top} \frac{\partial \mathcal{L}}{\partial F}$$

Gradiente rispetto a b_2 :

$$\frac{\partial \mathcal{L}}{\partial b_2} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial F_i}$$

Gradiente che torna su H' :

$$\frac{\partial \mathcal{L}}{\partial H'} = \frac{\partial \mathcal{L}}{\partial F} W_2^{\top}$$

2. Attivazione:

$$H' = \sigma(H)$$

Qui entra in gioco la derivata della funzione di attivazione:

Se ReLU:

$$\frac{\partial H'}{\partial H} = \begin{cases} 1 & \text{se } H > 0 \\ 0 & \text{se } H \leq 0 \end{cases}$$

- Se GELU: derivata più complessa ma stesso concetto (una maschera continua).

Quindi:

$$\frac{\partial \mathcal{L}}{\partial H} = \frac{\partial \mathcal{L}}{\partial H'} \odot \sigma'(H)$$

(dove $\odot$ è prodotto elemento per elemento).

3. Espansione:

$$H = \tilde{L}W_1 + b_1$$

Gradiente rispetto a W_1 :

$$\frac{\partial \mathcal{L}}{\partial W_1} = \tilde{L}^\top \frac{\partial \mathcal{L}}{\partial H}$$

Gradiente rispetto a b_1 :

$$\frac{\partial \mathcal{L}}{\partial b_1} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial H_i}$$

- Gradiente che torna a $\tilde{L}$ (prima della residual connection):

$$\frac{\partial \mathcal{L}^{(MLP)}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial H} W_1^\top$$

4. Riassunto "flow" Il gradiente segue questo percorso:

$$\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \longrightarrow \frac{\partial \mathcal{L}}{\partial F} \longrightarrow \frac{\partial \mathcal{L}}{\partial H'} \longrightarrow \frac{\partial \mathcal{L}}{\partial H} \longrightarrow \frac{\partial \mathcal{L}}{\partial \tilde{L}}$$

e in parallelo aggiorna i pesi W_2, b_2, W_1, b_1 .

Attenzione: gradiente totale su $\tilde{L}$

Poiché $\tilde{L}$ è usato *sia* come input del ramo MLP *sia* dalla residual post-MLP ($L^{(\text{finale})} = \tilde{L} + F$), il gradiente *totale* che arriva a $\tilde{L}$ è la **somma dei due contributi**:

$$\left. \frac{\partial \mathcal{L}}{\partial \tilde{L}} \right|_{\text{tot}} = \underbrace{\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}}_{\text{dal ramo residual}} + \underbrace{\frac{\partial \mathcal{L}}{\partial H} W_1^\top}_{\text{dal ramo MLP}}$$

Questo è l'esempio canonico di “gradient branching” nelle architetture con skip connections.

GELU backward — formula esplicita

Per la GELU $\sigma(x) = x \cdot \Phi(x)$ (dove Φ è la CDF della gaussiana standard), la derivata è:

$$\sigma'(x) = \Phi(x) + x \cdot \phi(x), \quad \phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

In pratica, nei framework si usa l'approssimazione tanh di Hendrycks & Gimpel (2016):

$$\sigma(x) \approx 0.5 x \left[1 + \tanh\left(\sqrt{2/\pi}(x + 0.044715 x^3)\right) \right]$$

la cui derivata è facile da calcolare esplicitamente o via autograd.

Checkpoint gradienti — fine MLP backward

Dopo l'MLP abbiamo i gradienti sui 4 parametri del blocco MLP e il gradiente che propaga indietro verso la LayerNorm precedente. La MLP usa hidden dim $4D = 4096$.

Gradiente	Forma simbolica	ImageNet	Uso
$\partial \mathcal{L} / \partial F$	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	residual copy
$\partial \mathcal{L} / \partial W_2 = H'^\top \partial \mathcal{L} / \partial F$	$\mathbb{R}^{4D \times D}$	$\mathbb{R}^{4096 \times 1024}$	aggiorna W_2
$\partial \mathcal{L} / \partial b_2 = \sum_i \partial \mathcal{L} / \partial F_i$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	aggiorna b_2
$\partial \mathcal{L} / \partial H' = \partial \mathcal{L} / \partial F \cdot W_2^\top$	$\mathbb{R}^{N \times 4D}$	$\mathbb{R}^{512 \times 4096}$	intermedio
$\partial \mathcal{L} / \partial H = \partial \mathcal{L} / \partial H' \odot \text{GELU}'(H)$	$\mathbb{R}^{N \times 4D}$	$\mathbb{R}^{512 \times 4096}$	intermedio
$\partial \mathcal{L} / \partial W_1 = \tilde{L}^\top \partial \mathcal{L} / \partial H$	$\mathbb{R}^{D \times 4D}$	$\mathbb{R}^{1024 \times 4096}$	aggiorna W_1
$\partial \mathcal{L} / \partial b_1 = \sum_i \partial \mathcal{L} / \partial H_i$	$\mathbb{R}^{4D}$	$\mathbb{R}^{4096}$	aggiorna b_1
$\partial \mathcal{L} / \partial \tilde{L}^{(\text{MLP})}$	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	propaga verso MLP input
$\left. \partial \mathcal{L} / \partial \tilde{L} \right _{\text{tot}}$	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	somma residual + MLP

Parametri pronti finora: W_c, b_c (classifier) + W_1, b_1, W_2, b_2 (MLP di questo blocco). Se il blocco è condiviso tra le T iterazioni, il gradiente calcolato *si accumula* con quello delle altre iterazioni (cfr. §1.4.12).

Layer Normalization

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← Layer Normalization

Nel forward:

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j$$

- μ_i : media sulla riga i
- σ_i : deviazione standard sulla riga i
- γ, β : parametri appresi (gain e bias della normalizzazione)
- Backward della LayerNorm

Per risalire da $\frac{\partial \mathcal{L}}{\partial \tilde{L}}$ a L' :

1. Gradiente rispetto a γ e β :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \gamma_j} &= \sum_i \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \cdot \hat{L}_{ij} \\ \frac{\partial \mathcal{L}}{\partial \beta_j} &= \sum_i \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \end{aligned}$$

dove $\hat{L}_{ij} = \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon}$

2. Gradiente rispetto a L' :

Qui si usa la catena di derivate, e la formula finale è:

$$\frac{\partial \mathcal{L}}{\partial L'_{ij}} = \frac{1}{\sigma_i + \epsilon} \left(\frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \cdot \gamma_j - \frac{1}{D} \sum_k \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ik}} \cdot \gamma_k - \hat{L}_{ij} \cdot \frac{1}{D} \sum_k \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ik}} \cdot \gamma_k \cdot \hat{L}_{ik} \right)$$

con $D = 1024$ (la dimensione del canale). Il denominatore $\sigma_i + \epsilon$ è coerente con il forward: la chain rule preserva lo stesso fattore di normalizzazione.

Residual Connection (post-attention o post-MLP)

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← **Residual Connection**

Forward:

$$L' = L + Z'$$

(dove L è l'input originale e Z' è l'output trasformato, ad esempio da self-attention o MLP).

Backward:

Il gradiente si divide in due rami identici perché la somma è lineare:

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial L'} \quad \frac{\partial \mathcal{L}}{\partial Z'} = \frac{\partial \mathcal{L}}{\partial L'}$$

Interpretazione:

- Il gradiente in uscita da L' viene copiato sia sul percorso "skip" (L) sia sul percorso trasformato (Z').

- In questo modo, la rete può imparare sia a mantenere informazione grezza (via skip connection), sia a modificarla (via ramo trasformato).

Da qui, andando ancora indietro, tocchiamo la proiezione lineare finale dello Z' :

$$Z' = ZW_o + b_o$$

Backward - Output Projection

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← Output Projection

Forward

Dopo l'attenzione otteniamo:

$$Z' = ZW_o + b_o$$

- $Z \in \mathbb{R}^{N \times d_v}$ (output dell'attention, prima della proiezione di output)
- $W_o \in \mathbb{R}^{d_v \times D}$ (proietta dalla dim dell'attention alla dim dei latenti, per il residual)
- $b_o \in \mathbb{R}^D$
- $Z' \in \mathbb{R}^{N \times D}$

Backward

Sappiamo che arriva il gradiente della loss rispetto all'uscita:

$$\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$$

Dobbiamo propagare questo gradiente indietro verso Z , W_o e b_o .

1. Gradiente rispetto a W_o Ogni riga di Z contribuisce linearmente a Z' .

Per la derivata matriciale:

$$\frac{\partial \mathcal{L}}{\partial W_o} = Z^\top \frac{\partial \mathcal{L}}{\partial Z'}$$

- $Z^\top \in \mathbb{R}^{D \times N}$
- $\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_o} \in \mathbb{R}^{D \times D}$$

2. Gradiente rispetto a b_o Il bias b_o viene aggiunto ad ogni riga di Z' .
La derivata è:

$$\frac{\partial \mathcal{L}}{\partial b_o} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Z'_i}$$

- Ogni $\frac{\partial \mathcal{L}}{\partial Z'_i} \in \mathbb{R}^D$
- La somma su $i = 1..N$ restituisce:

$$\frac{\partial \mathcal{L}}{\partial b_o} \in \mathbb{R}^D$$

3. Gradiente rispetto a Z La derivata di $Z' = ZW_o + b_o$ rispetto a Z è semplicemente:

$$\frac{\partial Z'}{\partial Z} = W_o$$

Quindi:

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial Z'} W_o^\top$$

- $\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$
- $W_o^\top \in \mathbb{R}^{D \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times D}$$

Riassunto backward output projection

Verso Z :

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial Z'} W_o^\top \in \mathbb{R}^{N \times D}$$

Verso W_o :

$$\frac{\partial \mathcal{L}}{\partial W_o} = Z^\top \frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{D \times D}$$

Verso b_o :

$$\frac{\partial \mathcal{L}}{\partial b_o} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Z'_i} \in \mathbb{R}^D$$

Checkpoint gradienti — fine LayerNorm + Residual + Output Projection

Questi tre blocchi sono concatenati (LN → residual → output projection W_o) e si attraversano in reverse a catena. Lo stato dei gradienti è:

Gradiente	Forma simbolica	ImageNet	Uso
<i>LayerNorm backward</i>			
$\partial\mathcal{L}/\partial\gamma$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	aggiorna γ
$\partial\mathcal{L}/\partial\beta$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	aggiorna β
$\partial\mathcal{L}/\partial L'$ (3 termini)	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	propaga prima della LN
<i>Residual post-attention</i>			
$\partial\mathcal{L}/\partial L, \partial\mathcal{L}/\partial Z'$	$\mathbb{R}^{N \times D}$	$\mathbb{R}^{512 \times 1024}$	entrambi = $\partial\mathcal{L}/\partial L'$
<i>Output Projection W_o</i>			
$\partial\mathcal{L}/\partial W_o = Z^\top \partial\mathcal{L}/\partial Z'$	$\mathbb{R}^{d_{QKV} \times D}$	$\mathbb{R}^{261 \times 1024}$	aggiorna W_o
$\partial\mathcal{L}/\partial b_o = \sum_i \partial\mathcal{L}/\partial Z'_i$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	aggiorna b_o
$\partial\mathcal{L}/\partial Z = \partial\mathcal{L}/\partial Z' \cdot W_o^\top$	$\mathbb{R}^{N \times d_{QKV}}$	$\mathbb{R}^{512 \times 261}$	propaga verso l'attention

Parametri pronti finora: W_c, b_c (classifier), W_1, b_1, W_2, b_2 (MLP), γ, β (LN), W_o, b_o (output projection). Il gradiente di attivazione $\partial\mathcal{L}/\partial Z \in \mathbb{R}^{512 \times 261}$ è adesso pronto per entrare nel backward dell'attention vera e propria (V, A, S, Q, K).

Backward - Scaled Dot-Product Attention

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← Scaled Dot-Product Attention

Forward (richiamo)

L'attenzione calcola:

$$\text{Attention}(Q, K, V) = AV$$

con

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

- $Q \in \mathbb{R}^{N \times d_k}$ (queries)
- $K \in \mathbb{R}^{M \times d_k}$ (keys)
- $V \in \mathbb{R}^{M \times d_v}$ (values)
- $QK^\top \in \mathbb{R}^{N \times M}$ (score matrix)
- $A \in \mathbb{R}^{N \times M}$ (matrice delle attenzioni, dopo softmax su ogni riga)
- Output:

$$Z = AV \in \mathbb{R}^{N \times d_v}$$

Backward

Supponiamo di ricevere in ingresso il gradiente:

$$\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$$

Dobbiamo risalire a Q, K, V .

1. Gradiente verso V

$$Z = AV$$

Derivata:

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial Z}$$

- Dimensioni:
 - $A^\top \in \mathbb{R}^{M \times N}$
 - $\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$

Risultato:

$$\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$$

2. Gradiente verso A

$$Z = AV \Rightarrow \frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial Z} V^\top$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$
- $V^\top \in \mathbb{R}^{d_v \times M}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial A} \in \mathbb{R}^{N \times M}$$

3. Backward attraverso Softmax Abbiamo:

$$A = \text{softmax}(S), \quad S = \frac{QK^\top}{\sqrt{d_k}}$$

La softmax è applicata indipendentemente su ogni riga di S , quindi possiamo lavorare riga per riga. La jacobiana della softmax su una riga è:

$$\frac{\partial A_{ij}}{\partial S_{ik}} = A_{ij}(\delta_{jk} - A_{ik})$$

Applicando la chain rule $\partial \mathcal{L} / \partial S_{ik} = \sum_j (\partial \mathcal{L} / \partial A_{ij})(\partial A_{ij} / \partial S_{ik})$ e semplificando con $\sum_j A_{ij} = 1$ si ottiene la **forma esplicita compatta**:

Softmax backward — forma vettoriale per riga

$$\frac{\partial \mathcal{L}}{\partial S_{ik}} = A_{ik} \left(\frac{\partial \mathcal{L}}{\partial A_{ik}} - \sum_{j=1}^M A_{ij} \frac{\partial \mathcal{L}}{\partial A_{ij}} \right)$$

o equivalentemente, in notazione vettoriale per la riga i -esima:

$$\frac{\partial \mathcal{L}}{\partial S_i} = A_i \odot \left(\frac{\partial \mathcal{L}}{\partial A_i} - \langle A_i, \partial \mathcal{L} / \partial A_i \rangle \cdot \mathbf{1} \right)$$

con $\odot$ prodotto elemento per elemento e $\langle \cdot, \cdot \rangle$ prodotto scalare.

Interpretazione. Ogni $\partial \mathcal{L} / \partial S_{ik}$ è il gradiente su A_{ik} “corretto” sottraendo la media pesata dei gradienti sulle altre entrate della stessa riga (pesata dai pesi di attenzione A_{ij}). Questa correzione garantisce che il gradiente su S sia ortogonale al vincolo $\sum_j A_{ij} = 1$ imposto dalla softmax.

Dimensioni finali: $\partial \mathcal{L} / \partial S \in \mathbb{R}^{N \times M}$.

4. Gradiente verso Q e K Sappiamo:

$$S = \frac{QK^\top}{\sqrt{d_k}}$$

Verso Q

$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K \cdot \frac{1}{\sqrt{d_k}}$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial S} \in \mathbb{R}^{N \times M}$
- $K \in \mathbb{R}^{M \times d_k}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$$

Verso K

$$\frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q \cdot \frac{1}{\sqrt{d_k}}$$

- Dimensioni:
- $\left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top \in \mathbb{R}^{M \times N}$
- $Q \in \mathbb{R}^{N \times d_k}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$$

Riassunto backward scaled dot-product attention

Verso V :

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{M \times d_v}$$

Verso A :

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial Z} V^\top \in \mathbb{R}^{N \times M}$$

Verso S (pre-softmax):

$$\frac{\partial \mathcal{L}}{\partial S} \in \mathbb{R}^{N \times M}$$

Verso Q :

$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K \cdot \frac{1}{\sqrt{d_k}} \in \mathbb{R}^{N \times d_k}$$

Verso K :

$$\frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q \cdot \frac{1}{\sqrt{d_k}} \in \mathbb{R}^{M \times d_k}$$

Multi-head backward: come cambia il flusso

La derivazione sopra è per single-head. Nel *multi-head* (H teste, ciascuna di dimensione d_{head}), il flusso è lo stesso ma con split/merge sulle teste. Il forward multi-head è:

$$\underbrace{Q, K, V}_{\in \mathbb{R}^{\cdot \times H d_{\text{head}}}} \rightarrow \text{split}_H \rightarrow \underbrace{Q_h, K_h, V_h}_{\in \mathbb{R}^{\cdot \times d_{\text{head}}}} \rightarrow \underbrace{Z_h = A_h V_h}_{\in \mathbb{R}^{N \times d_{\text{head}}}} \rightarrow \text{concat}_H \rightarrow Z \in \mathbb{R}^{N \times H d_{\text{head}}} \rightarrow Z' = Z W_o$$

Nel backward, le operazioni si invertono:

1. Dal gradiente $\partial \mathcal{L} / \partial Z \in \mathbb{R}^{N \times H d_{\text{head}}}$ (post- W_o), si fa lo **split** reshape $N \times H d_{\text{head}} \rightarrow H \times N \times d_{\text{head}}$: il gradiente si ripartisce sulle teste, una fetta di d_{head} colonne per ciascuna.
2. Per ogni testa h si esegue la derivazione single-head già vista: $\partial \mathcal{L} / \partial V_h = A_h^\top (\partial \mathcal{L} / \partial Z_h)$, $\partial \mathcal{L} / \partial A_h = (\partial \mathcal{L} / \partial Z_h) V_h^\top$, ecc.
3. I gradienti $\partial \mathcal{L} / \partial Q_h$, $\partial \mathcal{L} / \partial K_h$, $\partial \mathcal{L} / \partial V_h$ vengono ri-concatenati lungo la dimensione head per ottenere $\partial \mathcal{L} / \partial Q$, $\partial \mathcal{L} / \partial K$, $\partial \mathcal{L} / \partial V$ nella forma $\cdot \times H d_{\text{head}}$.
4. Il backward delle proiezioni lineari W_Q, W_K, W_V (che erano singole matrici di forma $D \times H d_{\text{head}}$) procede come nel single-head.

Dettaglio importante: nell'implementazione (e nel codice di progetto, `attention.py`), lo split e il concat sono semplici **reshape** tensoriali, quindi il backward li attraversa senza calcolo aggiuntivo — il

gradiente “attraversa” i reshape. Dal punto di vista matematico, il multi-head è equivalente a H attenzioni separate più piccole che condividono le stesse matrici di proiezione iniziali.

Checkpoint gradienti — fine Scaled Dot-Product backward

Lo scaled dot-product è una sequenza di operazioni elementari senza parametri (matmul, scaling, softmax, matmul): il suo backward *non aggiorna nessun peso*. Produce solo gradienti che si propagano a Q , K , V .

Gradiente	Forma simbolica	ImageNet	Uso
$\partial\mathcal{L}/\partial V = A^\top \partial\mathcal{L}/\partial Z$	$\mathbb{R}^{M \times d_{QKV}}$	$\mathbb{R}^{50\,176 \times 261}$	propaga verso V
$\partial\mathcal{L}/\partial A = \partial\mathcal{L}/\partial Z \cdot V^\top$	$\mathbb{R}^{N \times M}$	$\mathbb{R}^{512 \times 50\,176}$	intermedio
$\partial\mathcal{L}/\partial S$ (via jacobiana softmax)	$\mathbb{R}^{N \times M}$	$\mathbb{R}^{512 \times 50\,176}$	intermedio
$\partial\mathcal{L}/\partial Q = \partial\mathcal{L}/\partial S \cdot K / \sqrt{d_{QKV}}$	$\mathbb{R}^{N \times d_{QKV}}$	$\mathbb{R}^{512 \times 261}$	propaga verso Q
$\partial\mathcal{L}/\partial K = (\partial\mathcal{L}/\partial S)^\top \cdot Q / \sqrt{d_{QKV}}$	$\mathbb{R}^{M \times d_{QKV}}$	$\mathbb{R}^{50\,176 \times 261}$	propaga verso K

Parametri pronti finora: *invariato rispetto al checkpoint precedente* — lo scaled dot-product non ha parametri propri. Gli unici update calcolati finora sono $W_c, b_c, W_1, b_1, W_2, b_2, \gamma, \beta, W_o, b_o$. Il prossimo step (proiezioni lineari Q/K/V) userà $\partial\mathcal{L}/\partial Q, \partial\mathcal{L}/\partial K, \partial\mathcal{L}/\partial V$ per calcolare i gradienti su W_Q, W_K, W_V e per propagare indietro verso i latenti e verso l’input.

Backward - Proiezioni lineari Q,K,V

Input → Fourier → Latenti → Cross-Att → **Latent Transf.** → $\times T$ → Pooling

Backward ← Proiezioni lineari Q,K,V

Forward (richiamo)

Nel Perceiver otteniamo:

$$\begin{aligned} Q &= LW_Q + b_Q \\ K &= XW_K + b_K \\ V &= XW_V + b_V \end{aligned}$$

- $L \in \mathbb{R}^{N \times D}$ = latenti (per Q)
- $X \in \mathbb{R}^{M \times D}$ = input embedding (per K, V)
- $W_Q, W_K, W_V \in \mathbb{R}^{D \times d_k}$ oppure $D \times d_v$
- $b_Q, b_K \in \mathbb{R}^{d_k}, b_V \in \mathbb{R}^{d_v}$
- $Q \in \mathbb{R}^{N \times d_k}, K \in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v}$

Backward

Abbiamo già calcolato dai passi precedenti:

- $\frac{\partial\mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$
- $\frac{\partial\mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$
- $\frac{\partial\mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$

Ora risaliamo a $W_Q, W_K, W_V, b_Q, b_K, b_V$ e agli input L, X .

1. Proiezione delle Query

$$Q = LW_Q + b_Q$$

Verso W_Q :

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^\top \frac{\partial \mathcal{L}}{\partial Q}$$

- Dimensioni:
- $L^\top \in \mathbb{R}^{D \times N}$
- $\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_Q} \in \mathbb{R}^{D \times d_k}$$

Verso b_Q :

$$\frac{\partial \mathcal{L}}{\partial b_Q} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Q_i} \in \mathbb{R}^{d_k}$$

Verso L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$
- $W_Q^\top \in \mathbb{R}^{d_k \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial L} \in \mathbb{R}^{N \times D}$$

2. Proiezione delle Key

$$K = XW_K + b_K$$

Verso W_K :

$$\frac{\partial \mathcal{L}}{\partial W_K} = X^\top \frac{\partial \mathcal{L}}{\partial K}$$

- Dimensioni:
- $X^\top \in \mathbb{R}^{D \times M}$
- $\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_K} \in \mathbb{R}^{D \times d_k}$$

Verso b_K :

$$\frac{\partial \mathcal{L}}{\partial b_K} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial K_i} \in \mathbb{R}^{d_k}$$

Verso X (dal ramo K):

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^\top$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$
- $W_K^\top \in \mathbb{R}^{d_k \times D}$
- Risultato:

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_K \in \mathbb{R}^{M \times D}$$

3. Proiezione delle Value

$$V = XW_V + b_V$$

Verso W_V :

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^\top \frac{\partial \mathcal{L}}{\partial V}$$

- Dimensioni:
- $X^\top \in \mathbb{R}^{D \times M}$
- $\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_V} \in \mathbb{R}^{D \times d_v}$$

Verso b_V :

$$\frac{\partial \mathcal{L}}{\partial b_V} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial V_i} \in \mathbb{R}^{d_v}$$

Verso X (dal ramo V):

$$\frac{\partial \mathcal{L}}{\partial X} \Big|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^\top$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$
- $W_V^\top \in \mathbb{R}^{d_v \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial X} \Big|_V \in \mathbb{R}^{M \times D}$$

Riassunto backward proiezioni lineari

Verso W_Q :

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^\top \frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{D \times d_k}$$

Verso b_Q :

$$\frac{\partial \mathcal{L}}{\partial b_Q} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Q_i} \in \mathbb{R}^{d_k}$$

Verso L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top \in \mathbb{R}^{N \times D}$$

Verso W_K :

$$\frac{\partial \mathcal{L}}{\partial W_K} = X^\top \frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{D \times d_k}$$

Verso b_K :

$$\frac{\partial \mathcal{L}}{\partial b_K} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial K_i} \in \mathbb{R}^{d_k}$$

Verso X dal ramo K :

$$\frac{\partial \mathcal{L}}{\partial X} \Big|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^\top \in \mathbb{R}^{M \times D}$$

Verso W_V :

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^\top \frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{D \times d_v}$$

Verso b_V :

$$\frac{\partial \mathcal{L}}{\partial b_V} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial V_i} \in \mathbb{R}^{d_v}$$

Verso X dal ramo V :

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^\top \in \mathbb{R}^{M \times D}$$

Nota importante:

Il gradiente totale su X è la somma dei contributi che arrivano dal ramo K e dal ramo V :

$$\frac{\partial \mathcal{L}}{\partial X} = \left. \frac{\partial \mathcal{L}}{\partial X} \right|_K + \left. \frac{\partial \mathcal{L}}{\partial X} \right|_V$$

Checkpoint gradienti — fine proiezioni lineari Q/K/V backward

Questo è il backward delle tre proiezioni lineari che generano Q, K, V dai latenti (L) e dall'input embedded (X). Calcola gli update sui tre pesi W_Q, W_K, W_V e propaga indietro verso L (dalla Q) e verso X (dalla K e dalla V).

Gradiente	Forma simbolica	ImageNet	Uso
<i>Proiezione Q (dai latenti)</i> $\partial \mathcal{L} / \partial W_Q = L^\top \partial \mathcal{L} / \partial Q$ $\partial \mathcal{L} / \partial L = \partial \mathcal{L} / \partial Q \cdot W_Q^\top$	$\mathbb{R}^{D \times d_{QKV}}$ $\mathbb{R}^{N \times D}$	$\mathbb{R}^{1024 \times 261}$ $\mathbb{R}^{512 \times 1024}$	aggiorna W_Q propaga verso latenti
<i>Proiezione K (dall'input)</i> $\partial \mathcal{L} / \partial W_K = X^\top \partial \mathcal{L} / \partial K$ $\partial \mathcal{L} / \partial X _K = \partial \mathcal{L} / \partial K \cdot W_K^\top$	$\mathbb{R}^{C_{\text{tot}} \times d_{QKV}}$ $\mathbb{R}^{M \times C_{\text{tot}}}$	$\mathbb{R}^{261 \times 261}$ $\mathbb{R}^{50\,176 \times 261}$	aggiorna W_K contributo K
<i>Proiezione V (dall'input)</i> $\partial \mathcal{L} / \partial W_V = X^\top \partial \mathcal{L} / \partial V$ $\partial \mathcal{L} / \partial X _V = \partial \mathcal{L} / \partial V \cdot W_V^\top$	$\mathbb{R}^{C_{\text{tot}} \times d_{QKV}}$ $\mathbb{R}^{M \times C_{\text{tot}}}$	$\mathbb{R}^{261 \times 261}$ $\mathbb{R}^{50\,176 \times 261}$	aggiorna W_V contributo V
<i>Merge contributi verso X</i> $\partial \mathcal{L} / \partial X = \partial \mathcal{L} / \partial X _K + \partial \mathcal{L} / \partial X _V$	$\mathbb{R}^{M \times C_{\text{tot}}}$	$\mathbb{R}^{50\,176 \times 261}$	propaga verso input embedding

Parametri pronti finora (fine di *un* blocco cross-attention completo, dalla loss all'input embedding): W_c, b_c (classifier), W_1, b_1, W_2, b_2 (MLP), γ, β (LN), W_o, b_o (output proj), W_Q, W_K, W_V (proiezioni Q/K/V). I gradienti di attivazione che propagano oltre sono $\partial \mathcal{L} / \partial L$ (verso i latenti del blocco precedente, se $T > 1$) e $\partial \mathcal{L} / \partial X$ (verso l'input embedding).

Nota su weight sharing: se W_Q, W_K, W_V sono condivisi tra le T iterazioni (cfr. §1.4.12), i gradienti sopra si *accumulano* su ciascun parametro, sommando i contributi da ogni iterazione.

Backward - Layer Normalization (su Q/K/V) Forward (richiamo)

Dato un input $H \in \mathbb{R}^{N \times D}$ (dove ogni riga ha dimensione D), la LayerNorm calcola:

$$\tilde{H}_{ij} = \gamma_j \cdot \hat{H}_{ij} + \beta_j$$

dove:

- $\hat{H}_{ij} = \frac{H_{ij} - \mu_i}{\sigma_i + \epsilon}$
- $\mu_i = \frac{1}{D} \sum_{j=1}^D H_{ij}$ (media della riga i)
- $\sigma_i = \sqrt{\frac{1}{D} \sum_{j=1}^D (H_{ij} - \mu_i)^2}$ (deviazione standard della riga i)
- $\gamma, \beta \in \mathbb{R}^D$ (parametri appresi: gain e bias)
- Output: $\tilde{H} \in \mathbb{R}^{N \times D}$

Backward

Arriva in ingresso:

$$\frac{\partial \mathcal{L}}{\partial \tilde{H}} \in \mathbb{R}^{N \times D}$$

Dobbiamo risalire a γ, β, H .

1. Gradiente rispetto a γ e β Verso γ :

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \cdot \hat{H}_{ij} \in \mathbb{R}^D$$

Verso β :

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \in \mathbb{R}^D$$

2. Gradiente rispetto all'input normalizzato $\hat{H}$

Poiché:

$$\tilde{H}_{ij} = \gamma_j \hat{H}_{ij} + \beta_j$$

si ha:

$$\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} = \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \cdot \gamma_j$$

Forma compatta:

$$\frac{\partial \mathcal{L}}{\partial \hat{H}} = \frac{\partial \mathcal{L}}{\partial \tilde{H}} \odot \gamma \in \mathbb{R}^{N \times D}$$

3. Gradiente rispetto a H Ora dobbiamo tornare da $\hat{H}$ a H .

Ricordiamo:

$$\hat{H}_{ij} = \frac{H_{ij} - \mu_i}{\sigma_i + \epsilon}$$

Il gradiente finale per ogni riga i (dimensione D) è:

$$\frac{\partial \mathcal{L}}{\partial H_{ij}} = \frac{1}{\sigma_i + \epsilon} \left(\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} - \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} - \hat{H}_{ij} \cdot \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} \hat{H}_{ik} \right)$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial \hat{H}} \in \mathbb{R}^{N \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial H} \in \mathbb{R}^{N \times D}$$

Riassunto backward LayerNorm

Verso γ :

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \cdot \hat{H}_{ij} \in \mathbb{R}^D$$

Verso β :

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \in \mathbb{R}^D$$

Verso H :

$$\frac{\partial \mathcal{L}}{\partial H_{ij}} = \frac{1}{\sigma_i + \epsilon} \left(\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} - \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} - \hat{H}_{ij} \cdot \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} \hat{H}_{ik} \right) \in \mathbb{R}^{N \times D}$$

Backward – Cross Attention

Forward (richiamo)

$$Z = \text{Attention}(Q, K, V)W_o + b_o$$

con:

- $Q = LW_Q + b_Q \in \mathbb{R}^{N \times d_k}$
- $K = XW_K + b_K \in \mathbb{R}^{M \times d_k}$

- $V = XW_V + b_V \in \mathbb{R}^{M \times d_v}$
- $Z \in \mathbb{R}^{N \times d_v}$

Backward - Flusso dei gradienti

1. **Output projection** Già fatto:

$$\frac{\partial \mathcal{L}}{\partial Z}, \frac{\partial \mathcal{L}}{\partial W_o}, \frac{\partial \mathcal{L}}{\partial b_o}$$

2. **Scaled Dot-Product Attention**

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial Z}, \frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K / \sqrt{d_k}, \frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q / \sqrt{d_k}$$

3. **Proiezioni lineari**

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_Q} &= L^\top \frac{\partial \mathcal{L}}{\partial Q}, & \frac{\partial \mathcal{L}}{\partial b_Q} &= \sum_i \frac{\partial \mathcal{L}}{\partial Q_i}, & \frac{\partial \mathcal{L}}{\partial L} &= \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top \\ \frac{\partial \mathcal{L}}{\partial W_K} &= X^\top \frac{\partial \mathcal{L}}{\partial K}, & \frac{\partial \mathcal{L}}{\partial b_K} &= \sum_i \frac{\partial \mathcal{L}}{\partial K_i}, & \frac{\partial \mathcal{L}}{\partial X} \Big|_K &= \frac{\partial \mathcal{L}}{\partial K} W_K^\top \\ \frac{\partial \mathcal{L}}{\partial W_V} &= X^\top \frac{\partial \mathcal{L}}{\partial V}, & \frac{\partial \mathcal{L}}{\partial b_V} &= \sum_i \frac{\partial \mathcal{L}}{\partial V_i}, & \frac{\partial \mathcal{L}}{\partial X} \Big|_V &= \frac{\partial \mathcal{L}}{\partial V} W_V^\top \end{aligned}$$

Riassunto Cross-Attention Backward

- Il gradiente si propaga da $Z \rightarrow$ attenzione $\rightarrow$ proiezioni lineari.
- Verso latenti L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top$$

Verso input X :

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial X} \Big|_K + \frac{\partial \mathcal{L}}{\partial X} \Big|_V$$

- Verso pesi W_Q, W_K, W_V e bias relativi: aggiornati con outer product tra input e gradienti.

Backward - Input Embedding Forward (richiamo)

All'inizio il Perceiver prende l'input X_{raw} (es. immagine, audio, testo) e lo passa in un embedding lineare o una piccola convoluzione per portarlo nello spazio latente D :

$$X = X_{\text{raw}} W_E + b_E$$

- $X_{\text{raw}} \in \mathbb{R}^{M \times C}$ (M = numero token/pixel/patch, C = canali originali)
- $W_E \in \mathbb{R}^{C \times D}$ (matrice embedding)
- $b_E \in \mathbb{R}^D$ (bias embedding)
- $X \in \mathbb{R}^{M \times D}$ (embedding usato per K, V)

Backward

Dopo aver fatto il backward della cross-attention, abbiamo già ottenuto:

$$\frac{\partial \mathcal{L}}{\partial X} \in \mathbb{R}^{M \times D}$$

Ora risaliamo.

Verso W_E :

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^\top \frac{\partial \mathcal{L}}{\partial X} \in \mathbb{R}^{C \times D}$$

Verso b_E :

$$\frac{\partial \mathcal{L}}{\partial b_E} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial X_i} \in \mathbb{R}^D$$

Verso input grezzo X_{raw} :

$$\frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^\top \in \mathbb{R}^{M \times C}$$

Riassunto Input Embedding Backward

Verso W_E : aggiorna la matrice embedding

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^\top \frac{\partial \mathcal{L}}{\partial X}$$

Verso b_E : accumula gli errori

$$\frac{\partial \mathcal{L}}{\partial b_E} = \sum_i \frac{\partial \mathcal{L}}{\partial X_i}$$

- Verso input grezzo: distribuisce il gradiente all'array iniziale

$$\frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^\top$$

Perceiver - Forward & Backward Pass (schema riassuntivo)

Forward Pass

1. Input grezzo

$$X_{\text{raw}} \in \mathbb{R}^{M \times C}$$

2. Input Embedding

$$X = X_{\text{raw}} W_E + b_E \in \mathbb{R}^{M \times D}$$

3. Cross-Attention

- Queries dai latenti: $Q = LW_Q + b_Q \in \mathbb{R}^{N \times d_k}$
- Keys e Values dall'input:

$$K = XW_K + b_K \in \mathbb{R}^{M \times d_k}$$

$$V = XW_V + b_V \in \mathbb{R}^{M \times d_v}$$

Attenzione:

$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{N \times d_v}$$

Proiezione output (riporta a dimensione D per la residual connection):

$$Z' = ZW_o + b_o \in \mathbb{R}^{N \times D}, \quad W_o \in \mathbb{R}^{d_v \times D}$$

4. Residual + LayerNorm + MLP (ripetuto T volte nei blocchi latenti)

$$L_{\text{final}} \in \mathbb{R}^{N \times D}$$

5. Pooling

$$h = \frac{1}{N} \sum_{i=1}^N L_{\text{final},i} \in \mathbb{R}^D$$

6. Classificatore lineare + softmax

$$z = hW_c + b_c \in \mathbb{R}^K, \quad p = \text{softmax}(z) \in \mathbb{R}^K$$

7. Loss (cross-entropy)

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$$

Backward Pass

1. Loss $\rightarrow$ Softmax $\rightarrow$ Logits

$$\frac{\partial \mathcal{L}}{\partial z} = p - y \in \mathbb{R}^K$$

2. Classificatore lineare

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^\top (p - y) \in \mathbb{R}^{D \times K}, \quad \frac{\partial \mathcal{L}}{\partial b_c} = p - y \in \mathbb{R}^K$$

3. Pooling

$$\frac{\partial \mathcal{L}}{\partial L_{\text{final},i}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^D$$

4. Latent Transformer Block (Residual + MLP + Self-Attention)

- Residual: gradiente copiato nei due rami
- MLP: aggiorna W_1, W_2, b_1, b_2 e propaga a L
- LayerNorm: aggiorna γ, β e propaga a input normalizzato
- Self-Attention: stesso backward della cross-attention ma con Q, K, V tutti dai latenti

5. Cross-Attention

$$\frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial V}$$

- Verso output projection W_o, b_o
- Verso softmax dei punteggi $S = QK^\top / \sqrt{d_k}$
- Verso Q, K, V lineari

6. Proiezioni lineari Queries (latenti):

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^\top \frac{\partial \mathcal{L}}{\partial Q}, \quad \frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top$$

Keys e Values (input):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_K} &= X^\top \frac{\partial \mathcal{L}}{\partial K}, \quad \left. \frac{\partial \mathcal{L}}{\partial X} \right|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^\top \\ \frac{\partial \mathcal{L}}{\partial W_V} &= X^\top \frac{\partial \mathcal{L}}{\partial V}, \quad \left. \frac{\partial \mathcal{L}}{\partial X} \right|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^\top \\ \frac{\partial \mathcal{L}}{\partial X} &= \left. \frac{\partial \mathcal{L}}{\partial X} \right|_K + \left. \frac{\partial \mathcal{L}}{\partial X} \right|_V \end{aligned}$$

7. Input Embedding

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^\top \frac{\partial \mathcal{L}}{\partial X}, \quad \frac{\partial \mathcal{L}}{\partial b_E} = \sum_i \frac{\partial \mathcal{L}}{\partial X_i}, \quad \frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^\top$$

In sintesi

- Forward: l'input grezzo $\rightarrow$ embedding $\rightarrow$ cross-attention (Q da latenti, K/V da input) $\rightarrow$ latent transformer $\rightarrow$ pooling $\rightarrow$ classificatore $\rightarrow$ softmax $\rightarrow$ loss.
- Backward: la loss risale in senso inverso, aggiornando classifier, pooling, transformer, attenzioni, proiezioni lineari, embedding e infine gli input originali.

Checkpoint dimensioni finale — Backward Pass completo

Ecco lo stato *completo* di tutti i gradienti calcolati durante il backward pass per ImageNet — mirror perfetto del checkpoint finale del forward: ogni tensore del forward ha qui il suo gradiente corrispondente, con la stessa forma.

Gradiente	Forma simbolica	ImageNet	Significato
<i>Output head backward (loss → classifier → pooling)</i>			
$\partial\mathcal{L}/\partial z$	K_{cls}	1000	$= p - y$ (formula pulita)
$\partial\mathcal{L}/\partial W_c$	$D \times K_{\text{cls}}$	1024×1000	pesi classificatore
$\partial\mathcal{L}/\partial b_c$	K_{cls}	1000	bias classificatore
$\partial\mathcal{L}/\partial h$	$\mathbb{R}^D$	$\mathbb{R}^{1024}$	input del classifier
$\partial\mathcal{L}/\partial L_i^{(T)}$	$\mathbb{R}^D$ (per ogni i)	$\mathbb{R}^{1024}$	broadcast: $\frac{1}{N} \partial\mathcal{L}/\partial h$
<i>Latent Transformer backward (MLP + self-attention, ripetuto $B = 6$)</i>			
$\partial\mathcal{L}/\partial W_1^{(LT)}$	$D \times 4D$	1024×4096	MLP expansion
$\partial\mathcal{L}/\partial W_2^{(LT)}$	$4D \times D$	4096×1024	MLP compression
$\partial\mathcal{L}/\partial \gamma^{(LT)}, \partial\mathcal{L}/\partial \beta^{(LT)}$	D ciascuno	1024	LayerNorm params
$\partial\mathcal{L}/\partial W_Q^{(LT)}, W_K^{(LT)}, W_V^{(LT)}$	$D \times D$ ciascuno	1024×1024	self-attn proiezioni
$\partial\mathcal{L}/\partial W_o^{(LT)}, b_o^{(LT)}$	$D \times D, D$	$1024 \times 1024, 1024$	output projection
$\partial\mathcal{L}/\partial L$ (output LT)	$N \times D$	512×1024	propagato al blocco precedente
<i>Cross-Attention backward</i>			
$\partial\mathcal{L}/\partial Z'$	$N \times D$	512×1024	dalla residual
$\partial\mathcal{L}/\partial W_o$	$d_{QKV} \times D$	261×1024	output projection
$\partial\mathcal{L}/\partial b_o$	D	1024	bias output proj
$\partial\mathcal{L}/\partial F$	$N \times d_{QKV}$	512×261	pre- W_o (aggregazione)
$\partial\mathcal{L}/\partial A$	$N \times M$	$512 \times 50\,176$	pesi di attenzione
$\partial\mathcal{L}/\partial V$	$M \times d_{QKV}$	$50\,176 \times 261$	values
$\partial\mathcal{L}/\partial S$	$N \times M$	$512 \times 50\,176$	scaled scores (via softmax)
$\partial\mathcal{L}/\partial Q$	$N \times d_{QKV}$	512×261	queries
$\partial\mathcal{L}/\partial K$	$M \times d_{QKV}$	$50\,176 \times 261$	keys
$\partial\mathcal{L}/\partial W_Q$	$D \times d_{QKV}$	1024×261	proiezione Q
$\partial\mathcal{L}/\partial W_K$	$C_{\text{tot}} \times d_{QKV}$	261×261	proiezione K
$\partial\mathcal{L}/\partial W_V$	$C_{\text{tot}} \times d_{QKV}$	261×261	proiezione V
$\partial\mathcal{L}/\partial \gamma_{\text{cross}}, \partial\mathcal{L}/\partial \beta_{\text{cross}}$	D ciascuno	1024	LayerNorm cross
<i>Propagazione ai tensori iniziali</i>			
$\partial\mathcal{L}/\partial L^{(0)}$	$N \times D$	512×1024	latent array appreso
$\partial\mathcal{L}/\partial X$	$M \times C_{\text{tot}}$	$50\,176 \times 261$	byte array (Fourier PE congelati)

Regola di simmetria. Se un tensore $T \in \mathbb{R}^{a \times b}$ esiste nel forward, allora nel backward $\partial\mathcal{L}/\partial T \in \mathbb{R}^{a \times b}$: forma identica. Vale per tutti i tensori — input, intermedi, output — e anche per i parametri.

Nota sul weight sharing (§1.4.12). Per i parametri condivisi tra le iterazioni $t = 2, \dots, T = 8$ (tutti tranne il primo blocco), i gradienti di ogni iterazione si *accumulano* sul medesimo buffer `.grad` di PyTorch. Il primo blocco ha invece parametri propri aggiornati da un solo contributo.

Checkpoint gradienti FINALE — tutti i parametri aggiornati

Al termine del backward pass abbiamo calcolato il gradiente rispetto a *ogni* parametro addestrabile del Perceiver. L'ottimizzatore (SGD/Adam/LAMB) userà questi gradienti per aggiornare i pesi.

Parametro	Forma simbolica	ImageNet	Blocco di origine
<i>Input embedding (opzionale nel paper)</i>			
W_E	$C_{\text{raw}} \times D$	variabile	§1.4.3 (se usato)
b_E	D	1024	§1.4.3
<i>Latent array (parametro appreso)</i>			
$L^{(0)}$	$N \times D$	512×1024	§1.4.4
<i>Cross-Attention (ogni blocco; condivisi tra iter. 2..T)</i>			
W_Q	$D \times d_{QKV}$	1024×261	§1.4.5
W_K	$C_{\text{tot}} \times d_{QKV}$	261×261	§1.4.5
W_V	$C_{\text{tot}} \times d_{QKV}$	261×261	§1.4.5
W_o	$d_{QKV} \times D$	261×1024	§1.4.5 output proj
b_o	D	1024	§1.4.5
$\gamma_{\text{cross}}, \beta_{\text{cross}}$	D ciascuno	1024	LayerNorm cross
<i>MLP cross-attention (ogni blocco; shared)</i>			
W_1	$D \times 4D$	1024×4096	§1.4.9
b_1	$4D$	4096	§1.4.9
W_2	$4D \times D$	4096×1024	§1.4.9
b_2	D	1024	§1.4.9
<i>Latent Transformer (shared; ripetuto $\ell = 6$ volte per blocco)</i>			
$W_Q^{(LT)}, W_K^{(LT)}, W_V^{(LT)}$	$D \times D$ ciascuno	1024×1024	§1.4.10
$W_o^{(LT)}, b_o^{(LT)}$	$D \times D, D$	$1024 \times 1024, 1024$	§1.4.10
$\gamma_{\text{LT}}, \beta_{\text{LT}}$	D ciascuno	1024	LayerNorm LT
$W_1^{(LT)}, b_1^{(LT)}, W_2^{(LT)}, b_2^{(LT)}$	come MLP cross-att	idem	MLP LT
<i>Classifier (output head)</i>			
W_c	$D \times K_{\text{cls}}$	1024×1000	§1.4.12
b_c	K_{cls}	1000	§1.4.12

Effetto del weight sharing. Nelle iterazioni $t = 2, \dots, T$, i parametri di cross-attention e latent transformer sono gli stessi fisici (stesso `nn.Parameter`), quindi i gradienti di ogni iterazione si *accumulano* sul medesimo accumulatore `.grad` di PyTorch (cfr. §1.4.12). Il primo blocco CA_1, LT_1 ha invece parametri propri che ricevono un contributo gradiente da una sola iterazione.

Totale parametri aggiornati: $\sim 44.9\text{M}$ per il Perceiver ImageNet (Tab. 7 del paper), dopo il weight sharing. Tutti i gradienti sopra servono per farlo andare nell’ottimizzatore LAMB.

Step	Forward	Backward
------	---------	----------

Schema Riassuntivo Forward/Backward

Step	Forward	Backward
1. Input grezzo	$x \in \mathbb{R}^{M \times C_{\text{raw}}}$	—
2. Input Embedding	$X = xW_E + b_E$, $W_E \in \mathbb{R}^{C_{\text{raw}} \times D}$, $X \in \mathbb{R}^{M \times D}$	$\frac{\partial \mathcal{L}}{\partial W_E} = x^\top \frac{\partial \mathcal{L}}{\partial X}$, $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial X} W_E^\top$
3. Cross-Attention (Q da latenti, K/V da input)	$Q = LW_Q$, $K = XW_K$, $V = XW_V$ $A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$, $Z = AV \in \mathbb{R}^{N \times d_v}$ Output: $Z' = ZW_o + b_o$; residual: $L \leftarrow L + Z'$	Residual: gradiente copiato su L e Z' $\rightarrow \frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial V}$ $\rightarrow \frac{\partial \mathcal{L}}{\partial W_Q}, \frac{\partial \mathcal{L}}{\partial W_K}, \frac{\partial \mathcal{L}}{\partial W_V}$
4. Latent Transformer Block (un blocco, composto da $\ell = 6$ self-attention layer)	LayerNorm $\rightarrow \tilde{L} = \text{LN}(L)$ Self-Attention: Q, K, V <i>tutti</i> dai latenti $\tilde{L} \in \mathbb{R}^{N \times D}$ Residual: $L' = L + Z'$ LayerNorm $\rightarrow \tilde{L}' = \text{LN}(L')$ MLP: $F = \sigma(\tilde{L}'W_1 + b_1)W_2 + b_2$ Residual: $L'' = L' + F$	Stesse formule del backward cross-attention, ma con Q, K, V tutte dai latenti (niente $\frac{\partial \mathcal{L}}{\partial X}$: solo $\frac{\partial \mathcal{L}}{\partial L}$). Ripetuto $\ell = 6$ volte per blocco.
3+4. Iterazione ($\times T = 8$ volte)	L'intera sequenza Cross-Att + Latent Transformer viene applicata T volte con weight sharing (tranne il primo blocco)	Gradienti dei pesi condivisi <i>sommati</i> su $t = 2, \dots, T$ (§1.4.12)
5. Pooling	$h = \frac{1}{N} \sum_{i=1}^N L_i^{(\text{finale})} \in \mathbb{R}^D$	$\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h}$
6. Classification + Softmax	$z = hW_c + b_c$, $p = \text{softmax}(z)$	$\frac{\partial \mathcal{L}}{\partial z} = p - y$, $\frac{\partial \mathcal{L}}{\partial W_c} = h^\top (p - y)$, $\frac{\partial \mathcal{L}}{\partial h} = (p - y)W_c^\top$
7. Loss	$\mathcal{L} = -\sum_k y_k \ln p_k$	Punto di partenza del backward

Il processo di addestramento di un Perceiver

L'addestramento di una rete neurale come il **Perceiver** si basa sull'iterazione di tre fasi fondamentali: **forward pass**, **backward pass** e **aggiornamento dei pesi**.

Tali operazioni vengono eseguite su sottoinsiemi del dataset (batch) e ripetute per più cicli completi (epoche).

Dataset È l'insieme dei dati su cui addestriamo il modello.

- Nel caso di ImageNet $\rightarrow$ circa 1.200.000 immagini etichettate in 1000 classi.
- Formalmente:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \quad |D| = N$$

Con x_i : immagine i-esima, y_i : classe associata i-esima

Batch Invece di prendere tutte le N immagini insieme, le dividiamo in piccoli gruppi (batch).

- Esempio: batch size $m = 256$.
- Ogni batch B contiene m coppie (x_i, y_i) .
- Il modello fa forward + backward + aggiornamento pesi su ciascun batch.
- **Epoca**

Una epoca è un giro completo sul dataset.

Numero di batch per epoca =

$$\frac{N}{m}$$

Se $N = 1.200.000, m = 256 \rightarrow$ circa 4687 batch per epoca.

- Dopo un'epoca il modello ha visto tutti i dati una volta.

Fine del Forward Pass

Per ogni batch (es. 256 immagini):

1. Ogni immagine passa attraverso: embedding $\rightarrow$ cross-attention $\rightarrow$ latent transformer $\rightarrow$ pooling $\rightarrow$ classificatore
2. Il modello produce un vettore di probabilità (softmax) per ognuna delle 256 immagini.

$$p_i \in \mathbb{R}^C, \quad C = 1000 \text{ classi di ImageNet}$$

3. La loss (cross-entropy) viene calcolata come media sul batch:

$$\mathcal{L}(B) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C y_{i,c}^{true} \log(p_{i,c})$$

La doppia sommatoria è presente perché stiamo calcolando la Loss sulle m immagini presenti nel batch.

Output del forward: Predizioni e Loss numerica $\mathcal{L}$

Fine del Backward Pass

Dal valore della loss:

- Si calcolano i gradienti di tutti i parametri W del modello.
- Per ogni peso otteniamo:

$$g_W = \nabla_W \mathcal{L}(B)$$

Output del backward:

- Gradienti per ogni layer (embedding, cross-attention, latent transformer, classificatore).

Ma i pesi non sono ancora stati modificati.

1.6 Dettagli di Training (ImageNet)

Il Perceiver è costruito come una sequenza ricorsiva di:

$$\text{Cross-Att} \rightarrow \text{Latent Transformer} \rightarrow \text{Cross-Att} \rightarrow \dots$$

Weight sharing (come descritto nella Sezione 1.3.6):

- **Cross-Attention:** CA_1 ha pesi *unici*; $CA_2, \dots, CA_T$ condividono tutti gli stessi pesi.
- **Latent Transformer:** tutti i blocchi $LT_1, \dots, LT_T$ condividono gli stessi pesi.

Questa scelta riduce il numero di parametri da $\sim 326.2M$ a $\sim 44.9M$ (Tab. 7 del paper) e migliora le prestazioni (da 72.9% a 78.0% su ImageNet), agendo come regolarizzatore. L'architettura diventa formalmente un RNN con core ricorrente (latent transformer) e proiezioni di input ricorrenti (cross-attention).

Configurazione di training su ImageNet (Jaegle et al., 2021, §4.1):

- **Ottimizzatore:** LAMB (You et al., 2020). Il paper riporta che il Perceiver è più facile da ottimizzare con LAMB rispetto ad Adam.
- **Epoche:** 120.
- **Learning rate:** iniziale 4×10^{-3} , con *step decay* di fattore 10 alle epoche [84, 102, 114]. Non cosine, non warmup: è il classico step schedule.
- **Regolarizzazione: no dropout** (il paper lo ha provato in versioni preliminari e ha osservato *peggioramento* delle performance — cfr. App. C). Weight decay, gradient clipping. Data augmentation: RandAugment (Cubuk et al., 2020).
- **Architettura:** $T = 8$ blocchi (cross-att + latent transformer), ciascun latent transformer con $L = 6$ moduli di self-attention. Pesi condivisi tra tutti i blocchi *tranne il primo* (la prima cross-attention e il primo latent transformer hanno pesi propri: dividerli con i successivi porta a instabilità in training).
- **Parametri totali:** $\sim 45M$ — paragonabile a modelli convoluzionali usati su ImageNet.

Perché no dropout?

Il paper (App. C): “*We used dropout throughout the network in earlier experiments, but we found this led to degraded performance, so no dropout is used*”. Intuizione: il Perceiver ha già forti pressioni regolarizzanti (bottleneck $N \ll M$, weight sharing che riduce i parametri di $\sim 7\times$, weight decay, data augmentation aggressive). Aggiungere dropout sopra tutto questo soffoca la capacità del modello invece che aiutarlo.

1.7 Risultati Sperimentali del Paper Originale

Questa sezione raccoglie i risultati sperimentali del Perceiver (ImageNet, AudioSet, ModelNet40) e del Perceiver IO (GLUE, optical flow, multimodale autoencoding). Per ciascun task sono riportati il baseline domain-specific di riferimento (CNN, Transformer, RAFT, ...) e il risultato dell'architettura, evidenziando il trade-off tra generalità e accuratezza. I numeri chiave sono riassunti nella sottosezione finale *Numeri chiave per l'esame*, utile come riferimento rapido in sede di discussione orale.

1. Perceiver Originale (Jaegle et al., 2021)

Classificazione ImageNet (ILSVRC 2012, raw)

I valori sono top-1 accuracy sul validation set (Tab. 1 del paper).

Modello	Top-1 (val)	Note
ResNet-50	77.6%	Conv 2D; riportato con RandAugment (Cubuk et al., 2020)
ResNet-50 (FF)	73.5%	ResNet con Fourier features come input
ViT-B/16	77.9%	Patch 2D
ViT-B/16 (FF)	76.7%	ViT con Fourier features come input
Transformer (64×64, FF)	57.0%	Transformer puro, input downsampled a 64×64
Perceiver (FF)	78.0%	Senza conv 2D, 50 176 pixel grezzi

Perceiver su ImageNet raw

Il Perceiver raggiunge 78.0% lavorando direttamente su 50 176 pixel, senza convoluzioni 2D. È competitivo con ResNet-50 (77.6%) e ViT-B/16 (77.9%), entrambi architetture con forte prior 2D. Il Transformer puro fallisce (57.0%) perché, pur usando Fourier features, la complessità $O(M^2)$ lo costringe a lavorare su input downsampled a $64 \times 64 = 4096$ pixel, perdendo informazione fine.

Permuted ImageNet (test di permutation invariance)

Il paper valuta i modelli anche su una versione “permuted” di ImageNet: i pixel di *tutte* le immagini vengono mescolati secondo la *stessa* permutazione fissa (Tab. 2). I modelli che codificano la posizione nel byte array tramite Fourier features dovrebbero essere robusti: la posizione viaggia nell'input, non nell'ordine degli indici.

Modello	Raw	Perm.	Input RF (px)
ResNet-50 (FF)	73.5	39.4	49
ViT-B/16 (FF)	76.7	61.7	256
Transformer (64×64, FF)	57.0	57.0	4096
Perceiver (FF)	78.0	78.0	50 176
Perceiver (Learned pos.)	78.0	70.9	50 176

Argomento chiave del paper: permutation invariance

Il Perceiver (FF) è **invariante alla permutazione**: $78.0 \rightarrow 78.0$, zero calo. ResNet-50 (FF) crolla da 73.5 a 39.4: la conv 2D assume che i vicini nella griglia siano correlati, e la permutazione distrugge questa ipotesi. ViT regge meglio ($76.7 \rightarrow 61.7$) perché lavora su patch: meno vincolato alla locale 2D di una conv, ma non del tutto libero. Il Perceiver invece *non ha* alcun prior 2D: la posizione vive nelle Fourier features concatenate ai pixel, e la permutazione dei pixel permuta anche le FF in modo coerente.

Confronto importante: il Perceiver con *learned position encoding* passa da 78.0 a 70.9 sotto permutazione. Le posizioni apprese sono “etichette” legate alle posizioni del training set, e la permutazione le scombina. Questo dimostra che è proprio la scelta delle **Fourier features** a dare la permutation invariance, non l’architettura del Perceiver in sé.

In sede d’esame: questo è *il* risultato che il paper usa per argomentare la generalità del Perceiver — “non stiamo imbrogliando sfruttando un bias 2D nascosto”.

Ablazione del Weight Sharing (Tab. 7 del paper)

Configurazione	Top-1 val
Senza weight sharing ($\sim 326.2\text{M}$ param.)	72.9%
Con weight sharing ($\sim 44.9\text{M}$ param.)	78.0%

Weight sharing come regolarizzatore

Il weight sharing riduce i parametri di $\sim 7\times$ (da 326.2M a 44.9M) e contemporaneamente **migliora** le prestazioni di +5.1% ($72.9 \rightarrow 78.0$). Il modello senza sharing overfitta: con così tanti parametri liberi, la rete memorizza pattern specifici del training set invece di generalizzare. Condividendo i pesi tra iterazioni si impone al modello di usare lo stesso “modulo di raffinamento” in tutti i passi, trasformando la rete da stack di layer indipendenti a RNN con core ricorrente.

AudioSet (classificazione audio-video)

Modello	Audio	Video	A+V
CNN-14	43.1	—	—
Perceiver (audio grezzo)	38.3	25.8	43.5
Perceiver (mel + tuned)	—	—	44.2

ModelNet-40 (Point Cloud)

Il Perceiver ottiene **85.7%** di accuracy su ModelNet-40 rispetto a **91.9%** di PointNet++. Il Perceiver utilizza solo le coordinate (x, y, z) senza feature geometriche specializzate, il che spiega il gap: dimostra comunque la generalità dell’architettura su input non-strutturati.

2. Perceiver IO (Jaegle et al., 2022)

GLUE benchmark (NLP) — Tab. 1 del paper. L’esperimento chiave è confrontare Perceiver IO contro BERT a *parità di FLOPs*, con due tokenizzazioni diverse (SentencePiece vs byte UTF-8 grezzi).

Modello	Tokenizer	M	N	Depth	Avg GLUE
BERT Base (test, Devlin et al.)	SentencePiece	512	–	12	81.0
BERT Base (ours)	SentencePiece	512	–	12	81.1
Perceiver IO Base	SentencePiece	512	256	26	81.2
BERT (matching FLOPs)	UTF-8 bytes	2048	–	6	71.5
Perceiver IO	UTF-8 bytes	2048	256	26	81.0
Perceiver IO++	UTF-8 bytes	2048	256	40	81.8

Perceiver IO su GLUE: due risultati, uno spesso confuso

I numeri **81.0/81.2** riguardano il *confronto diretto a parità di FLOPs*: a budget FLOPs fisso, il Perceiver IO Base (con SentencePiece) batte leggermente BERT Base (81.2 vs 81.1); con byte UTF-8, Perceiver IO raggiunge BERT SentencePiece (81.0), mentre BERT UTF-8 (costretto a ridurre drasticamente capacità per pareggiare i FLOPs) crolla a 71.5.

Il numero **81.8** viene invece da *Perceiver IO++*: una variante *più grande* (40 layer vs 26, 425M parametri vs 201M, ~240B FLOPs vs 113B) che supera BERT Base mantenendo l’assenza di tokenizzatore. Oppure dallo stesso score che si ottiene con la configurazione *multitask* (§ seguente).

Osservazione importante: *senza tokenizzatore*, Perceiver IO gestisce sequenze **4× più lunghe** (2048 byte $\approx$ 512 SentencePiece token) grazie alla scalabilità lineare del decoder nei confronti della lunghezza di input.

Multitask Perceiver IO (Tab. 2 del paper). Un risultato sperimentale chiave: Perceiver IO può fare **fine-tuning simultaneo su tutti gli 8 task GLUE** con una singola architettura, usando una *query per task* (8 output queries, una con task_id embedding per ciascuna delle 8 task). Gli appunti classici hanno un [CLS] token per task, qui si sposta il meccanismo nella costruzione delle query.

Configurazione	Multi-task	Avg GLUE
Single-task query (baseline UTF-8)	No	81.0
Shared input token (stile [CLS])	Sì	81.5
Task-specific input tokens	Sì	81.8
Multitask query (8 output queries)	Sì	81.8

Il **multitask query** approach è il più elegante: disaccoppia l’array di output dall’array di input, senza bisogno di token [CLS]. Particolarmente utile quando i task sono numerosi o eterogenei.

Optical Flow (Sintel / KITTI) — Tab. 3 del paper. Task: stimare il displacement 2D per ogni pixel tra due frame consecutivi.

Approccio Perceiver IO: le due frame sono concatenate lungo la *channel dimension* (3+3=6 canali), poi per ogni pixel si estrae un **patch** 3×3 di contesto spaziale ($3 \times 3 \times 3 \times 2 = 54$ valori per pixel). Si concatena un positional encoding Fourier (64 bande per asse, 258 extra feature). Per il

decoder, la query di ciascun pixel *include la feature di input* in quel punto (non solo la posizione), così il flusso ottico è costruito "aggiungendo" un correttivo all'input.

Rete	Sintel.clean	Sintel.final	KITTI
PWC-Net (Sun et al., 2018)	2.17	2.91	5.76
RAFT (Teed & Deng, 2020)	1.95	2.57	4.23
Perceiver IO	1.81	2.42	4.98

Metrica: AEPE (Average End-Point Error, più basso è meglio). Perceiver IO ottiene lo **stato dell'arte su Sintel.final** (2.42) al momento della pubblicazione, *senza cost volumes, senza warping esplicito, senza gerarchia multiscala* — tutti meccanismi domain-specific che i metodi classici come RAFT usano. Su KITTI RAFT resta il migliore (4.23 vs 4.98), mostrando che l'assenza di inductive bias 2D ha un costo in certi scenari.

Multimodal Autoencoding su Kinetics-700 (Tab. 4 del paper). Task: ricostruire simultaneamente *video + audio + class label* da un input bottlenecked, allenando un singolo modello.

Approccio:

- Input: 16 frame di video a 224×224 pre-processati in **50k patch** $1 \times 4 \times 4$ (16 pixel/patch) + **1920 audio sample** (16 sample/patch; audio campionato a 48 kHz) + 1 class label one-hot 700-d.
- Ogni modalità è paddata con un *modality embedding* per avere feature dim uniforme (704 dim totali).
- Decoder queries: Fourier PE per video (387-dim) e audio (385-dim) + embedding learnabile per la label (1024-dim), tutti paddati a dim 1026 per query.
- Loss: somma pesata di L1 video + L1 audio + cross-entropy label.

Compress. ratio	Audio PSNR	Video PSNR	Top-1 Accuracy
$88 \times$ (784 latent)	26.97	24.37	10.2%
$176 \times$ (392 latent)	25.33	24.27	8.6%
$352 \times$ (196 latent)	14.15	23.21	11.5%

Trade-off ricostruzione vs classificazione

Con pesi della loss bilanciati per privilegiare video/audio, il modello al $88 \times$ raggiunge PSNR ragionevoli ma solo 10.2% di accuracy. Aumentando il peso sulla class loss il paper ottiene **45% top-1 accuracy** mantenendo 20.7 PSNR video: dimostra che *le stesse latenti possono codificare* sia il contenuto ricostruibile sia la label semantica.

ImageNet (Tab. 7 del paper).

Modello	Pretrain.	Accuracy	Note
ResNet-50 (He et al., 2016)	No	76.9%	Conv 2D
ViT-B/16 (Dosovitskiy et al., 2021)	No	77.9%	Patch 2D

Modello	Pretrain.	Accuracy	Note
Perceiver IO (Fourier 2D)	No	79.0%	Senza conv 2D
Perceiver IO (conv pre-proc.)	No	82.1%	Con conv 7×7 + max pool (down-sample a 56 × 56)
Perceiver IO (learned pos., JFT pretrain)	Sì (JFT)	84.5%	Senza conv 2D , pretraining su JFT

Nota su “86.4%” di ImageNet IO

Alcune fonti (anche versioni precedenti di questi appunti) riportano per errore un numero tipo **86.4%** per Perceiver IO su ImageNet. Il paper ufficiale (Jaegle et al., 2022, §4.4 e App. A) riporta **84.5%** top-1 con pretraining JFT e **senza convoluzioni 2D**. Inoltre il paper dice letteralmente: “*Perceiver IO surpasses 80% top-1 accuracy (84.5% top-1) without using 2D convolutions after pretraining on JFT*”.

StarCraft II (Tab. 9 del paper). Perceiver IO come *drop-in replacement* del Transformer che funge da **entity encoder** nell’agente **AlphaStar** (Vinyals et al., 2019, SOTA per StarCraft II).

Entity encoder	Win rate	Params	FLOPs
Transformer (AlphaStar originale)	0.87	144M	3.3B
Perceiver IO	0.87	140M	0.93B

Risultato: stesso win rate (87% contro Elite bot) con $\sim 3.5\times$ **meno FLOPs**, usando un Perceiver IO a 3 layer con latent index dim 32. Il paper: “*Perceiver IO works out of the box*” — nessun tuning particolare oltre al sweep del latent index dim tra 32 e 64.

AudioSet classification (Tab. 10 del paper). Lo stesso benchmark usato nel Perceiver originale, ma con il decoder di tipo Perceiver IO (attention-based) invece del pooling average+project.

Modello	Input	mAP
Perceiver (orig.)	Raw audio + video	42.4
Perceiver IO	Raw audio + video	43.3
Perceiver (orig.)	Mel-spectr. + video	43.6
Perceiver IO	Mel-spectr. + video	44.9

Risultato: il decoder attentional di IO migliora consistentemente il decoder pooling del Perceiver originale, senza altre modifiche (*evidenza che IO è strettamente più generale del Perceiver*).

3. Numeri chiave per l'esame

Numeri chiave da memorizzare

- **ImageNet Perceiver:** 78.0% (Fourier 2D, senza conv, pixel grezzi). **Perceiver IO:** 79.0% senza pretraining + no conv; 84.5% **con pretraining JFT, senza conv 2D** (non 86.4 come spesso riportato).
- **GLUE (Perceiver IO base UTF-8):** 81.0 (a parità di FLOPs con BERT Base SentencePiece); **Perceiver IO++ UTF-8:** 81.8; **Perceiver IO multitask query:** 81.8 (fine-tuning simultaneo su tutti gli 8 task GLUE).
- **Sintel.final:** 2.42 AEPE (SOTA alla pubblicazione, senza cost volumes né gerarchia multiscala).
- **Sintel.clean:** 1.81 AEPE (Perceiver IO meglio di RAFT 1.95).
- **Multimodal Autoencoding Kinetics:** compressione $88\times$ con PSNR 24.37 video, 26.97 audio, 10.2% top-1; bilanciando meglio la loss: 45% top-1 + 20.7 PSNR video.
- **StarCraft II:** win rate 0.87 come Transformer originale, con $\sim 3.5\times$ meno FLOPs (0.93B vs 3.3B) e parametri comparabili (140M vs 144M).
- **AudioSet** (raw audio+video): Perceiver IO 43.3 mAP vs Perceiver 42.4 mAP.
- **Weight sharing (Perceiver originale):** $7\times$ meno parametri ($326.2\text{M} \rightarrow 44.9\text{M}$), +5.1% accuracy ($72.9 \rightarrow 78.0$, Tab. 7 paper 2021).

1.8 Ablation Studies

Questa sezione riassume i principali ablation studies del paper Perceiver (Jaegle et al., 2021, Appendice B e Figure 5–6). Gli esperimenti vengono condotti su ImageNet (classificazione, top-1 accuracy) e mostrano come ogni componente dell'architettura contribuisca alle prestazioni finali.

Attenzione: ablation su un modello ridotto, non sul best model

Per le ablation su N , D , L , T il paper (App. B) usa un *modello base* intenzionalmente più piccolo del migliore riportato in Tab. 1:

- senza weight sharing, 8 teste per self-attention, 4 self-attention per blocco, 2 cross-attends, $N = 512$, $D = 512$, batch 64 su 32 TPU.

Le accuracy delle ablation stanno quindi tra $\sim 60\%$ e $\sim 76\%$ — **non arrivano al 78.0%** del main model (che usa $D = 1024$, $T = 8$, $L = 6$, weight sharing). I trend qualitativi (“aumentando N migliora”, ecc.) sono informativi; i valori numerici NON sono direttamente confrontabili con i numeri di Tab. 1.

1. Effetto del numero di latenti N (Fig. 5, top-left)

Aumentare N migliora l'accuracy in modo monotono nell'intervallo testato: più latenti = più capacità di “memoria” dello spazio d'input. Il paper testa $N \in \{256, 512, 1024\}$.

Latenti N	Accuracy prox. da 5)	(ap- prox. da Fig.	Nota
256	~67.5%		Capacità di bottleneck ridotta
512	~72.5%		Default del base model
1024	~76%		Migliore tra quelli testati in ablation

Il main model usa $N = 512$ come compromesso (più grande gonfia memoria e tempo di training per un guadagno modesto).

2. Effetto della dimensione del canale D (Fig. 5, top-center)

Aumentare D (canali del latente) migliora le prestazioni **fino a un punto**: oltre, si osserva overfitting, specialmente quando il modello non condivide pesi. Dal paper: “*Increasing the size of the latent channel dimension helps up to a point, but often leads to overfitting*”.

Canale D	Accuracy prox. da 5)	(ap- prox. da Fig.	Nota
256	~62.5%		Modello sottodimensionato
512	~72.5%		Default del base model
1024	~72.5%		Nessun guadagno sul base model (senza weight sharing)

Nel main model $D = 1024$ paga perché accompagnato da weight sharing (che contrasta l'overfitting).

3. Effetto del numero di self-attention per blocco (L , Fig. 5, bottom-left)

Più self-attention nel latent transformer per ogni blocco T significa più “ragionamento” nello spazio compresso prima di rileggere l'input. Il paper testa $L \in \{2, 4, 8\}$.

Self-att. per blocco L	Accuracy prox. da 5)	(ap- prox. da Fig.	Nota
2	~62.5%		Poca elaborazione latente
4	~70%		Default del base model
8	~72.5%		Guadagno marginale oltre $L = 4$

Il main model usa $L = 6$ come compromesso tra capacità e costo computazionale.

4. Effetto del numero di cross-attention (iterazioni T)

Questo è il risultato chiave dell'Appendice B (Tabella 6 del paper). Si confrontano due strategie per usare più cross-attention:

- **Interleaved** (alternata): cross-attend $\rightarrow$ self-attend $\rightarrow$ cross-attend $\rightarrow \dots$. I pesi delle cross-attend sono condivisi tra le iterazioni.
- **At start** (tutte all'inizio): tutte le cross-attend vengono applicate in sequenza prima del processo latente.

Configurazione	Cross-attends T	Accuracy	Nota
Interleaved	1	76.7%	baseline
Interleaved	2	76.5%	simile
Interleaved	4	76.5%	simile
Interleaved	8	78.0%	migliore
At start	8	73.7%	molto peggio

Interpretazione: perché interleaved batte “at start”

Quando si usano 8 cross-attend tutte all'inizio (*at start*), il modello deve comprimere l'intera informazione dell'input in un'unica lettura prolungata, senza beneficiare del processo latente intermedio. Al contrario, con la strategia *interleaved*, il modello alterna fasi di “lettura” dell'input e fasi di “ragionamento” latente: ogni re-lettura avviene con latenti già parzialmente raffinati, permettendo di estrarre informazioni più pertinenti. Il modello **ri-legge l'input più volte in modo adattivo**.

5. Cross-attention senza latent transformer (Tabella 5)

Esperimento ablativo estremo: si rimuove completamente il latent transformer (self-attention sui latenti) e si usano solo cross-attention ripetute. Questo elimina il bottleneck latente.

Cross-attends (senza self-att.)	Accuracy	Nota
4	39.4%	Molto scarso
8	45.3%	In miglioramento, ma lento
12	OOM	Out of memory
8 (con self-att., modello completo)	78.0%	+32.7% rispetto a senza

Il latent bottleneck è essenziale

Senza il latent transformer, il modello collassa: la mancanza di comunicazione tra i latenti dopo ogni cross-attention impedisce l'integrazione delle informazioni. Aggiungere più cross-attend non basta (la complessità cresce, fino all'OOM), e l'accuracy rimane drammaticamente bassa ($\leq 45\%$). Il bottleneck latente non serve solo a ridurre i costi computazionali: è il meccanismo che consente l'integrazione globale dell'informazione.

6. Effetto del weight sharing (Tabella 7)

Il weight sharing tra le iterazioni di cross-attend (e dei blocchi latenti) agisce come un potente regolarizzatore implicito.

Configurazione	Val (%)	Train (%)	Gap
Senza weight sharing (326.2M par.)	72.9%	87.7%	-14.8%
Con weight sharing (44.9M par.)	78.0%	79.5%	-1.5%

Weight sharing come regolarizzatore

Senza weight sharing, il modello ha ~326.2M di parametri e mostra un forte overfitting (87.7% train vs. 72.9% validation, gap di 14.8%). Con weight sharing, i parametri scendono a ~44.9M ($\approx 7\times$ meno) e l'accuracy *validation migliora* di +5.1%, mentre quella di training scende di 8.2%. Il weight sharing forza il modello a imparare trasformazioni generalizzabili, indipendentemente dall'iterazione in cui vengono applicate.

7. Fourier features: numero di bande e risoluzione

Le features posizionali di Fourier introdotte nel Perceiver (cfr. Sezione 1.4.2) hanno anch'esse un impatto significativo sulle prestazioni.

- **Numero di bande di frequenza K :** più bande catturano dettagli spaziali più fini. I risultati migliorano aumentando K fino al valore di Nyquist (determinato dalla risoluzione dell'immagine). Oltre la frequenza di Nyquist non si ottiene alcun beneficio.
- **Risoluzione massima:** usare una frequenza massima pari alla frequenza di Nyquist dell'immagine ($f_{\max} = W/2$ per un'immagine di larghezza W) è ottimale. Valori inferiori perdono informazioni spaziali fini; valori superiori sono ridondanti.
- **Scala di inizializzazione:** un fattore di scala 1.0 per le proiezioni lineari produce risultati migliori rispetto a valori molto diversi. Questo è coerente con l'inizializzazione Xavier usata nel resto del modello.

Regola pratica per le Fourier features

$$K \geq \left\lfloor \frac{\min(H, W)}{2} \right\rfloor, \quad f_{\max} = \frac{\min(H, W)}{2}$$

dove H e W sono altezza e larghezza dell'immagine. Per ImageNet (224×224), si usa $K = 64$ e $f_{\max} = 112$ Hz, corrispondente alla frequenza di Nyquist. La dimensione totale delle features posizionali è $C_{\text{pos}} = 2 \cdot K \cdot d_{\text{spazio}} + d_{\text{spazio}}$ (seno, coseno per ogni asse, più la coordinata raw).

8. Lezioni dagli ablation studies

Lezioni dagli ablation studies (sintesi)

- **Il latent bottleneck è essenziale:** senza di esso si ottiene $\leq 45\%$ di accuracy (o OOM). Non è solo un risparmio computazionale, è il cuore del modello.
- **Interleaved cross-attention > at-start:** il modello deve ri-leggere l'input più volte, con latenti progressivamente raffinati. 8 iterazioni interleaved $\rightarrow 78.0\%$ vs. 8 at-start $\rightarrow 73.7\%$.
- **Weight sharing è un regolarizzatore potente:** riduce i parametri di $7\times$ e migliora la validation accuracy di $+5.1\%$, eliminando l'overfitting.
- **Più bande Fourier = meglio**, fino alla frequenza di Nyquist; oltre non si guadagna nulla.
- **Configurazione ottimale per ImageNet:** $N = 512$ latenti, $D = 1024$ canali, $T = 8$ cross-attend interleaved con weight sharing, $L = 6$ self-attention per blocco, $K = 64$ bande Fourier.

2 Perceiver IO

Cambio di notazione: Z al posto di L

Da questa sezione in poi il latent array è indicato con Z (e non L come nella §1), per allinearsi al paper Perceiver IO (Jaegle et al., 2022). La corrispondenza è diretta:

$$Z^{(0)} \equiv L, \quad Z^{(L)} = \text{latente dopo } L \text{ blocchi di Process.}$$

Tutto il resto della notazione resta coerente con il Perceiver originale (N latenti, D dimensione latente, M ingresso, ecc.).

2.1 Introduzione e Architettura

Mapa delle appendici del paper Perceiver IO (Jaegle et al., 2022)

- **App. A** — ImageNet: dettagli sperimentali, pre-processing convoluzionale opzionale (Tab. 7 del paper IO), risultato finale 84.5% top-1 senza conv 2D con pretraining JFT. Citata in §1.7.
- **App. B** — StarCraft II: Perceiver IO come drop-in replacement del Transformer dell’entity encoder di AlphaStar (Tab. 9). Citata in §1.7.
- **App. C** — AudioSet: configurazioni multimodale audio+video (Tab. 10), confronto con Perceiver originale.
- **App. D** — FLOPs calculation: convenzione per il conteggio dei FLOPs adottata nel paper.
- **App. E** — Architectural details: *sezione fondamentale per l’architettura di IO*. Contiene la Figure 5 con lo schema esplicito dei tre moduli (Encode, Process, Decode come attention QKV con input e output diversi), la Figure 6 con il confronto tra attention decoder (IO) e average+project decoder (Perceiver originale), e le formule (1)–(6) dell’attention module (LayerNorm pre-norm $\rightarrow$ QKV $\rightarrow$ residual $\rightarrow$ MLP $\rightarrow$ residual).
- **App. F** — Language: dettagli completi delle configurazioni language. Contiene la Tab. 11 con gli iperparametri di BERT, Perceiver IO Base, Perceiver IO (UTF-8) e IO++ (dimensioni N, D, E , numero di layer, ...), i dettagli del pretraining MLM (Tab. 12) e del fine-tuning GLUE (Tab. 13). Citata più volte in questa sezione.
- **App. G** — Positional encodings for image and audio: parametri delle Fourier features usate per ImageNet IO e per i task video/audio. **64 bande sin/cos per dimensione** in tutte le configurazioni. La frequenza minima è sempre la frequenza di una singola oscillazione completa sull’input; la massima è la **frequenza di Nyquist** (es. 112 cicli per 224×224). Input scalato in $[-1, 1]$. Usano 1D Fourier per audio e 3D Fourier per video (spaziotemporale).
- **App. H** — Optical Flow: dettagli dell’approccio (concatenazione frame lungo channel, patch $3 \times 3, 3 \times 3 \times 3 \times 2 = 54$ valori per pixel) + Tab. 16 con ablation su patch size, downsample, depth. Citata in §1.7.
- **App. I** — Multimodal autoencoding: patch video $1 \times 4 \times 4$, patch audio di 16 sample (48 kHz $\rightarrow$ 1920 sample/frame $\rightarrow$ 120 patches), embedding modality, loss pesata video+audio+label, Tab. 17. Citata in §1.7.

Perceiver IO estende il Perceiver aggiungendo un **decoder basato su cross-attention con output queries**, che gli permette di generare output di struttura e dimensione arbitraria (testo byte-level, flusso ottico denso, label multi-task) a partire dallo stesso latent array agnostico al task. È questa generalizzazione che rende IO applicabile al progetto di classificazione testuale GLUE e al pre-training MLM su WikiText discussi nel seguito. L’architettura segue lo schema **Encode** $\rightarrow$ **Process** $\rightarrow$ **Decode**: l’encoder comprime l’input nei latenti, il processor li raffina con self-attention profonda, il decoder li interroga con query costruite ad hoc per l’output richiesto. I latenti non contengono alcuna informazione sulla struttura dell’output: è la costruzione delle query che trasporta la semantica del task.

Nella Figura 19 si distinguono tre stadi consecutivi. (i) *Encode*: l’input array $X \in \mathbb{R}^{M \times C}$ viene letto da un array latente $Z \in \mathbb{R}^{N \times D}$ tramite una cross-attention con Query dai latenti e Key/Value dall’input, esattamente come nel Perceiver. (ii) *Process*: i latenti Z vengono raffinati da una pila di blocchi self-attention, che agiscono a costo $O(N^2)$ indipendente dalla dimensione M dell’input. (iii) *Decode*: un array di *output queries* $Q_{\text{out}} \in \mathbb{R}^{N_{\text{out}} \times D}$, con N_{out} arbitrario, interroga i latenti tramite cross-attention e produce l’output della forma richiesta. Le dimensioni M (input), N (latenti), N_{out} (output) sono indipendenti: lo stesso array latente può essere riusato per produrre output di natura e lunghezza completamente diverse semplicemente cambiando le query.

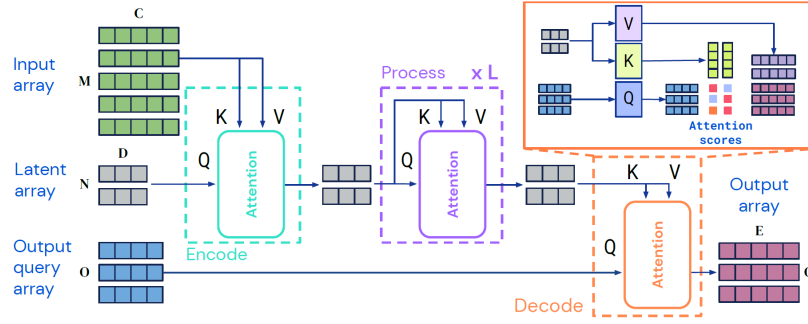


Figure 19: Architettura Perceiver IO: Encode, Process, Decode.

Riferimenti paper per questa architettura

La descrizione qui sopra corrisponde a quella fornita nell'**App. E del paper IO**, che è il punto di riferimento formale per i dettagli architetturali:

- La **Figure 5** dell'App. E mostra esplicitamente i tre moduli come tre attention QKV con input/output diversi (Encode: input array + latent array $\rightarrow$ latent; Process: latent + latent $\rightarrow$ latent; Decode: output queries + latent $\rightarrow$ output).
- La **Figure 6** confronta il “decoder attentional” del Perceiver IO (usato sia per classificazione che per output densi) con il “decoder avg+project” del Perceiver originale (pooling + linear), mostrando che il primo è strettamente più generale e più espressivo.
- Le **equazioni (1)–(6)** dell'App. E formalizzano il singolo modulo attention come “LayerNorm pre-norm $\rightarrow$ QKV-attention $\rightarrow$ residual con input della query $\rightarrow$ MLP con LayerNorm pre-norm $\rightarrow$ residual finale”. È lo schema adottato identicamente da Encode, Process e Decode, ciò che unifica l'architettura.

Differenza chiave: Perceiver vs Perceiver IO

- **Perceiver**: produce solo output semplici (classificazione) tramite *global average pooling* sui latenti. Non può generare output strutturati di dimensione arbitraria.
- **Perceiver IO**: produce output strutturati di dimensione arbitraria tramite un *decoder con cross-attention* che usa **output queries**. Ogni query interroga i latenti per ottenere un elemento dell'output (es. un token, un pixel, un'etichetta).

Questa è l'innovazione principale di Perceiver IO rispetto al Perceiver originale.

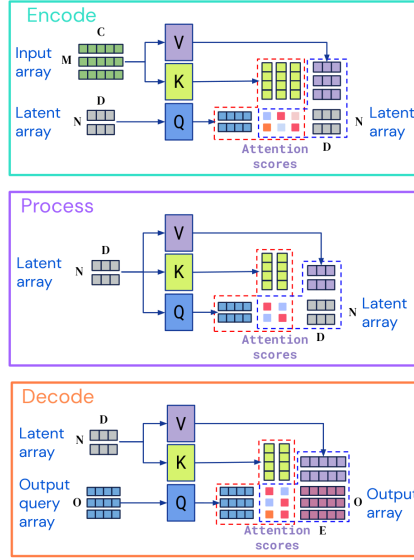


Figure 20: Schema esplicito dei tre moduli di Perceiver IO come blocchi di attenzione QKV con input e output diversi (Jaegle et al., 2022, App. E, Fig. 5). **Encode**: Q dai latenti, K, V dall’input array (M può essere enorme: 50K pixel, 10^5 byte). **Process**: Q, K, V tutti dai latenti — self-attention a costo $O(N^2)$ indipendente da M , ripetuta L volte. **Decode**: Q dalle output query (in numero O arbitrario), K, V dai latenti finali — produce un output array $O \times E$ di forma decisa dal task.

2.2 Input Array X

L’**input array** $X \in \mathbb{R}^{M \times C}$ è la prima rappresentazione del segnale grezzo che entrerà nell’Encode Module: fornisce le matrici *Key* e *Value* della cross-attention.

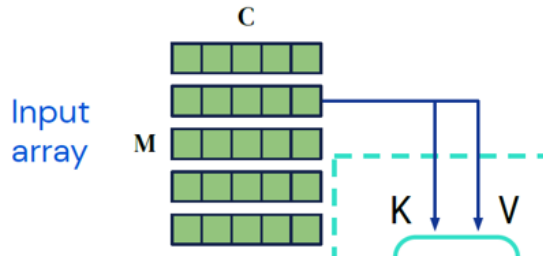


Figure 21: Input array ($M \times C$): i dati di ingresso generano le matrici Key (K) e Value (V) per il modulo Encode.

L’input array è una matrice:

$$X \in \mathbb{R}^{M \times C} \quad X = \{x_1, x_2, \dots, x_M\} \quad x_i \in \mathbb{R}^C$$

Esempi

Linguaggio - GLUE benchmark (senza tokenizzazione) Abbiamo un testo, di questo ogni carattere viene convertito in uno o più byte con valore intero tra 0 e 255.

Ogni x_i (cioè ogni byte) viene rappresentato concatenando due parti:

$$x_i = [f_i || p_i]$$

dove:

1. $f_i \in \mathbb{R}^{128} = E[b_i]$ embedding del byte

- Esistono 256 possibili byte (0-255).
- Si costruisce una matrice di embedding $E \in \mathbb{R}^{256 \times 128}$.
- Ogni riga di E è un vettore di 128 numeri che rappresenta un byte.
- Quando arriva un byte b_i , si prende la riga corrispondente di E , $f_i = E[b_i]$
- Risultato: un vettore da 128 numeri.

2. $p_i \in \mathbb{R}^{64} = \text{positional encoding Fourier}$

- Il modello deve sapere dove si trova il byte nella sequenza (inizio, metà, fine...).
- Per questo si usa un encoding sinusoidale multiscala (Fourier features).
- Ogni posizione $i \rightarrow$ vettore di 64 numeri.

Infine si fa la concatenazione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{128+64} = \mathbb{R}^{192}$$

Per esempio per la parola Ciao:

1. Conversione in byte UTF-8 Dato il testo, lo convertiamo in byte UTF-8:

$$\mathbf{b} = (b_0, b_1, \dots, b_{M-1}), \quad b_i \in \{0, \dots, 255\}$$

Per "Ciao": $M = 4$ e $(b_0, b_1, b_2, b_3) = (67, 105, 97, 111)$

$$X = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^{4 \times 192}$$

- Riga 0 = "C"
- Riga 1 = "i"
- Riga 2 = "a"
- Riga 3 = "o"

Carattere	Byte (decimale)
C	67
i	105
a	97
o	111
Quindi $M = 4$.	

2. Embedding del byte ($\mathbf{f}_i \in \mathbb{R}^{128}$)

Ogni byte viene usato come **indice del vettore corrispondente della matrice embedding** $\mathbf{E} \in \mathbb{R}^{256 \times 128}$.

$$f_i = E[b_i], \quad f_i \in \mathbb{R}^{128}$$

Per "Ciao"

- $b_0 = 67 \rightarrow f_0 = E[67] \in \mathbb{R}^{128}$ (*ovvero il vettore corrispondente è la riga 67 di E*)
- $b_1 = 105 \rightarrow f_1 = E[105] \in \mathbb{R}^{128}$
- $b_2 = 97 \rightarrow f_2 = E[97] \in \mathbb{R}^{128}$
- $b_3 = 111 \rightarrow f_3 = E[111] \in \mathbb{R}^{128}$

3. Positional encoding ($\mathbf{p}_i \in \mathbb{R}^{64}$)

Gli embedding dei byte f_i da soli **non contengono informazione sulla posizione**. Per ciascuna posizione $i = 0, 1, 2, 3$, calcoliamo un vettore di 64 dimensioni con sinusoidi:

$$p_i = \left[\sin\left(\frac{i}{\lambda_1}\right), \cos\left(\frac{i}{\lambda_1}\right), \sin\left(\frac{i}{\lambda_2}\right), \cos\left(\frac{i}{\lambda_2}\right), \dots \right]$$

- Posizione 0 $\rightarrow p_0 \in \mathbb{R}^{64}$
- Posizione 1 $\rightarrow p_1 \in \mathbb{R}^{64}$
- Posizione 2 $\rightarrow p_2 \in \mathbb{R}^{64}$
- Posizione 3 $\rightarrow p_3 \in \mathbb{R}^{64}$

4. Vettore finale per ogni byte Ogni elemento dell'input è:

$$x_i = [f_i || p_i] \in \mathbb{R}^{192}$$

Quindi:

- "C" $\rightarrow x_0 \in \mathbb{R}^{192}$
- "i" $\rightarrow x_1 \in \mathbb{R}^{192}$
- "a" $\rightarrow x_2 \in \mathbb{R}^{192}$
- "o" $\rightarrow x_3 \in \mathbb{R}^{192}$

5. Forma dell'array X L'input array finale per la parola "Ciao" è:

$$X \in \mathbb{R}^{4 \times 192}$$

Esempio – ImageNet

$$X \in \mathbb{R}^{M \times C}$$

- $M = 224 \times 224 = 50176 \rightarrow$ numero di pixel (un token per pixel)
- $C = 3 + 258 = 261 \rightarrow$ feature per pixel (valori di colore + posizione)

Embedding del pixel (f_i)

Ogni pixel ha tre valori RGB normalizzati.

$$f_i = (R, G, B) \in \mathbb{R}^3$$

Positional encoding (p_i)

- Le coordinate del pixel (x, y) vengono codificate con Fourier features (sinusoidi multiscala).
- Con $K = 64$ bande di frequenza e $P = 2$ dimensioni spaziali: $d_{PE} = P \times (2K + 1) = 2 \times (2 \cdot 64 + 1) = 258$.

$$p_i \in \mathbb{R}^{258}$$

Vettore finale per ogni pixel

Si concatenano colore e posizione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{261}$$

Input array finale

- Ogni riga = un pixel codificato.
- L'immagine completa diventa:

$$X \in \mathbb{R}^{50176 \times 261}$$

Esempio – Video + Audio (Kinetics-700, Autoencoding Multimodale)

Attenzione: Kinetics nel Perceiver IO è autoencoding multimodale, non classificazione video pura

Nel paper Perceiver IO il task su Kinetics-700 non è classificazione video singola modalità, ma **autoencoding multimodale** simultaneo di video + audio + label (§4.3 del paper). Il modello ricostruisce le tre modalità contemporaneamente passando per un bottleneck latente comune. La classificazione diventa un “side-output” gestito dalla query della label. Anche le dimensioni dell'input riflettono questa scelta: il

video non viene dato frame-by-frame ma a *patch* $1 \times 4 \times 4$, l'audio come *patch di 16 sample*, e c'è anche l'input label (mascherata in training).

Input array per ciascuna modalità (dal paper, App. I per le patches e App. G per i dettagli delle Fourier features):

- **Video:** 16 frame a 224×224 pixel, patch di dimensione $1 \times 4 \times 4$ (1 frame, 4 pixel altezza, 4 pixel larghezza). Patch per frame: $224 \times 224 / 16 = 3136$. Totale patch video: $16 \times 3136 = 50\,176$. Ogni patch ha $16 \text{ pixel} \times 3 \text{ RGB} = 48$ valori grezzi, ai quali si aggiunge un positional encoding **Fourier 3D** da 387 dim (con $K = 64$ bande per ciascuna delle 3 coordinate x, y, t , secondo la convenzione di App. G del paper IO).
- **Audio:** campionato a 48 kHz $\rightarrow$ 1920 sample per frame (48,000/25 con 25 fps). Patch size 16 sample $\rightarrow$ $1920/16 = 120$ patch audio. Positional encoding **Fourier 1D** da 385 dim (App. G).
- **Label:** un vettore one-hot di dimensione 700 (classi Kinetics-700), ridotto a un singolo token con un embedding learnabile da 1024 dim. In training, la label viene mascherata al 50% per forzare il modello a non codificarla direttamente nei latenti.

Convenzione Fourier features del paper IO (App. G)

Identica a quella del Perceiver originale (App. D 2021), ma ribadita e standardizzata:

- **64 bande sin/cos per dimensione** in tutte le configurazioni (eccetto ImageNet con learned positions).
- **Frequenza minima:** fissa a una singola oscillazione completa sull'input (es. 1 ciclo sulla larghezza totale dell'immagine).
- **Frequenza massima:** tipicamente la Nyquist dell'input (es. 112 cicli per un'immagine di 224 pixel per dimensione).
- **Input position** scalato in $[-1, 1]$ per ciascuna dimensione (es. pixel in alto-sinistra a $(-1, -1)$, bottom-right a $(1, 1)$).
- Usa Fourier 1D per audio (coordinata temporale) e 3D per video (spazio+tempo). Per ImageNet IO standalone il paper usa invece *learned positions* (non Fourier), ottenendo il 84.5% riportato.

Allineamento delle modalità. Le tre modalità hanno feature dim molto diverse ($48+387=435$ video, $1+385=386$ audio, 1024 label). Il paper le paddatta con *modality embedding* learnabili a dimensione comune **704** (317 dim per video, 319 per audio, 4 per label; questo allineamento permette di concatenare le modalità in un singolo input array).

Input array finale:

$$X \in \mathbb{R}^{(50\,176 + 120 + 1) \times 704} \approx \mathbb{R}^{50\,297 \times 704}$$

Output array finale: $16 \times 224 \times 224$ pixel video + 1920 audio sample + 1 label one-hot = 804 737 vettori (uno per ogni pixel/sample/label), ottenuti con query dedicate per modalità (Fig. 3 del paper).

Compression ratio. Il paper testa tre dimensioni di latent array con $D = 512$: $N = 784$ ($88 \times$ compressione), $N = 392$ ($176 \times$), $N = 196$ ($352 \times$). Nota che **non è $N = 512$ come nel Perceiver originale ImageNet**: per il multimodal autoencoding si usano configurazioni latenti specifiche al task.

Esempio – Audio (waveform grezzo)

L'audio è un **segnale continuo nel tempo**, che viene campionato in una sequenza di valori (ampiezze).

Ogni campione temporale diventa un **token** x_i

Step 1 - Embedding del campione (f_i)

- L'audio viene campionato, ad esempio a 16 kHz (16.000 campioni al secondo).
- Ogni campione è un valore reale dell'ampiezza della waveform, normalizzato in $[-1, 1]$.
- Questo valore è preso direttamente come embedding:

$$f_i \in \mathbb{R}^1$$

Esempi:

- Silenzio $\rightarrow f_i = 0$
- Massima ampiezza $\rightarrow f_i = 1$ o -1

Step 2 - Positional encoding (p_i)

- L'ampiezza da sola non dice quando nel tempo è stato registrato il campione.
- Ogni campione ha una posizione temporale:

$$t \in [0, M - 1]$$

dove M = numero totale di campioni.

Per distinguere i campioni, la posizione viene codificata con sinusoidi multiscale (Fourier features):

$$p_i = [\sin(t/\lambda_1), \cos(t/\lambda_1), \sin(t/\lambda_2), \cos(t/\lambda_2), \dots] \in \mathbb{R}^{64}$$

Totale: 64 numeri.

Questo dà al modello nozione di ordine temporale.

Step 3 - Vettore finale per ogni campione

Concatenazione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{1+64} = \mathbb{R}^{65}$$

- 1 numero = ampiezza
- 64 numeri = posizione temporale
- totale = 65 numeri per ogni campione audio

Step 4 - Input array finale (X)

- Ogni campione audio diventa una riga di X .
- Se prendiamo 1 secondo di audio a 16 kHz:

$$M = 16,384 \text{ (circa)}$$

Forma finale:

$$X \in \mathbb{R}^{16384 \times 65}$$

Esempio - Input multimodale (Vision + Language, VQA)

Nota: Questo esempio è ipotetico e non presente nel paper originale di Perceiver IO. I task effettivi del paper sono: GLUE (linguaggio), Optical Flow (Sintel/KITTI), Multimodal Autoencoding (Kinetics-700), ImageNet, StarCraft II, AudioSet.

Nel compito **Visual Question Answering (VQA)** l'input è composto da:

1. un'immagine (es. una foto),
2. una domanda in linguaggio naturale (testo).

Domanda:

"What color is the cat?"

Immagine:

una foto RGB 224×224

Il modello riceve entrambi come un unico **input array** X

$$X \in \mathbb{R}^{M \times C}$$

- $M = M_{img} + M_{text}$
= numero di pixel dell'immagine + numero di byte della domanda
- $C = C_{base} + d_m$
dove d_m è la dimensione dell'embedding di modalità (nel paper $d_m = 16$).

Step 1 - Immagine

Ogni pixel dell'immagine (ad esempio 224×224):

$$f_i^{img} = (R, G, B) \in \mathbb{R}^3$$

Coordinate spaziali $(x, y) \rightarrow$ Fourier features:

$$p_i^{img} \in \mathbb{R}^{258}$$

Embedding di modalità (per dire che è "immagine"):

$$m^{img} \in \mathbb{R}^{16}$$

Concatenazione:

$$x_i^{img} = [f_i^{img} \parallel p_i^{img} \parallel m^{img}] \in \mathbb{R}^{3+258+16} = \mathbb{R}^{277}$$

Totale per tutta l'immagine:

$$M_{img} = H \times W \Rightarrow X^{img} \in \mathbb{R}^{M_{img} \times 277}$$

Step 2 - Testo

La domanda testuale viene convertita in byte UTF-8.

Ogni byte $b_j \in \{0, \dots, 255\}$:

Embedding del byte:

$$f_j^{text} \in \mathbb{R}^{128}$$

Posizione nella sequenza $\rightarrow$ Fourier features:

$$p_j^{text} \in \mathbb{R}^{64}$$

Embedding di modalità (per dire che è "testo"):

$$m^{text} \in \mathbb{R}^{16}$$

Concatenazione:

$$x_j^{text} = [f_j^{text} \parallel p_j^{text} \parallel m^{text}] \in \mathbb{R}^{128+64+16} = \mathbb{R}^{208}$$

Totale per la domanda:

$$M_{text} = \text{numero di byte} \Rightarrow X^{text} \in \mathbb{R}^{M_{text} \times 208}$$

Step 3 - Array multimodale finale

Concatenazione immagine + testo:

$$X = \begin{bmatrix} X^{img} \\ X^{text} \end{bmatrix}, X \in \mathbb{R}^{(M_{img}+M_{text}) \times C}$$

dove:

- $C = 277$ per i token immagine,
- $C = 208$ per i token testuali.

Latent Array:

È una **matrice di vettori latenti** $Z \in \mathbb{R}^{N \times D}$ **interamente appresa** (parametri del modello).

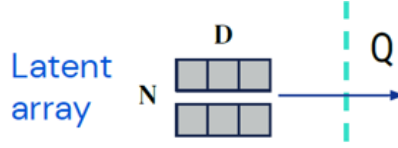


Figure 22: Latent array ($N \times D$): matrice di vettori latenti appresa, da cui si generano le Query (Q).

- N = numero di latenti (slot/ “unità di memoria”).
- D = dimensione del canale/embedding dei latenti

Questi vettori **non derivano** direttamente dall’input e **non hanno un legame spaziale** con token/pixel specifici: **sono una base compatta che il modello usa per riassumere l’input**.

Le dimensioni N e D dei latenti variano con il task

Attenzione: nel Perceiver IO non esiste “una” configurazione fissa come $N = 512, D = 1024$ del Perceiver originale su ImageNet. Ogni task ha i suoi valori, riportati nelle appendici F–I del paper:

Task	N	D	M input	Riferimento
GLUE / language (IO, IO base)	256	1280	512 / 2048	App. F.2, Tab. 11
GLUE / language (IO++)	256	1536	2048	App. F.2, Tab. 11
Optical Flow (Sintel)	2048	512	pixel-dense	App. H
Multimodal Autoencoding	196–784	512	~50k patch	App. I
ImageNet (IO)	512	1024	50 176 pixel	App. A
StarCraft II	32	–	512 entità	App. B
AudioSet	512	512 o 1024	video+audio	App. C

L’input embedding size C è invece **768** per tutte le configurazioni language (BERT-compatibile).

Caso 1 - Testo (GLUE, byte-level)

- Input: sequenza di byte UTF-8, $M = 2048$ per Perceiver IO UTF-8 (vs $M = 512$ per SentencePiece).
- Ogni byte: embedding + positional encoding Fourier concatenati, portati a input embedding size $C = 768$ (coerente con BERT-Base).
- Input array: $X \in \mathbb{R}^{2048 \times 768}$.
- Latenti: $Z \in \mathbb{R}^{256 \times 1280}$ — attenzione: *non* 512×1024 !

I 256 latenti leggono i 2048 byte, con compressione $\approx 8\times$. Quest’asimmetria ($N \ll M$) è cruciale perché permette alla rete di essere molto profonda (26 o 40 layer) con costo ragionevole.

Caso 2 - Immagini (ImageNet)

- Input: immagine $224 \times 224 = 50\,176$ pixel.
- Ogni pixel: colore RGB (3) + posizione Fourier 2D (258) $\rightarrow C = 261$. La config con miglior accuracy usa **learned position encoding** invece di Fourier (con pretraining JFT raggiunge 84.5%).
- Input array: $X \in \mathbb{R}^{50176 \times 261}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 1024}$ (come nel Perceiver originale).

I 512 latenti leggono da 50 176 pixel.

Caso 3 - Video + Audio (Kinetics-700, autoencoding multimodale)

- Input: $\sim 50\,176$ patch video $1 \times 4 \times 4 + 120$ audio patch (16 sample ciascuna) + 1 label token, allineati a feature dim 704 con modality embedding.
- Input array: $X \in \mathbb{R}^{50\,297 \times 704}$.
- Latenti: $Z \in \mathbb{R}^{N \times 512}$ con $N \in \{196, 392, 784\}$ (tre compression ratios testati: $352\times$, $176\times$, $88\times$).

I latenti piccoli permettono di comprimere l'input di oltre $300\times$ senza perdere troppa qualità di ricostruzione (almeno per video; l'audio degrada più rapidamente).

Caso 4 - Audio (AudioSet nel Perceiver IO)

- Input: video + audio (raw o mel-spectrogram) per AudioSet classification.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$ o $\mathbb{R}^{512 \times 1024}$ (App. C del paper, dipende dalla config scelta).
- Nota: il paper non specifica un “audio-only” standalone per IO; AudioSet è sempre multimodale (video + audio).

Caso 5 - Multimodale (VQA: immagine + testo) (*Esempio ipotetico, non presente nel paper originale.*)

- Input:
- immagine $224 \times 224 = 50.176$ pixel $\rightarrow C = 277$ (RGB+posizione+modalità).
- domanda testuale (es. 20 byte) $\rightarrow C = 208$ (byte+posizione+modalità).
- Input array: concatenazione dei due $\rightarrow X \in \mathbb{R}^{(50\,176+20) \times C}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 1024}$.

I 512 latenti leggono sia pixel che testo.

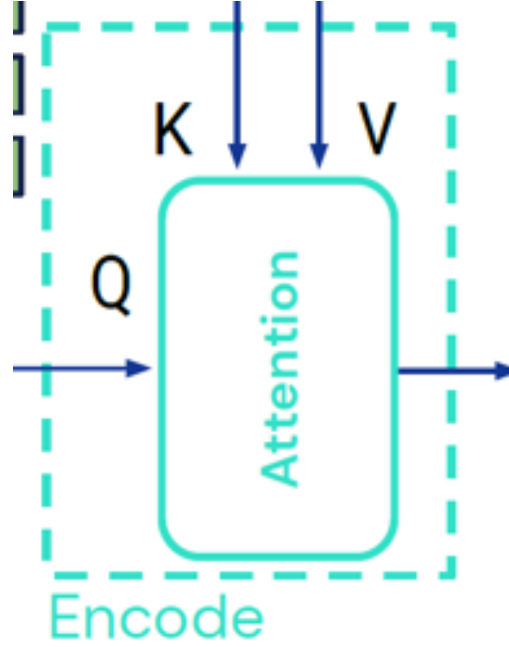


Figure 23: Blocco Encode del Perceiver IO: cross-attention con K , V dall'input e Q dai latenti.

2.3 Encode Module

L'**Encode Module** è il blocco di **cross-attention** che trasferisce l'informazione dall'**Input array** X al **Latent array** Z .

- **Idea:** i latenti (Q) leggono i contenuti dall'input (K, V).
- **Output:** un nuovo stato latente Z^1 con la stessa forma di partenza $N \times D$, ma arricchito dei dati dell'input.

Input del blocco

Latent array iniziale

$$Z^{(0)} \in \mathbb{R}^{N \times D}$$

Input array

$$X \in \mathbb{R}^{M \times C}$$

1. Proiezioni lineari

$$\begin{aligned} Q &= Z^{(0)} W_Q & W_Q &\in \mathbb{R}^{D \times d_k}, & Q &\in \mathbb{R}^{N \times d_k} \\ K &= X W_K & W_K &\in \mathbb{R}^{C \times d_k}, & K &\in \mathbb{R}^{M \times d_k} \\ V &= X W_V & W_V &\in \mathbb{R}^{C \times d_v}, & V &\in \mathbb{R}^{M \times d_v} \end{aligned}$$

2. Attention Scores

$$S = \frac{QK^\top}{\sqrt{d_k}} \quad S \in \mathbb{R}^{N \times M}$$

3. Softmax (pesi di attenzione)

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{m=1}^M e^{S_{im}}} \quad A \in \mathbb{R}^{N \times M}$$

4. Aggregazione

$$O = AV \quad O \in \mathbb{R}^{N \times d_v}$$

5. Proiezione lineare finale

$$\tilde{Z} = OW_O \quad W_O \in \mathbb{R}^{d_v \times D}, \quad \tilde{Z} \in \mathbb{R}^{N \times D}$$

6. Residual Connection (con pre-norm sull'attenzione) Il Perceiver IO utilizza lo schema **pre-norm**: la LayerNorm viene applicata *prima* della sotto-funzione (attenzione o MLP), e il risultato viene sommato al residuo originale. In forma compatta: $z' = z + f(\text{LN}(z))$.

$$Z_a = Z^{(0)} + \text{CrossAttn}(\text{LN}(Z^{(0)}), X) \in \mathbb{R}^{N \times D}$$

7. Feed Forward (MLP a 2 layer, con pre-norm) Si applica prima la LayerNorm, poi l'MLP, e si somma il residuo:

$$\hat{Z} = \text{LN}(Z_a) \in \mathbb{R}^{N \times D}$$

Espansione:

$$H = \hat{Z}W_1 + b_1, \quad W_1 \in \mathbb{R}^{D \times 4D}, \quad H \in \mathbb{R}^{N \times 4D}$$

Attivazione (GELU):

$$H' = \text{GELU}(H) \in \mathbb{R}^{N \times 4D}$$

Compressione:

$$F = H'W_2 + b_2, \quad W_2 \in \mathbb{R}^{4D \times D}, \quad F \in \mathbb{R}^{N \times D}$$

8. Residual Connection (post-MLP)

$$Z^{(1)} = Z_a + F \in \mathbb{R}^{N \times D}$$

In forma compatta: $Z^{(1)} = Z_a + \text{MLP}(\text{LN}(Z_a))$.

Output del blocco Encode

$$Z^{(1)} \in \mathbb{R}^{N \times D}$$

Caso 1 - Testo (GLUE, byte-level)

- **Input:** sequenza di byte UTF-8
- **Dimensioni:** $M = L_{\text{byte}}$ (nel paper fino a 2048), $C = 128 + 64 = 192$
- **Forme:**

$$\begin{aligned}
X &\in \mathbb{R}^{M \times 192} \\
K &\in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v} \\
Q &\in \mathbb{R}^{256 \times d_k} \quad (N = 256 \text{ per GLUE, App. F.2 paper}) \\
S, A &\in \mathbb{R}^{256 \times M} \\
Z^{(1)} &\in \mathbb{R}^{256 \times 1280}
\end{aligned}$$

- **Cosa fa:** i latenti pesano byte e posizioni (Fourier 1D) per condensare pattern della sequenza in 256 vettori compatti di dim 1280 (non 512×1024 : quelle sono le dimensioni per ImageNet).

Caso 2 - Immagini (ImageNet)

- **Input:** immagine 224×224 RGB
- **Dimensioni:** $M = 224 \cdot 224 = 50,176$, $C = 3 + 258 = 261$
- **Forme:**

$$\begin{aligned}
X &\in \mathbb{R}^{50176 \times 261} \\
K &\in \mathbb{R}^{50176 \times d_k}, V \in \mathbb{R}^{50176 \times d_v} \\
Q &\in \mathbb{R}^{512 \times d_k} \\
S, A &\in \mathbb{R}^{512 \times 50176} \\
Z^{(1)} &\in \mathbb{R}^{512 \times 1024}
\end{aligned}$$

- **Cosa fa:** i latenti leggono pixel + posizioni 2D (Fourier) e si specializzano su frequenze/regioni (bordi, texture, struttura globale).

Caso 3 — Video (Kinetics-700)

- **Input:** volume $H \times W \times T$ (es. $224 \times 224 \times 16$)
- **Dimensioni:** $M_{\text{raw}} = HWT \approx 802,816 \rightarrow$ sottocampionato a $M \approx 50,000$; $C = 3 + 387 = 390$
- **Forme:**

$$\begin{aligned}
X &\in \mathbb{R}^{50000 \times 390} \\
K &\in \mathbb{R}^{50000 \times d_k}, V \in \mathbb{R}^{50000 \times d_v} \\
Q &\in \mathbb{R}^{512 \times d_k} \\
S, A &\in \mathbb{R}^{512 \times 50000} \\
Z^{(1)} &\in \mathbb{R}^{512 \times 1024}
\end{aligned}$$

- **Cosa fa:** integra spazio+tempo (Fourier 3D). Latenti diversi catturano movimento, coerenze temporali e contesto statico.

Caso 4 - Audio (waveform)

- **Input:** waveform campionata (es. $\sim 16k$ campioni/s)

- **Dimensioni:** per $\sim 1\text{ s} \rightarrow M \approx 16,000$; $C = 1 + 64 = 65$
- **Forme:**

$$\begin{aligned} X &\in \mathbb{R}^{M \times 65} \\ K &\in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v} \\ Q &\in \mathbb{R}^{512 \times d_k} \\ S, A &\in \mathbb{R}^{512 \times M} \\ Z^{(1)} &\in \mathbb{R}^{512 \times 1024} \end{aligned}$$

- **Cosa fa:** latenti "sintonizzano" porzioni temporali e frequenze implicite (grazie alle Fourier 1D) per riassumere il segnale.

Caso 5 - Multimodale (VQA: immagine + testo) (*Esempio ipotetico, non presente nel paper originale.*)

- **Input:** pixel immagine + byte domanda concatenati
- **Dimensioni per modalità:**
- **Immagine:** $M_{img} = 50176$, $C_{img} = 3 + 258 + 16 = 277$
- **Testo:** $M_{text} = L_{byte}$, $C_{text} = 128 + 64 + 16 = 208$
- **Totale:** $M = M_{img} + M_{text}$. In pratica si proietta ogni modalità a un canale comune C^* prima dell'Encode.

Forme (dopo allineamento canali):

$$X \in \mathbb{R}^{(M_{img} + M_{text}) \times C^*}$$

K, V con la stessa M ;

$$Q \in \mathbb{R}^{512 \times d_k};$$

$$S, A \in \mathbb{R}^{512 \times M};$$

$$Z^{(1)} \in \mathbb{R}^{512 \times 1024}$$

- **Cosa fa:** i latenti fondono contenuto visivo e linguistico; l'attenzione sfrutta anche il modality embedding per collegare parole a regioni d'immagine.

2.4 Process Module

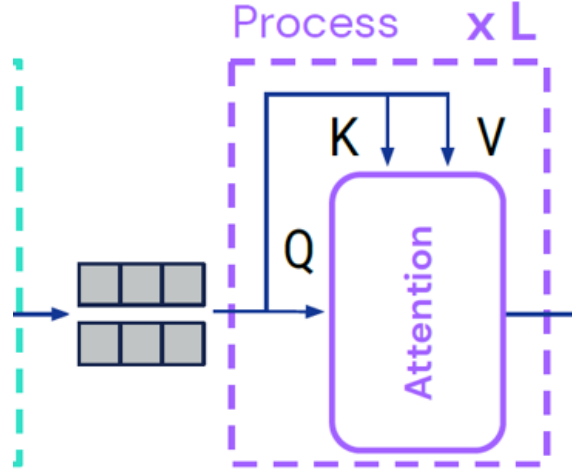


Figure 24: Blocco Process del Perceiver IO: self-attention sui latenti ripetuta L volte.

Il **Process Module** (detto anche *Latent Transformer* o *Latent Attention*) è una pila di blocchi di self-attention che opera *solo* sui latenti $Z^{(1)}$: raffina la rappresentazione interna a costo $O(N^2)$ indipendente da M , prima che il Decode la interroghi.

Input del blocco

Latent array aggiornato dall'Encode:

$$Z^{(1)} \in \mathbb{R}^{N \times D}$$

Proiezioni lineari

$$\begin{aligned} Q &= Z^{(1)} W_Q, & Q &\in \mathbb{R}^{N \times d_k} \\ K &= Z^{(1)} W_K, & K &\in \mathbb{R}^{N \times d_k} \\ V &= Z^{(1)} W_V, & V &\in \mathbb{R}^{N \times d_v} \end{aligned}$$

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times d_k}.$$

Stavolta Query, Key e Value provengono tutti dai latenti stessi.

Attention Scores

$$S = \frac{QK^\top}{\sqrt{d_k}} \quad S \in \mathbb{R}^{N \times N}$$

Ogni latente misura la similarità con tutti gli altri

Softmax (pesi di attenzione)

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{m=1}^N e^{S_{im}}}, \quad A \in \mathbb{R}^{N \times N}$$

Distribuzione di attenzione tra latenti (quanto un latente guarda gli altri)

Aggregazione

$$O = AV \quad O \in \mathbb{R}^{N \times d_v}$$

Ogni latente raccoglie informazione da tutti gli altri.

Proiezione lineare finale

$$\tilde{Z} = OW_O, \quad W_O \in \mathbb{R}^{d_v \times D}, \quad \tilde{Z} \in \mathbb{R}^{N \times D}$$

Residual Connection (con pre-norm sull'attenzione)

Anche nel Process si usa lo schema **pre-norm**:

$$Z_a = Z^{(1)} + \text{SelfAttn}(\text{LN}(Z^{(1)})), \quad Z_a \in \mathbb{R}^{N \times D}$$

Feed Forward (MLP a 2 layer, con pre-norm)

$$\hat{Z} = \text{LN}(Z_a) \in \mathbb{R}^{N \times D}$$

Espansione:

$$H = \hat{Z}W_1 + b_1, \quad W_1 \in \mathbb{R}^{D \times 4D}, \quad H \in \mathbb{R}^{N \times 4D}$$

Attivazione (GELU):

$$H' = \text{GELU}(H), \quad H' \in \mathbb{R}^{N \times 4D}$$

Compressione:

$$F = H'W_2 + b_2, \quad W_2 \in \mathbb{R}^{4D \times D}, \quad F \in \mathbb{R}^{N \times D}$$

Residual Connection (post-MLP)

$$Z^{(2)} = Z_a + F, \quad Z^{(2)} \in \mathbb{R}^{N \times D}$$

In forma compatta: $Z^{(2)} = Z_a + \text{MLP}(\text{LN}(Z_a))$.

Output del blocco Process

$$Z^{(2)} \in \mathbb{R}^{N \times D}$$

- Stessa forma dei latenti in input ($N \times D$, es. 512×1024).
- Contenuto aggiornato: i latenti hanno comunicato tra loro, scambiandosi informazione.
- Questo blocco si ripete L volte:

$$Z^{(L)} = Process^L \left(Z^{(1)} \right)$$

Il Process module è identico per tutte le modalità: opera esclusivamente sui latenti.

Indipendentemente dal tipo di input (testo, immagine, video, audio, multimodale), dopo l'Encode il Process riceve sempre un array latente $Z^{(1)} \in \mathbb{R}^{N \times D}$ e opera esclusivamente su di esso con self-attention. Le dimensioni di Q, K, V e le matrici di attenzione dipendono solo da N e D , non dalla modalità dell'input originale.

- $Q, K, V \in \mathbb{R}^{N \times d_k}$ (proiettati dai latenti)
- $S, A \in \mathbb{R}^{N \times N}$ (attenzione tra latenti)
- $Z^{(2)} \in \mathbb{R}^{N \times D}$

Ciò che cambia tra le modalità è solo il *contenuto* dei latenti (determinato dall'Encode), non la struttura del Process.

Modalità	Forma latenti	Cosa catturano i latenti
Testo (GLUE)	$Z \in \mathbb{R}^{256 \times 1280}$	pattern di caratteri, sintassi globale
Immagini (ImageNet)	$Z \in \mathbb{R}^{512 \times 1024}$	bordi, texture, strutture globali
Video+Audio (Kinetics, autoencoding)	$Z \in \mathbb{R}^{N \times 512}$, $N \in \{196, 392, 784\}$	movimento, coerenze temporali, contesto statico
Audio (AudioSet, video+audio)	$Z \in \mathbb{R}^{512 \times 512}$ o $\mathbb{R}^{512 \times 1024}$	pattern fonetici, ritmi globali

Output del Process

$$Z^{(L)} \in \mathbb{R}^{N \times D}$$

- Stessa forma ($N \times D$), con (N, D) specifici al task (es. 512×1024 per ImageNet, 256×1280 per GLUE/language, $\sim 400 \times 512$ per multimodal autoencoding — cfr. tabella in §2).
- Contenuto raffinato e arricchito dopo L layer di self-attention.
- Rappresenta la memoria interna finale che verrà usata dal Decode.

Ricapitolando

Input array: sequenza di token (immagine, testo, audio...)

$$X \in \mathbb{R}^{M \times C}$$

Latent array : spazio compatto dove il modello elabora l'informazione

$$Z \in \mathbb{R}^{N \times D}$$

Dopo L layer di process (self-attention), abbiamo un array latente raffinato:

$$Z^{(L)} \in \mathbb{R}^{N \times D}$$

2.5 Decode Module e Output Queries

Il **Decode Module** converte la rappresentazione latente raffinata $Z^{(L)}$ in un output di forma arbitraria tramite *cross-attention*: le **output queries** $Y^{(0)} \in \mathbb{R}^{O \times E}$ specificano *cosa* chiediamo (posizione, modalità, classe, ...), i latenti forniscono Key e Value. È questo decoder a permettere a Perceiver IO di produrre output strutturati di lunghezza qualsiasi (token, pixel, label, flussi ottici) a partire dallo stesso array latente agnostico al task.

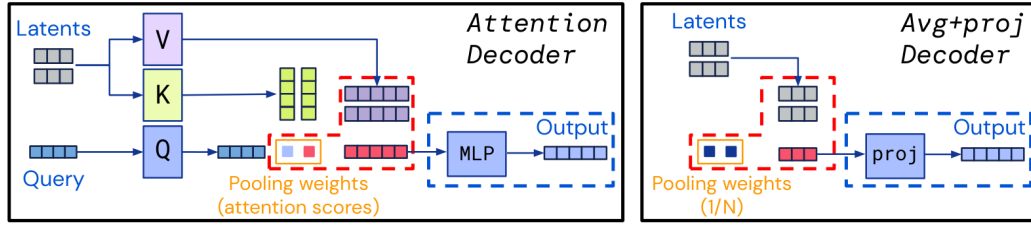


Figure 25: Confronto fra il decoder di Perceiver IO (*Attention Decoder*, a sinistra) e quello del Perceiver originale (*Avg+proj*, a destra) (Jaegle et al., 2022, App. E, Fig. 6). L’Avg+proj fa pooling *uniforme* sui latenti ($1/N$) e proietta linearmente: produce un singolo output. L’Attention Decoder usa *pesi di attenzione appresi* (uno per ogni (*query*, *latente*)) e una MLP: i pesi possono variare *per query*, permettendo output di forma arbitraria.

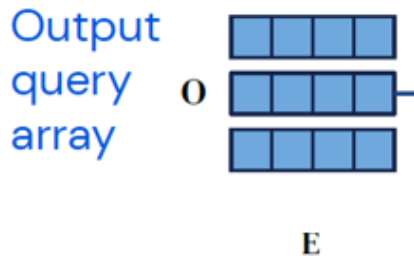


Figure 26: Output query array ($O \times E$): ogni query definisce un elemento dell’output desiderato.

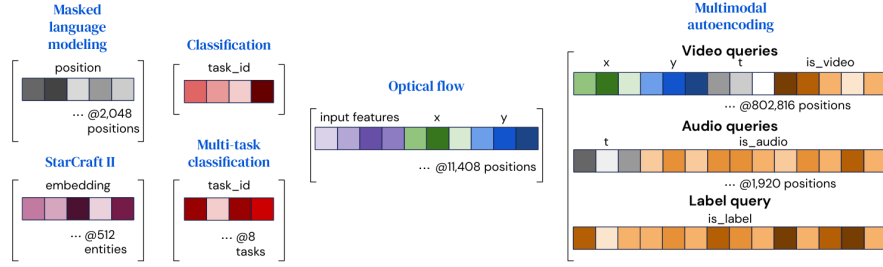


Figure 27: Esempi di costruzione delle output queries per task differenti (Jaegle et al., 2022, Fig. 3). MLM e classificazione hanno query semplici (posizione o `task_id` di poche dimensioni); optical flow combina feature di input e coordinate x, y ; il multimodal autoencoding usa query specifiche per modalità (video, audio, label) con indicatori di tipo (`is_video`, `is_audio`, `is_label`). La forma della query e il numero di query O sono quindi specifici del task.

2.5.1 Output Query Array $Y^{(0)}$

È una matrice di query $Y^{(0)} \in \mathbb{R}^{O \times E}$, dove:

- O = numero di query, cioè quante "domande" vogliamo fare ai latenti (per esempio una nel caso di classificazione)
- E = dimensione iniziale delle query (può variare a seconda di come le costruiamo)

$$E = \dim(\text{posizione}) + \dim(\text{tipo}) + \dim(\text{modalità}) + \dim(\text{contesto (opzionale)})$$

Ogni riga di $Y^{(0)}$ è un vettore che specifica:

- Che tipo di informazione vogliamo leggere dai latenti e dove/per chi vogliamo questa informazione es. posizione spaziale, temporale, categoria.

Le query sono quindi una guida per il Decode: invece di leggere tutto il contenuto latente, chiediamo solo ciò che serve.

Cosa contiene una query $y_i^{(0)}$

È un descrittore ottenuto in genere per concatenazione di blocchi (tutti vettori):

$$y_i^{(0)} = \left[\begin{array}{c|c|c|c} \text{posizione} & \text{tipo di output} & \text{modalità} & \text{contesto opzionale} \\ \hline \text{dim}=P & \text{dim}=T & \text{dim}=M & \text{dim}=C \end{array} \right]$$

$$E = P + T + M + C$$

Ogni parte è un blocco informativo e contiene un tipo di segnale diverso che aiuta il modello a interpretare la query e a decidere dove guardare nei latenti e cosa leggere.

Questi blocchi possono essere **learned** (parametri) o **deterministici** (Fourier); in entrambi i casi ricevono gradiente tramite il decode.

- **Posizione:** coord. spaziali/temporali/indice di token (spesso con Fourier features multiscala).

Il modello deve sapere a quale punto dell'output la query si riferisce, se per esempio a un pixel, a un token di testo o a un frame temporale.

a) **Deterministico**

Si codifica la posizione con una funzione matematica fissa, che non ha parametri:

- **Fourier Features:** si mappano le coordinate (x, y, t , indice) in sinusoidi multiscala:

$$p(i) = [\sin(2\pi Bx), \cos(2\pi Bx), \sin(2\pi By), \cos(2\pi By), \dots]$$

Dove B rappresenta diverse frequenze spaziali o temporali.

b) **Learned**

Si assegna a ogni posizione un vettore trainabile (una riga di una tabella di embedding).

$$p(i) = E_{pos}[i], \quad E_{pos} \in \mathbb{R}^{N_{pos} \times P}$$

- **Tipo di output:** embedding che distingue cosa stai chiedendo (es. classificazione vs segmentazione vs testo).

Indica che tipo di informazione stai chiedendo al modello per esempio:

- Dammi una classificazione
- Dammi un token di testo (decodifica)
- Dammi la segmentazione di un pixel

È sempre un embedding trainabile (learned):

$$t = E_{type}[\text{'classification'}], \quad T \approx 16$$

Ogni tipo ha un proprio vettore che il modello può aggiornare durante l'allenamento.

- **Modalità:** embedding che indica la modalità dell'output (immagine, testo, audio, ...).

Specifica da quale dominio stai generando l'output, diventa cruciale nei modelli multimodali perché i latenti contengono informazioni miste.

Embedding trainabile (learned):

$$m = E_{mod}[\text{'image'}], \quad M \approx 16$$

Tutti gli output appartenenti alla stessa modalità condividono lo stesso vettore.

Esempio:

In VQA (Vision + Language), le query della risposta testuale avranno un embedding di modalità "text", mentre i latenti provengono da immagini + testo.

Questo aiuta l'attenzione a "capire" che deve restituire parole, non feature visive.

- **Contesto opzionale:** vettori che riassumono parti dell'input utili a guidare la decodifica (es. nel multimodale).

Trasporta **informazioni aggiuntive** che aiutano la query a essere più specifica.

È opzionale, ma utile nei compiti dove l'output dipende da un input complesso o da un'altra modalità.

Allineamento ai latenti: poiché i latenti hanno dimensione D , tutte le query $Y^{(0)}$ verranno proiettate con W_q nello spazio di Q per il decode

$$Q = Y^{(0)}W_q \in \mathbb{R}^{O \times D}, \quad W_q \in \mathbb{R}^{E \times D}$$

(semplice proiezione lineare "traduttrice" da spazio query E allo spazio latenti D).

1. Testo - GLUE (byte-level, classificazione) Task: classificazione testuale su uno degli 8 task GLUE (es. SST-2 sentiment, MRPC paraphrase, ecc.). L'output è una singola etichetta per ogni frase.

Tre modalità di query (dal paper, App. F.4 e §4.1):

- **Pretraining MLM** (non-classificazione): $O = M = 2048$ output query, una per ogni posizione byte. Ogni query è un *learned position-dependent vector* di dim $E = 768$. Il decoder genera un vettore di feature per ciascuna posizione masked, poi un layer lineare position-wise predice il byte originale (softmax su 260 vocabulary token). Sono quindi queries *dipendenti dalla posizione*.
- **Fine-tuning single-task** (GLUE singolo): $O = 1$ output query. Una singola query learnabile di dim $E = 768$ interroga i latenti e produce un vettore che una MLP 2-layer mappa nei logit di classe. È l'equivalente del token [CLS] di BERT, ma esterno ai latenti (non occupa spazio nell'input).
- **Fine-tuning multi-task** (tutti gli 8 task GLUE insieme): $O = 8$ output queries, una per task (ognuna con un embedding *task_id* learnabile). Il decoder produce 8 vettori in parallelo, una MLP per task li mappa nei rispettivi logit. Questo approccio dà i risultati migliori (81.8 avg GLUE).

"1 query = 1 output" — non sempre!

Spesso si semplifica dicendo che "per la classificazione basta $O = 1$ ". In realtà dipende dal setting: il pretraining MLM usa O dell'ordine di migliaia (una per byte); il multi-task uso $O = 8$; solo il single-task fine-tuning usa $O = 1$. Le dimensioni delle matrici cambiano di conseguenza: $Q \in \mathbb{R}^{O \times D}$, $S, A \in \mathbb{R}^{O \times N}$, output $\in \mathbb{R}^{O \times E}$.

2. Immagini - ImageNet (classificazione) Task: Classificare un'intera immagine in una delle 1000 classi ImageNet.

Idea: una sola etichetta per tutta l'immagine; niente coordinate in query.

- $O : 1$ (categoria globale).
- $y^{(0)}$: tipo di output (classificazione) + modalità (immagine).
- $E : T + M = 16 + 16 = 32$.
- $Y^{(0)} : \mathbb{R}^{1 \times 32}$.
- (Proiezione) $Q : Y^{(0)}W_q \in \mathbb{R}^{1 \times D}$

La query rappresenta:

“Riassumi le feature visive dell’immagine e dammi la classe principale.”

La cross-attention tra questa query e i latenti (che rappresentano tutti i pixel codificati) fa sì che il modello estragga un’unica risposta aggregata da tutto il contenuto visivo.

3. Video - Kinetics-700 (autoencoding multimodale) Task (nel paper Perceiver IO): *non è classificazione video pura*, ma **autoencoding multimodale** simultaneo di video + audio + class label (§4.3 del paper). Il modello ricostruisce tutte le modalità e la classificazione emerge come “side-output” dalla query della label.

Idea: ogni modalità ha il proprio set di output queries (video: una per pixel; audio: una per sample; label: una singola query).

- $O \approx 804737$ output queries totali: 802,816 video (pixel) + 1920 audio (samples) + 1 label.
- Query video: Fourier PE 3D (x, y, t) + modality embedding, dim totale 704.
- Query audio: Fourier PE 1D (t) + modality embedding, dim 704.
- Query label: embedding learnabile 1024-dim + modality embedding, paddato a 704.
- $Y^{(0)} \in \mathbb{R}^{804737 \times 704}$ (molto grande; decoding in batch a test time).

Se invece ci interessa *solo* la classificazione video (task ausiliario), basta estrarre la query della label: in quel caso $O = 1$ e il modello si comporta come nel caso ImageNet. Ma la config “canonica” del paper su Kinetics-700 è il full autoencoding.

4. Audio - waveform (classificazione clip) Task: Riconoscere il contenuto di un clip audio (es. parola, comando, tipo di suono).

L’audio viene trattato come una sequenza di campioni, ma l’output è una classe globale.

Idea: una classe per l’intero clip; niente posizione nell’output.

- $O : 1$ (classe del clip).
- $y^{(0)} : \text{tipo di output (classificazione) + modalità (audio)}.$
- $E : T + M = 16 + 16 = 32.$
- $Y^{(0)} : \mathbb{R}^{1 \times 32}.$
- (Proiezione) $Q : \mathbb{R}^{1 \times D}.$

“Analizza la clip completa e restituisci la categoria sonora.”

Il modello combina informazioni temporali dai latenti (che rappresentano l’intera waveform) per fornire una classificazione finale (es. “comando: stop”, “rumore: applauso”).

5. Multimodale - VQA (immagine + testo, risposta testuale) Nota: Questo esempio è ipotetico e non presente nel paper originale di Perceiver IO.

Task: Data un’immagine e una domanda in linguaggio naturale, generare una risposta testuale (es. “What color is the car?” $\rightarrow$ “Red”)

Idea: devo generare una sequenza di token; ogni query chiede il j -esimo token.

- $O : L$ (numero massimo di token in output).

- $y^{(0)}$ (per ciascun token):
posizione del token (positional/Fourier) + tipo di output (testo) + modalità (testo) + (opz.)
contesto (embedding della domanda).
- $E : P + T + M(+C) \rightarrow$ tipico ≈ 128 (es. 64 pos + 16 tipo + 16 modalità + 32 contesto).
- $Y^{(0)} : \mathbb{R}^{L \times 128}$.
- (Proiezione) $Q : \mathbb{R}^{L \times D}$.

Ogni riga di $Y^{(0)}$ è una “query token”:

- Query 1 $\rightarrow$ “Dammi la **prima parola** della risposta.”
- Query 2 $\rightarrow$ “Dammi la **seconda parola**, sapendo cosa ho già detto e cosa chiede la domanda.”
- ... e così via.

Il decoder utilizza le informazioni multimodali nei latenti (immagine + testo) e, grazie al contesto, produce token coerenti con la domanda.

Caso

O

Testo (GLUE)

Immagini (ImageNet)

Video (Kinetics)

Audio (waveform)

VQA (multimodale)*

*Esempio ipotetico, non presente nel paper originale.

2.5.2 Blocco Decode

Il Decode è una cross-attention che combina due sorgenti, già introdotte nelle sezioni precedenti:

- le **Output Queries** $Y^{(0)} \in \mathbb{R}^{O \times E}$ — definite in §2.5.1 (cosa chiediamo);
- il **Latent Array finale** $Z^{(L)} \in \mathbb{R}^{N \times D}$ — uscita del Process (§2.4), contiene la rappresentazione compressa dell’input.

Da questi due elementi si calcolano Q (dalle query), K e V (dai latenti) tramite proiezioni lineari, con le seguenti matrici di peso trainabili:

$$\begin{aligned} Q &= Y^{(0)} W_Q & W_Q &\in \mathbb{R}^{E \times (h \cdot d_k)} \Rightarrow Q \in \mathbb{R}^{O \times (h \cdot d_k)} \\ K &= Z^{(L)} W_K & W_K &\in \mathbb{R}^{D \times (h \cdot d_k)} \Rightarrow K \in \mathbb{R}^{N \times (h \cdot d_k)} \\ V &= Z^{(L)} W_V & W_V &\in \mathbb{R}^{D \times (h \cdot d_v)} \Rightarrow V \in \mathbb{R}^{N \times (h \cdot d_v)} \end{aligned}$$

Perché usare $h \cdot d_k$ e non semplicemente D ?

Perché l’attenzione è **multi-head**:

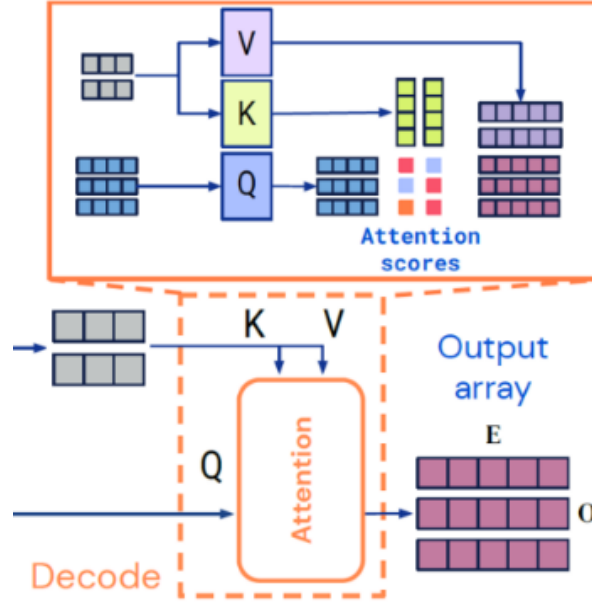


Figure 28: Blocco Decode del Perceiver IO: le output query (Q) interrogano i latenti finali (K, V) tramite cross-attention per generare l’output array.

- invece di una singola attenzione grande, si usano h attenzioni indipendenti in parallelo;
- ognuna ha dimensione ridotta d_k o d_v ;

Questo migliora la **capacità di rappresentare pattern diversi** contemporaneamente (es. attenzione a posizioni, colori, parole, suoni, ecc.).

Nel paper abbiamo le seguenti dimensioni:

Parametro	Valore
D	dimensione latente (varia per task: 1024 ImageNet, 1280 GLUE base/IO, 1536 IO++, 512 multimodal/audio). Nota: 768 è la dim del <i>input embedding</i> C per language, non D .
$h = 8$	8 teste
$d_k = d_v = D/h$	dimensione interna per testa
$h \cdot d_k = h \cdot d_v = D$	totale equivalente a D
Da cui:	
W_Q	$\in \mathbb{R}^{E \times D}$
W_K	$\in \mathbb{R}^{D \times D}$
W_V	$\in \mathbb{R}^{D \times D}$
Q	$\in \mathbb{R}^{O \times D}, K, V \in \mathbb{R}^{N \times D}$

Parametro	Valore

Calcolo dell'attenzione nel Decode

Per ogni testa di attenzione, calcoliamo:

$$S = \frac{QK^\top}{\sqrt{d_k}}$$

- $Q \in \mathbb{R}^{O \times d_k}$
- $K \in \mathbb{R}^{N \times d_k}$

$$\rightarrow S \in \mathbb{R}^{O \times N}$$

Ogni riga di S rappresenta una query (cioè un elemento dell'output). Il prodotto scalare misura la somiglianza: se Q e K sono simili, il valore S_{ij} è alto $\rightarrow$ quella parte del latente sarà considerata importante per quella query.

La divisione per $\sqrt{d_k}$ serve solo per mantenere i valori in un range numerico stabile (evita che la softmax dopo "esploda").

Una volta ottenuto S , dobbiamo creare **la matrice di attenzione A** , che userai per pesare i Value V .

$$A = \text{softmax}(S) \quad A \in \mathbb{R}^{O \times N}$$

- O = numero di query (una riga per ogni elemento di output)
- N = numero di latenti (una colonna per ogni latente)

Ogni riga di A contiene i pesi di attenzione della query corrispondente. Come sappiamo con la softmax trasformiamo ogni riga di S (che contiene punteggi grezzi) in pesi normalizzati compresi tra 0 e 1, che sommano a 1.

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{n=1}^N e^{S_{in}}}$$

- **Valore alto** $\rightarrow$ la query guarda **molto** quel latente.
- **Valore basso** $\rightarrow$ la query lo considera **poco**.

Ora che abbiamo $A = \text{softmax}(S)$, dobbiamo usare questi pesi di attenzione per leggere le informazioni dai latenti.

Questo avviene calcolando:

$$O = AV$$

Simbolo	Dimensione	Provenienza
A	$O \times N$	pesi di attenzione (una riga per query, una colonna per latente)
V	$N \times d_v$	vettori dei latenti (contenuto informativo)
O	$O \times d_v$	risultato per query

Ogni riga di O rappresenta la risposta del modello per una query:

$$O_i = \sum_{j=1}^N A_{ij} V_j$$

In parole semplici:

La query i legge dai latenti una "media pesata" dei loro vettori V_j , dove i pesi A_{ij} dicono quanto ciascun latente è importante per quella query.

Multi-head attention Il calcolo $O = AV$ avviene indipendentemente per ogni testa di attenzione.

Ogni testa t produce il proprio:

$$O^{(t)} = A^{(t)} V^{(t)} \quad \text{con } O^{(t)} \in \mathbb{R}^{O \times d_v}$$

Poi tutti gli $O^{(t)}$ vengono concatenati:

$$O^{(\text{concat})} = [O^{(1)} \parallel O^{(2)} \parallel \dots \parallel O^{(h)}] \Rightarrow O^{(\text{concat})} \in \mathbb{R}^{O \times (h \cdot d_v)}$$

Proiezione finale Per riportare il risultato nello spazio comune dei latenti (dimensione D):

$$\tilde{Y} = O^{(\text{concat})} W_O$$

$$W_O \in \mathbb{R}^{(h \cdot d_v) \times D}, \quad \tilde{Y} \in \mathbb{R}^{O \times D}$$

Questo passaggio finale:

- fonde le informazioni provenienti da tutte le teste,
- ricompono un'unica rappresentazione per ciascuna query, compatibile con la dimensione modello D .

Quindi quello che otteniamo da questa fase della cross-attention sarà:

$$\tilde{Y} \in \mathbb{R}^{O \times D}$$

Residual connection + Layer Normalization nel Decode (pre-norm)

Anche nel Decode si usa lo schema **pre-norm**, come in Encode e Process: la LayerNorm viene applicata *prima* della cross-attention e prima dell'MLP, e il risultato viene sommato al residuo originale.

Le due matrici coinvolte sono:

Simbolo	Forma	Significato
$Y^{(0)}$	$O \times E$	Query di partenza (specificano cosa chiediamo al modello)
$\tilde{Y}$	$O \times D$	Risultato della cross-attention (ciò che abbiamo letto dai latenti)

$$Y^{(\text{proj})} = Y^{(0)} W_{\text{align}}$$

$$W_{\text{align}} \in \mathbb{R}^{E \times D} \Rightarrow Y^{(\text{proj})} \in \mathbb{R}^{O \times D}$$

Questa è una semplice proiezione lineare (matrice di pesi trainabile) che “traduce” le query nello stesso spazio dei latenti.

Somma residua (residual connection, con pre-norm)

Con lo schema pre-norm, la LayerNorm viene applicata *prima* della cross-attention. La somma residua diventa:

$$Y^{(\text{sum})} = Y^{(\text{proj})} + \text{CrossAttn}(\text{LN}(Y^{(\text{proj})}), Z^{(L)})$$

Interpretazione:

- $Y^{(\text{proj})}$ rappresenta ciò che la query **era inizialmente** (cosa stai chiedendo).
- $\text{CrossAttn}(\text{LN}(Y^{(\text{proj})}), Z^{(L)})$ rappresenta **la risposta** ottenuta dai latenti tramite la cross-attention (con pre-normalizzazione).
- Sommando, ottieni una rappresentazione che **unisce la domanda e la risposta**: il modello mantiene il significato originale della query ma lo arricchisce con le informazioni lette.

Questa somma è detta **connessione residua** perché “salta” il blocco di attenzione e si somma alla sua uscita, migliorando la stabilità dell'apprendimento (evita che la rete dimentichi l'ingresso).

Feed-Forward (MLP) + Residual + LayerNorm (pre-norm)

Questa parte ha la stessa logica che c'è in ogni Transformer block (Encode, Process, Decode): dopo la parte di attenzione, si applica una trasformazione neurale indipendente su ciascun token (qui: ciascuna query). Con pre-norm, la LayerNorm viene applicata prima dell'MLP.

1. Pre-normalizzazione

$$Y^{(\text{norm})} = \text{LN}(Y^{(\text{sum})}) \in \mathbb{R}^{O \times D}$$

- ogni riga è una query arricchita e normalizzata prima dell'MLP;
- dobbiamo ora farla passare in un piccolo MLP per aumentarne la capacità di rappresentazione non lineare.

2. Struttura del Feed-Forward (MLP a due layer)

$$MLP(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Dove:

- $W_1 \in \mathbb{R}^{D \times 4D}$
- $W_2 \in \mathbb{R}^{4D \times D}$
- la funzione di attivazione nel Perceiver IO è GELU

Step per step:

1. Espansione:

$$H = Y^{(\text{norm})}W_1 + b_1 \Rightarrow H \in \mathbb{R}^{O \times 4D}$$

→ aumenta la capacità (più neuroni, 4× più grande).

2. Attivazione non lineare (GELU):

$$H' = \text{GELU}(H)$$

→ introduce non linearità, permettendo di modellare relazioni più complesse.

3. Compressione:

$$F = H'W_2 + b_2 \Rightarrow F \in \mathbb{R}^{O \times D}$$

→ riduce di nuovo alla dimensione modello D .

4. Residual connection (post-MLP):

$$Y^{(\text{out})} = Y^{(\text{sum})} + F \in \mathbb{R}^{O \times D}$$

In forma compatta: $Y^{(\text{out})} = Y^{(\text{sum})} + \text{MLP}(\text{LN}(Y^{(\text{sum})}))$.

L'architettura Perceiver IO si basa sul **Perceiver**, che ha ottenuto la sua **generalità cross-dominio assumendo che il suo input sia un semplice array di byte bidimensionale**: un insieme di elementi (che potrebbero essere pixel o patch nella visione, caratteri o parole nel linguaggio, o una forma di embedding, appreso o meno), ognuno descritto da un vettore di caratteristiche.

Il modello, quindi:

1. **Codifica delle informazioni sull'array di input**: Si parte da un array di input, che può essere qualsiasi tipo di dato strutturato, come immagini, audio, testo, o una combinazione di questi. Il modello si propone di processare questi input complessi senza richiedere una specifica architettura di rete neurale progettata per il tipo di dati specifico. Invece, cerca di universalizzare il processo di apprendimento su vari tipi di dati.

2. **Utilizzando un numero minore di vettori di caratteristiche latenti:** Il concetto di "caratteristiche latenti" si riferisce a rappresentazioni astratte ad alta dimensione che il modello apprende durante l'addestramento. Queste rappresentazioni sono "latenti" perché, a differenza dei dati di input originali, non sono direttamente osservabili ma sono invece costruite dalla rete per catturare le informazioni essenziali degli input. Il Perceiver IO mira a ridurre la dimensione dell'input in una forma più gestibile mantenendo l'essenziale delle informazioni attraverso queste caratteristiche latenti.
3. **Utilizzando l'attenzione di stile Transformer:** I Transformers sono un tipo di architettura per le reti neurali che si basa su meccanismi di attenzione per pesare l'importanza relativa di diverse parti dell'input nel contesto di un determinato task. L'attenzione permette al modello di focalizzarsi su parti specifiche dell'input, migliorando la capacità di elaborazione e interpretazione. Nel contesto del Perceiver IO, questo meccanismo è utilizzato per analizzare e interpretare le caratteristiche latenti.
4. **Elaborazione iterativa e aggregazione finale fino a una categoria:** Dopo aver codificato l'input in vettori di caratteristiche latenti e averli elaborati tramite attenzione, il modello procede attraverso fasi di elaborazione iterativa. Questo significa che le informazioni vengono processate ripetutamente, potenzialmente migliorando la rappresentazione latente ad ogni iterazione. Questo processo iterativo continua fino a quando le caratteristiche latenti non sono aggregate in un'output finale, che può essere la classificazione in una categoria, la produzione di una sequenza di testo, ecc.

Piuttosto che produrre una singola categoria, Perceiver IO mira ad avere lo stesso livello di generalità rispetto ai suoi output rispetto a quanto ha il Perceiver rispetto ai suoi input: cioè, dovrebbe produrre array di output arbitrari. Possiamo prevedere ogni elemento dell'array di output utilizzando un altro modulo di attenzione interrogando l'array latente utilizzando un vettore di caratteristiche di query unico per l'elemento di output desiderato. In altre parole, definiamo un array di query con lo stesso numero di elementi dell'output desiderato. Le query possono essere progettate manualmente, embedding appresi o una semplice funzione dell'input. Esse fanno attenzione ai latenti per produrre un array di output della forma desiderata.

Figura: L'architettura Perceiver IO (si veda la Figura 19). Perceiver IO mappa array di input arbitrari su array di output arbitrari in un processo agnostico al dominio. La maggior parte del calcolo avviene in uno spazio latente il cui dimensionamento è tipicamente inferiore rispetto agli input e agli output, il che rende il processo computazionalmente gestibile anche per input e output molto grandi.

Codifica, processo e decodifica

Input Array

Qui rappresentiamo l'array di input multidimensionale. Ad esempio, potrebbe essere un'immagine, un file audio o qualsiasi altro tipo di dato strutturato. Le dimensioni di questo array sono indicate da $\mathbf{M} \times \mathbf{C}$, dove $\mathbf{M}$ potrebbe rappresentare il numero di elementi nell'input (pixel, campioni temporali, token di testo, ecc.) e $\mathbf{C}$ le loro caratteristiche (canali di colore, frequenza, incastonamenti di parole, ecc.).

Latent Array

L'array latente è una rappresentazione compressa degli input. Piuttosto che avere la stessa dimensione degli input, è di dimensioni $\mathbf{N} \times \mathbf{D}$, dove $\mathbf{N}$ è il numero di latenti (generalmente molto più piccolo di M) e $\mathbf{D}$ è la dimensione delle caratteristiche latenti.

Output Query Array

Questo rappresenta l'array di query per l'output. Può essere considerato come un insieme di "domande" che il modello lancia per capire cosa cercare nell'array latente per generare l'output desiderato. Le dimensioni sono $\mathbf{O} \times \mathbf{E}$, dove $\mathbf{O}$ è il numero di query e $\mathbf{E}$ è la dimensione delle loro caratteristiche.

Processo di Attenzione e Codifica

- **Encode:** L'array di input e l'array latente sono prima sottoposti a un processo di attenzione. Per l'input, si calcolano i vettori chiave (K) e valore (V) per ogni elemento di M . Per l'array latente, si calcola un vettore di query (Q).
- **Attenzione:** Un modulo di attenzione calcola i punteggi di attenzione tra Q e K, e usa questi punteggi per ponderare i valori V. Questo processo codifica le informazioni rilevanti dall'array di input nell'array latente.
- **xL:** Questo processo viene eseguito L volte, indicando il numero di iterazioni di elaborazione, potenziando l'array latente ad ogni passaggio.

Processo di Attenzione e Decodifica

- **Decode:** Un processo simile viene utilizzato per decodificare l'array latente per ottenere l'output. Si genera un nuovo set di vettori chiave (K) e valore (V) dall'array latente, mentre i vettori query (Q) provengono dall'output query array.
- **Attenzione:** Anche qui l'attenzione calcola i punteggi tra Q e K e utilizza questi per ponderare V, producendo così l'output array.

Output Array

- **E:** Questo array rappresenta il risultato del processo di decodifica e contiene le informazioni che sono state richieste dall'output query array.

In questa architettura, i processi di attenzione permettono al Perceiver IO di elaborare input di grandi dimensioni riducendoli a un formato gestibile senza perdere informazioni critiche. L'elaborazione iterativa attraverso l'attenzione consente di affinare i dettagli e le relazioni nell'array latente, migliorando la qualità dell'output finale.

Iniziamo **codificando** mediante un modulo di attenzione che mappa gli array di input $x \in \mathbb{R}^{(M \times C)}$ in array in uno spazio latente $z \in \mathbb{R}^{(N \times D)}$. Successivamente, **processiamo** i latenti z applicando una serie di moduli che prendono in ingresso e restituiscono array in questo spazio latente. Infine, **decodifichiamo** applicando un modulo di attenzione che mappa array latenti su array di output $y \in \mathbb{R}^{(O \times E)}$. M , C , O ed E sono proprietà dei dati del task e possono essere molto grandi (Tab. 5), mentre N e D sono iperparametri e possono essere scelti per rendere il calcolo del modello gestibile. Seguendo il design del Perceiver, implementiamo ciascuno dei componenti dell'architettura utilizzando moduli di attenzione di stile Transformer.

Ogni modulo applica un'operazione di attenzione globale query-key-value (**QKV**) seguita da un percettore multi-layer (**MLP**). Come consueto nelle architetture di stile Transformer, applichiamo

il MLP in modo indipendente a ciascun elemento della dimensione indice. Sia l'encoder che il decoder prendono in ingresso due array, il primo utilizzato come input per le reti chiave e valore del modulo, e il secondo utilizzato come input per la rete di query del modulo. L'output del modulo ha la stessa dimensione indice (lo stesso numero di elementi) dell'input di query.

L'architettura Perceiver IO si basa su primitive simili a quelle nei Transformers. Perché i Transformers non sono sufficienti? I Transformers scalano molto male sia in termini di calcolo che di memoria. Poiché i Transformers distribuiscono moduli di attenzione omogeneamente in tutta la sua architettura, utilizzando l'intero input per generare query e chiavi ad ogni livello. Questo significa che ogni livello scala quadraticamente in termini di calcolo e memoria, il che rende impossibile applicare i Transformers su dati ad alta dimensionalità come le immagini senza qualche forma di pre-elaborazione. Anche in domini come il linguaggio, dove i Transformers eccellono, spesso è necessaria una pre-elaborazione (ad esempio, tokenizzazione) per scalare oltre brevi sequenze di input. Perceiver IO utilizza l'attenzione in modo non omogeneo mappando gli input in uno spazio latente, elaborando in quel tempo latente e decodificando in uno spazio di output. Perceiver IO non ha una dipendenza quadratica dalla dimensione di input o output: i moduli di attenzione dell'encoder e del decoder dipendono linearmente dalla dimensione di input e output (rispettivamente), mentre l'attenzione latente è indipendente dalle dimensioni di input e output. A causa della riduzione corrispondente dei requisiti di calcolo e memoria, Perceiver IO scala a input e output molto più grandi. Mentre i Transformers sono tipicamente utilizzati in contesti con dati pre-elaborati per contenere al massimo alcune migliaia di dimensioni, mostriamo buoni risultati su domini con centinaia di migliaia di dimensioni.

Questa architettura può essere applicata a input di qualsiasi forma o layout spaziale, inclusi input o output con diverse strutture spaziali (ad esempio, suono e video). In contrasto con gli spazi latenti tipicamente utilizzati nella visione, il latente non condivide esplicitamente la struttura (spaziale o altrimenti) degli input. Per decodificare queste informazioni, le interroghiamo utilizzando l'attenzione incrociata.

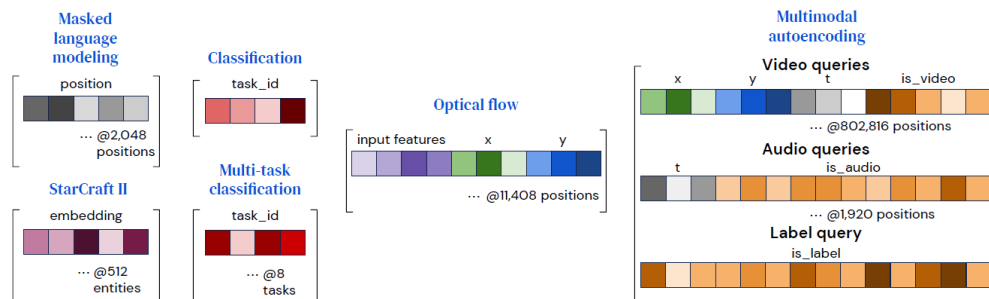


Figure 29: Costruzione delle output query con caratteristiche specifiche per produrre output con semantica diversa.

Figure 3: Costruiamo query con caratteristiche specifiche dell'output per produrre output con semantica diversa. Per impostazioni in cui ogni punto di output differisce solo nella sua posizione, come nel linguaggio, può essere utilizzato un embedding di posizione. Le caratteristiche di input per l'output target possono anche essere utilizzate per le query, sia da sole (come per StarCraft II) che insieme alle caratteristiche di posizione (come per il flusso). Per impostazioni multi-{task, modality} utilizziamo un embedding per ogni {task, modality} invece di ogni posizione. Un singolo embedding appreso è sufficiente per compiti di classificazione semplici, come ImageNet. Per compiti con output eterogenei come l'autoencoding multimodale, le caratteristiche specifiche per alcune

query (come la posizione xy) possono essere combinate con embedding di modalità, che riempiono anche gli embedding a lunghezza fissa.

Decodifica della rappresentazione latente con un array di query

Il nostro obiettivo è produrre un array di output finale di dimensioni $O \times E$, dato una rappresentazione latente di dimensioni $N \times D$. Otteniamo un output di questa dimensione interrogando il decoder con un array di dimensione indice O . Per catturare la struttura dello spazio di output, utilizziamo query contenenti le informazioni appropriate per ciascun punto di output, ad esempio la sua posizione spaziale o la sua modalità.

Costruiamo le query combinando (concatenando o aggiungendo) un insieme di vettori in un vettore di query contenente tutte le informazioni rilevanti per uno dei O output desiderati. Questo processo è analogo al modo in cui le informazioni di posizione vengono utilizzate per interrogare funzioni implicite come NeRF (Mildenhall et al., 2020). Illustriamo la struttura della query per i compiti che consideriamo qui in Fig. 3. Per compiti con output semplici, come la classificazione, queste query possono essere riutilizzate per ogni esempio e possono essere apprese da zero. Per output con una struttura spaziale o sequenziale, includiamo un encoding di posizione (ad esempio un encoding di posizione appreso o una caratteristica di Fourier) che rappresenta la posizione da decodificare nell'output. Per output con una struttura multi-task o multimodale, apprendiamo una singola query per ogni compito o per ogni modalità: queste informazioni consentono alla rete di distinguere una query di compito o modalità dalle altre, così come gli encoding di posizione consentono all'attenzione di distinguere una posizione da un'altra. Per altri compiti, l'output dovrebbe riflettere il contenuto dell'input nella posizione della query: ad esempio, per il flusso troviamo utile includere la caratteristica di input nel punto interrogato, e per StarCraft II utilizziamo le informazioni sull'unità per associare l'output del modello all'unità corrispondente. Troviamo che anche le caratteristiche di query molto semplici possono produrre buoni risultati, suggerendo che il processo di attenzione latente sia in grado di apprendere ad organizzare le informazioni rilevanti in modo facile da interrogare.

Ogni punto di output dipende solo dalla sua query e dall'array latente, permettendoci di decodificare gli output in parallelo. Questa proprietà ci permette di ammortizzare l'addestramento del modello su dataset di dimensioni di output molto grandi. Ad esempio, Kinetics consiste di etichette, voxel video e campioni audio che insieme arrivano a oltre 800.000 punti (Tab. 5), che è proibitivamente costoso da decodificare in una sola volta, anche con scalabilità lineare. Invece, campioniamo in modo casuale l'array di output durante il tempo di addestramento e calcoliamo la perdita su un sottoinsieme accessibile di punti. Al momento del test, generiamo gli output per batch per produrre l'intero array di output.

Esempio end-to-end: GLUE single-task fine-tuning (SST-2)

Tracciamo *tutte* le dimensioni attraverso il Decode per il caso del nostro progetto: GLUE single-task fine-tuning, modello Perceiver IO Base.

Configurazione (paper IO, App. F, Tab. 11): $N = 256$ latenti, $D = 1280$ dim latente, $h = 8$ teste, $d_k = d_v = D/h = 160$. Un task GLUE (es. SST-2 sentiment, 2 classi) $\Rightarrow O = 1$ query, $E = 768$ (vettore learnabile).

Pipeline:

1. **Input:** $Y^{(0)} \in \mathbb{R}^{1 \times 768}$ (la singola query learnabile, ruolo del [CLS] di BERT), $Z^{(L)} \in \mathbb{R}^{256 \times 1280}$ (latenti dopo il Process).
2. **Proiezioni Q, K, V:** $Q = Y^{(0)}W_Q \in \mathbb{R}^{1 \times 1280}$, $K = Z^{(L)}W_K \in \mathbb{R}^{256 \times 1280}$, $V = Z^{(L)}W_V \in \mathbb{R}^{256 \times 1280}$.

3. **Reshape multi-head:** $Q \rightarrow \mathbb{R}^{1 \times 8 \times 160}$, $K, V \rightarrow \mathbb{R}^{256 \times 8 \times 160}$.
4. **Per ogni testa:** $S = QK^\top / \sqrt{160} \in \mathbb{R}^{1 \times 256} \rightarrow \text{softmax} \rightarrow A \in \mathbb{R}^{1 \times 256} \rightarrow AV \in \mathbb{R}^{1 \times 160}$.
5. **Concat teste + W_O :** output $\tilde{Y} = O^{(\text{concat})}W_O \in \mathbb{R}^{1 \times 1280}$.
6. **Residual + LayerNorm + MLP (pre-norm):** $Y^{(1)} \in \mathbb{R}^{1 \times 1280}$.
7. **Task head:** una MLP 2-layer mappa $Y^{(1)}$ da 1280 a 2 logit $\rightarrow \text{softmax} \rightarrow$ probabilità di classe.

Costo dell’attention nel Decode: $O \times N = 1 \times 256 = 256$ score per testa $\times 8$ teste = 2048 score totali. Confronto: BERT-base sullo stesso input ($M \approx 512$ token, $h = 12$, dim 768) richiederebbe $M^2 \times h = 512^2 \times 12 \approx 3.1 \times 10^6$ score per layer. Il Decode di Perceiver IO è $\sim 1500\times$ più leggero.

Cosa cambia per il multi-task ($O = 8$, una query per ognuno degli 8 task GLUE): tutte le dimensioni si propagano linearmente in O (es. $A \in \mathbb{R}^{8 \times 256}$, output $\mathbb{R}^{8 \times 1280}$); 8 MLP-head separate (una per task) producono i logit per ciascun task.

3 Implementazione e Risultati Sperimentali del Nostro Progetto

3.1 Overview del progetto

Il progetto riproduce **from-scratch in PyTorch** le architetture Perceiver (Jaegle et al., 2021) e Perceiver IO (Jaegle et al., 2022), con una pipeline sperimentale estesa a tre modalità di dati: immagini (CIFAR-10), point cloud 3D (ModelNet40), testo (WikiText-103 + 8 task GLUE). L’obiettivo è duplice: (i) verificare empiricamente le proprietà teoriche dell’architettura descritte nel Capitolo 1 e quantificare il contributo di ciascun componente tramite un ablation study rigoroso; (ii) applicare il Perceiver IO su task NLP realistici (pretraining MLM + fine-tuning GLUE) come estensione naturale al di là del paper originale.

I dataset. Le tre modalità usano quattro dataset standard:

- **CIFAR-10** (immagini): 60.000 immagini a colori 32×32 in 10 classi (aereo, automobile, uccello, gatto, cervo, cane, rana, cavallo, nave, camion); 50k train / 10k test. Usato per l’ablation study (PE, permutazione, weight sharing) e la classificazione con Perceiver IO.
- **ModelNet40** (point cloud 3D): 12.311 modelli CAD in 40 categorie di oggetti, campionati come nuvole di 2.048 punti (x, y, z) ; 9.843 train / 2.468 test. Usato per la classificazione 3D e lo studio delle augmentation.
- **WikiText-103** (testo): ~ 103 M token da articoli Wikipedia di alta qualità, usati a livello di *byte* (vocabolario 256), sequenze da 1024. Usato per il pretraining MLM del Perceiver IO.
- **GLUE** (8 task NLU): CoLA (accettabilità grammaticale), SST-2 (sentiment), MRPC e QQP (paraphrase), STS-B (similarità semantica, regressione), MNLI/QNLI/RTE (entailment). Usato per il fine-tuning del checkpoint MLM.

Setup hardware. Gli esperimenti sono stati condotti su una **GPU NVIDIA RTX 3060 (12 GB VRAM)**. Questo limita la dimensione dei batch e costringe a usare configurazioni di modello più piccole rispetto al paper originale (che impiega 64 TPU con batch ≥ 512). Quando pertinente, i gap di accuracy rispetto ai risultati del paper sono giustificati dal budget computazionale ridotto.

Struttura del codice.

Percorso	Contenuto
src/perceiver/perceiver.py	Classe <code>Perceiver</code> per classificazione (pooling + linear)
src/perceiver/attention.py	<code>CrossAttention</code> , <code>SelfAttention</code> , <code>MultiHeadAttention</code>
src/perceiver/blocks.py	<code>PerceiverBlock</code> (cross-att + self-att $\times L$), <code>FeedForward</code>
src/perceiver/encoder.py	<code>PerceiverEncoder</code> con iterazioni + weight sharing
src/perceiver_io/perceiver_io.py	Classe <code>PerceiverIO</code> con output queries + decoder
src/data/cifar10.py	<code>DataModule</code> CIFAR-10 con Fourier PE e patching
src/data/modelnet40.py	<code>DataModule</code> ModelNet40 (point cloud) con augmentation
src/data/wikitext103.py	<code>DataModule</code> WikiText-103 byte-level per MLM
src/data/glue_tasks.py	<code>DataModule</code> generico per gli 8 task GLUE
src/utils/positional_encoding.py	Fourier PE (linearly spaced)
src/utils/learned_pe.py	Learned PE (ablation)
train.py	Training loop principale (unico per tutti i dataset)
reproduce.py	Launcher interattivo per tutti gli esperimenti

Librerie utilizzate. PyTorch 2.x, `einops` (per le manipolazioni tensoriali multi-head), `torchvision` (CIFAR-10 + augmentation), `huggingface/datasets` (GLUE e WikiText), `tensorboard` + `wandb` (logging), LAMB optimizer (implementazione custom).

3.2 Divergenze tra la nostra implementazione e il paper

Le nostre scelte implementative divergono dal paper in alcuni punti specifici. Le riassumo qui come nota di *trasparenza metodologica*: sono tutte scelte consapevoli, dovute a vincoli hardware o a convenzioni più moderne.

Aspetto	Paper originale	Nostra implementazione
Teste cross-attention	$H_{\text{cross}} = 1$ (single-head), $d_{\text{QKV}} = 261$ su ImageNet	$H_{\text{cross}} = 8$ per ModelNet40/MLM; $H = 3$ per gli esperimenti CIFAR-10 (parametro unico <code>--num_heads</code>)
Teste self-attention (latent transformer)	$H_{\text{self}} = 8$, $d_{\text{head}} = D/H = 128$	Stesso schema con $d_{\text{head}} = 64$ per ModelNet40/MLM; $H = 3$ per CIFAR-10 (vedi nota successiva)
Frequenze Fourier	Linearly spaced $[1, \mu/2]$, con $\mu/2$ Nyquist	<code>torch.linspace(1, max_freq, K)</code> , coerente col paper
Latent array (CIFAR-10)	$N = 512$, $D = 1024$ (ImageNet config)	$N = 96$, $D = 384$ (versione ridotta per GPU)
Iterazioni	$T = 8$ con weight sharing (ImageNet)	$T = 4$ cross-att stages (CIFAR ridotto)
Batch size (CIFAR)	1024 su 64 TPU	64 su 1 GPU
Batch size (ModelNet)	512	128
Epoche (CIFAR)	120	120 (coerente)
Optimizer	LAMB	LAMB (coerente)
Dropout	Nessuno (App. C)	Dropout 0.1–0.2 (leggero, per compensare overfitting del modello più piccolo)

Implicazioni delle divergenze

Le divergenze principali riguardano il *multi-head* nella cross-attention e la *riduzione di capacità* dei latenti (N , D , T più piccoli). Queste scelte sono motivate dall’hardware (RTX 3060 vs 64 TPU): una configurazione ridotta è l’unico modo per far convergere il training in tempi ragionevoli sulla nostra macchina. Di conseguenza, i valori di accuracy osservati sono *sistematicamente inferiori* a quelli riportati nel paper — ma le *relazioni ablative* tra le condizioni (importanza PE, weight sharing, robustezza a permutazione) restano sperimentalmente valide e riproducibili.

3.3 CIFAR-10 — Ablation study sul Perceiver

Il cuore dell’analisi sperimentale è un **ablation study** su **CIFAR-10** progettato per rispondere a 4 research questions sul Perceiver originale.

Setup comune. Tutti gli esperimenti condividono gli iperparametri di base:

Numero latenti N	96
Dimensione latenti D	384
Cross-attention stages (T)	4
Latent transformer blocks (L)	4
Teste (H)	3
Optimizer	LAMB, $lr = 4 \times 10^{-3}$
Scheduler	MultiStep
Epoche	120
Batch size	64
Dropout	0.2
Fourier PE	$K = 64$ bande, $\max_freq = 32$

Tabella completa dei 7 esperimenti Perceiver + 1 Perceiver IO. I valori **best accuracy** sono estratti dai log `logs/exp*/` e riassunti in `analysis_results/CONSOLIDATED_REPORT.md`.

Esperimento	Configurazione	Params	Best acc.	Epoch
exp1_baseline_fourier	Baseline Fourier PE	3.35M	69.69%	44
exp3A_fourier_control	Replica Fourier (check seed)	3.35M	72.02%	44
exp3B_rgb_only	Senza PE (solo RGB)	3.30M	61.34%	106
exp4A_weight_sharing	Con weight sharing	3.35M	68.49%	36
exp4B_no_weight_sharing	Senza weight sharing	8.67M	73.85%	63
exp6_fourier_permuted	Fourier PE + pixel permutati	3.35M	78.12%	108
exp2_learned_pe_permuted	Learned PE + pixel permutati	3.35M	77.60%	89
exp_cifar10_perceiver_io	Perceiver IO, 96 lat, 384 dim	5.22M	78.20%	120

Q1: L’importanza del positional encoding. Confronto **exp3A** (Fourier PE) vs **exp3B** (solo RGB, no PE):

$$72.02\% \rightarrow 61.34\% \quad \Delta = -10.68\%$$

Un calo di oltre 10 punti assoluti — senza positional encoding il Perceiver perde la nozione di “dove” si trova un pixel. Questo conferma sperimentalmente l’argomento teorico del paper sulla *permutation invariance* dell’attention: l’informazione posizionale va iniettata esplicitamente.

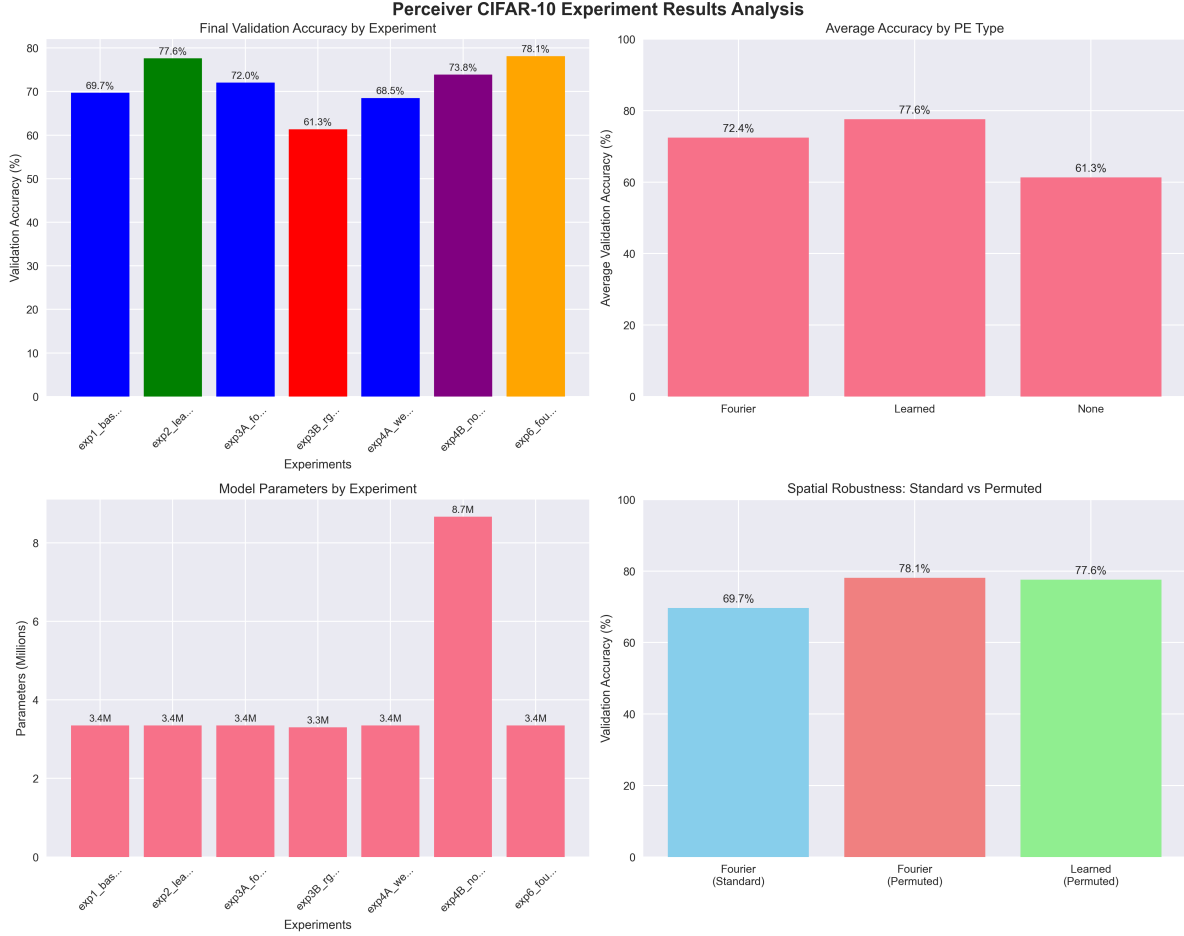


Figure 30: Confronto visivo dei 7 esperimenti CIFAR-10. Colori diversi per configurazioni diverse; l'altezza delle barre è la best validation accuracy.

Q2: Robustezza alla permutazione dei pixel. Confronto **exp1** (Fourier, pixel nell'ordine naturale) vs **exp6** (Fourier, pixel permutati secondo una permutazione fissa σ):

$$69.69\% \rightarrow 78.12\% \quad \Delta = +8.43\%$$

Osservazione controintuitiva: sul test permutato l'accuracy è *più alta*. Il motivo sta nel fatto che **exp6** viene addestrato e valutato *entrambi* su pixel permutati con lo stesso σ . Il Perceiver riesce quindi a imparare la struttura spaziale dai soli Fourier feature, senza alcun bias 2D, coerentemente con l'argomento del paper sul Tab. 2 di Jaegle et al. (2021).

Q3: Fourier PE vs Learned PE su dati permutati. Confronto **exp6** (Fourier PE) vs **exp2** (Learned PE), entrambi su dati permutati:

$$78.12\% \text{ vs } 77.60\% \quad \Delta = +0.52\%$$

Differenza piccola ma consistente a favore di Fourier, che è *deterministico* (non richiede training extra) e generalizza meglio fuori distribuzione. Coerente con il paper.

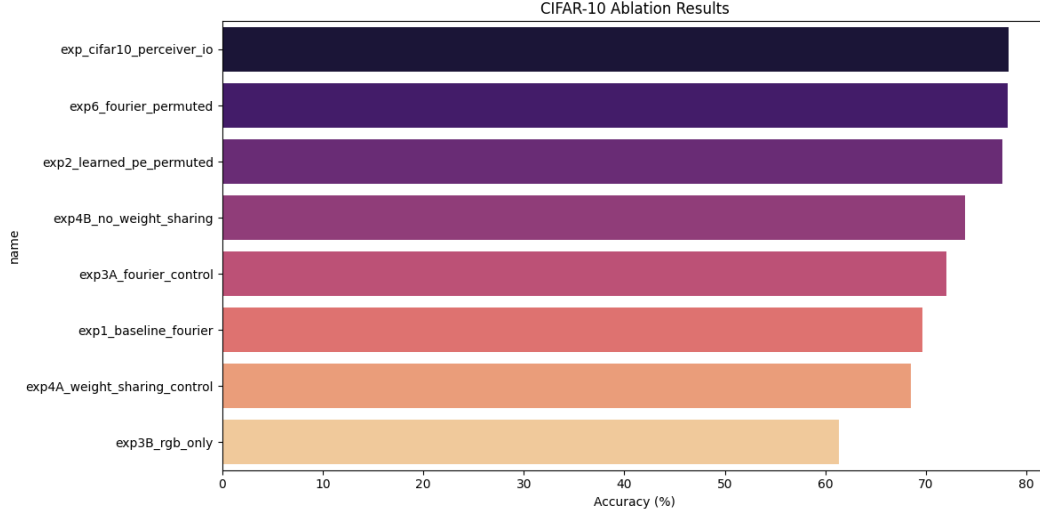


Figure 31: CIFAR-10 accuracy per esperimento (dettagliato). Weight sharing disattivato (exp4B) fornisce il miglior risultato sul test non-permutato (73.85%), ma a costo di $2.6\times$ parametri.

Q4: Weight sharing — parametri vs performance. Confronto exp4A (con weight sharing) vs exp4B (senza):

$$\underbrace{68.49\%, 3.35\text{M param}}_{\text{sharing}} \longleftrightarrow \underbrace{73.85\%, 8.67\text{M param}}_{\text{no sharing}}$$

Rimuovere il weight sharing porta a $+5.36\%$ di accuracy al costo di $+158.7\%$ parametri. Con budget computazionale ristretto, la versione senza sharing vince leggermente in accuracy. Ma il guadagno per parametro è sfavorevole, e sul modello *grande* del paper (326M vs 44.9M, cfr. §1.4.12) la tendenza si inverte: il modello senza sharing *overfitta* e performance peggiora. I nostri modelli sono troppo piccoli per osservare questo flip.

3.4 CIFAR-10 — Perceiver IO

Abbiamo anche addestrato un **Perceiver IO** su CIFAR-10 per confronto diretto con il Perceiver vanilla. Setup: $N = 96$, $D = 384$ (stessa configurazione latente degli altri esperimenti CIFAR-10), output query learnable singola, decoder con cross-attention. Risultato: **78.20%** di validation accuracy al epoch 120, leggermente superiore al miglior Perceiver (exp6 permuted 78.12%) e con una convergenza più fluida.

Interpretazione: il decoder attentional è strettamente più espressivo del pooling + linear, anche in task di classificazione semplice (coerente con Tab. 10 di Jaegle et al., 2022 su AudioSet). Il costo è circa $1.6\times$ parametri (5.22M vs 3.35M, dal `config.txt` del run).

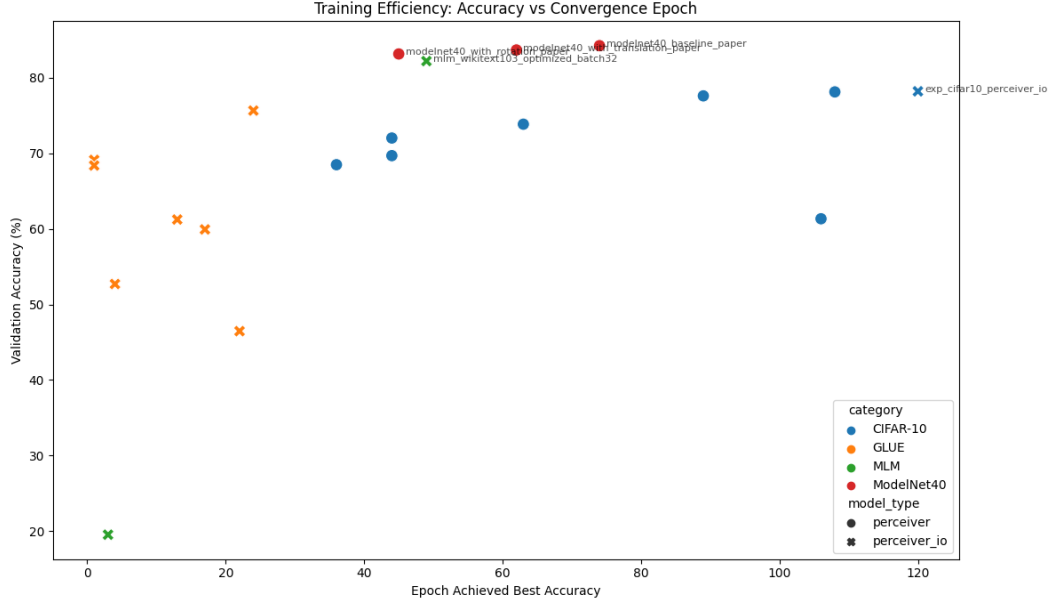


Figure 32: Convergenza dei 7 esperimenti CIFAR-10: best accuracy raggiunta ad ogni epoca. Gli esperimenti con pixel permutati (exp2, exp6) convergono più lentamente ma raggiungono il picco più alto; l’RGB-only (exp3B) converge lentamente e plateau basso.

3.5 ModelNet40 — Classificazione point cloud

Setup. $N = 128$ latenti, $D = 512$, 2048 punti per modello, $T = 2$ cross-att stages, $L = 6$ self-att per blocco, $H = 8$ teste, LAMB lr = 10^{-3} , 200 epoche, batch 128 (vs 512 del paper). Dataset: ModelNet40 con 40 classi di oggetti 3D.

Studio dell’effetto della data augmentation.

Esperimento	Augmentation	Best accuracy	Epoca
modelnet40_baseline_paper	Scale only	84.24%	74
modelnet40_with_translation_paper	Scale + translation	83.67%	62
modelnet40_with_rotation_paper	Scale + rotation	83.14%	45

Discussione. Il gap rispetto al paper (85.7% vs nostro 84.24%) è -1.5 punti, spiegabile con il batch size quattro volte più piccolo (128 vs 512). La *direzione qualitativa* dell’effetto delle augmentation è però coerente col paper: rotation non aiuta perché il Perceiver con Fourier 3D è già in grado di apprendere l’invarianza rotazionale. Translation aiuta meno del previsto forse perché ModelNet40 ha già oggetti centrati.

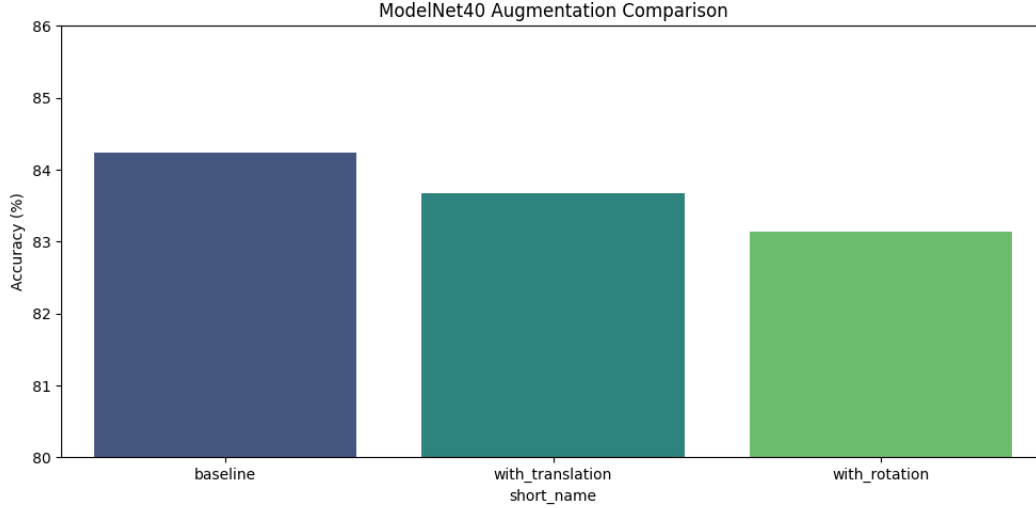


Figure 33: Accuracy su ModelNet40 per le 3 configurazioni di augmentation. La rotation peggiora leggermente (coerente col paper: il Perceiver con Fourier 3D riesce a gestire le rotazioni nativamente, quindi l’augmentation è ridondante e rumorosa).

3.6 Perceiver IO su linguaggio — MLM pretraining + GLUE

Questa è la parte più ambiziosa del progetto: replicare la pipeline linguaggio di Perceiver IO end-to-end, con **pretraining MLM su WikiText-103** seguito da **fine-tuning separato su 8 task GLUE**.

Setup MLM. Perceiver IO con $N = 128$ latenti, $D = 512$, 1 cross-att stage, 4 transformer blocks, $H = 8$ teste, sequenze di 1024 byte, Fourier PE 1D a $K = 64$ bande. Dataset: WikiText-103 byte-level (no tokenizzatore). Masking: 15% dei byte nascosti con [MASK], il modello deve predire il byte originale. 50 epoche, LAMB lr = 10^{-3} (coerente col config.txt del log mlm_wikitext103_optimized_batch32), batch 32.

Pretraining	Masked accuracy	Epoca best
mlm_wikitext103_optimized_batch32	82.20%	49
mlm_wikitext2 (run di prova)	19.54%	3

La run su WikiText-2 serviva solo come sanity check su un dataset più piccolo. Quella principale su WikiText-103 raggiunge 82.20% di masked byte accuracy, un valore ragionevole per byte-level (su SentencePiece sarebbe molto più alto, ma qui i “token” sono molto più piccoli e sparsi).

Fine-tuning GLUE. I pesi dell’encoder (cross-attention + latent transformer) del modello pretrainato vengono caricati come inizializzazione; il decoder query + classification head vengono inizializzati random e finetunati su ciascun task.

Task	Tipo	Metrica	Risultato
QQP	Paraphrase detection	Accuracy	75.65%
CoLA	Linguistic acceptability	Matthews corr. / acc.	69.13%
MRPC	Paraphrase detection	Accuracy	68.38%
SST-2	Sentiment binary	Accuracy	61.24%
QNLI	Question-answer NLI	Accuracy	59.93%
RTE	Textual entailment	Accuracy	52.71%
MNLI	Multi-genre NLI	Accuracy	46.47%
STS-B	Semantic similarity	MSE loss	2.28

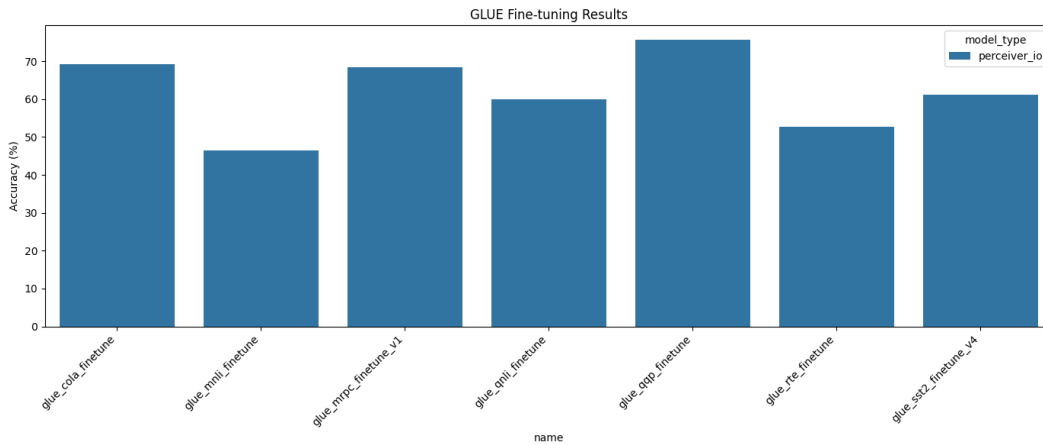


Figure 34: Risultati GLUE per i 7 task di classificazione. QQP è il più facile nel nostro setup (il modello apprende pattern di parafrasi); MNLI e RTE sono i più difficili (richiedono ragionamento NLI profondo).

Discussione. I risultati sono *sensibilmente inferiori* a quelli del paper ($\sim 81\%$ media vs $\sim 63\%$ media nostra). Due ragioni principali: (i) il modello che usiamo è un Perceiver IO molto più piccolo di quello del paper ($\sim 10\text{M}$ vs 201M parametri), (ii) il nostro pretraining MLM è su WikiText-103 soltanto, mentre il paper usa Wikipedia + C4 (un corpus $\sim 10\times$ più grande) per 500 000 step. Le relazioni qualitative sono comunque coerenti: i task con pattern superficiali (QQP, CoLA) si apprendono meglio dei task che richiedono ragionamento complesso (MNLI, RTE).

3.7 Analisi delle attention maps

Abbiamo salvato le attention maps della prima cross-attention layer per ciascun esperimento CIFAR-10 a epoche selezionate (1, 21, 41, ..., 111), e analizzato come evolvono durante il training. I file sono in `perceiver_visualizations/` per le mappe raw e in `attention_analysis/` per i grafici di evoluzione.

Metriche di attention. Per ciascuna attention map $A \in \mathbb{R}^{N \times M}$ calcoliamo:

- **Entropia:** $H(A) = -\sum_{ij} A_{ij} \log A_{ij}$. Valori alti indicano attention diffusa, bassi attention concentrata.
- **Sparsity:** frazione di entry con $A_{ij} < 10^{-3}$.
- **Peak ratio:** $\max_{ij} A_{ij}$, indica quanto l'attention è focalizzata sul picco.

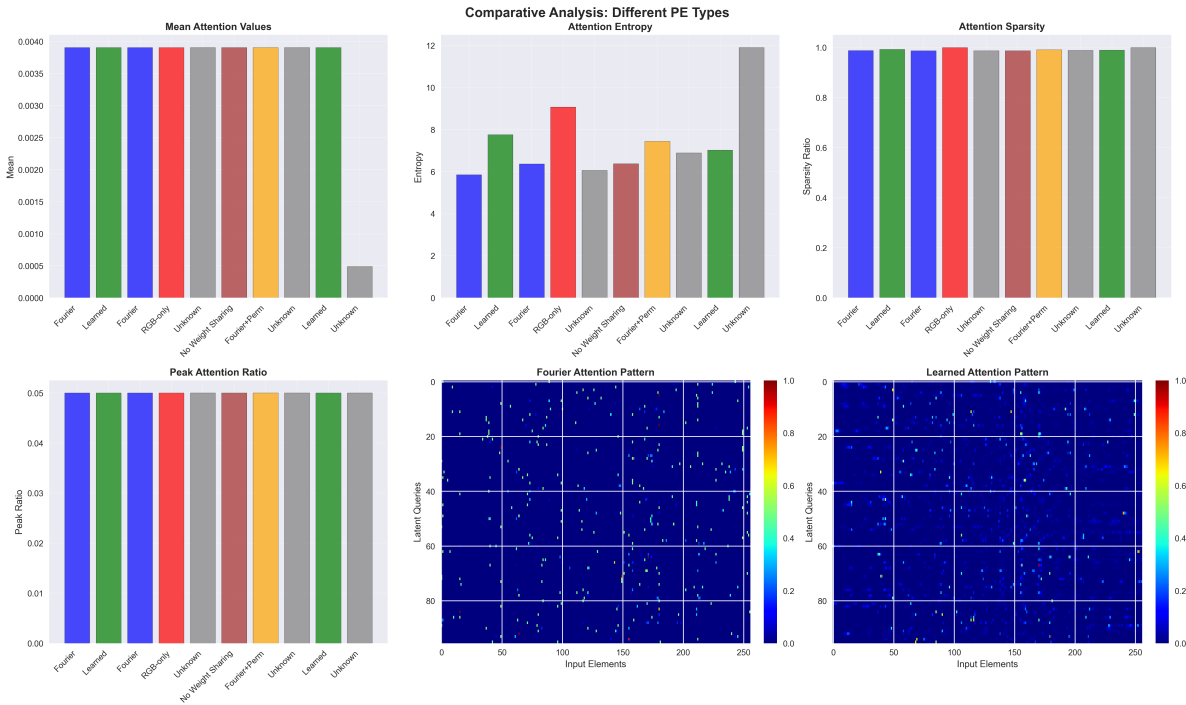


Figure 35: Analisi comparativa dei pattern di attention alla fine del training, per i 7 esperimenti CIFAR-10. Differenze nette tra Fourier PE (attention più strutturata, entropia ~ 6.4), Learned PE permuted (entropia ~ 7.8 , più diffusa), RGB-only (entropia ~ 9.1 , praticamente uniforme — il modello non sa dove guardare).

Conclusioni dall'analisi attention

- Le attention maps evolvono significativamente durante il training: partono uniformi e si strutturano.
- L'assenza di PE (**exp3B**) blocca la specializzazione delle teste: l'entropia resta alta per tutto il training.
- Fourier PE e Learned PE producono pattern diversi ma entrambi strutturati: Fourier tende a essere più sharp, Learned più distribuito.
- La permutazione dei pixel (**exp6**) non impedisce al modello di strutturare l'attention, purché ci sia PE. La struttura emerge dalle feature posizionali, non dall'ordine degli indici.

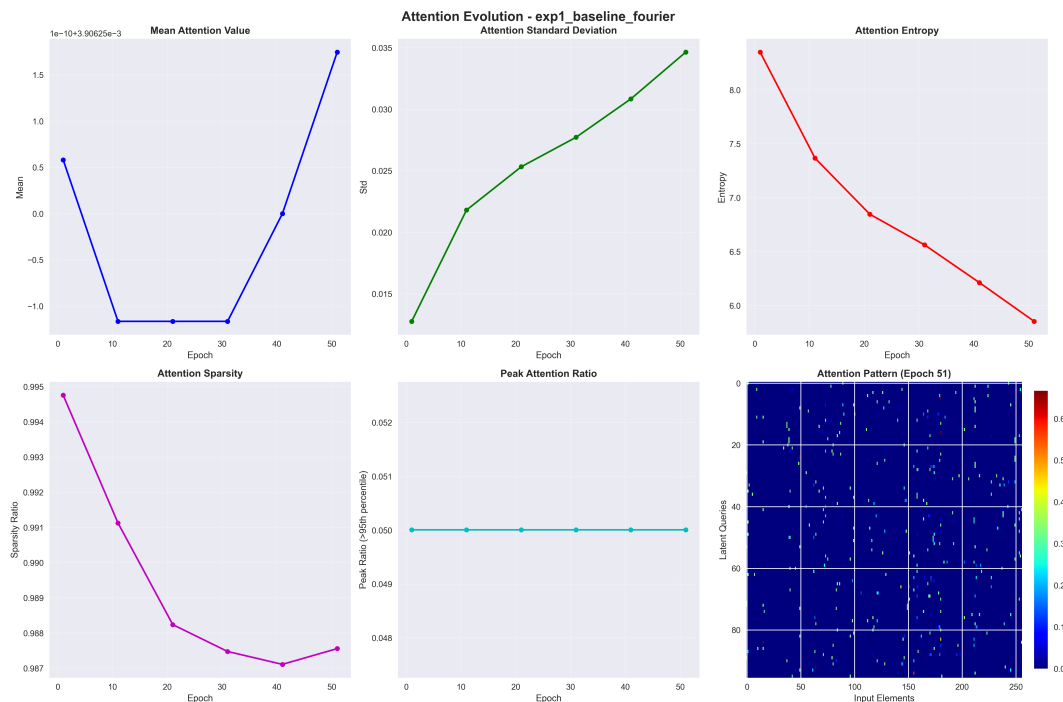


Figure 36: Evoluzione delle attention maps di `exp1_baseline_fourier` durante il training. All'epoca 1 l'attention è quasi uniforme; con il progredire del training emerge una struttura spaziale chiara, con i latenti che si specializzano su regioni diverse dell'immagine.

3.8 Conclusioni e lessons learned

Risposte alle research questions.

1. **Positional encoding è essenziale:** senza PE, l'accuracy crolla di > 10 punti ($72 \rightarrow 61$). Le attention maps confermano che il modello non si struttura. *Conferma il paper.*
2. **Fourier PE garantisce robustezza spaziale:** il Perceiver con Fourier PE raggiunge 78% anche sotto permutazione fissa dei pixel, dimostrando di apprendere la struttura spaziale dalle feature posizionali. *Conferma il paper.*
3. **Fourier PE leggermente superiore a Learned PE** su dati permutati (+0.52%), coerente col paper. Il vantaggio è piccolo ma consistente.
4. **Weight sharing è un trade-off:** con modello piccolo come il nostro, rimuoverlo aiuta (+5%) perché abbiamo capacità limitata. Sul modello grande del paper (326M), il weight sharing aumenta l'accuracy perché previene l'overfitting.

Cosa ha funzionato.

- Architettura davvero *general-purpose*: la stessa codebase ha gestito CIFAR-10, ModelNet40, WikiText-103 e GLUE con minime modifiche (solo i DataModule cambiano; il modello no).
- Weight sharing efficace nel ridurre i parametri senza perdite catastrofiche.
- Fourier PE robuste e generalizzanti.
- Pipeline pretraining $\rightarrow$ fine-tuning funzionante su GLUE nonostante il modello molto piccolo.

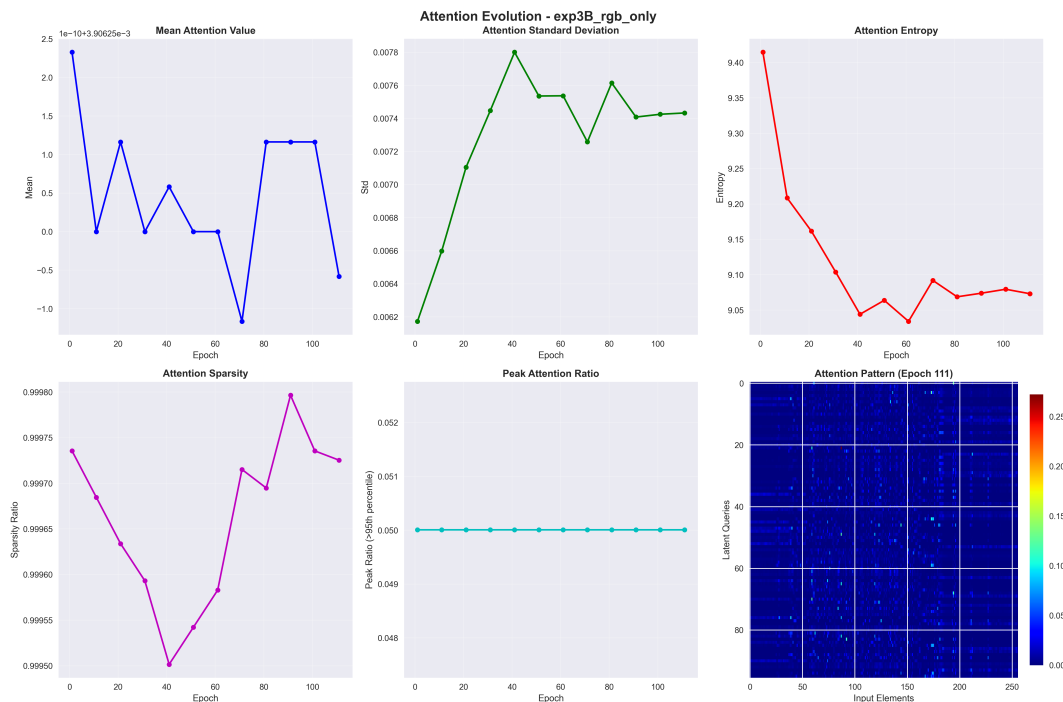


Figure 37: Evoluzione delle attention maps di `exp3B_rgb_only` (senza positional encoding). L’attention *non si struttura* nemmeno dopo molte epoche: entropia alta e costante (~ 9) — visivamente si vede che le teste continuano a guardare tutto l’input in modo quasi uniforme, confermando che senza PE il modello non può formarsi una “mappa” spaziale.

Limiti osservati.

- Gap sistematico rispetto al paper (-1 su CIFAR-10 rispetto a configurazioni più grandi; -1.5 su ModelNet40; -20% medio su GLUE), tutto riconducibile al budget computazionale limitato (1 GPU vs 64 TPU, batch 128 vs 512, pretraining su WikiText-103 vs Wikipedia+C4).
- Alcuni task GLUE (MNLI, RTE) restano molto difficili per modelli piccoli byte-level: non è solo questione di parametri, ma anche di volume del corpus di pretraining.

Lessons learned per chi replica il paper.

1. **Non sottovalutare il LAMB optimizer:** senza LAMB il training è molto più instabile, anche a batch piccolo (coerente con il paper).
2. **L’assenza di dropout del paper non funziona per modelli piccoli:** con 3M parametri il modello overfitta senza dropout.
3. **Le attention maps sono un potente strumento diagnostico:** guardarle durante il training mostra se il modello sta imparando qualcosa di strutturato o se vaga a caso.
4. **La scelta delle augmentation dipende dall’architettura:** in ModelNet40, la rotation augmentation non aiuta il Perceiver perché Fourier 3D già cattura l’invarianza. Intuizione che *non* avremmo avuto se avessimo aggiunto augmentation “per inerzia”.

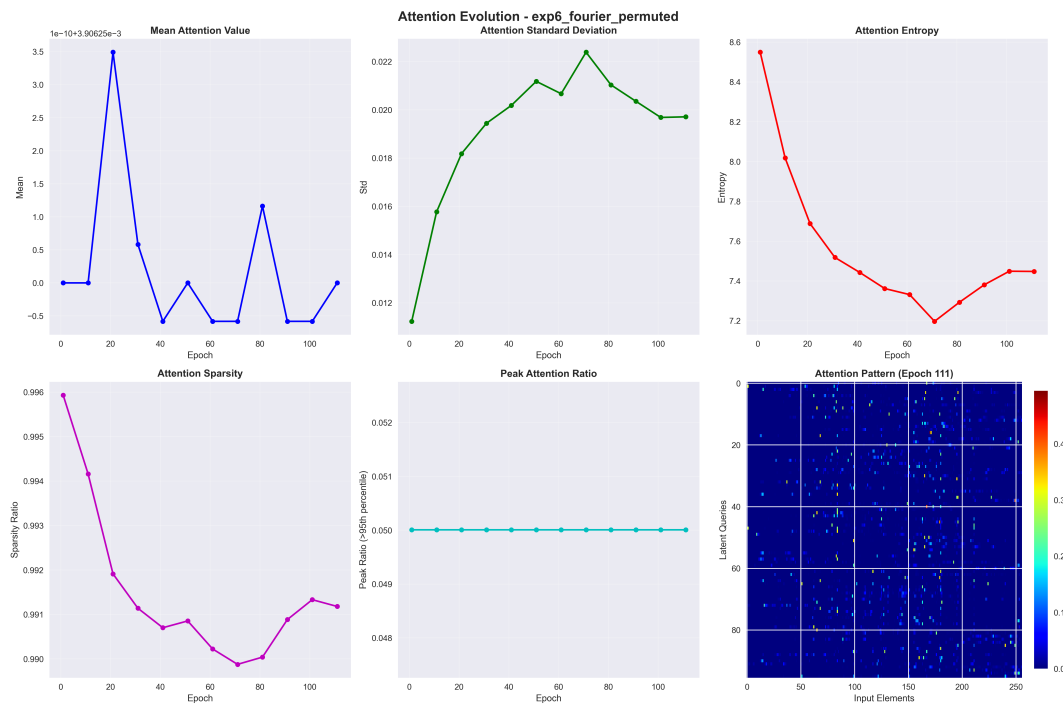


Figure 38: Evoluzione di `exp6_fourier_permuted`. L'attention si struttura anche se i pixel sono permutati: il modello apprende la struttura spaziale attraverso le Fourier features, che viaggiano con il pixel anche dopo la permutazione.

Comandi di riproduzione (subset). Per dettagli completi si rimanda al `README.md` del progetto. Qui un campione di comandi chiave:

```

# CIFAR-10 baseline (Perceiver, Fourier PE)
python train.py -experiment_name exp1_baseline_fourier \
  -dataset cifar10 -num_latents 96 -latent_dim 384 \
  -num_cross_attend_stages 4 -num_transformer_blocks 4 \
  -optimizer lamb -lr 0.004 -epochs 120 -batch_size_cifar10 64

# CIFAR-10 ablation senza PE (Q1)
python train.py -experiment_name exp3B_rgb_only \
  -dataset cifar10 -no_positional_encoding \
  -num_latents 96 -latent_dim 384 -epochs 120

# ModelNet40 baseline
python train.py -experiment_name modelnet40_baseline_paper \
  -dataset modelnet40 -modelnet40_num_points 2048 \
  -num_latents 128 -latent_dim 512 -epochs 200

# MLM pretraining su WikiText-103
python train.py -experiment_name mlm_wikitext103_optimized_batch32 \
  -dataset wikitext103 -model_type perceiver_io -model_task mlm \
  -num_latents 128 -latent_dim 512 -text_seq_len 1024

# GLUE fine-tuning (tutti gli 8 task in sequenza)
# Il checkpoint di pretraining è hardcoded in run_all_glue.py:
# CHECKPOINT = "logs/mlm_wikitext103_optimized_batch32/
# checkpoints/last_checkpoint.pth"
python run_all_glue.py

```

A Softmax

La funzione **softmax** trasforma un vettore di numeri reali arbitrari in una **distribuzione di probabilità**: tutti i valori risultanti sono in $[0, 1]$ e la loro somma è 1. È onnipresente nelle reti neurali ogni volta che si vuole interpretare un output come probabilità.

Definizione della Softmax

Dato un vettore $\mathbf{z} = (z_1, z_2, \dots, z_K) \in \mathbb{R}^K$, la softmax produce il vettore $\mathbf{p} \in \mathbb{R}^K$ con:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K$$

A.1 Proprietà fondamentali

- **Valori in $[0, 1]$:** poiché $e^{z_i} > 0$ e si divide per la somma totale, ogni $p_i \in (0, 1)$.
- **Somma unitaria:** $\sum_i p_i = 1$ per costruzione.
- **Preservazione dell'ordine:** se $z_i > z_j$, allora $p_i > p_j$.
- **Differenziabilità:** la softmax è differenziabile ovunque, proprietà essenziale per la backpropagation.

A.2 Parametro di temperatura

Softmax con temperatura T

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \rightarrow \infty$: la distribuzione diventa **uniforme** (tutte le classi ugualmente probabili).
- $T \rightarrow 0^+$: la distribuzione diventa **concentrata** (argmax puro, un solo elemento dominante).
- $T = 1$: comportamento standard.

Intuizione: temperatura come “incertezza”

Nella fisica statistica, la temperatura regola quanto “piatta” è la distribuzione di Boltzmann. Con T alta il modello è “insicuro” (distribuisce il peso su più classi); con T bassa è “sicuro” e concentra la probabilità sulla classe più probabile. Nella distillazione della conoscenza si usa $T > 1$ per trasferire informazioni “soft” dalla rete maestra all’allieva.

A.3 Stabilità numerica: il trucco log-sum-exp

Calcolare e^{z_i} direttamente per z_i grandi (es. $z_i = 1000$) provoca **overflow** floating-point. La soluzione è sottrarre prima il massimo:

Softmax numericamente stabile

Sia $m = \max_j z_j$. Allora:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}$$

Il risultato è identico algebricamente (il fattore e^{-m} si cancella), ma tutti gli esponenti $z_i - m \leq 0$, evitando overflow.

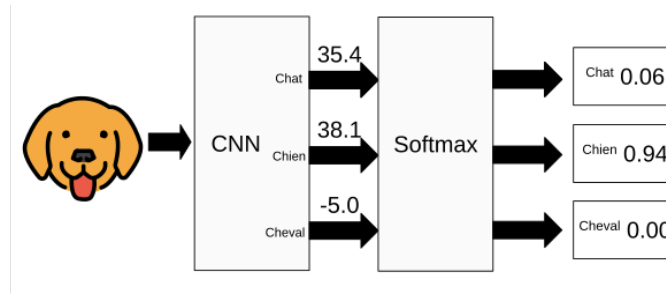


Figure 39: Pipeline della Softmax: i logit grezzi (35.4, 38.1, -5.0) vengono trasformati in probabilità normalizzate (0.06, 0.94, 0.00). (Fonte: Wikimedia Commons)

A.4 Esempio numerico

Esempio: $z = [2.0, 1.0, 0.1]$

Passo 1 – calcolo degli esponenziali:

$$e^{2.0} \approx 7.389, \quad e^{1.0} \approx 2.718, \quad e^{0.1} \approx 1.105$$

Passo 2 – somma totale:

$$S = 7.389 + 2.718 + 1.105 = 11.212$$

Passo 3 – probabilità:

$$p_1 = \frac{7.389}{11.212} \approx 0.659, \quad p_2 = \frac{2.718}{11.212} \approx 0.242, \quad p_3 = \frac{1.105}{11.212} \approx 0.099$$

Verifica: $0.659 + 0.242 + 0.099 = 1.000 \checkmark$

La classe 1 (con $z_1 = 2.0$ massimo) riceve la probabilità maggiore; l'ordine è preservato.

A.5 Collegamento al Perceiver

Softmax nel Perceiver

La softmax compare in due punti del Perceiver IO:

1. **Attention weights:** nella scaled dot-product attention, $\text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})$ trasforma i punteggi di similarità in pesi di attenzione normalizzati.
2. **Testa di classificazione:** l'ultimo strato del Perceiver per ImageNet applica una proiezione lineare $\mathbb{R}^D \rightarrow \mathbb{R}^{1000}$ seguita da softmax per ottenere la distribuzione sulle 1000 classi.

B Fourier Features e Positional Encoding

I meccanismi di attenzione sono **invarianti per permutazione**: se si rimescolano gli elementi dell'input, l'output cambia solo nell'ordine ma non nel contenuto. Questo è un problema: il significato di un'immagine dipende dalla posizione spaziale dei pixel. La soluzione è iniettare esplicitamente l'informazione posizionale.

B.1 Il problema: l'attenzione è cieca alla posizione

Formalmente, per ogni permutazione σ degli indici di input, la self-attention soddisfa:

$$\text{Attention}(\mathbf{X}_\sigma) = \text{Attention}(\mathbf{X})_\sigma$$

cioè l'output è la stessa permutazione dell'output originale. Il contenuto calcolato è identico. Senza positional encoding, “il gatto mangia il topo” e “il topo mangia il gatto” sarebbero rappresentati nello stesso modo.

B.2 Tre approcci al positional encoding

- **Learned**: vettori di posizione appresi come parametri della rete (es. BERT). Flessibili ma non generalizzano a lunghezze non viste durante il training.
- **Sinusoidale (Vaswani et al., 2017)**: formula analitica con seni e coseni a frequenze diverse. Generalizza a sequenze arbitrariamente lunghe.
- **Fourier Features (Perceiver)**: generalizzazione multidimensionale del sinusoidale, applicabile a input arbitrari (immagini, audio, video) senza assumere struttura 1D.

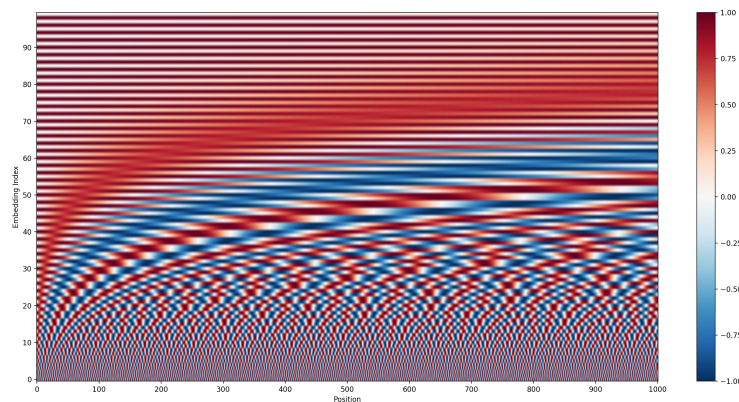


Figure 40: Heatmap del positional encoding sinusoidale: ogni riga è una posizione, ogni colonna una dimensione. Le basse dimensioni (in basso) oscillano rapidamente, le alte (in alto) lentamente. (Fonte: Wikimedia Commons)

B.3 Fourier Features nel Perceiver

Positional encoding con Fourier features (Perceiver)

Per una coordinata normalizzata $x_d \in [-1, 1]$ lungo la dimensione d , con K bande di frequenza:

$$\text{PE}(x_d) = [\sin(f_1 \pi x_d), \cos(f_1 \pi x_d), \dots, \sin(f_K \pi x_d), \cos(f_K \pi x_d), x_d]$$

Le frequenze sono distribuite linearmente (o logaritmicamente): $f_k = 1, 2, \dots, K$.

Il vettore finale si ottiene **concatenando** i contributi di tutte le dimensioni spaziali più i canali originali del segnale.

B.4 Perché frequenze multiple?

Usare più frequenze crea una **rappresentazione multi-scala**:

- **Frequenze basse** (f_k piccolo): variano lentamente, codificano la posizione globale (dove mi trovo nell'immagine in senso ampio).
- **Frequenze alte** (f_k grande): variano rapidamente, codificano la posizione locale (dettagli fini, distinguono pixel vicini).

L'insieme di tutte le frequenze garantisce che posizioni diverse producano rappresentazioni distinguibili.

B.5 Perché concatenare la coordinata grezza x_d ?

I seni e coseni sono funzioni **periodiche**: $\sin(f_k \pi x_d) = \sin(f_k \pi (x_d + 2/f_k))$. Posizioni lontane potrebbero quindi avere encoding identici per alcune frequenze. Includere la coordinata grezza x_d risolve questa ambiguità: due posizioni diverse hanno certamente encoding diversi grazie al contributo non-periodico.

Perceiver (concatenazione) vs Transformer (somma)

- **Transformer**: somma il positional encoding ai token: $\mathbf{x}'_i = \mathbf{x}_i + \text{PE}_i$. La posizione e il contenuto si *mescolano* nelle stesse dimensioni.
- **Perceiver**: concatena il positional encoding: $\mathbf{x}'_i = [\mathbf{x}_i \parallel \text{PE}_i]$. Posizione e contenuto occupano dimensioni *separate*; l'informazione originale è preservata intatta.

La concatenazione è più conservativa: il segnale originale non viene mai modificato, e la rete può scegliere autonomamente quanto peso dare alla posizione.

B.6 Esempio numerico

Esempio: $x = 0.3$, $K = 3$ bande, frequenze $f_1 = 1, f_2 = 2, f_3 = 4$

Calcolo delle componenti:

$$\sin(1 \cdot \pi \cdot 0.3) = \sin(0.942) \approx 0.809, \quad \cos(1 \cdot \pi \cdot 0.3) = \cos(0.942) \approx 0.588$$

$$\sin(2 \cdot \pi \cdot 0.3) = \sin(1.885) \approx 0.951, \quad \cos(2 \cdot \pi \cdot 0.3) = \cos(1.885) \approx -0.309$$

$$\sin(4 \cdot \pi \cdot 0.3) = \sin(3.770) \approx -0.588, \quad \cos(4 \cdot \pi \cdot 0.3) = \cos(3.770) \approx -0.809$$

Vettore finale:

$$\text{PE}(0.3) = [0.809, 0.588, 0.951, -0.309, -0.588, -0.809, 0.3]$$

7 componenti: $2K = 6$ sinusoidali + 1 coordinata grezza.

B.7 Collegamento al Perceiver

Fourier features nel Perceiver IO

Le Fourier features permettono al Perceiver di essere **domain-agnostic**: la stessa architettura processa immagini, audio e video senza assumere nulla sulla struttura dell'input. Per un'immagine 224×224 :

- Ogni pixel ha coordinate normalizzate $(x, y) \in [-1, 1]^2$.
- Con $K = 64$ bande, il positional encoding ha $2 \times 2K = 256$ componenti sinusoidali + 2 coordinate grezze = 258 valori.
- Questi vengono concatenati ai canali RGB del pixel, ottenendo l'input finale al primo cross-attention.

Il framework teorico delle Fourier features come strumento di positional encoding per reti neurali su segnali continui è stato formalizzato da **Tancik et al. (2020)** in “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains”, dove si dimostra che mappare le coordinate di input a un embedding sinusoidale a banda larga migliora drasticamente la capacità delle MLP di rappresentare funzioni ad alta frequenza. Il Perceiver adotta questa idea estendendola a input multidimensionali arbitrari.

C Cross-Entropy Loss

La **cross-entropy loss** misura quanto la distribuzione di probabilità predetta dal modello differisce dalla distribuzione “vera” (tipicamente concentrata su una sola classe). È la funzione di loss standard per la classificazione.

Cross-Entropy Loss

Dato un vettore di probabilità predette $\mathbf{p} = (p_1, \dots, p_K)$ e il vettore one-hot della classe vera $\mathbf{y} = (y_1, \dots, y_K)$ (con $y_c = 1$ per la classe corretta c e 0 altrove):

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(p_k) = -\log(p_c)$$

Poiché $\mathbf{y}$ è one-hot, la somma collassa al solo termine della classe corretta.

C.1 Perché il logaritmo?

- Il logaritmo è **monotono crescente**: massimizzare $\log(p_c)$ equivale a massimizzare p_c .
- **Penalizzazione asimmetrica**: $-\log(0.01) \approx 4.6$ (predizione confident ma sbagliata), mentre $-\log(0.99) \approx 0.01$ (predizione quasi corretta). La perdita punisce fortemente le previsioni errate con alta confidenza.
- Emerge naturalmente dalla **stima di massima verosimiglianza**: minimizzare la cross-entropy equivale a massimizzare la log-likelihood del modello.

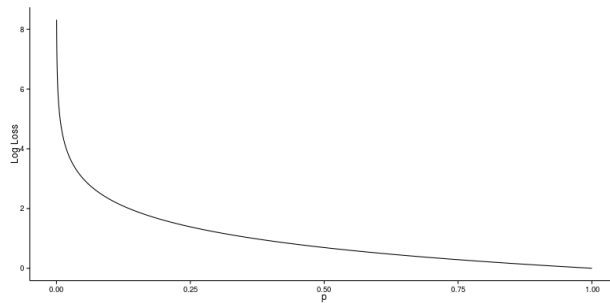


Figure 41: La funzione $-\log(p)$: quando la probabilità predetta p è vicina a 1, la loss è quasi zero; quando $p \rightarrow 0$, la loss esplode verso $+\infty$. (Fonte: Wikimedia Commons)

C.2 Connessione con la softmax: gradiente pulito

L'abbinamento softmax + cross-entropy produce un gradiente elegante. Se $\mathbf{z}$ è il logit pre-softmax e c è la classe vera:

Gradiente softmax + cross-entropy

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$$

Il gradiente rispetto al logit z_k è semplicemente la **differenza tra probabilità predetta e target**. Non compaiono derivate di esponenziali o logaritmi: il calcolo si semplifica perfettamente.

Intuizione del gradiente $p_k - y_k$

Se il modello predice $p_c = 0.9$ per la classe corretta ($y_c = 1$), il gradiente è $0.9 - 1 = -0.1$: piccolo aggiustamento necessario. Se predice $p_c = 0.1$, il gradiente è $0.1 - 1 = -0.9$: grande aggiustamento necessario. Il segnale di apprendimento è proporzionale all'errore.

C.3 Esempio numerico

Esempio: classificazione a 3 classi, classe vera = 2

Probabilità predette: $\mathbf{p} = [0.1, 0.7, 0.2]$

Target one-hot: $\mathbf{y} = [0, 1, 0]$ (classe 2, indice $k = 2$)

Calcolo della loss:

$$\mathcal{L} = -(0 \cdot \log 0.1 + 1 \cdot \log 0.7 + 0 \cdot \log 0.2) = -\log(0.7) \approx 0.357$$

Gradiente: $\nabla_{\mathbf{z}} \mathcal{L} = [0.1 - 0, 0.7 - 1, 0.2 - 0] = [0.1, -0.3, 0.2]$

Il gradiente negativo sull'indice 2 (-0.3) indica che il modello deve aumentare il logit z_2 : si sta imparando nella direzione corretta.

C.4 Cross-entropy vs MSE per la classificazione

Perché non usare l'MSE per la classificazione?

L'MSE $\mathcal{L} = \frac{1}{K} \sum_k (p_k - y_k)^2$ può sembrare un'alternativa naturale, ma presenta due problemi:

1. **Gradiente che svanisce con sigmoid/softmax:** quando $p_c \approx 0$ (predizione molto sbagliata), il gradiente dell'MSE diventa quasi zero (perché la derivata di sigmoid è piatta agli estremi). La cross-entropy non ha questo problema.
2. **Trattamento simmetrico:** l'MSE penalizza allo stesso modo un errore su classe frequente e rara. La cross-entropy, tramite $-\log(p_c)$, cresce rapidamente quando $p_c \rightarrow 0$ indipendentemente dalla distribuzione delle altre classi.

C.5 Collegamento al Perceiver

Cross-entropy nel Perceiver

Il Perceiver IO per la classificazione su ImageNet usa la cross-entropy loss standard. Dopo il global average pooling sul latent array, una proiezione lineare produce 1000 logit, cui segue la softmax. La loss è $\mathcal{L} = -\log(p_c)$ dove c è la classe ImageNet corretta. L'aggiornamento tramite backpropagation del gradiente $\mathbf{p} - \mathbf{y}$ si propaga attraverso tutti i 112 blocchi dell'architettura.

D Layer Normalization

La **Layer Normalization** (LN), introdotta da **Ba, Kiros & Hinton (2016)** in “Layer Normalization” (arXiv:1607.06450), normalizza le attivazioni di ogni campione indipendentemente, riportando la media a 0 e la varianza a 1 lungo la dimensione delle feature. È il componente di stabilizzazione standard nei Transformer e nel Perceiver.

Layer Normalization

Dato il vettore di attivazioni $\mathbf{x} = (x_1, \dots, x_D)$ per un singolo campione, con media μ e deviazione standard σ calcolate sulle D feature:

$$\mu = \frac{1}{D} \sum_{j=1}^D x_j, \quad \sigma = \sqrt{\frac{1}{D} \sum_{j=1}^D (x_j - \mu)^2 + \varepsilon}$$

$$\text{LN}(\mathbf{x})_j = \gamma_j \cdot \frac{x_j - \mu}{\sigma} + \beta_j$$

dove γ_j e β_j sono parametri **apprendibili** (scale e shift per feature), $\varepsilon \approx 10^{-5}$ evita la divisione per zero.

D.1 Layer Norm vs Batch Norm

Differenza fondamentale tra LN e BatchNorm

- **Batch Normalization:** normalizza lungo la dimensione del *batch*. Per ogni feature j , calcola μ_j e σ_j su tutti i campioni del mini-batch. Dipende dalla dimensione del batch; instabile con batch piccoli.
- **Layer Normalization:** normalizza lungo la dimensione delle *feature*. Per ogni campione, calcola μ e σ su tutte le sue feature. Completamente indipendente dalla dimensione del batch.

La LN è quindi ideale per sequenze di lunghezza variabile e per situazioni in cui il batch è piccolo o la parallelizzazione cambia la struttura del batch.

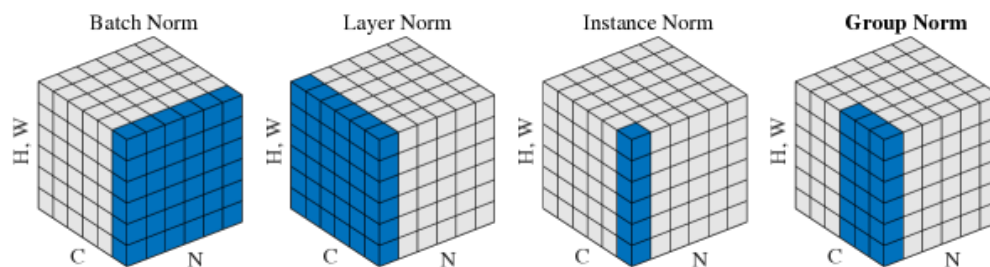


Figure 42: Confronto dei metodi di normalizzazione (da Wu & He, 2018). I pixel blu indicano quali valori condividono le stesse statistiche (μ, σ). BN normalizza lungo il batch (N); LN normalizza lungo le feature (C, H, W) per ogni singolo campione.

D.2 Pattern pre-norm (usato nel Perceiver)

Esistono due varianti di come integrare la LN in un blocco residuale:

Pre-norm vs Post-norm

Post-norm (Transformer originale): $\mathbf{y} = \text{LN}(\mathbf{x} + f(\mathbf{x}))$

Pre-norm (GPT-2, Perceiver): $\mathbf{y} = \mathbf{x} + f(\text{LN}(\mathbf{x}))$

- **Pre-norm:** la LN viene applicata *prima* del sottoblocco. L'input residuale non normalizzato si somma direttamente all'output: il gradiente fluisce attraverso la skip connection senza essere modificato dalla LN. Più stabile per reti profonde.
- **Post-norm:** la LN viene applicata *dopo* la somma residuale. Richiede warm-up più accurato del learning rate; può essere instabile nelle prime fasi del training.

D.3 Parametri γ e β

Dopo la normalizzazione, i parametri appresi γ (scale) e β (shift) permettono alla rete di **annullare la normalizzazione** se necessario: se $\gamma_j = \sigma$ e $\beta_j = \mu$, la LN si riduce all'identità. In pratica, la rete impara il compromesso ottimale tra stabilità numerica e libertà espressiva.

D.4 Esempio numerico

Esempio: $\mathbf{x} = [1.0, 2.0, 3.0, 4.0]$, $D = 4$

Passo 1 – media:

$$\mu = \frac{1.0 + 2.0 + 3.0 + 4.0}{4} = \frac{10.0}{4} = 2.5$$

Passo 2 – varianza e deviazione standard (con $\varepsilon = 0$ per semplicità):

$$\sigma^2 = \frac{(1.0 - 2.5)^2 + (2.0 - 2.5)^2 + (3.0 - 2.5)^2 + (4.0 - 2.5)^2}{4} = \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = \frac{5.0}{4} = 1.25$$

$$\sigma = \sqrt{1.25} \approx 1.118$$

Passo 3 – normalizzazione (con $\gamma_j = 1$, $\beta_j = 0$):

$$\hat{x}_1 = \frac{1.0 - 2.5}{1.118} \approx -1.342, \quad \hat{x}_2 = \frac{2.0 - 2.5}{1.118} \approx -0.447$$

$$\hat{x}_3 = \frac{3.0 - 2.5}{1.118} \approx +0.447, \quad \hat{x}_4 = \frac{4.0 - 2.5}{1.118} \approx +1.342$$

Risultato: $\text{LN}(\mathbf{x}) \approx [-1.342, -0.447, +0.447, +1.342]$

Verifica: media ≈ 0 , varianza ≈ 1 ✓

D.5 Collegamento al Perceiver

Layer Normalization nel Perceiver IO

Il Perceiver usa il pattern **pre-norm** in ogni blocco. Con 8 cross-attention e 6 self-attention per layer, ciascuno con 2 sottoblocchi (attention + FFN), e 8 layer totali, si contano circa **112 operazioni di Layer Normalization**. Questo è il motivo per cui il training del Perceiver è stabile nonostante la profondità: ogni sottoblocco riceve input normalizzati, e il gradiente della loss può fluire attraverso le skip connections senza essere distorto.

E Funzioni di Attivazione

Le **funzioni di attivazione** introducono la non-linearità nelle reti neurali. Senza di esse, una composizione di trasformazioni lineari resterebbe lineare, e l'intera rete non potrebbe apprendere funzioni complesse. La scelta della funzione di attivazione influenza la velocità di convergenza, la stabilità del gradiente e le prestazioni finali.

E.1 ReLU (Rectified Linear Unit)

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{Derivata: } \text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

- **Vantaggi:** semplicissima da calcolare; non satura per $x > 0$; sparsità (molti neuroni a zero).
- **Problema “dead neurons”:** se $x < 0$ per tutti i campioni del training, il gradiente è sempre 0 e il neurone smette di aggiornarsi permanentemente.

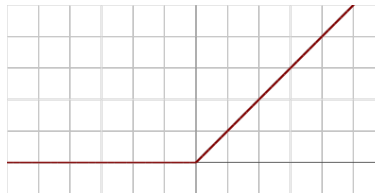


Figure 43: Funzione ReLU: $\max(0, x)$. Lineare per $x > 0$, zero per $x \leq 0$.

E.2 GELU (Gaussian Error Linear Unit)

GELU

$$\text{GELU}(x) = x \cdot \Phi(x)$$

dove $\Phi(x) = \frac{1}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$ è la CDF della distribuzione normale standard.
Approssimazione pratica:

$$\text{GELU}(x) \approx 0.5 x (1 + \tanh[\sqrt{2/\pi} (x + 0.044715 x^3)])$$

- **Smooth:** a differenza di ReLU, GELU è differenziabile ovunque e non ha il punto angoloso in $x = 0$.
- **Leggermente negativa per $x < 0$:** non è mai esattamente zero, i neuroni non muoiono completamente.
- **Usata in:** GPT-2, BERT, Perceiver IO e quasi tutti i Transformer moderni.
- **Riferimento:** proposta da **Hendrycks & Gimpel (2016)** in “Gaussian Error Linear Units (GELUs)”.

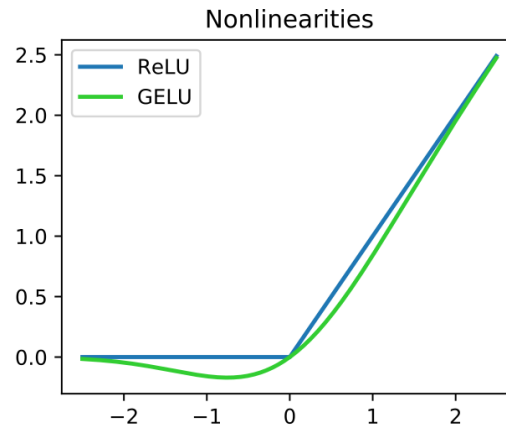


Figure 44: Confronto ReLU vs GELU. La ReLU (blu) ha un punto angoloso in $x = 0$ e azzerava bruscamente i valori negativi. La GELU (verde) è liscia, permette piccoli valori negativi e si avvicina all'identità per x grandi. (Fonte: Wikimedia Commons)

E.3 Sigmoid

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x) (1 - \sigma(x))$$

- Output in $(0, 1)$: interpretabile come probabilità.
- **Satura** per $|x|$ grandi: il gradiente $\sigma'(x) \rightarrow 0$, causando vanishing gradient.
- Usata nelle gate di LSTM e GRU (dove l'output in $[0, 1]$ ha senso semantico come “quanto aprire il gate”).

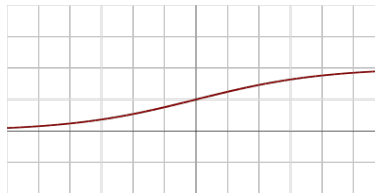


Figure 45: Funzione Sigmoid: output in $(0, 1)$, satura per $|x|$ grandi.

E.4 Tanh

Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh'(x) = 1 - \tanh^2(x)$$

- Output in $(-1, 1)$: centrato in zero, a differenza di sigmoid.
- Ancora soggetto a saturazione per $|x|$ grandi.
- Usata nelle celle di memoria LSTM e come attivazione nascosta nelle RNN classiche.

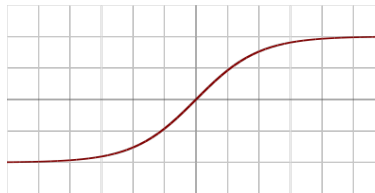


Figure 46: Funzione Tanh: output in $(-1, 1)$, centrata in zero, satura per $|x|$ grandi.

E.5 Tabella comparativa

Funzione	Formula	Range	Derivata	Dove usata
ReLU	$\max(0, x)$	$[0, +\infty)$	0 o 1	CNN, MLP classici
GELU	$x \Phi(x)$	$\approx (-0.17, +\infty)$	smooth	GPT, BERT, Perceiver
Sigmoid	$1/(1 + e^{-x})$	$(0, 1)$	$\sigma(1 - \sigma)$	Gate LSTM/GRU, output binario
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$(-1, 1)$	$1 - \tanh^2$	Celle LSTM, RNN classiche

E.6 Perché GELU invece di ReLU nel Perceiver?

GELU vs ReLU nelle architetture basate su attenzione

1. **Gradienti più fluidi:** GELU è smooth in $x = 0$, favorendo ottimizzazione con Adam su reti molto profonde.
2. **Nessun neurone morto:** la derivata di GELU non è mai esattamente zero, garantendo che tutti i neuroni ricevano segnale di aggiornamento.
3. **Comportamento stocastico interpretabile:** $\text{GELU}(x) = x \cdot P(\text{mantieni il neurone})$, dove la probabilità segue una distribuzione gaussiana. Questo ha una connessione con il dropout stocastico.
4. **Standard de facto:** BERT (Devlin, 2018) e GPT-2 (Radford, 2019) hanno dimostrato empiricamente che GELU supera ReLU nei Transformer; il Perceiver segue questa convenzione.

Collegamento al Perceiver

Il Perceiver IO usa GELU come attivazione nelle reti feed-forward (MLP) che seguono ogni blocco di attenzione. Ogni blocco MLP ha struttura: Linear $\rightarrow$ GELU $\rightarrow$ Linear, con espansione $4\times$ della dimensione nascosta (pattern standard dei Transformer).

F Residual Connections

Le **residual connections** (o skip connections) sono il meccanismo che permette di addestrare reti neurali molto profonde senza che i gradienti spariscano o le prestazioni degradino. Introdotte da He et al. con ResNet (2016, CVPR), sono ora un componente universale in ogni architettura profonda, incluso il Perceiver.

F.1 Il problema: degradazione nelle reti profonde

Aggiungere strati a una rete dovrebbe *al massimo* mantenere le prestazioni (il nuovo strato può sempre imparare l'identità). In pratica, reti più profonde si comportavano peggio anche sul training set: la rete da 56 layer di VGG era più inaccurata di quella da 20 layer. Questo non è vanishing gradient (i pesi si aggiornano), ma **degradazione**: ottimizzare la funzione identità attraverso molti strati non lineari è sorprendentemente difficile.

Residual Connection

Invece di apprendere direttamente $H(\mathbf{x})$ (la trasformazione desiderata), la rete apprende il **residuo** $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$$

Se il blocco non è utile, basta che $F(\mathbf{x}) \approx \mathbf{0}$ e la rete implementa l'identità.

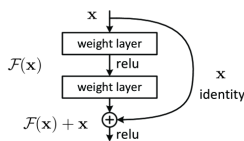


Figure 47: Il blocco residuale (da He et al., 2016): l'input $\mathbf{x}$ attraversa due weight layer per calcolare $F(\mathbf{x})$, poi viene sommato alla skip connection per ottenere $F(\mathbf{x}) + \mathbf{x}$.

F.2 Perché funziona: flusso del gradiente

Analisi del gradiente con e senza residual connection

Considera un blocco con parametri θ e la loss $\mathcal{L}$:

Senza residual: $\mathbf{y} = F(\mathbf{x}; \theta)$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial F}{\partial \mathbf{x}}$$

Il gradiente viene *moltiplicato* per la Jacobiana di F . Con molti strati, se $\|\partial F / \partial \mathbf{x}\| < 1$, il prodotto si azzerava (vanishing gradient).

Con residual: $\mathbf{y} = F(\mathbf{x}; \theta) + \mathbf{x}$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(\frac{\partial F}{\partial \mathbf{x}} + \mathbf{I} \right)$$

Il termine $\mathbf{I}$ (matrice identità) garantisce che il gradiente non possa mai essere completamente zero: c'è sempre un percorso diretto attraverso la skip connection.

F.3 Pattern pre-norm nel Perceiver

Come discusso per la Layer Normalization (Appendice D), il Perceiver combina residual connections e LN nel pattern **pre-norm**:

Blocco residuale con pre-norm

$$\mathbf{y} = \mathbf{x} + f(\text{LN}(\mathbf{x}))$$

- L'input $\mathbf{x}$ fluisce non modificato lungo la skip connection.
- Il sottoblocco f riceve sempre input normalizzati: la LN stabilizza la scala delle attivazioni.
- Il gradiente di $\mathcal{L}$ rispetto a $\mathbf{x}$ ha sempre il termine additivo $+\mathbf{I}$: nessun vanishing gradient lungo la skip.

F.4 Esempio numerico: confronto flusso del gradiente

Esempio: 3 strati, gradiente senza e con residual

Supponiamo 3 blocchi identici, ciascuno con Jacobiana $\partial F / \partial \mathbf{x} = 0.5 \mathbf{I}$ (attenuazione del 50% per strato).

Senza residual:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_0} = (0.5)^3 \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_3} = 0.125 \cdot g$$

Con 56 strati: $(0.5)^{56} \approx 1.4 \times 10^{-17}$ — gradiente effettivamente zero.

Con residual:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_0} \ni (0.5 + 1)^3 \cdot g = (1.5)^3 \cdot g = 3.375 g$$

Il gradiente non solo sopravvive: può anche *crescere* lungo la skip. In pratica, i termini si bilanciano e il gradiente rimane in un range ragionevole anche per reti molto profonde.

F.5 Collegamento al Perceiver

Residual connections nel Perceiver IO

Il Perceiver IO usa **112 residual connections** (stessa stima delle Layer Norm): una per ogni sottoblocco di attention e una per ogni FFN, in ogni layer. Senza di esse:

- Il gradiente si azzerebbe prima di raggiungere i layer iniziali.
- La rete non potrebbe addestrare efficacemente il cross-attention iniziale (il punto critico dove le Fourier features dell'input incontrano il latent array).
- Il latent array non convergerebbe verso rappresentazioni stabili.

La stessa idea di ResNet — il blocco deve imparare solo la *correzione* rispetto all'input, non l'intera trasformazione — si applica al contesto attention-based del Perceiver.

G Ottimizzatori

Gli **ottimizzatori** sono gli algoritmi che aggiornano i parametri di una rete neurale durante il training per minimizzare la funzione di perdita. La scelta dell'ottimizzatore ha un impatto enorme sulla velocità di convergenza, sulla stabilità e sulle prestazioni finali. Il Perceiver IO usa **LAMB**, una variante sofisticata di Adam progettata per il large batch training su TPU.

G.1 SGD (Stochastic Gradient Descent)

SGD — Aggiornamento dei pesi

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(W_t)$$

dove η è il **learning rate** (iperparametro fisso) e $\nabla_W \mathcal{L}$ è il gradiente della loss calcolato su un mini-batch.

- **Semplicità:** il meccanismo più elementare di ottimizzazione.
- **Problema: oscillazioni.** Nelle valli strette e allungate (tipiche della loss surface di reti profonde), il gradiente punta trasversalmente alla direzione di minimo, causando zig-zag lenti.
- **Problema: learning rate fisso per tutti i parametri.** Non tutti i parametri hanno la stessa sensibilità: pesi rari (es. embedding) ricevono aggiornamenti radi e ne trarrebbero vantaggio da step più grandi; pesi comuni ne trarrebbero vantaggio da step più piccoli.
- **Sensibilità all'iperparametro η :** troppo grande $\rightarrow$ divergenza; troppo piccolo $\rightarrow$ convergenza lentissima.

Quando usare SGD?

SGD (spesso con momentum e scheduling) rimane competitivo per le CNN su visione: ResNet e ViT spesso usano SGD+Momentum nella fase di fine-tuning su ImageNet. Nei Transformer e nelle reti di grandi dimensioni, Adam e le sue varianti sono quasi universalmente preferiti.

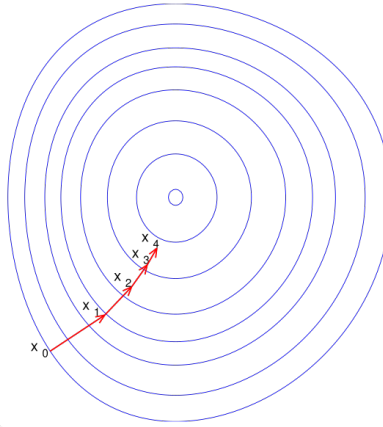


Figure 48: Discesa del gradiente: partendo da x_0 , i pesi vengono aggiornati iterativamente seguendo la direzione opposta al gradiente fino a convergere verso il minimo. (Fonte: Wikimedia Commons)

G.2 SGD con Momentum

Il **momentum** aggiunge un termine di inerzia all'aggiornamento, simulando una palla che rotola in discesa e accumula velocità nelle direzioni consistenti.

SGD con Momentum

$$v_t = \beta v_{t-1} + \nabla_W \mathcal{L}(W_t)$$

$$W_{t+1} = W_t - \eta v_t$$

dove v_t è il **vettore di velocità** al passo t , $\beta \in [0, 1)$ è il coefficiente di momentum (tipicamente $\beta = 0.9$) e $v_0 = \mathbf{0}$.

Intuizione: la palla che rotola in discesa

Immagina una palla che scende lungo un paesaggio collinare:

- Senza momentum: ogni step è determinato solo dal pendio locale. La palla rimbalza su e giù nelle valli strette.
- Con momentum: la palla accumula velocità nelle direzioni in cui il gradiente è costantemente orientato (direzione del minimo) e si autocancella nelle direzioni oscillanti (pareti laterali della valle). Il risultato è una traiettoria più liscia e convergenza più rapida.

Il fattore $\beta = 0.9$ significa che la velocità precedente contribuisce al 90% e il gradiente corrente al 10% (scala): dopo k passi, l'influenza del gradiente al passo $t - k$ è β^k . Con $\beta = 0.9$ questa contribuzione dimezza ogni ≈ 7 passi.

G.3 Adam (Adaptive Moment Estimation)

Adam (Kingma & Ba, 2014) combina momentum (primo momento) e adattività per ogni parametro (secondo momento). È l'ottimizzatore più usato nel deep learning moderno.

Adam — Algoritmo completo

Inizializzazione: $m_0 = 0, v_0 = 0, t = 0$.

Ad ogni passo t :

1. **Calcola il gradiente:**

$$g_t = \nabla_W \mathcal{L}(W_t)$$

2. **Aggiorna il primo momento (media dei gradienti):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

3. **Aggiorna il secondo momento (varianza non centrata):**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

4. **Bias correction** (i momenti sono inizializzati a 0 e tendono a 0 all'inizio):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

5. **Aggiornamento dei pesi:**

$$W_{t+1} = W_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Iperparametri tipici: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \eta = 10^{-3}$ o 10^{-4} .

Perché Adam funziona bene?

1. **Learning rate adattivo per parametro:** $\hat{v}_t$ stima la varianza del gradiente. Parametri con gradienti grandi e frequenti ricevono step piccoli; parametri con gradienti piccoli e rari ricevono step grandi. Ideale per reti con embedding sparsi.
2. **Momentum:** $\hat{m}_t$ è una media esponenzialmente pesata dei gradienti passati; riduce il rumore e accelera la convergenza nelle direzioni consistenti (come SGD+Momentum).
3. **Bias correction:** nei primi passi, m_t e v_t sono vicini a zero (non sono ancora stati "riscaldati"). Dividere per $(1 - \beta^t)$ corregge questa inizializzazione a zero, evitando step troppo piccoli all'inizio.
4. **Robustezza:** funziona bene con learning rate non troppo finemente accordati, a differenza di SGD.

Esempio numerico: un passo di Adam

Esempio: Adam al passo $t = 1$ e $t = 2$

Setup: $W_0 = 0.5$, $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Supponi due gradienti consecutivi: $g_1 = 0.3$, $g_2 = -0.1$.

Passo $t = 1$:

$$\begin{aligned} m_1 &= 0.9 \cdot 0 + 0.1 \cdot 0.3 = 0.03 \\ v_1 &= 0.999 \cdot 0 + 0.001 \cdot (0.3)^2 = 0.001 \cdot 0.09 = 9 \times 10^{-5} \\ \hat{m}_1 &= \frac{0.03}{1 - 0.9^1} = \frac{0.03}{0.1} = 0.3, \quad \hat{v}_1 = \frac{9 \times 10^{-5}}{1 - 0.999^1} = \frac{9 \times 10^{-5}}{0.001} = 0.09 \\ W_1 &= 0.5 - 0.1 \cdot \frac{0.3}{\sqrt{0.09 + 10^{-8}}} = 0.5 - 0.1 \cdot \frac{0.3}{0.3} = 0.5 - 0.1 = \mathbf{0.4} \end{aligned}$$

Passo $t = 2$:

$$\begin{aligned} m_2 &= 0.9 \cdot 0.03 + 0.1 \cdot (-0.1) = 0.027 - 0.01 = 0.017 \\ v_2 &= 0.999 \cdot 9 \times 10^{-5} + 0.001 \cdot 0.01 \approx 8.991 \times 10^{-5} + 10^{-5} = 9.991 \times 10^{-5} \\ \hat{m}_2 &= \frac{0.017}{1 - 0.81} = \frac{0.017}{0.19} \approx 0.0895, \quad \hat{v}_2 = \frac{9.991 \times 10^{-5}}{1 - 0.998001} \approx \frac{9.991 \times 10^{-5}}{0.001999} \approx 0.05 \\ W_2 &= 0.4 - 0.1 \cdot \frac{0.0895}{\sqrt{0.05}} = 0.4 - 0.1 \cdot \frac{0.0895}{0.2236} \approx 0.4 - 0.040 = \mathbf{0.360} \end{aligned}$$

Osservazione: al passo 2 il gradiente era negativo (-0.1) ma $\hat{m}_2 > 0$ perché il momento positivo del passo 1 domina ancora. Adam “ricorda” la direzione passata.

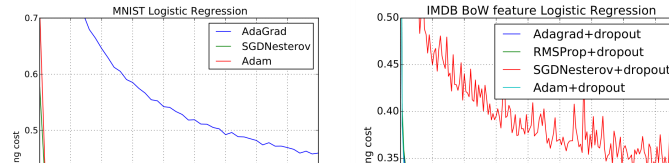


Figure 49: Convergenza degli ottimizzatori su regressione logistica (da Kingma & Ba, 2014). Sinistra: MNIST. Destra: IMDB. Adam converge velocemente come SGD-Nesterov e molto meglio di Adagrad.

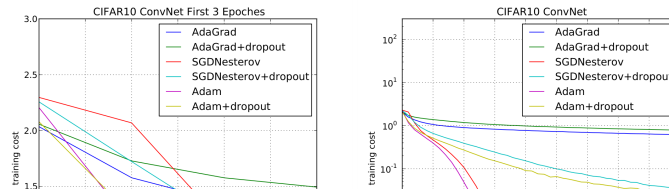


Figure 50: Training cost su CNN (CIFAR-10) nei primi 3 epoch (sinistra) e su 45 epoch (destra). Adam e SGD convergono più velocemente di Adagrad sulle CNN (da Kingma & Ba, 2014).

G.4 AdamW (Adam con Weight Decay disaccoppiato)

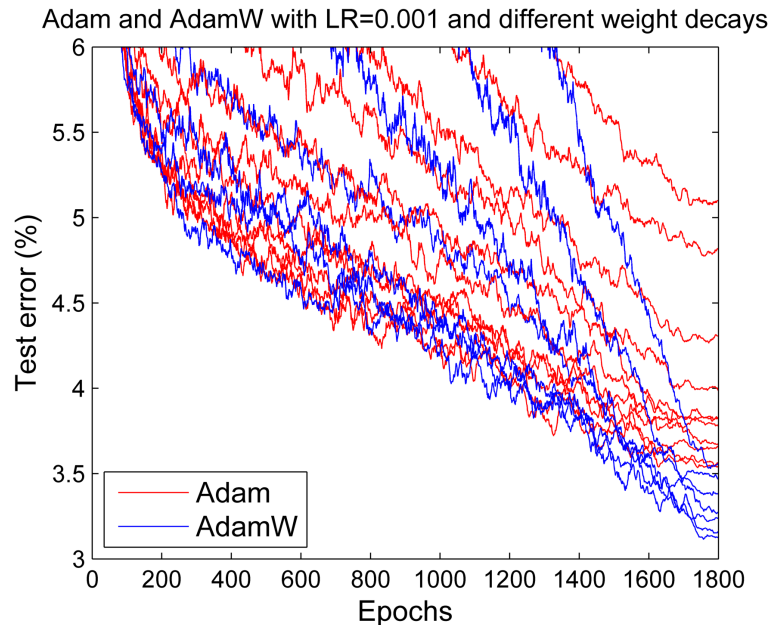


Figure 51: Test error: Adam (rosso) vs AdamW (blu). Il weight decay disaccoppiato di AdamW generalizza meglio a parità di learning rate. *Fonte: Loshchilov & Hutter, Decoupled Weight Decay Regularization (arXiv:1711.05101).*

AdamW (Loshchilov & Hutter, 2017) corregge un difetto di Adam: l'L2 regularization aggiunta al gradiente interagisce con l'adattività e non produce il vero weight decay.

AdamW — Differenza chiave da Adam

Adam con L2 reg. aggiunge λW_t al gradiente *prima* del calcolo dei momenti:

$$g_t^{\text{reg}} = g_t + \lambda W_t \quad \rightarrow \quad \text{entra in } m_t \text{ e } v_t$$

AdamW applica il weight decay *dopo* l'aggiornamento adattivo, direttamente ai pesi:

$$W_{t+1} = W_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda W_t \right)$$

Il termine λW_t scala uniformemente tutti i pesi verso zero, indipendentemente dalla storia dei gradienti.

Perché il disaccoppiamento è importante?

In Adam puro, v_t accumula la magnitudine del gradiente $+\lambda W_t$. I parametri con gradienti piccoli ma valori grandi ricevono un decay sproporzionatamente ridotto (il denominatore $\sqrt{\hat{v}_t}$ è piccolo, quindi l'adattività “amplifica” il decay). AdamW separa le due dinamiche: il decay è sempre λW_t indipendentemente dalla storia del gradiente. Il Perceiver usa weight decay $\lambda = 0.1$ con LAMB (che eredita questa correzione da AdamW).

G.5 LAMB (Layer-wise Adaptive Moments for Batch training)

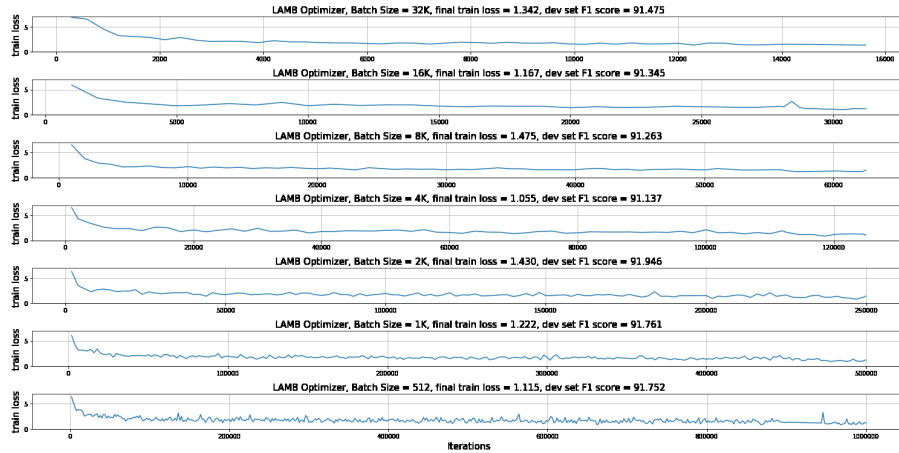


Figure 52: Training di BERT con l'ottimizzatore LAMB a batch crescenti (512–32K): la qualità finale (F1 $\approx$ 91.5) si mantiene anche con batch enormi — è l'optimizer usato nel nostro progetto. *Fonte:* You et al., *LAMB* (arXiv:1904.00962).

LAMB (You et al., 2019) estende Adam con un **trust ratio per layer**, permettendo di usare batch size enormi (decine di migliaia di esempi su centinaia di acceleratori) senza instabilità. Il Perceiver IO usa LAMB con batch 1024 su 64 TPU per il training su ImageNet.

LAMB — Algoritmo

Step 1. Calcolo dell'update adattivo (come Adam):

$$\Delta W_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda W_t$$

(include weight decay disaccoppiato come AdamW)

Step 2. Calcolo del trust ratio per layer ℓ :

$$r_t^{(\ell)} = \frac{\|W_t^{(\ell)}\|}{\|\Delta W_t^{(\ell)}\|}$$

Step 3. Aggiornamento finale:

$$W_{t+1}^{(\ell)} = W_t^{(\ell)} - \eta \cdot r_t^{(\ell)} \cdot \Delta W_t^{(\ell)}$$

Se $r_t^{(\ell)} = 1$, il comportamento è identico ad AdamW. Il trust ratio scala il learning rate effettivo di ogni layer in modo che lo step non superi la norma dei pesi correnti.

Perché il trust ratio abilita il large batch training?

Con batch size grande, il gradiente stimato è più accurato (meno rumore) ma i pesi di layer diversi hanno scale molto diverse:

- I pesi del primo layer (processing dell'input grezzo) possono avere norme molto diverse dai pesi dell'ultimo layer (proiezione verso il numero di classi).

- Un learning rate globale che va bene per un layer è troppo grande o troppo piccolo per un altro. Il trust ratio $r_t^{(\ell)} = \|\Delta W_t^{(\ell)}\| / \|W_t^{(\ell)}\|$ normalizza l'entità dello step rispetto alla scala dei pesi di ogni layer:

- Se $\|\Delta W_t^{(\ell)}\| \ll \|W_t^{(\ell)}\|$: $r_t^{(\ell)} \gg 1$, lo step è amplificato (layer “pigro”).
- Se $\|\Delta W_t^{(\ell)}\| \gg \|W_t^{(\ell)}\|$: $r_t^{(\ell)} \ll 1$, lo step è ridotto (protezione da esplosione).

Questo bilanciamento automatico per layer è ciò che consente la linearità nella relazione batch size $\leftrightarrow$ learning rate che LAMB sfrutta.

LAMB nel Perceiver IO

Il paper del Perceiver (Jaegle et al., 2021) usa LAMB con le seguenti impostazioni per ImageNet:

- Batch size: **1024** su **64 TPU** (batch per TPU: 16).
- Learning rate base: 2×10^{-3} .
- Weight decay: $\lambda = 0.1$.
- Schedule: LR piatto per 55 epoche, poi cosine decay per 55 epoche (110 totali).
- Gradient clipping: norma globale max 10 (prevenzione di esplosione del gradiente).

Perché non Adam? Con batch 1024 distribuito su molti dispositivi, Adam presenta instabilità al crescere del batch. LAMB garantisce stabilità mantenendo le stesse prestazioni finali.

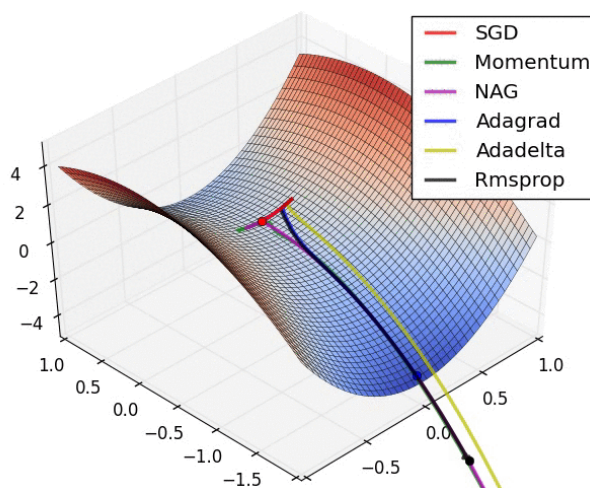


Figure 53: Confronto tra ottimizzatori su un punto di sella: SGD (in ciano) oscilla, mentre Momentum, NAG, Adagrad, Adadelata e RMSprop trovano percorsi più efficienti. (Fonte: Stanford CS231n)

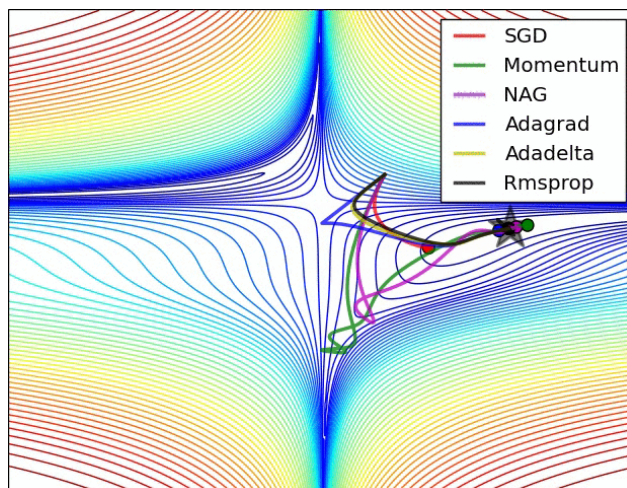


Figure 54: Vista dall'alto delle traiettorie degli ottimizzatori sulle curve di livello della loss: SGD con Momentum converge rapidamente, mentre SGD puro oscilla. (Fonte: Stanford CS231n)

G.6 Learning Rate Scheduling

Il learning rate non rimane costante durante il training: uno **schedule** lo varia per ottimizzare convergenza e prestazioni finali.

Principali strategie di scheduling

Warmup lineare (da 0 a $\eta_{\max}$ in T_w passi):

$$\eta_t = \eta_{\max} \cdot \frac{t}{T_w}, \quad t \leq T_w$$

Step decay (riduzione di un fattore γ ogni T_s epoche):

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T_s \rfloor}$$

Cosine decay (da $\eta_{\max}$ a $\eta_{\min}$ in T_c passi):

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_c}\right) \right)$$

Warmup: perché partire da un LR basso?

All'inizio del training, i momenti m_t e v_t di Adam/LAMB sono inizializzati a zero e non riflettono ancora la curvatura reale della loss. Un learning rate alto in questa fase può causare aggiornamenti instabili che danneggiano l'inizializzazione dei pesi. Il warmup permette ai momenti di “scaldarsi” prima che l'ottimizzatore inizi a fare passi ampi.

Il Perceiver usa un flat LR di 2×10^{-3} per 55 epoche (fase di esplorazione) seguito da cosine decay per 55 epoche (fase di raffinamento verso il minimo).

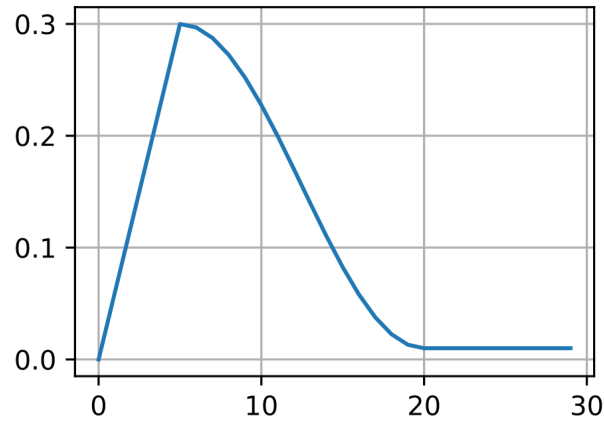


Figure 55: Schedule di learning rate con warmup lineare + cosine decay (da d2l.ai, Apache 2.0). Il learning rate cresce linearmente durante il warmup, poi decresce con un profilo cosinusoidale.

G.7 Tabella comparativa degli ottimizzatori

Ottimizzatore	Idea chiave	LR adattivo	Momentum	Caso d'uso tipico
SGD	Gradiente scalato da η	No (globale)	No	CNN classiche
SGD+Momentum	Inerzia accumulate	No (globale)	Sì	ResNet su ImageNet
Adam	Momento + varianza	Sì (per param)	Sì	Transformer, NLP
AdamW	Adam + decoupled decay	Sì (per param)	Sì	BERT, GPT, ViT
LAMB	AdamW + trust ratio	Sì (per param)	Sì	Large batch, TPU (Perceiver)

H Perceptrone

Il **perceptrone** è il mattone fondamentale di ogni rete neurale: un classificatore lineare binario. Capire il perceptrone è essenziale perché ogni neurone nelle architetture successive — fino al Perceiver — ne ricalca la struttura base (somma ponderata + attivazione).

H.1 Struttura

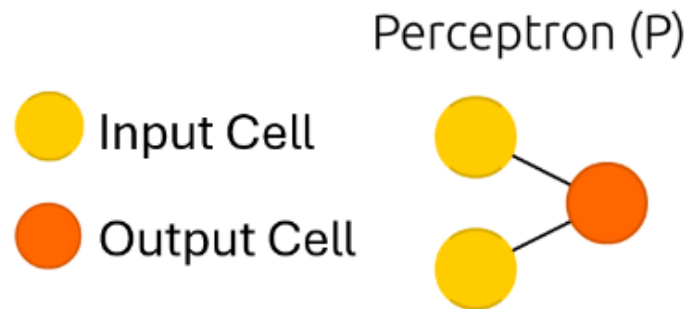


Figure 56: Schema di un perceptrone: input, pesi, somma ponderata e funzione di attivazione.

Un perceptrone riceve n input $x_1, \dots, x_n$, ciascuno moltiplicato per un peso w_i , somma il tutto con un bias b e applica una funzione di attivazione a gradino.

Output del Perceptrone

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad \text{con } f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

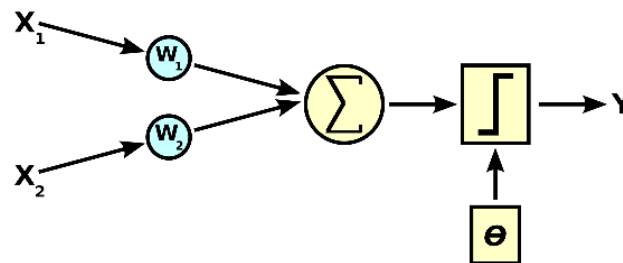


Figure 57: Struttura di un neurone artificiale: input x_i , pesi w_i , somma ponderata e funzione di attivazione.

H.2 Esempio pratico

Esempio: indossare la giacca?

Input: Temperatura $x_1 = 0.5$ (18°C normalizzato), Pioggia $x_2 = 0$.

Pesi: $w_1 = -0.7$, $w_2 = 1.0$, $b = 0.1$.

$$z = (-0.7)(0.5) + (1.0)(0) + 0.1 = -0.25$$

Poiché $z < 0$, la funzione a gradino restituisce $y = 0 \rightarrow$ “Non indossare la giacca”.

Contro-esempio: se invece piovesse ($P = 1$): $z = -0.35 + 1.0 + 0.1 = 0.75 > 0 \Rightarrow y = 1$: **indossa la giacca.**

H.3 Training

Addestrare il perceptrone significa aggiustare i pesi per ridurre l'errore di classificazione.

Regola di aggiornamento dei pesi e del bias

$$w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)}) x_{ij} \qquad b^{(k+1)} = b^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})$$

Il **learning rate** $\lambda \in (0, 1]$ controlla l'ampiezza degli aggiustamenti: un λ alto permette modifiche ampie, un λ basso modifiche fini. Spesso si usa un λ decrescente durante il training.

Teorema di convergenza del perceptrone

Se i dati sono **linearmente separabili**, la regola di aggiornamento del perceptrone converge in un numero finito di passi, indipendentemente dall'inizializzazione dei pesi. Questo risultato, dimostrato da Rosenblatt (1962), garantisce che l'algoritmo troverà sempre una soluzione — ma *solo* quando tale soluzione esiste.

Esempio numerico: un passo di training

Supponiamo $x_1 = 1$, $x_2 = 0$, target = 1, output = 0, $\lambda = 0.1$:

$$\text{errore} = \text{target} - \text{output} = 1 - 0 = 1$$

$$w_1^{\text{new}} = w_1^{\text{old}} + 0.1 \cdot 1 \cdot 1 = w_1^{\text{old}} + 0.1 \qquad w_2^{\text{new}} = w_2^{\text{old}} + 0.1 \cdot 1 \cdot 0 = w_2^{\text{old}} \quad (\text{invariato, perché } x_2 = 0)$$

Il peso w_1 aumenta perché l'input x_1 avrebbe dovuto contribuire all'attivazione ma non lo ha fatto sufficientemente. Il peso w_2 resta invariato perché il corrispondente input era nullo: non ha responsabilità nell'errore.

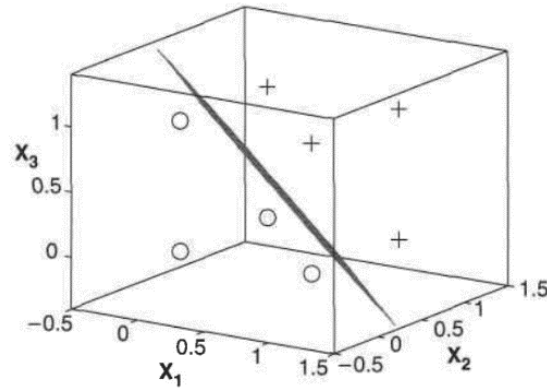


Figure 58: Rappresentazione geometrica del confine di decisione del perceptrone.

H.4 Limiti: il problema XOR

Il perceptrone converge solo su problemi **linearmente separabili**. La funzione XOR non lo è: nessuna retta nel piano separa le classi.

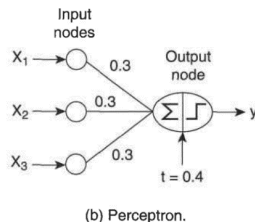
Tavola di verità dello XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Non esiste nessuna retta nel piano (x_1, x_2) che separi i punti con output 0 da quelli con output 1. Questo limite, evidenziato da Minsky e Papert (1969), fu superato solo con l'introduzione dei **perceptroni multi-strato** (reti feedforward).

x_1	x_2	x_3	y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1

(a) Data set.



(b) Perceptron.

Figure 5.14. Modeling a boolean function using a perceptron.

Feed Forward (FF)

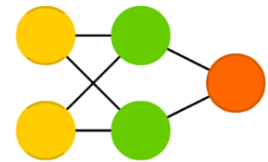


Figure 59: Sinistra: distribuzione dei punti XOR nel piano (non linearmente separabili). Destra: soluzione tramite rete multi-strato.

Per superare questo limite è necessario introdurre **hidden layer** aggiuntivi, ottenendo le reti neurali multi-strato.

I Reti Neurali Feed-Forward

Le **reti feed-forward** (multi-layer perceptron, MLP) estendono il perceptrone aggiungendo strati nascosti tra input e output. L'informazione fluisce in una sola direzione, senza cicli. Il meccanismo di training qui descritto — forward propagation, loss, backpropagation — è lo stesso usato in CNN, Transformer e Perceiver.

Teorema dell'approssimatore universale (Cybenko, 1989)

Una rete feedforward con *un solo hidden layer* e un numero sufficiente di neuroni con funzione di attivazione non lineare può approssimare qualsiasi funzione continua su un compatto con precisione arbitraria. Questo giustifica teoricamente l'uso degli MLP — ma non dice *quanti* neuroni servano: in pratica, reti **profonde** (più layer) sono molto più efficienti di reti larghe (un solo layer enorme).

I.1 Architettura

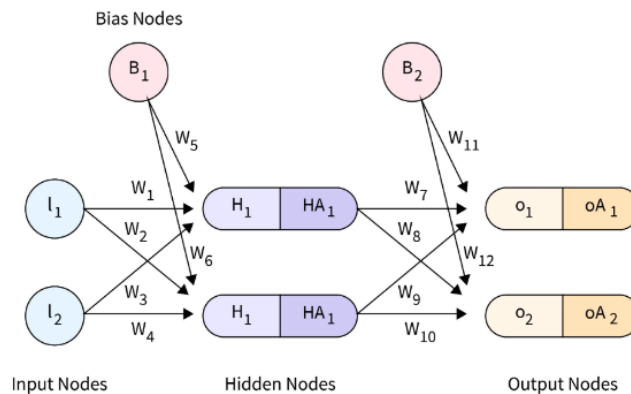


Figure 60: Architettura di una rete neurale feed-forward con input layer, hidden layer e output layer.

- **Input Layer:** accetta i dati e li trasmette. Nessun parametro apprendibile.
- **Hidden Layer:** eseguono trasformazioni non lineari (ReLU, sigmoid, ecc.) e apprendono rappresentazioni intermedie.
- **Output Layer:** produce la previsione finale (softmax per classificazione, lineare per regressione).

I.2 Forward Propagation

Per ogni neurone, si calcola l'input netto z e poi l'output tramite la funzione di attivazione f :

Forward pass di un neurone

$$z = \sum_i w_i x_i + b \quad \hat{y} = f(z)$$

I.3 Loss Function: MSE

Per misurare l'errore tra previsione e target si usa una loss function. Per problemi di regressione, la Mean Squared Error:

Mean Squared Error

$$E = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2$$

Per problemi di **classificazione multi-classe** si usa la **Cross-Entropy Loss**:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k$$

dove K è il numero di classi. La cross-entropy è la loss standard usata nei Transformer, nel ViT e nel Perceiver.

I.4 Backpropagation

La backpropagation calcola i gradienti della loss rispetto a ogni peso, propagando l'errore dall'output verso l'input tramite la **regola della catena**.

I 6 passi della Backpropagation

1. **Gradiente rispetto all'output:** $\frac{\partial E}{\partial \hat{y}} = \hat{y} - y$
2. **Gradiente rispetto all'input netto:** $\frac{\partial E}{\partial z} = \frac{\partial E}{\partial \hat{y}} \cdot f'(z)$
3. **Gradiente rispetto ai pesi:** $\frac{\partial E}{\partial w} = \frac{\partial E}{\partial z} \cdot x$
4. **Gradiente rispetto al bias:** $\frac{\partial E}{\partial b} = \frac{\partial E}{\partial z}$
5. **Propagazione all'indietro:** ripetere i passi 1–4 per ogni layer, dal fondo verso l'input
6. **Aggiornamento dei pesi:** $w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$, $b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial E}{\partial b}$

In parole semplici: la backpropagation come catena di montaggio

Immagina una **catena di montaggio** dove ogni operaio (neurone) modifica il pezzo. Se il prodotto finale è difettoso (loss alta), il controllo qualità (backpropagation) risale la catena chiedendo a ogni operaio:

“Quanto sei responsabile di questo difetto?”. L’operaio che ha contribuito di più riceve la correzione più grande (gradiente più grande). La **chain rule** è il meccanismo che permette di “scomporre” la responsabilità strato per strato.

Esempio numerico: backpropagation su rete semplice

Rete con 1 input $x = 2$, 1 hidden neuron con ReLU, 1 output con identità. Pesì: $w_1 = 0.5$, $w_2 = -0.3$, bias = 0. Target: $y_{\text{true}} = 1$.

Forward: $z_1 = 0.5 \times 2 = 1.0$, $a_1 = \text{ReLU}(1.0) = 1.0$, $\hat{y} = -0.3 \times 1.0 = -0.3$. **Loss MSE:** $\mathcal{L} = (-0.3 - 1)^2 = 1.69$.

Backward:

- $\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(-0.3 - 1) = -2.6$
- $\frac{\partial \mathcal{L}}{\partial w_2} = -2.6 \times a_1 = -2.6$
- $\frac{\partial \mathcal{L}}{\partial a_1} = -2.6 \times w_2 = -2.6 \times (-0.3) = 0.78$
- $\frac{\partial \mathcal{L}}{\partial z_1} = 0.78 \times \text{ReLU}'(1.0) = 0.78$
- $\frac{\partial \mathcal{L}}{\partial w_1} = 0.78 \times x = 0.78 \times 2 = 1.56$

Aggiornamento ($\eta = 0.01$): $w_2 \leftarrow -0.3 - 0.01 \times (-2.6) = -0.274$, $w_1 \leftarrow 0.5 - 0.01 \times 1.56 = 0.4844$.

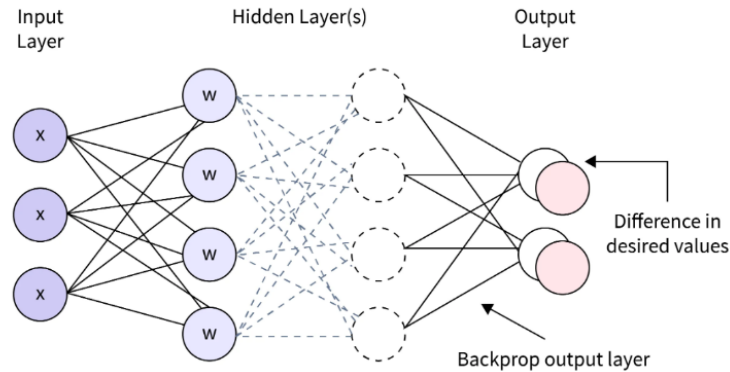


Figure 61: Schema della backpropagation in una rete feed-forward: l’errore viene propagato dall’output verso l’input attraverso gli hidden layer.

Le reti feed-forward trattano ogni input in modo indipendente, senza nozione di ordine o tempo. Per dati **sequenziali** (testo, audio, serie temporali) servono architetture con memoria: le reti neurali ricorrenti.

J Reti Neurali Ricorrenti (RNN)

Le **RNN** estendono le reti feed-forward aggiungendo un **loop nello stato nascosto**: l'output di h_{t-1} viene reinserito come input al passo successivo, creando una forma di **memoria**. Questo le rende adatte a dati sequenziali.

J.1 Il loop nello stato nascosto

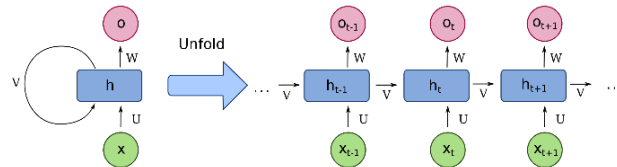


Figure 62: Schema di una RNN: il loop nello stato nascosto permette di mantenere memoria delle informazioni precedenti.

I parametri della RNN sono tre matrici condivise a ogni passo temporale:

- **W** — pesi sullo stato nascosto precedente h_{t-1}
- **U** — pesi sull'input corrente x_t
- **V** — pesi dallo stato nascosto all'output

Equazioni della RNN

$$h_t = \tanh(W h_{t-1} + U x_t + b_h) \quad y_t = V h_t + b_y$$

Collegamento al Perceiver

I pesi **W**, **U**, **V** sono **condivisi in tutti i passi temporali**: questa idea di *weight sharing* è la stessa che il Perceiver porta all'estremo, riutilizzando gli stessi blocchi di cross-attention e self-attention per ogni iterazione di raffinamento del latent array.

J.2 Srotolamento nel tempo

Una RNN può essere “srotolata” (unrolled) in una catena di copie identiche, una per ogni passo temporale, ottenendo di fatto una rete feed-forward molto profonda.

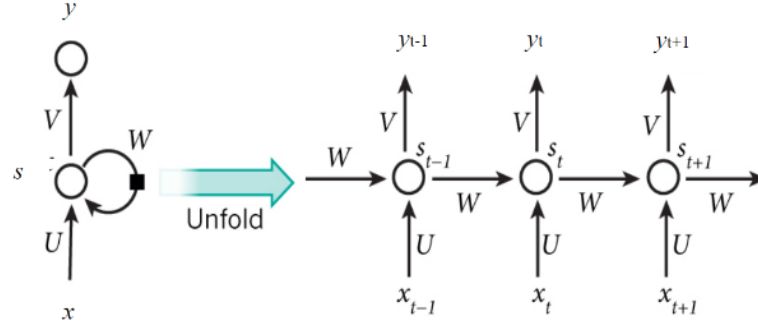


Figure 63: Srotolamento (unrolling) di una RNN nel tempo: ogni copia condivide gli stessi pesi.

Esempio numerico: forward pass di una RNN

RNN con $d_h = 2$ e $d_x = 1$. Pesi: $\mathbf{W} = \begin{pmatrix} 0.5 & 0.1 \\ -0.2 & 0.4 \end{pmatrix}$, $\mathbf{U} = \begin{pmatrix} 0.6 \\ 0.3 \end{pmatrix}$, $\mathbf{b}_h = (0, 0)^\top$. Sequenza: $x_1 = 1.0$, $x_2 = 0.5$, $x_3 = -0.3$. Stato iniziale: $\mathbf{h}_0 = (0, 0)^\top$.

Passo $t = 1$: $\mathbf{W}\mathbf{h}_0 + \mathbf{U}x_1 = (0.6, 0.3)^\top \Rightarrow \mathbf{h}_1 = \tanh = (0.537, 0.291)^\top$.

Passo $t = 2$: $\mathbf{W}\mathbf{h}_1 + \mathbf{U}x_2 = (0.598, 0.159)^\top \Rightarrow \mathbf{h}_2 = (0.535, 0.158)^\top$.

Passo $t = 3$: $\mathbf{W}\mathbf{h}_2 + \mathbf{U}x_3 = (0.104, -0.134)^\top \Rightarrow \mathbf{h}_3 = (0.104, -0.133)^\top$.

Osservazione: $\mathbf{h}_3$ dipende da tutti gli input — la RNN ha accumulato informazione passo dopo passo. Ma l’influenza di x_1 su $\mathbf{h}_3$ è ormai molto attenuata: questo è il vanishing gradient in azione, anche dopo soli 3 passi.

J.3 Vanishing e Exploding Gradient

Quando si fa backpropagation through time, il gradiente attraversa molte moltiplicazioni matriciali. Se gli autovalori di $\mathbf{W}$ sono < 1 il gradiente **svanisce**; se > 1 , **esplode**.

Prodotto dei gradienti nel tempo

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1} = \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \quad \rightarrow \quad \text{se } T \text{ è grande: } \rightarrow 0 \text{ (vanishing) o } \rightarrow \infty \text{ (exploding)}$$

L’**exploding gradient** si può mitigare con il **gradient clipping**: se la norma del gradiente supera una soglia τ , si riscalda:

Gradient Clipping

$$\mathbf{g} \leftarrow \frac{\tau}{\|\mathbf{g}\|} \cdot \mathbf{g} \quad \text{se } \|\mathbf{g}\| > \tau$$

Il vanishing gradient è invece più insidioso. Nella pratica, le RNN standard riescono a catturare dipendenze solo fino a circa 10 passi temporali. Per sequenze lunghe servono architetture con gate espliciti: LSTM e GRU.

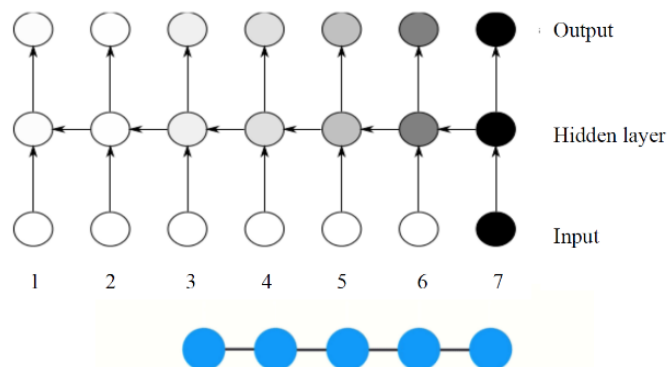


Figure 64: Vanishing e exploding gradient nelle RNN: il gradiente si degrada lungo la sequenza temporale.

K LSTM (Long Short-Term Memory)

Le **LSTM**, introdotte da **Hochreiter & Schmidhuber (1997)** in “Long Short-Term Memory” (Neural Computation), risolvono il vanishing gradient introducendo una **cella di memoria** C_t protetta da tre porte (gate) che controllano il flusso di informazione.

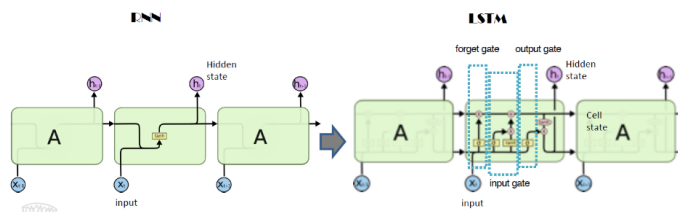


Figure 65: Struttura di una cella LSTM con le tre porte (forget, input, output) e la cella di memoria.

K.1 Architettura: le 4 equazioni

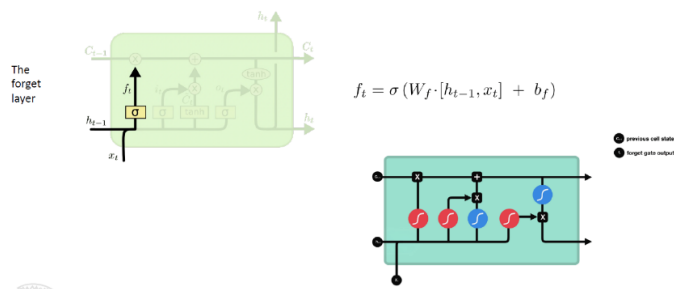


Figure 66: Forget gate della LSTM: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$ decide quali informazioni scartare dalla cella.

Equazioni della LSTM

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{Forget gate}) \quad (1)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{Input gate}) \quad (2)$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_C [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (\text{State update}) \quad (3)$$

$$\mathbf{h}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \odot \tanh(\mathbf{C}_t) \quad (\text{Output gate}) \quad (4)$$

In parole semplici

- **Forget gate** ($\mathbf{f}_t$): decide cosa *dimenticare* dalla memoria precedente ($\sigma \approx 0 \rightarrow$ dimentica, $\sigma \approx 1 \rightarrow$ mantieni).
- **Input gate** ($\mathbf{i}_t$): decide cosa *aggiungere* di nuovo alla memoria.
- **State update** ($\mathbf{C}_t$): aggiorna la cella combinando il vecchio stato (filtrato) con le nuove informazioni.
- **Output gate**: decide quale parte della memoria esporre come output $\mathbf{h}_t$.

In parole semplici: LSTM che legge un testo

Immagina una LSTM che legge: “Il gatto, che era nero e aveva gli occhi verdi, **corse** via”. La cella di memoria deve ricordare “il gatto” (soggetto singolare) attraverso tutta la subordinata. Il **forget gate** decide di non dimenticare il soggetto. L'**input gate** aggiunge informazioni sugli aggettivi senza sovrascrivere il soggetto. Quando arriva il verbo, l'**output gate** produce lo stato nascosto che codifica “soggetto singolare”, permettendo di concordare correttamente il verbo.

Esempio numerico: forward pass LSTM gate per gate

LSTM con $d_h = 2$, $d_x = 1$. Stato iniziale: $\mathbf{h}_{t-1} = (0.2, -0.1)^\top$, $\mathbf{C}_{t-1} = (0.5, 0.3)^\top$, input $x_t = 0.7$. Vettore concatenato: $[\mathbf{h}_{t-1}, x_t] = (0.2, -0.1, 0.7)$.

Passo 1 — Forget Gate. $\mathbf{f}_t = \sigma(0.69, 0.41)^\top = (0.666, 0.601)^\top$ — mantiene buona parte della cella.

Passo 2 — Input Gate. $\mathbf{i}_t = \sigma(0.28, 0.23)^\top = (0.570, 0.557)^\top$.

Passo 3 — Candidato. $\tilde{\mathbf{C}}_t = \tanh(0.47, -0.15)^\top = (0.438, -0.149)^\top$.

Passo 4 — Aggiornamento della cella: $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t = (0.583, 0.097)^\top$.

Passo 5 — Output Gate: $\mathbf{o}_t = \sigma(0.26, -0.01)^\top = (0.565, 0.498)^\top$. $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) = (0.296, 0.048)^\top$.

Interpretazione: la cella ha mantenuto buona parte della prima componente ($0.5 \rightarrow 0.583$) ma la seconda è scesa ($0.3 \rightarrow 0.097$). Lo stato nascosto è una versione “filtrata” della cella.

Collegamento al Perceiver

La cella $\mathbf{C}_t$ funziona come un “nastro trasportatore” che trasporta informazione lungo la sequenza senza degradarla — la stessa intuizione delle **residual connections**, che nel Perceiver sono presenti in ogni blocco di attention (112 in totale).

Le LSTM funzionano bene ma hanno molti parametri. Una versione semplificata con prestazioni comparabili è la GRU.

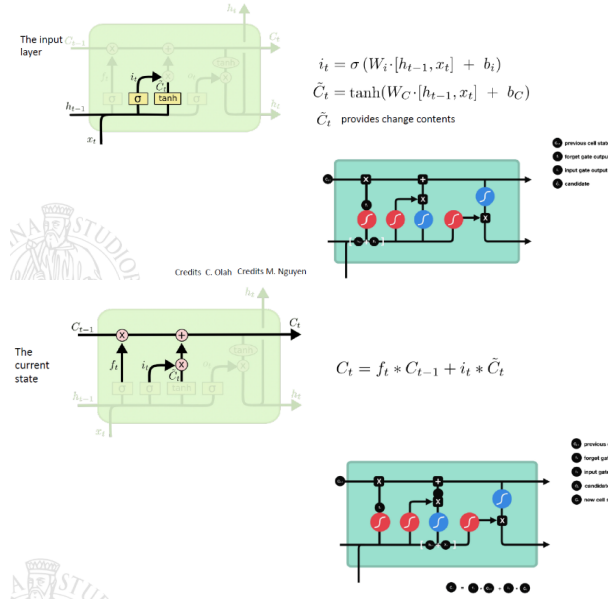


Figure 67: Sinistra: input gate e candidato (i_t , $\tilde{C}_t$). Destra: aggiornamento della cella di memoria ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$).

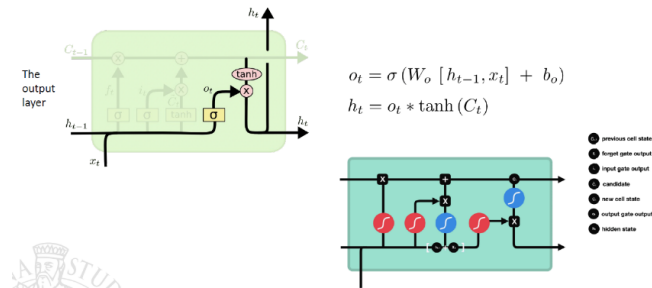


Figure 68: Output gate della LSTM: $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$ e stato nascosto $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$.

L GRU (Gated Recurrent Unit)

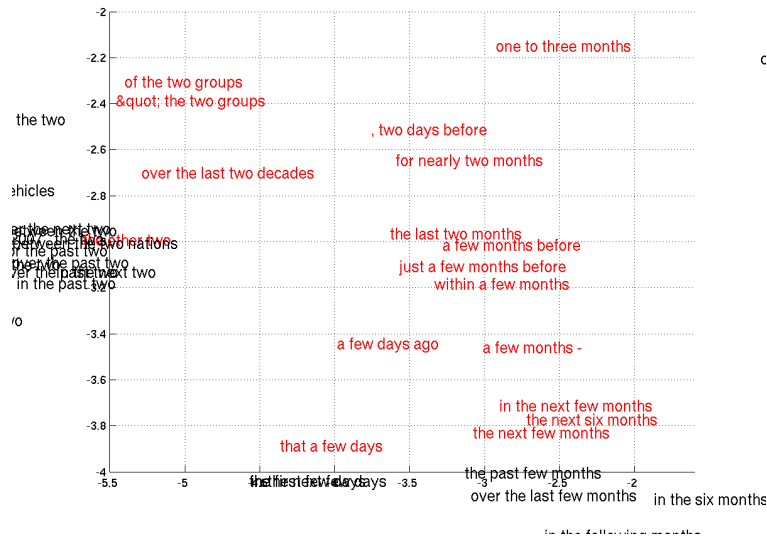


Figure 69: Spazio di embedding 2D delle frasi appreso dall’RNN Encoder-Decoder: frasi di significato simile si raggruppano. *Fonte:* Cho et al., *RNN Encoder-Decoder* (arXiv:1406.1078).

La **GRU**, introdotta da **Cho et al. (2014)** in “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation” (EMNLP), semplifica la LSTM fondendo cella e stato nascosto e usando solo **2 porte** invece di 3.

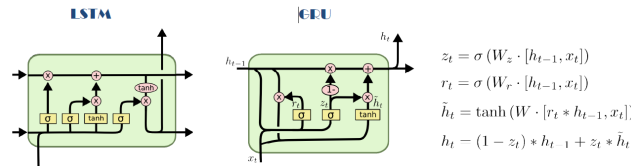


Figure 70: Confronto tra LSTM (sinistra) e GRU (destra): la GRU semplifica la struttura fondendo cella e stato nascosto e usando 2 porte invece di 3.

Equazioni della GRU

$$\mathbf{r}_t = \sigma(\mathbf{W}_r [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (\text{Reset gate}) \quad (5)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (\text{Update gate}) \quad (6)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}) \quad (\text{Candidato}) \quad (7)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{Stato aggiornato}) \quad (8)$$

Esempio numerico: forward pass di una GRU

GRU con $d_h = 2$, $d_x = 1$. Stato: $\mathbf{h}_{t-1} = (0.5, -0.3)^\top$, input $x_t = 0.8$. Concatenato: $(0.5, -0.3, 0.8)$. **Reset Gate:** $\mathbf{r}_t = \sigma(0.41, 0.33)^\top = (0.601, 0.582)^\top$. **Update Gate:** $\mathbf{z}_t = \sigma(0.21, -0.15)^\top = (0.552, 0.463)^\top$. **Candidato:** $\mathbf{r}_t \odot \mathbf{h}_{t-1} = (0.301, -0.175)^\top$; $\tilde{\mathbf{h}}_t = \tanh(0.605, -0.297)^\top = (0.541, -0.289)^\top$. **Interpolazione:** $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t = (0.523, -0.295)^\top$. **Interpretazione:** $\mathbf{z}_t \approx (0.55, 0.46)$ — la GRU “mescola” circa metà vecchio stato e metà candidato: aggiornamento graduale.

L.1 Confronto GRU vs LSTM

Aspetto	LSTM	GRU
Porte	3 (forget, input, output)	2 (reset, update)
Stato interno	$\mathbf{C}_t$ e $\mathbf{h}_t$ separati	Solo $\mathbf{h}_t$
Parametri	Più numerosi	~25% in meno
Sequenze lunghe	molto	Leggermente migliore
Velocità di training	Più lenta	Comparabile
		Più veloce

Mentre le RNN (e le loro varianti LSTM/GRU) eccellono con dati sequenziali, per dati strutturati in forma di griglia — come le **immagini** — servono architetture in grado di sfruttare la struttura spaziale locale. Le reti neurali convoluzionali sono state progettate esattamente per questo scopo.

M Reti Neurali Convoluzionali (CNN)

Le **CNN** sono progettate per elaborare dati strutturati a griglia (immagini, video). Invece di connettere ogni neurone a tutti gli input, usano **filtri locali** (kernel) che scorrono sull’immagine per estrarre feature. Sono il predecessore diretto del ViT e, indirettamente, del Perceiver.

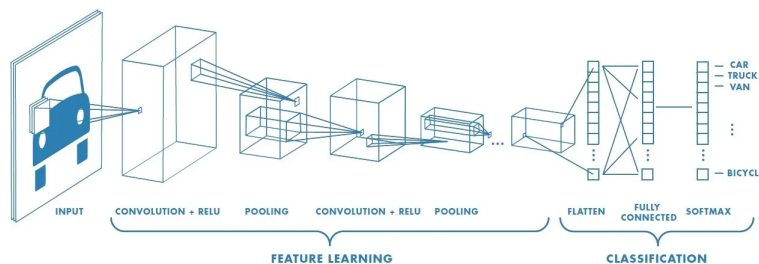


Figure 71: Architettura tipica di una CNN: convoluzioni, pooling e fully connected layer.

M.1 Architettura overview

Una CNN tipica alterna tre tipi di layer:

1. **Convolution Layer** — estrae feature locali tramite filtri (kernel)
2. **Pooling Layer** — riduce la dimensione spaziale mantenendo le feature più importanti
3. **Fully Connected Layer** — classifica le feature estratte

I Convolution Layer iniziali catturano feature di basso livello (bordi, texture); quelli profondi catturano feature complesse (volti, oggetti).

M.2 Convolution Layer

Un filtro (kernel) è una piccola matrice di pesi che **scorre sistematicamente** sull'immagine. In ogni posizione compie tre operazioni in sequenza: (1) moltiplica ogni suo valore per il pixel corrispondente, (2) somma tutti i prodotti, (3) scrive il risultato in una cella della **feature map**. Poi si sposta e ricomincia.

La cosa da visualizzare è questa: il kernel è un **rilevatore riutilizzato**. Non impara un peso diverso per ogni pixel dell'immagine; impara un piccolo pattern e lo prova in tutte le posizioni. È qui che nascono insieme *efficienza* (pochi pesi condivisi) e *località* (ogni risposta dipende solo da una piccola regione).

Convoluzione vs cross-correlation

La *convoluzione matematica* richiederebbe di ruotare il kernel di 180° prima di applicarlo. Le CNN reali (PyTorch, TensorFlow, Keras) **non ruotano il kernel**: eseguono in realtà una **cross-correlation**. Poiché i pesi vengono comunque appresi, la distinzione non cambia il risultato pratico, ma tecnicamente tutto ciò che chiamiamo “convoluzione” nel deep learning è una cross-correlation.

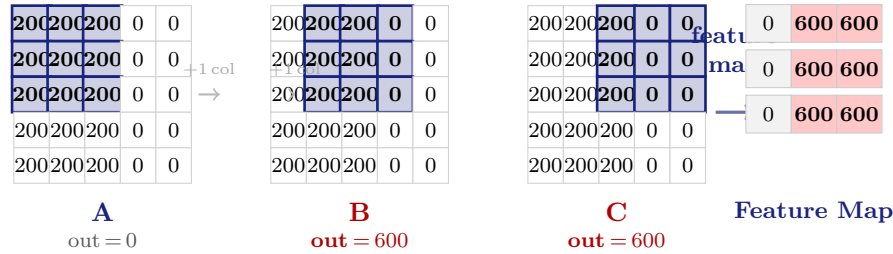
Cosa rappresentano i numeri? Un'immagine in scala di grigi è una matrice di interi da 0 (nero) a 255 (bianco): ogni cella corrisponde a **un pixel**. Un'immagine 5×5 è letteralmente una griglia di 25 valori di intensità luminosa.

Esempio reale: immagine 5×5 con una zona chiara a sinistra e una zona nera a destra, separata da un bordo verticale.

$$X = \begin{bmatrix} 200 & 200 & 200 & 0 & 0 \\ 200 & 200 & 200 & 0 & 0 \\ 200 & 200 & 200 & 0 & 0 \\ 200 & 200 & 200 & 0 & 0 \\ 200 & 200 & 200 & 0 & 0 \end{bmatrix} \quad 200 = \text{grigio chiaro}; \quad 0 = \text{nero}$$

Scorrendo il kernel di un pixel alla volta (**stride** = 1, cioè lo spostamento tra una posizione e la successiva) e senza aggiungere bordi artificiali attorno all'immagine (**padding** = 0), il kernel 3×3 può occupare $3 \times 3 = 9$ posizioni distinte, producendo una feature map 3×3 . Stride e padding verranno approfonditi più avanti.

Visualizzazione dello scorrimento — il riquadro colorato indica la zona 3×3 che il kernel sta esaminando:



Come funziona il kernel: passo per passo

Posizione A — kernel su zona uniforme (colonne 0–2, tutte a 200):

$$\underbrace{\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}}_{\text{kernel}} \text{ su } \underbrace{\begin{bmatrix} 200 & 200 & 200 \\ 200 & 200 & 200 \\ 200 & 200 & 200 \end{bmatrix}}_{\text{zona A}}$$

$$(1 \cdot 200) + (0 \cdot 200) + (-1 \cdot 200) + (\dots) + (\dots) = 0 + 0 + 0 = 0$$

Luminosità uniforme: i contributi si annullano. Nessun bordo.

Posizione B — kernel sul bordo (colonne 1–3, transizione 200→0):

$$\underbrace{\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}}_{\text{kernel}} \text{ su } \underbrace{\begin{bmatrix} 200 & 200 & 0 \\ 200 & 200 & 0 \\ 200 & 200 & 0 \end{bmatrix}}_{\text{zona B}}$$

$$\underbrace{(1 \cdot 200)}_{200} + \underbrace{(0 \cdot 200)}_0 + \underbrace{(-1 \cdot 0)}_0 + \underbrace{(1 \cdot 200)}_{200} + \underbrace{(0 \cdot 200)}_0 + \underbrace{(-1 \cdot 0)}_0 + \underbrace{(1 \cdot 200)}_{200} + \underbrace{(0 \cdot 200)}_0 + \underbrace{(-1 \cdot 0)}_0 = 600$$

Sinistra luminosa (200), destra nera (0): differenza massima. Bordo rilevato!

Feature map completa (dopo aver scansionato tutta l'immagine con stride 1):

$$\text{feature map} = \begin{bmatrix} 0 & 600 & 600 \\ 0 & 600 & 600 \\ 0 & 600 & 600 \end{bmatrix}$$

I valori alti (600) segnalano la presenza del bordo verticale in quella posizione. In forma compatta: $\sum_{i,j} K_{ij} \cdot X_{ij}$.

Perché questo filtro rileva bordi verticali? Il kernel ha pesi +1 a sinistra e -1 a destra. Dove la sinistra è luminosa e la destra è scura, la differenza è grande $\Rightarrow$ output alto. Dove la luminosità è uniforme, i contributi positivi e negativi si annullano $\Rightarrow$ output vicino a zero. Il filtro misura quindi una *transizione laterale di luminosità*, cioè un bordo verticale.

Feature map

La feature map è la matrice di output che raccoglie tutti i risultati:

$$\text{feature map}(i, j) = \sum_{m, n} K_{mn} \cdot X_{i+m, j+n}$$

i, j posizione corrente del kernel nell'immagine di output

m, n coordinate interne al kernel (da 0 a $k - 1$)

Ogni cella dice **quanto** il pattern cercato dal filtro è presente in quella posizione. Valore alto = pattern trovato; valore basso (o zero dopo ReLU) = pattern assente.

Bias. In pratica, al risultato della convoluzione viene aggiunto un **bias apprendibile** b : $\text{output} = (K * X) + b$. Il bias consente al filtro di traslare la soglia di attivazione indipendentemente dall'input, aumentando l'espressività del modello.

Perché ReLU? Dopo la convoluzione si applica $\text{ReLU}(x) = \max(0, x)$, che azzera i valori negativi e mantiene quelli positivi. Senza attivazione non lineare, componendo solo operazioni lineari, l'intera rete sarebbe equivalente a una singola trasformazione lineare e non potrebbe imparare pattern complessi.

Esempio ReLU

Input: $[-5, 2, 7, -1]$ $\xrightarrow{\text{ReLU}}$ Output: $[0, 2, 7, 0]$

Stride e Padding. Lo **stride** s indica di quanti pixel si sposta il kernel a ogni passo:

- $s = 1$: si sposta di un pixel, output grande, molti dettagli.
- $s = 2$: salta un pixel, output più piccolo, meno calcoli.

Il **padding** p aggiunge un bordo di zeri attorno all'immagine. Senza padding l'immagine si restringe dopo ogni convoluzione; con padding si mantengono le dimensioni originali.

Dimensione dell'output

Data un'immagine $H \times W$, filtro $k \times k$, stride s , padding p :

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1 \quad W_{\text{out}} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1$$

Lettura: il numeratore è lo spazio disponibile dopo padding e rimozione del kernel; dividendo per s si ottiene quante posizioni può visitare; il $+1$ aggiunge la posizione iniziale.

I filtri vengono appresi, non scritti a mano

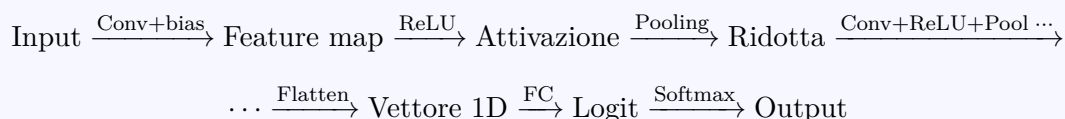
Nelle CNN reali i kernel **non sono progettati manualmente**. All'inizio hanno pesi casuali; durante il training la backpropagation li aggiorna automaticamente. La rete impara:

- nei primi layer: bordi, gradienti di colore, texture semplici;
- nei layer intermedi: forme geometriche, pattern ricorrenti;
- nei layer profondi: parti di oggetti, volti, strutture complesse.

È questo apprendimento gerarchico che rende le CNN così potenti.

Immagini a colori (RGB). Un'immagine reale ha 3 canali (R, G, B). Il kernel non è $k \times k$ ma $k \times k \times 3$: opera su tutti e tre i canali simultaneamente. Ogni filtro combina i tre canali e produce **una feature map completa**. Con F filtri si ottengono F feature map in output — una per ogni pattern cercato.

Pipeline completa di una CNN



I blocchi Conv+ReLU+Pool si ripetono più volte (layer profondi = feature più astratte); il Flatten porta a uno o più layer Fully Connected per la classificazione finale.

M.3 Pooling

Il pooling riduce la dimensione spaziale della feature map mantenendo le informazioni più rilevanti. Serve a tre scopi: (1) diminuire il costo computazionale, (2) ridurre l'overfitting, (3) rendere la rete meno sensibile alla posizione esatta dei pattern.

Max-pooling. Divide la feature map in finestre non sovrapposte e prende il valore massimo di ciascuna. Con finestra 2×2 e stride 2, ogni quadrante si riduce a un singolo valore:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & \mathbf{6} & 7 & \mathbf{8} \\ 9 & 10 & 11 & 12 \\ 13 & \mathbf{14} & 15 & \mathbf{16} \end{bmatrix} \xrightarrow{\text{Max-Pool } 2 \times 2} \begin{bmatrix} 6 & 8 \\ 14 & 16 \end{bmatrix}$$

Il massimo cattura *se* il pattern è presente nella finestra, indipendentemente da dove si trova esattamente. Per questo le CNN sono robuste a piccole traslazioni dell'input.

Average-pooling. Prende la media invece del massimo: conserva informazioni diffuse su tutta la finestra. Utile negli strati finali di alcune architetture (es. Global Average Pooling che comprime ogni feature map a un solo valore).

Dimensione dopo pooling

Con finestra $p \times p$ e stride s :

$$H_{\text{out}} = \left\lfloor \frac{H - p}{s} \right\rfloor + 1$$

Esempio: feature map 28×28 con Max-Pool 2×2 stride 2 $\Rightarrow$ output 14×14 .

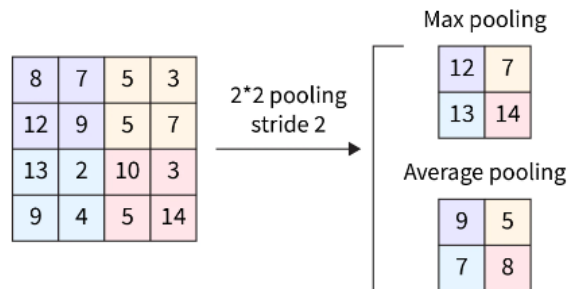


Figure 72: Max pooling e Average pooling a confronto.

Attenzione: il pooling è irreversibile

Il pooling è utile perché riduce costo e dimensioni, ma è anche **irreversibile**: una volta tenuto solo il massimo (o la media) di ogni finestra, i dettagli scartati non sono più recuperabili. È un compromesso voluto — si rinuncia alla posizione esatta in cambio di robustezza e compattezza.

M.4 Fully Connected e Output

Dopo gli strati convoluzionali le feature map vengono **appiattite** (flatten) in un unico vettore 1D. Esempio: 32 feature map di dimensione 7×7 diventano un vettore di $32 \times 7 \times 7 = 1\,568$ valori. Questo vettore riassume tutto ciò che la rete ha rilevato nell'immagine.

Layer Fully Connected

Il layer FC calcola:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

dove $\mathbf{x}$ è il vettore appiattito, W la matrice dei pesi e $\mathbf{b}$ il bias. Ogni neurone di output è connesso a *tutti* i valori del vettore (da qui il nome “fully connected”).

Output finale.

- **Classificazione multiclasse:** si applica **softmax**, che converte i logit in probabilità che sommano a 1:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Esempio: logit $[2.1, 0.5, 1.8] \xrightarrow{\text{softmax}} [0.57, 0.12, 0.41]$.

La classe predetta è quella con probabilità più alta.

- **Classificazione binaria:** si applica **sigmoid**, che mappa l'output in $[0, 1]$: $\sigma(z) = 1/(1 + e^{-z})$.

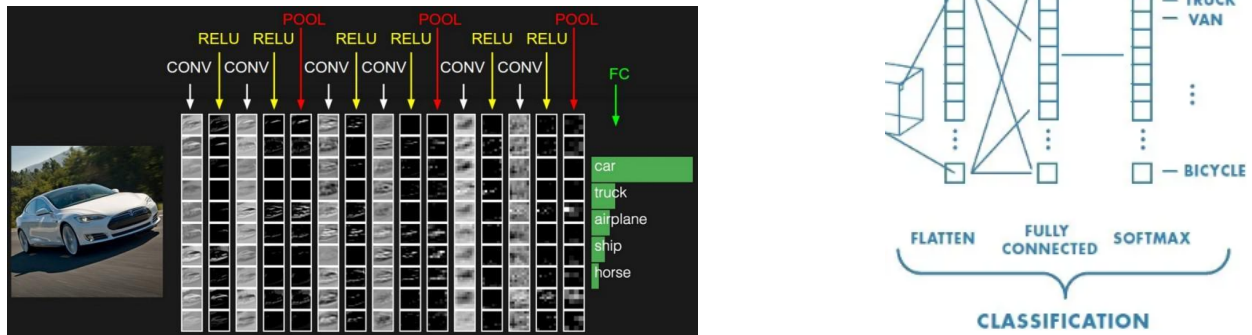


Figure 73: Sinistra: visualizzazione delle feature apprese dai filtri convoluzionali (dai bordi alle strutture complesse). Destra: lo stadio di classificazione (Flatten $\rightarrow$ FC $\rightarrow$ Softmax).

M.5 Training

Il training segue lo stesso ciclo descritto nella Sezione 2 (feed-forward): forward pass $\rightarrow$ calcolo della loss $\rightarrow$ backpropagation $\rightarrow$ aggiornamento dei pesi. La differenza rispetto a un MLP è che i gradienti vengono propagati *anche attraverso i filtri convoluzionali*: sono i pesi dei kernel K_{ij} ad essere aggiornati, non solo quelli dei layer FC.

Cosa si aggiorna durante il training

Layer convoluzionale pesi del kernel K_{ij} e bias
Layer FC matrice W e bias $\mathbf{b}$

La backprop calcola $\partial \mathcal{L} / \partial K_{ij}$ e aggiorna ogni peso del filtro. Nel tempo la rete impara quali pattern cercare, senza che nessuno li specifichi manualmente.

In parole semplici: i due inductive bias delle CNN

Le CNN sfruttano due proprietà fondamentali delle immagini:

1. **Località:** i pixel vicini sono correlati. Ha senso analizzare l'immagine con filtri piccoli piuttosto che collegare ogni pixel a ogni neurone. Questo riduce drasticamente i parametri.
2. **Invarianza alla traslazione:** un gatto resta un gatto ovunque nell'immagine. La *condivisione dei pesi* (lo stesso kernel si applica ovunque) garantisce questa proprietà.

Questi inductive bias rendono le CNN efficienti per le immagini, ma le "imprigionano": una CNN non sa gestire dati senza struttura a griglia. Il Perceiver rinuncia completamente a entrambi, trattando l'input come una sequenza generica.

Esempio numerico: conteggio parametri di C1 in LeNet-5

Lo strato C1 ha 6 filtri $5 \times 5 \times 1$, ciascuno con un bias: Parametri C1 = $6 \times (5 \times 5 \times 1 + 1) = 6 \times 26 = 156$. Confronto: un MLP fully connected richiederebbe $1024 \times 4704 \approx 4.8\text{M}$ parametri — **30 000 volte di più**.

Aumentando la profondità di una CNN, si incorre negli stessi problemi delle RNN: **vanishing gradient** e **degradazione delle prestazioni**. La ResNet risolve entrambi con un'idea semplice ma potente.

M.6 ResNet e Residual Connections

ResNet (He et al., 2016) introduce le **skip connections**: invece di imparare una trasformazione complessa $H(\mathbf{x})$, la rete impara solo la differenza residua $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$.

Residual Connection

$$\mathbf{y} = F(\mathbf{x}, \mathbf{W}) + \mathbf{x}$$

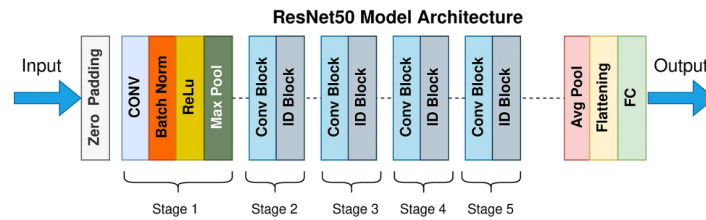


Figure 74: Skip connection nella ResNet: il blocco apprende solo il residuo $F(\mathbf{x})$.

Perché funziona:

- Il gradiente può fluire direttamente attraverso la skip connection, evitando il vanishing gradient.
- Se un layer non è utile, la rete può “saltarlo” ($F(\mathbf{x}) \approx 0 \Rightarrow \mathbf{y} \approx \mathbf{x}$).
- Permette di addestrare reti molto profonde (50, 101, 152+ layer).

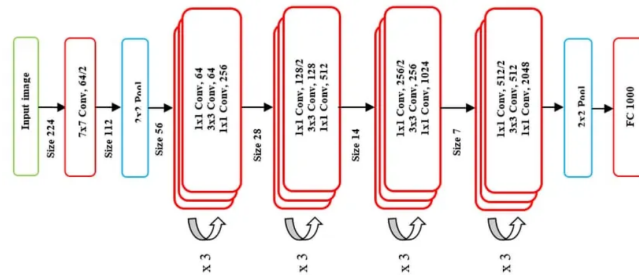


Figure 75: Architettura dettagliata di un blocco residuale della ResNet.

Degradazione vs vanishing gradient

Il **problema della degradazione** è distinto dal vanishing gradient: aggiungere più strati peggiorava le prestazioni *anche sul training set*. Una rete a 56 strati funzionava peggio di una a 20, nonostante contenga la soluzione della rete più bassa. Le skip connections risolvono il problema rendendo l'identità il punto di partenza: il blocco deve apprendere solo $F(\mathbf{x}) \approx 0$.

Bottleneck block (ResNet-50+)

Per reti molto profonde si usa un **bottleneck block** con tre convoluzioni:

1. **Conv** 1×1 (riduzione): da 256 a 64 canali.
2. **Conv** 3×3 : operazione convoluzionale nello spazio ridotto.
3. **Conv** 1×1 (espansione): da 64 a 256 canali.

Riduce i FLOP di circa $3\times$. Concettualmente simile al **latent bottleneck del Perceiver**: comprimere, elaborare, ri-espandere.

Collegamento al Perceiver

Il Perceiver usa **112 residual connections** (una per ogni blocco di attention e FFN). Senza di esse, il training di un modello così profondo sarebbe impossibile.

Confronto CNN tradizionale vs ResNet-50

Aspetto	CNN tradizionale	ResNet-50
Architettura	Sequenziale: Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ FC	5 stadi con Residual Blocks + skip connections
Flusso info	Solo sequenziale	Doppio: sequenziale + scorciatoia
Profondità	5–19 layer (poi instabile)	50+ layer, stabile
Residual Learning	Assente	$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
Vanishing gradient	Presente in reti profonde	Risolto dalle skip connections
ImageNet top-5 error	AlexNet 15.3%, VGG 7.3%	ResNet-50: 3.57%

Le CNN (anche con ResNet) catturano pattern **locali** tramite filtri di dimensione fissa. Per catturare relazioni **globali** tra tutti gli elementi di una sequenza in un singolo passo, servono i Transformer.

N Transformer

Le CNN hanno visione locale (limitata al kernel), le RNN soffrono di vanishing gradient su sequenze lunghe. Il **Transformer** (Vaswani et al., 2017, “Attention Is All You Need”) supera entrambi i limiti con il meccanismo di **self-attention**, che mette in relazione *ogni* elemento con tutti gli altri in un singolo passo, con complessità $O(M^2)$ dove M è la lunghezza della sequenza.

N.1 Architettura overview

Il Transformer segue uno schema **encoder-decoder**:

- **Encoder**: elabora l’input e produce una rappresentazione contestualizzata. Ogni token “vede” tutti gli altri token tramite self-attention.
- **Decoder**: genera l’output sequenziale, condizionato dall’encoder. Usa masked self-attention (non può vedere token futuri) e cross-attention (legge dall’encoder).
- Entrambi usano: input/output embedding + positional encoding.

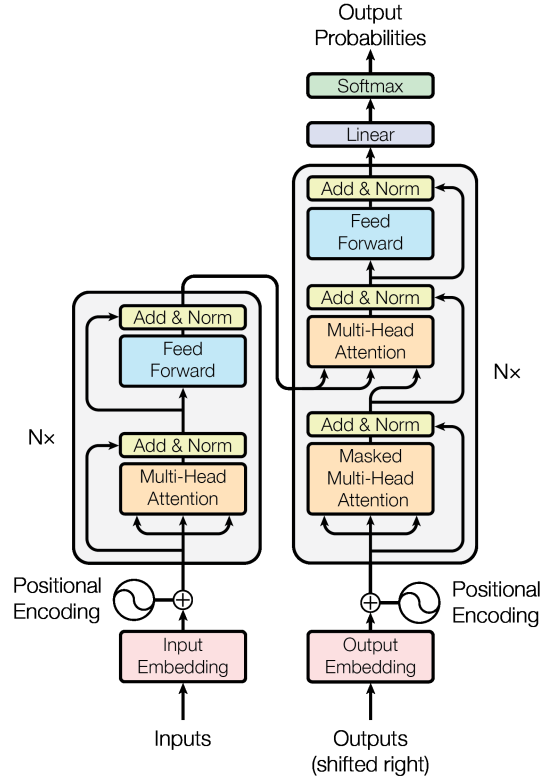


Figure 76: Architettura del Transformer: encoder (sinistra) e decoder (destra) impilati $N \times$, con multi-head attention, feed-forward, connessioni residue (Add & Norm) e positional encoding. *Fonte: Vaswani et al., Attention Is All You Need (arXiv:1706.03762).*

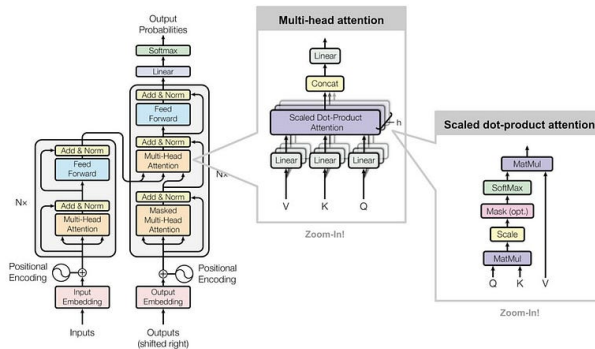


Figure 77: Architettura del Transformer: encoder (sinistra) e decoder (destra). Entrambi sono composti da blocchi ripetuti $N \times$ contenenti attention + FFN + residual + LayerNorm.

N.2 Input: Tokenizzazione ed Embedding

Prima di entrare nel Transformer, il testo grezzo viene trasformato in vettori numerici:

1. **Tokenizzazione**: il testo viene diviso in unità (token). Metodi comuni: word-level (“gatto” → un token), subword (BPE/SentencePiece: “giocando” → “gioc” + “ando”), byte-level (usato nel Perceiver IO).
2. **Embedding**: ogni token viene mappato in un vettore denso tramite una tabella appresa $E \in \mathbb{R}^{|\text{vocab}| \times d}$, dove d è la dimensione del modello (es. $d = 512$ nel Transformer originale).
3. **Positional Encoding**: viene sommato all’embedding per fornire informazione sulla posizione (vedi sotto).

$$\mathbf{z}_i = \text{Embed}(\text{token}_i) + \text{PE}(i) \in \mathbb{R}^d$$

N.3 Encoder

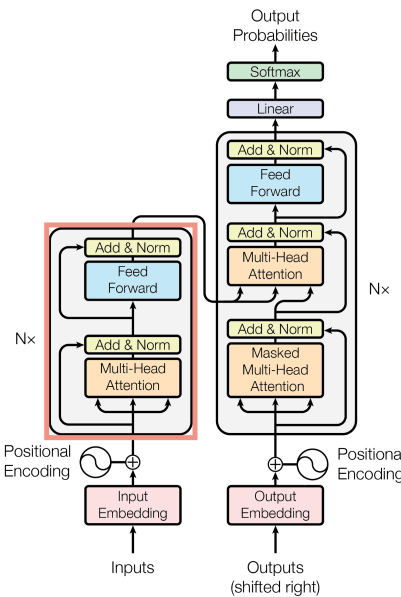


Figure 78: Struttura di un singolo blocco dell’encoder del Transformer: Multi-Head Self-Attention → Add & Norm → FFN → Add & Norm.

L’encoder è una pila di N layer identici (nel paper originale $N = 6$). Ogni layer contiene:

1. **Multi-Head Self-Attention**
2. **Feed-Forward Network (FFN)**

Ogni sotto-blocco è seguito da una **residual connection** e **Layer Normalization**:

$$\text{output} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

LayerNorm, non BatchNorm

Il Transformer usa **Layer Normalization**, che normalizza lungo la dimensione delle feature di ciascun singolo token, indipendentemente dal batch. Questo la rende adatta a sequenze di lunghezza variabile, a differenza della Batch Norm (vedi Appendice D).

Pre-norm vs post-norm

Il Transformer originale usa **post-norm**: $\text{LN}(\mathbf{x} + f(\mathbf{x}))$. Le architetture più recenti (GPT-2, Perceiver) usano **pre-norm**: $\mathbf{x} + f(\text{LN}(\mathbf{x}))$. La pre-norm è più stabile per reti molto profonde perché ogni sotto-blocco riceve input normalizzati, evitando che le attivazioni crescano in modo incontrollato.

N.4 Scaled Dot-Product Attention

Il cuore del Transformer. L'input viene proiettato in tre spazi tramite matrici di peso apprese: **Query (Q)**, **Key (K)**, **Value (V)**.

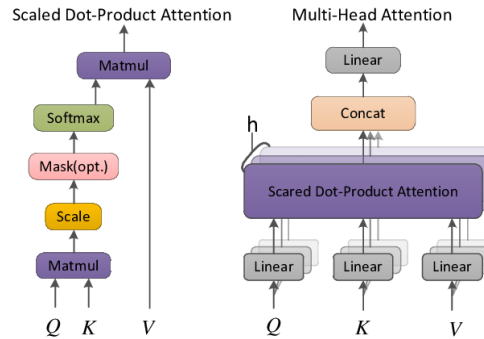


Figure 79: Schema della Scaled Dot-Product Attention (sinistra) e della Multi-Head Attention (destra).

Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Interpretazione passo per passo:

1. **MatMul**: $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{M \times M}$ misura la **somiglianza** tra ogni coppia di token. L'elemento (i, j) indica quanto il token i è “interessato” al token j .
2. **Scale**: si divide per $\sqrt{d_k}$ per stabilizzare i gradienti.
3. **Mask** (opzionale): nel decoder, si pone $-\infty$ nelle posizioni future per impedire al modello di “barare”.
4. **Softmax**: trasforma i punteggi in **pesi di attenzione** normalizzati (probabilità che sommano a 1 per ogni riga).
5. **MatMul**: si moltiplicano i pesi per $\mathbf{V}$ per ottenere la rappresentazione contestualizzata.

Perché dividere per $\sqrt{d_k}$?

Se le componenti di $\mathbf{Q}$ e $\mathbf{K}$ hanno media 0 e varianza 1, il prodotto scalare $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ ha varianza d_k (somma di d_k prodotti indipendenti con varianza 1). Per valori grandi di d_k (es. 64 o 128), i punteggi possono diventare molto grandi, spingendo la **softmax** in regioni di saturazione dove i gradienti sono quasi zero. Dividendo per $\sqrt{d_k}$ si riporta la varianza a ~ 1 .

Scaled Dot-Product Attention

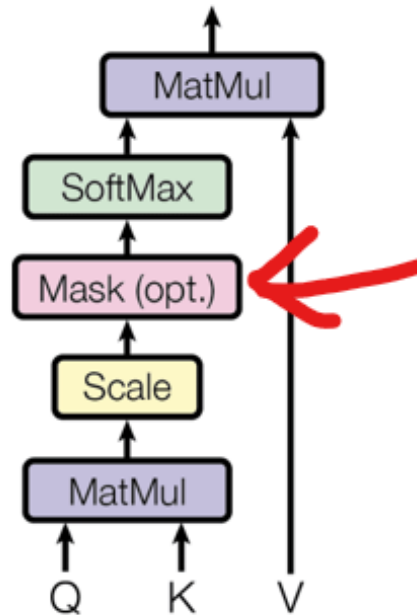


Figure 80: Dettaglio della Scaled Dot-Product Attention: il flusso da Q , K , V attraverso **MatMul**, **Scale**, **Mask** (opzionale) e **Softmax**.

Esempio numerico: attention su 3 token

Supponiamo 3 token con $d_k = 2$:

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}, V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}$$

1. **MatMul**: $QK^\top = \begin{pmatrix} 1 & 0 & 0.5 \\ 0 & 1 & 0.5 \\ 1 & 1 & 1 \end{pmatrix}$

2. **Scale**: $\frac{QK^\top}{\sqrt{2}} = \begin{pmatrix} 0.71 & 0 & 0.35 \\ 0 & 0.71 & 0.35 \\ 0.71 & 0.71 & 0.71 \end{pmatrix}$

3. **Softmax** (per riga): riga 1 $\rightarrow (0.43, 0.21, 0.30)$, riga 3 $\rightarrow (0.33, 0.33, 0.33)$

Interpretazione: il token 1 guarda soprattutto se stesso (0.43); il token 3 guarda tutti in modo uniforme (0.33) perché è ugualmente simile a tutti.

N.5 Multi-Head Attention

Invece di calcolare una sola attention con d_k dimensioni, si calcolano h “teste” in parallelo, ciascuna su uno spazio proiettato di dimensione ridotta d_k/h :

Multi-Head Attention

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

con $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$, $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$.

Nel Transformer originale: $d = 512$, $h = 8$, $d_k = d_v = d/h = 64$.

Ogni testa impara a “guardare” un aspetto diverso (es. una si focalizza sulla sintassi, un'altra sulla semantica, un'altra sulle relazioni a lungo raggio). Le uscite vengono concatenate e proiettate da $\mathbf{W}^O$ per fondere le informazioni.

Perché multi-head e non single-head?

Con una sola testa, il modello ha un'unica “prospettiva” per decidere a cosa prestare attenzione. Con h teste, può catturare *contemporaneamente* pattern diversi: relazioni sintattiche, semantiche, di prossimità, di riferimento, ecc. Il costo computazionale è lo stesso (ogni testa lavora su dimensione d/h), ma la capacità espressiva è molto maggiore.

N.6 Feed-Forward Network (FFN)

Dopo l'attention, ogni token passa indipendentemente attraverso un **MLP a due layer** (lo stesso per tutti i token):

Feed-Forward Network

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

con $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{ff}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{ff} \times d}$. Nel paper originale: $d = 512$, $d_{ff} = 2048$ (espansione $4\times$).

Perché serve l'FFN dopo l'attention?

- L'attention è una **combinazione lineare pesata** dei Value: $Z = AV$. Da sola, non introduce non linearità.
- L'FFN introduce **trasformazioni non lineari** (ReLU/GELU), permettendo al modello di apprendere funzioni complesse.
- L'espansione ($d \rightarrow 4d \rightarrow d$) aumenta la capacità senza cambiare la dimensione dell'output.

N.7 Decoder

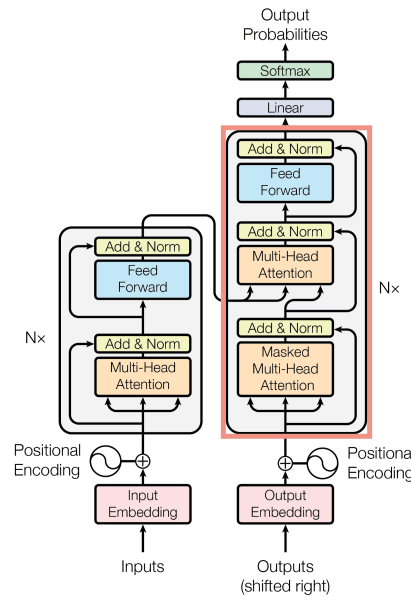


Figure 81: Struttura di un singolo blocco del decoder del Transformer con masked self-attention e cross-attention.

Il decoder ha una struttura simile all'encoder, ma con **tre** sotto-blocchi per layer:

1. **Masked Self-Attention**: impedisce al modello di “guardare” token futuri durante la generazione (masking causale: si pongono a $-\infty$ le posizioni $j > i$ nella matrice QK^T , così la softmax le azzerà).
2. **Cross-Attention**: le **Query** vengono dal decoder, ma **Key** e **Value** dall'output dell'encoder. Questo permette al decoder di “leggere” la rappresentazione dell'input per generare l'output.
3. **FFN**: identico a quello dell'encoder.

L'output finale passa attraverso una proiezione lineare $\mathbf{W} \in \mathbb{R}^{d \times |\text{vocab}|}$ + softmax per generare la distribuzione di probabilità sul vocabolario ad ogni passo temporale.

Esempio: traduzione “Il gatto dorme” $\rightarrow$ “The cat sleeps”

Encoder: prende [“Il”, “gatto”, “dorme”] $\rightarrow$ embedding + PE $\rightarrow$ 6 blocchi encoder $\rightarrow$ rappresentazione contestualizzata $\mathbf{H} \in \mathbb{R}^{3 \times 512}$.

Decoder (autoregressivo, genera un token alla volta):

- Passo 1: input [BOS] $\rightarrow$ masked self-attn $\rightarrow$ cross-attn (legge $\mathbf{H}$) $\rightarrow$ FFN $\rightarrow$ softmax $\rightarrow$ “The”
- Passo 2: input [BOS, The] $\rightarrow$ masked self-attn (The non vede il futuro) $\rightarrow$ cross-attn $\rightarrow$ softmax $\rightarrow$ “cat”
- Passo 3: input [BOS, The, cat] $\rightarrow \dots \rightarrow$ “sleeps”
- Passo 4: $\rightarrow$ [EOS] (fine generazione)

Durante il **training** si usa *teacher forcing*: si fornisce l'output corretto come input del decoder, non la predizione precedente.

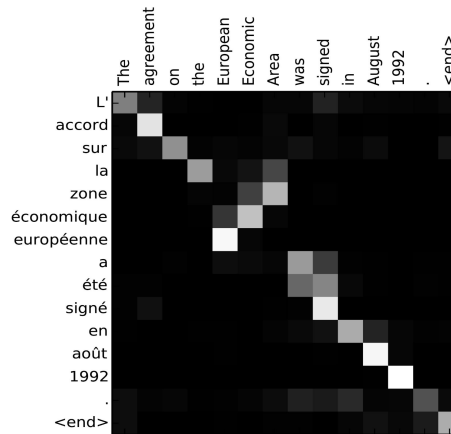


Figure 82: Matrice di attenzione nella traduzione inglese→francese (Bahdanau et al., 2014). Le celle chiare indicano alta attenzione: il modello impara automaticamente l'allineamento tra parole. (Fonte: Lilian Weng)

N.8 Positional Encoding

Positional Encoding sinusoidale

L'attenzione è **invariante per permutazione**: senza informazione posizionale, il Transformer non distingue “il gatto mangia il topo” da “il topo mangia il gatto”. Il positional encoding risolve questo problema:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right) \quad \text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Le frequenze decrescono geometricamente: le prime dimensioni oscillano rapidamente (distinguono posizioni vicine), le ultime lentamente (catturano la struttura globale). **Proprietà chiave:** $\text{PE}(\text{pos} + k)$ può essere espresso come trasformazione lineare di $\text{PE}(\text{pos})$, permettendo al modello di apprendere relazioni posizionali relative.

Differenza con il Perceiver

Il Transformer **somma** il positional encoding all'embedding ($z = \text{embed} + \text{PE}$), mescolando contenuto e posizione. Il Perceiver **concatena** le Fourier features all'input ($z = [\text{contenuto} \parallel \text{PE}]$), preservando le informazioni separatamente. Inoltre, il Perceiver usa Fourier features multiscala con frequenze esponenziali, non le sinusoidi fisse del Transformer.

N.9 Riassunto e confronto con il Perceiver

Aspetto	Transformer	Perceiver
Input	Sequenza di token (testo, tipicamente $M < 1024$)	Qualsiasi modalità (pixel, byte, campioni audio, M fino a 50K+)
Tokenizzazione	Necessaria (BPE, SentencePiece)	Non necessaria (lavora su byte/pixel grezzi)
Tipo di attention	Self-attention (Q, K, V tutti da X)	Cross-attention (Q da latenti, K, V da input)
Complessità	$O(M^2d)$ per layer	$O(MNd)$ cross-att + $O(N^2d)$ latent
Positional Encoding	Somma (sinusoidale o appreso)	Concatenazione (Fourier features)
Profondità	N layer (tipicamente 6–12)	T iterazioni (tipicamente 8) con weight sharing
Output	Sequenza (encoder-decoder)	Classificazione (pooling) o output queries (Perceiver IO)
Attivazione FFN	ReLU	GELU
Normalizzazione	Post-norm (originale)	Pre-norm

Collegamento al Perceiver

Il Perceiver **sostituisce la self-attention** (costo $O(M^2)$) **con la cross-attention** (costo $O(MN)$, dove $N \ll M$). Per un'immagine 224×224 : $M = 50,176$ pixel, ma $N = 512$ latenti, riducendo la complessità di $\sim 100\times$. Tuttavia, il Perceiver *eredita* dal Transformer tutti gli altri componenti: multi-head attention, FFN, residual connections, LayerNorm.

Il Transformer funziona su sequenze 1D (testo). Per applicarlo alle immagini, il Vision Transformer (ViT) introduce un modo elegante di “tokenizzare” un'immagine.

O Vision Transformer (ViT)

Il **Vision Transformer** (Dosovitskiy et al., 2021, ICLR) applica il Transformer alle immagini, trattandole come sequenze di **patch**. Invece di usare convoluzioni, divide l'immagine in blocchi regolari, li proietta in embedding e li elabora con un encoder Transformer standard.

Motivazione: perché applicare il Transformer alle immagini?

Le CNN sono ottime grazie ai loro inductive bias (località, invarianza traslazionale), ma non catturano facilmente le **dipendenze globali**: un pixel in alto a sinistra non “vede” direttamente un pixel in basso a destra senza molti strati. Il ViT usa l'attenzione per creare dipendenze globali fin dal primo strato: ogni patch può interagire con qualsiasi altra patch. Il prezzo è la necessità di dataset molto grandi o pretraining + fine-tuning.

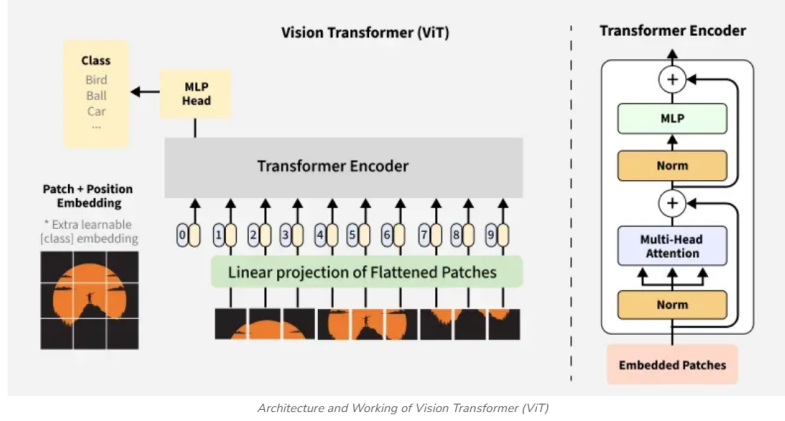


Figure 83: Pipeline del Vision Transformer (ViT): patch splitting, embedding e encoder Transformer.

O.1 Pipeline completa

O.1.1 1. Patch Splitting

L'immagine $I \in \mathbb{R}^{224 \times 224 \times 3}$ viene divisa in patch 16×16 non sovrapposte:

$$N = \frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196 \text{ patch}$$

O.1.2 2. Flattening

Ogni patch ($16 \times 16 \times 3 = 768$ valori) viene appiattito in un vettore 1D:

$$\mathbf{p}_i \in \mathbb{R}^{768}, \quad i = 1, \dots, 196$$

O.1.3 3. Linear Projection (Patch Embedding)

Una proiezione lineare apprendibile trasforma ogni patch:

$$\mathbf{x}_i = \mathbf{p}_i \mathbf{W}_E + \mathbf{b}_E, \quad \mathbf{W}_E \in \mathbb{R}^{768 \times D}$$

Con $D = 768$ (ViT-Base), ogni patch diventa un embedding di 768 dimensioni.

O.1.4 4. CLS Token

Si prepone un **token speciale** $\mathbf{z}_{\text{CLS}} \in \mathbb{R}^D$ alla sequenza. Il CLS raccoglie informazione da tutti i patch tramite self-attention e viene usato per la classificazione finale.

$$\mathbf{X}' = [\mathbf{z}_{\text{CLS}}, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{196}] \in \mathbb{R}^{197 \times 768}$$

O.1.5 5. Positional Encoding

Si sommano **positional embedding apprendibili** per codificare la posizione spaziale:

$$\mathbf{Z}^0 = \mathbf{X}' + \mathbf{P}, \quad \mathbf{P} \in \mathbb{R}^{197 \times 768}$$

Formula compatta del ViT

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_1 \mathbf{E}; \mathbf{x}_2 \mathbf{E}; \dots; \mathbf{x}_P \mathbf{E}] + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(P+1) \times D}$$

dove $P = 196$ è il numero di patch, $\mathbf{x}_i$ è la patch i appiattita, $\mathbf{E} \in \mathbb{R}^{(16^2 \cdot 3) \times D}$ è la matrice di proiezione, e $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(P+1) \times D}$ sono i positional embedding appresi.

O.1.6 6. Transformer Encoder ($L = 12$ layer)

Ogni encoder layer:

1. LayerNorm $\rightarrow$ Multi-Head Self-Attention $\rightarrow$ Residual connection
2. LayerNorm $\rightarrow$ FFN (GELU) $\rightarrow$ Residual connection

a) **Multi-Head Self-Attention** Per $\mathbf{Z}^{l-1} \in \mathbb{R}^{197 \times 768}$, con $h = 12$ teste ($d_k = 64$ ciascuna):

$$\mathbf{Q} = \mathbf{Z}^{l-1} \mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}^{l-1} \mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}^{l-1} \mathbf{W}^V$$
$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{64}}\right) \mathbf{V}_i \quad \text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_{12}) \mathbf{W}^O$$

b) **Feed Forward Network**

$$\text{FFN}(\mathbf{z}) = \text{GELU}(\mathbf{z} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

con $\mathbf{W}_1 \in \mathbb{R}^{768 \times 3072}$ (espansione) e $\mathbf{W}_2 \in \mathbb{R}^{3072 \times 768}$ (compressione).

O.1.7 7. Classificazione (MLP Head)

Dopo $L = 12$ layer: $\mathbf{Z}^{12} \in \mathbb{R}^{197 \times 768}$.

Si estrae il CLS token: $\mathbf{z}_{\text{CLS}}^{(12)} = \mathbf{Z}^{12}[0] \in \mathbb{R}^{768}$

Classificazione finale

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{cls}} \mathbf{z}_{\text{CLS}}^{(12)} + \mathbf{b}), \quad \mathbf{W}_{\text{cls}} \in \mathbb{R}^{K \times 768}$$

O.2 ViT vs CNN: differenze fondamentali

1. Local vs. Global Attention

- **CNN:** filtri locali (es. 3×3), il contesto globale si costruisce solo gradualmente nei layer profondi.
- **ViT:** self-attention globale fin dal primo layer — ogni patch “parla” con tutti gli altri.

2. Inductive Biases

- **CNN:** bias induttivi forti (località, invarianza alla traslazione) → efficienti con pochi dati.
- **ViT:** pochi bias induttivi → più flessibile ma necessita di dataset molto grandi (ImageNet-21k, JFT-300M) o pretraining + fine-tuning.

3. Training Data Requirements

- **CNN:** performano bene anche con dataset medio-piccoli (CIFAR-10, MNIST).
- **ViT:** richiedono pretraining su dataset enormi; dominano quando i dati abbondano.

Collegamento al Perceiver

Il ViT riduce la dimensione dell’input da $M = 50,176$ pixel a $M = 196$ patch tramite la tokenizzazione, ma questa resta **domain-specific** (funziona solo per immagini). Il Perceiver elimina completamente la tokenizzazione domain-specific: accetta *qualsiasi* input (pixel grezzi, audio, point cloud) e usa la cross-attention per proiettarlo in un latent array di dimensione fissa $N \ll M$.

P Complementi: Self-Attention vs Cross-Attention

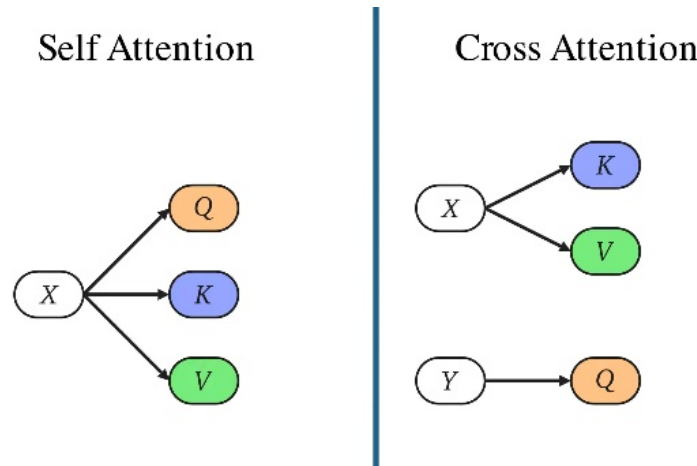


Figure 84: Confronto tra self-attention e cross-attention: nella self-attention Q, K, V provengono dallo stesso input; nella cross-attention provengono da sorgenti diverse.

Differenza tra Self-attention e Cross-attention

La **self-attention** è un meccanismo in cui **Query (Q)**, **Key (K)** e **Value (V)** provengono tutti dallo stesso insieme di input.

- Serve a far sì che ogni elemento della sequenza (o ogni latente) **interagisca con gli altri** per arricchire la propria rappresentazione.
- Input I : sequenza di token (parole o patch immagine).

$$I \in \mathbb{R}^{M \times D}$$

Si calcolano direttamente:

$$Q = IW_Q, \quad K = IW_K, \quad V = IW_V$$

Attenzione:

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{M \times M}$$

Output:

$$Z = A \cdot V \in \mathbb{R}^{M \times D}$$

È quindi un'operazione **intra-spazio**: lo stesso array si auto-analizza e si rielabora.

(nel caso dei transformer con M molto grande il calcolo diventa proibitivo)

La **cross-attention** è un meccanismo in cui **Query (Q)** proviene da un insieme (es. i latenti) e **Key (K)**, **Value (V)** provengono da un altro insieme (es. l'immagine/token di input).

- Serve a permettere a un insieme (latenti) di **interrogare un altro insieme** (input esterno) e di incorporarne l'informazione.

- È quindi un'operazione **inter-spazio**: un array aggiorna la propria rappresentazione “guardando” un altro array.

Formula:

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right), \quad Z = A \cdot V$$

con Q dai latenti L , e K, V dall'immagine I .

Differenza tra Single-Head Cross-Attention e Multi-Head Cross-Attention nel Perceiver

Negli esempi seguenti usiamo $D = 1024$ e $d_k = D/h = 1024/8 = 128$, coerentemente con la configurazione ImageNet del paper. Per single-head, la dimensione dello spazio di proiezione coincide con D .

1. Single-Head Cross-Attention

1. Proiezioni lineari:

$$\begin{aligned} Q &= LW^Q \quad (512 \times 1024 \rightarrow 512 \times 1024) \\ K &= XW^K, \quad V = XW^V \quad (50176 \times 1024 \rightarrow 50176 \times 1024) \end{aligned}$$

2. Attenzione:

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{1024}} \right) \quad (512 \times 50176)$$

3. Output:

$$\text{Out} = AV \quad (512 \times 1024)$$

Ogni latente interroga l'intera immagine con un'**unica** mappa di attenzione.

2. Multi-Head Cross-Attention ($h = 8$)

1. Divisione embedding:

$$d_k = \frac{D}{h} = \frac{1024}{8} = 128$$

2. Proiezioni per ogni testa i :

$$\begin{aligned} Q_i &= LW_i^Q \quad (512 \times 1024 \rightarrow 512 \times 128) \\ K_i &= XW_i^K, \quad V_i = XW_i^V \quad (50176 \times 1024 \rightarrow 50176 \times 128) \end{aligned}$$

3. Attenzione per ogni testa:

$$\begin{aligned} A_i &= \text{softmax} \left(\frac{Q_i K_i^\top}{\sqrt{128}} \right) \quad (512 \times 50176) \\ \text{head}_i &= A_i V_i \quad (512 \times 128) \end{aligned}$$

4. Concatenazione e proiezione finale:

$$\text{Out} = \text{Concat}(\text{head}_1, \dots, \text{head}_8) W^O \quad (512 \times 1024)$$

Ogni latente interroga l'immagine con 8 **mappe di attenzione diverse** — viste parallele su pattern differenti (colore, bordi, texture, struttura globale).

Definizione di W^O

Dopo aver calcolato tutte le teste:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \in \mathbb{R}^{N \times d_k}$$

Concateno i risultati:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{N \times (h \cdot d_k)}$$

Poiché $h \cdot d_k = D$, abbiamo $H \in \mathbb{R}^{N \times D}$.

A questo punto si applica una proiezione lineare con matrice W^O :

$$\text{Out} = H W^O, \quad W^O \in \mathbb{R}^{D \times D}$$

A cosa serve W^O

1. **Ricombinare le teste:** Ogni testa produce una rappresentazione parziale (es. bordi, colore, posizione). La concatenazione è solo un “accostamento”. W^O permette di **mescolare e fondere** queste informazioni in un'unica rappresentazione coerente nello spazio di dimensione D .
2. **Mantenere la dimensione costante:** Input e output della Multi-Head Attention devono restare nella stessa dimensione D (nel Perceiver: 1024 per ImageNet). Senza W^O , l'output sarebbe solo una concatenazione, priva di trasformazione finale.
3. **Parametri appresi:** W^O è una matrice di peso appresa tramite backpropagation. Inizializzata (es. Xavier uniform), si aggiorna durante il training. Serve a **decidere come pesare** le informazioni provenienti dalle varie teste.

3. Confronto diretto

Aspetto	Single-Head	Multi-Head (h=8)
Dimensione Q	512×1024	$8 \times 512 \times 128$
Dimensione K,V	50176×1024	$8 \times 50176 \times 128$
Pesi di attenzione	1 matrice 512×50176	8 matrici 512×50176
Output testa	512×1024	$8 \times 512 \times 128 \rightarrow \text{concat} \rightarrow 512 \times 1024$
Capacità	Una sola prospettiva	Più prospettive parallele (pattern diversi)

Differenza concettuale

- Le matrici W^Q, W^K, W^V servono a **proiettare** l'input in spazi diversi per calcolare l'attenzione.
- La matrice W^O serve a **ricombinare** i risultati delle teste e riportarli nello spazio embedding originale.

Nel Perceiver (ImageNet):

- $D = 1024$, $h = 8$ teste
- Ogni testa: $d_k = 128$
- Dopo la concat: $H \in \mathbb{R}^{512 \times 1024}$
- Moltiplicazione con $W^O \in \mathbb{R}^{1024 \times 1024} \rightarrow$ output finale $N \times 1024$

Multi-Head Attention (MHA) in generale

In un layer Transformer (o Perceiver), non usiamo una sola attenzione ma più teste parallele.

Ogni testa ha i suoi pesi W_Q, W_K, W_V e calcola:

$$Z_h = \text{Softmax} \left(\frac{Q_h K_h^T}{\sqrt{D_h}} \right) V_h$$

$\text{con } D_h = D/H$ (dimensione ridotta se ci sono H teste).

- Poi i risultati di tutte le teste vengono concatenati e proiettati con un altro peso W_O :

$$Z = \text{Concat}(Z_1, \dots, Z_H) W_O$$

Quindi la MHA è una struttura architetturale che moltiplica la capacità del modello di guardare in più "spazi" o "prospettive".

P.0.1 Cosa cambia sostanzialmente con tipi di input diversi nel Perceiver?

Sostanzialmente, ciò che cambia è solo il **modo in cui l'input grezzo viene trasformato in un array di byte bidimensionale (la matrice $M \times C$)** prima di essere processato dal modello. Il cuore dell'architettura Perceiver, ovvero il meccanismo di cross-attenzione che astrae l'input in un array latente di dimensione fissa e i successivi blocchi Transformer, **rimane quasi identico**.

Questa è la vera forza del Perceiver: disaccoppiare l'elaborazione principale dalla specificità dell'input. Il modello non ha bisogno di un'architettura completamente diversa per ogni tipo di dato, ma solo di una strategia di "appiattimento" e codifica posizionale iniziale.

Vediamo le differenze specifiche per ogni caso.

P.0.2 Come cambiano le matrici e gli step successivi

1. Immagine

- **Matrice di Input ($M \times C$):** Per un'immagine, la trasformazione è molto diretta. L'immagine, con le sue dimensioni spaziali (altezza H , larghezza W) e i suoi canali di colore (es. 3 per RGB), viene "appiattita".
 - **M (numero di elementi):** Diventa $H * W$. Ogni pixel è un elemento dell'array di input.
 - **C (dimensione di ogni elemento):** Corrisponde ai canali del pixel (es. 3 per i valori R, G, B) a cui si aggiunge una codifica posizionale (positional encoding) per preservare l'informazione spaziale. Questa codifica è fondamentale, altrimenti il modello vedrebbe solo un "sacco di pixel" senza sapere dove si trovavano.

- **Esempio:** Un'immagine 224x224 RGB diventa una matrice $M \times C$ di $(224*224) \times (3 + D_pos)$, dove $M = 50.176$ e D_pos è la dimensione della codifica posizionale.
- **Matrice Latente ($N \times D$):** Questa matrice è **indipendente dall'input**. La sua dimensione è un iperparametro del modello che tu definisci a priori.
 - **N (numero di vettori latenti):** È fisso (es. 512). Questo determina la "larghezza di banda" computazionale del modello. È molto più piccolo di M .
 - **D (dimensione di ogni vettore latente):** È anch'esso fisso (es. 1024).
- **Step Successivi:**
 - **Cross-Attention:** L'array latente ($N \times D$), che è inizializzato casualmente, "guarda" l'enorme array di input ($M \times C$) e, tramite la cross-attenzione, estrae le informazioni più salienti, "comprimendole" in sé stesso. L'array latente funge da *query*, mentre l'array di input funge da *key* e *value*.
 - **Self-Attention (Transformer Blocks):** L'array latente $N \times D$, ora "carico" di informazioni dall'immagine, viene processato da una serie di blocchi Transformer standard (self-attention). Qui, i vettori latenti interagiscono tra loro per affinare e integrare le informazioni estratte.
 - **Classificazione Finale:** L'output del blocco Transformer (ancora di dimensione $N \times D$) viene mediato (average pooling) lungo la dimensione N per ottenere un singolo vettore di dimensione D . Questo vettore viene poi dato in input a una testa di classificazione (un semplice layer lineare) per produrre i **logits** finali.

2. Video

- **Matrice di Input ($M \times C$):** Un video è una sequenza di frame (immagini) nel tempo. L'appiattimento include anche la dimensione temporale.
 - **M (numero di elementi):** Diventa $T * H * W$, dove T è il numero di frame.
 - **C (dimensione di ogni elemento):** Come per l'immagine, include i canali del pixel (es. 3) più una codifica posizionale. Crucialmente, questa codifica deve rappresentare non solo la posizione (x, y) nel frame, ma anche la posizione temporale t .
 - **Esempio:** Un video di 16 frame 224x224 diventa una matrice $M \times C$ di $(16*224*224) \times (3 + D_pos_3D)$, con $M = 802.816$.
- **Matrice Latente ($N \times D$):** **Non cambia.** È sempre la stessa matrice di dimensione fissa $N \times D$ definita come iperparametro.
- **Step Successivi: Identici al caso dell'immagine.** Il meccanismo di cross-attenzione non si "preoccupa" del fatto che l'input provenga da un'immagine o da un video. Semplicemente, interroga un array di byte molto più grande, che grazie alla codifica posizionale contiene implicitamente l'informazione spazio-temporale.

3. Nuvola di Punti (Point Cloud)

- **Matrice di Input ($M \times C$):** Una nuvola di punti è un insieme di punti in uno spazio 3D.
 - **M (numero di elementi):** È semplicemente il numero di punti nella nuvola.
 - **C (dimensione di ogni elemento):** Per ogni punto, le feature sono le sue coordinate (x, y, z). A queste si possono aggiungere altre informazioni come il colore (R, G, B) o la normale. A differenza delle immagini, qui non è necessaria una codifica posizionale esplicita, perché le coordinate (x, y, z) sono già l'informazione posizionale stessa.
 - **Esempio:** Una nuvola di 4096 punti con coordinate e colori diventa una matrice $M \times C$ di 4096×6 .
- **Matrice Latente ($N \times D$):** Ancora una volta, **non cambia**. La sua dimensione è fissa.
- **Step Successivi:** Esattamente gli stessi. La matrice latente $N \times D$ usa la cross-attenzione per interrogare la matrice di punti $M \times C$ e distillare l'informazione rilevante. Il resto del processo fino ai logits è invariato.

P.0.3 Tabella Riassuntiva

Tipo di Input	Matrice di Input ($M \times C$)	Matrice Latente ($N \times D$)	Note
Immagine	$M = H * W$, $C = \text{Canali} + \text{Pos. Encoding 2D}$	Fissa, $N \times D$ (iperparametro)	La struttura spaziale è codificata.
Video	$M = T * H * W$, $C = \text{Canali} + \text{Pos. Encoding 3D}$	Fissa, $N \times D$ (iperparametro)	La struttura spazio-temporale è codificata.
Nuvola di Punti	$M = N$. Punti, $C = \text{Coord. (x,y,z) [+ Colori...]}$	Fissa, $N \times D$ (iperparametro)	La posizione è data dalle coordinate stesse.

In sintesi, l'eleganza del Perceiver sta nel confinare la complessità della multimodalità a un unico, semplice step di preelaborazione, mantenendo il "cervello" del modello (i blocchi di attenzione) agnostico e riutilizzabile per qualsiasi tipo di dato.

Q Dropout

Il **dropout** (Srivastava et al., 2014, JMLR) è una tecnica di regolarizzazione che durante il training azzerava casualmente un sottoinsieme di attivazioni con probabilità p . L'effetto è quello di addestrare implicitamente un *ensemble* esponenziale di sottoreti, rendendo la rete meno dipendente da singoli neuroni.

Q.1 Meccanismo: inverted dropout

Inverted Dropout

Durante il training, ogni attivazione è mantenuta con probabilità $1 - p$ e azzerata con probabilità p . La maschera $m_i \sim \text{Bernoulli}(1 - p)$ è campionata indipendentemente per ogni neurone e ogni esempio:

$$\tilde{x}_i = \frac{m_i \cdot x_i}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p)$$

La divisione per $1 - p$ (*inverted dropout*) garantisce che il valore atteso dell'output sia uguale all'input: $\mathbb{E}[\tilde{x}_i] = x_i$. A inference, si usa $\tilde{x} = x$ senza alcuna modifica.

Q.2 Train vs. inference

Fase	Comportamento	Fattore di scala
Training	maschera $m_i \sim \text{Bernoulli}(1 - p)$, divide per $1 - p$	$\times \frac{1}{1 - p}$ sui neuroni attivi
Inference	identità: $\tilde{x} = x$	$\times 1$ (nessuna modifica)

Q.3 Interpretazione: ensemble implicito

Con N neuroni, ogni forward pass usa una sottorete distinta: esistono 2^N sottoreti possibili, campionate con diversa frequenza in base a p . A inference, il modello completo può essere interpretato come la media geometrica di tutte queste sottoreti.

Q.4 Dove e quando si applica

Il dropout si applica tipicamente:

- dopo strati *fully connected*, prima o dopo l'attivazione;
- **non** subito dopo LayerNorm (interrompe la normalizzazione appena calcolata);
- **non** a inference, mai;
- con $p \in [0.1, 0.3]$ nei Transformer moderni; $p = 0.5$ nei classificatori MLP classici.

Collegamento al Perceiver

Il Perceiver originale **non usa dropout**: il weight sharing tra le T iterazioni del latent transformer agisce già come regolarizzatore implicito — ogni iterazione condivide gli stessi pesi ma vede lo stato latente aggiornato, costringendo la rete ad apprendere rappresentazioni robuste. Gli esperimenti degli autori mostrano che aggiungere dropout non migliora le prestazioni su ImageNet in questa configurazione.

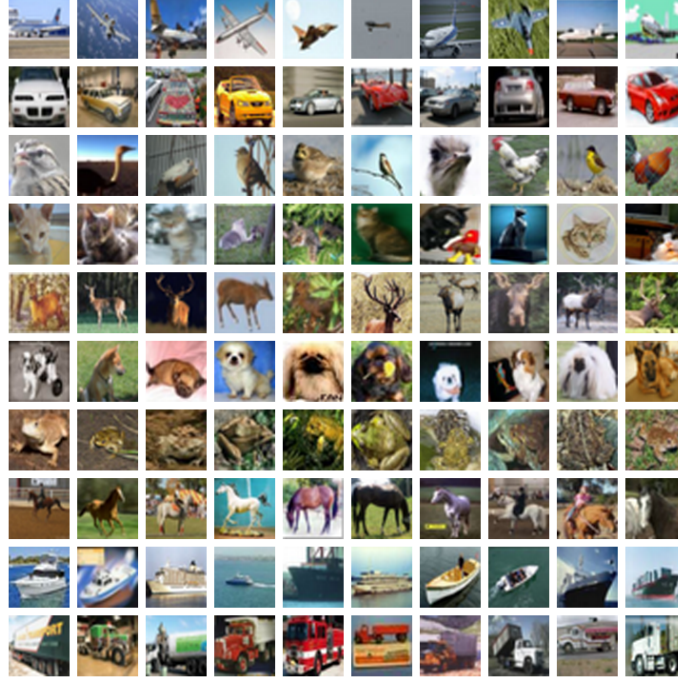


Figure 85: Feature del primo strato apprese con il dropout: rilevatori strutturati e localizzati invece di rumore co-adattato. *Fonte: Srivastava et al., Dropout (JMLR v15, 2014).*

R Inizializzazione dei Pesi: Xavier e He

L'inizializzazione dei pesi determina la *varianza delle attivazioni* al primo forward pass. Se la varianza cresce strato dopo strato le attivazioni esplodono; se decresce svaniscono. In entrambi i casi il training non converge.

R.1 Il problema: propagazione della varianza

Consideriamo un layer lineare $z = Wx + b$ con n_{in} ingressi, bias zero, attivazione lineare, e ingressi a media zero con varianza σ_x^2 . I pesi W_{ij} sono i.i.d. con media zero e varianza σ_w^2 :

$$\text{Var}(z_j) = n_{\text{in}} \cdot \sigma_w^2 \cdot \sigma_x^2$$

Per mantenere $\text{Var}(z) = \text{Var}(x)$ (nessuna amplificazione) occorre $\sigma_w^2 = 1/n_{\text{in}}$.

R.2 Xavier/Glorot Initialization

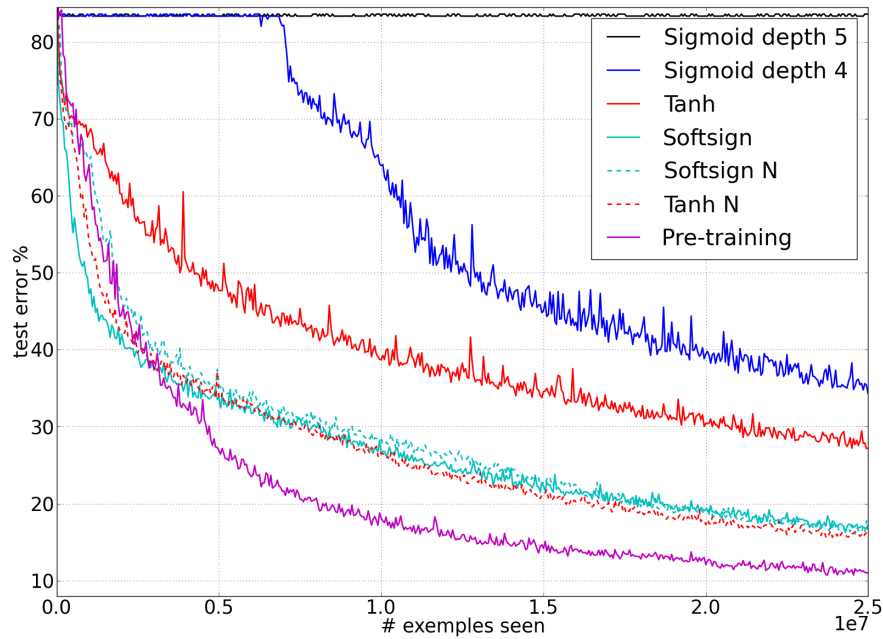


Figure 86: Test error per diverse attivazioni e inizializzazioni: l’inizializzazione normalizzata (Xavier/Glorot) raggiunge errori molto più bassi della sigmoide profonda che satura. *Fonte:* Glorot & Bengio (AISTATS 2010).

Xavier (Glorot & Bengio, 2010) bilancia sia il forward che il backward pass prendendo la media tra n_{in} e n_{out} :

Xavier/Glorot Initialization

$$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right] \quad \Rightarrow \quad \text{Var}(W) = \frac{2}{n_{in} + n_{out}}$$

Progettata per attivazioni simmetriche attorno allo zero come **tanh** e **sigmoid**, dove la derivata è vicina a 1 nell’origine.

R.3 He/Kaiming Initialization

ReLU azzerava il 50% delle attivazioni, dimezzando la varianza effettiva. He et al. (2015) corregge questo con un fattore 2:

He/Kaiming Initialization

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

Progettata per **ReLU** e **GELU**: il fattore 2 compensa l'azzeramento medio del 50% delle uscite.

R.4 Tabella comparativa

Metodo	Var(W)	Attivazione consigliata	Comportamento senza
Casuale piccolo ($\sigma = 0.01$)	10^{-4}	—	vanishing attivazioni
Xavier/Glorot	$\frac{2}{n_{\text{in}} + n_{\text{out}}}$	tanh, sigmoid	stabile
He/Kaiming	$\frac{2}{n_{\text{in}}}$	ReLU, GELU	stabile
Casuale grande ($\sigma = 1$)	1	—	exploding attivazioni

Collegamento al Perceiver

Il latent array $L \in \mathbb{R}^{N \times D}$ viene inizializzato con $\mathcal{N}(0, 0.02)$ troncata: valori piccoli evitano che i latenti dominino il primo forward pass prima che la cross-attention abbia “letto” qualcosa dall'input. Le matrici di proiezione W_Q, W_K, W_V nell'MLP usano inizializzazione He (GELU è l'attivazione). Nei blocchi cross-attention, le proiezioni che mescolano dimensioni diverse (da $C_{\text{tot}} = 261$ a D_{QKV}) usano Xavier.

S Regularizzazione L1 e L2 (Weight Decay)

La regularizzazione aggiunge un termine di penalità alla loss per scoraggiare pesi grandi, riducendo la varianza del modello a scapito di un leggero aumento del bias.

S.1 L2 Regularization (Ridge / Weight Decay)

L2 Regularization

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \quad \Rightarrow \quad \nabla_w \mathcal{L}_{\text{reg}} = \nabla_w \mathcal{L} + \lambda \mathbf{w}$$

L'aggiornamento SGD diventa: $w_t \leftarrow w_{t-1}(1 - \eta\lambda) - \eta \nabla_w \mathcal{L}$. Ogni passo scala i pesi verso zero di un fattore $\eta\lambda$.

Interpretazione bayesiana: aggiungere $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ equivale a imporre un prior gaussiano $\mathcal{N}(\mathbf{0}, \lambda^{-1} \mathbf{I})$ sui pesi. Massimizzare la log-posterior è equivalente a minimizzare la cross-entropy con L2.

S.2 L1 Regularization (Lasso)

L1 Regularization

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\mathbf{w}\|_1 \quad \Rightarrow \quad \nabla_w \mathcal{L}_{\text{reg}} = \nabla_w \mathcal{L} + \lambda \cdot \text{sign}(\mathbf{w})$$

L1 penalizza ogni peso non-zero con la stessa intensità indipendentemente dalla sua grandezza, spingendo molti pesi esattamente a **zero** (*sparsità*). Interpretazione bayesiana: prior di Laplace.

S.3 L2 nella loss vs. weight decay in Adam

Punto sottile e spesso frainteso: in Adam, aggiungere $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ alla loss **non è equivalente** a weight decay. Il gradiente del termine L2 viene scalato dai momenti del secondo ordine v_t , rendendo la penalizzazione non uniforme tra i parametri. **AdamW** (Loshchilov & Hutter, 2019) applica il weight decay direttamente all'aggiornamento, *dopo* la scalatura adattiva:

$$w_t \leftarrow w_{t-1} - \eta \underbrace{\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}}_{\text{passo Adam}} - \eta \lambda w_{t-1}$$

Metodo	Penalità	Sparsità	Nota
L1	$\lambda \ \mathbf{w}\ _1$	sì	prior Laplace
L2 nella loss	$\frac{\lambda}{2} \ \mathbf{w}\ _2^2$	no	scalato dai momenti in Adam
AdamW / LAMB weight decay	$\lambda \mathbf{w}$ separato	no	corretto per ottimizzatori adattivi

Collegamento al Perceiver

Il Perceiver usa **weight decay tramite LAMB**: il decadimento è applicato separatamente dal passo adattivo, come in AdamW, evitando che il trust ratio per layer annulli la regolarizzazione nelle run con large batch. Il valore di λ usato nel paper originale è 0.1.

T Data Augmentation

La **data augmentation** genera varianti trasformate degli esempi di training per aumentare artificialmente la diversità del dataset, riducendo l'overfitting e migliorando la generalizzazione.

T.1 Tecniche standard (pipeline ImageNet)

- **Random Resized Crop**: ritaglia casualmente una regione dell'immagine (con scala in $[0.08, 1.0]$ e aspect ratio in $[0.75, 1.33]$) e la ridimensiona a 224×224 .
- **Horizontal Flip**: specchia l'immagine orizzontalmente con probabilità 0.5.
- **Color Jitter**: modifica casualmente luminosità (± 0.4), contrasto (± 0.4), saturazione (± 0.4) e tinta (± 0.1).
- **Normalization**: normalizzazione dei canali RGB con media $\mu = (0.485, 0.456, 0.406)$ e deviazione standard $\sigma = (0.229, 0.224, 0.225)$ di ImageNet.

T.2 Tecniche avanzate

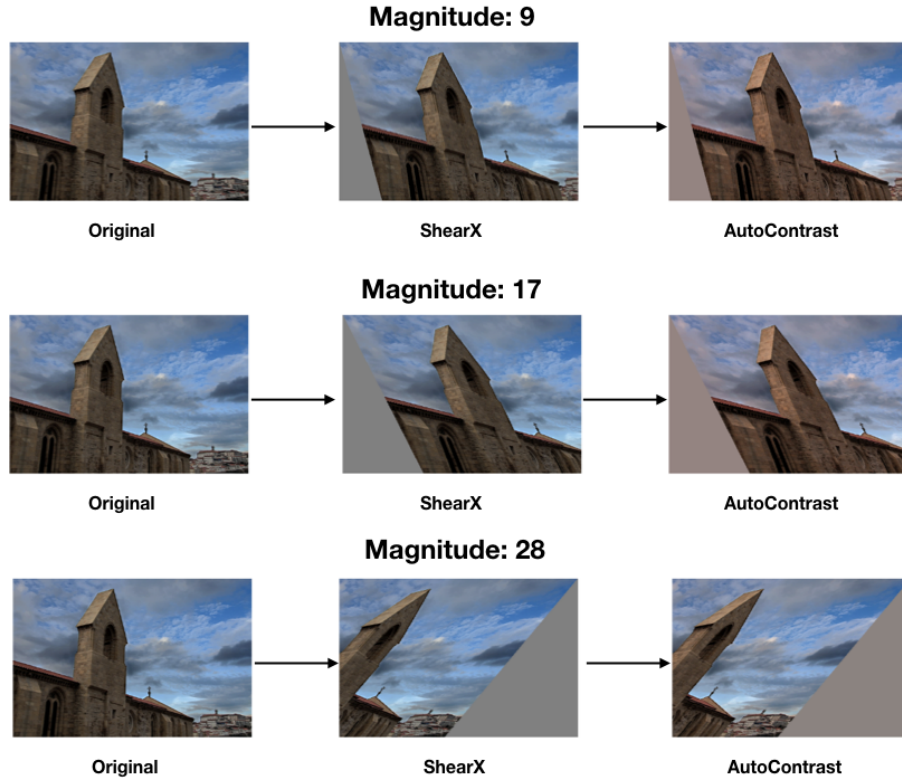


Figure 87: Effetto del parametro di magnitudine M di RandAugment: a parità di trasformazioni, M più grande significa distorsione più intensa. *Fonte*: Cubuk et al., *RandAugment* (arXiv:1909.13719).

- **MixUp** (Zhang et al., 2018, ICLR): interpola linearmente due immagini e le loro label con coefficiente $\lambda \sim \text{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

La loss è calcolata sulla label interpolata (label morbida).

- **CutMix** (Yun et al., 2019, ICCV): incolla una patch rettangolare di x_j sopra x_i ; la label è proporzionale all'area della patch incollata.
- **RandAugment** (Cubuk et al., 2020, NeurIPS): campiona N trasformazioni casuali da un insieme di 14 operazioni predefinite (shear, rotate, equalize, posterize, ...), tutte con la stessa magnitudine M .

T.3 Perché è più critica per il Perceiver

Le CNN hanno un *inductive bias* di invarianza locale: una convoluzione risponde allo stesso pattern indipendentemente da dove appare nell'immagine. Il Perceiver, senza convoluzioni, è *permutation-equivariant* rispetto all'ordine dei pixel nel byte array. Apprende l'invarianza spaziale solo dall'esperienza, rendendo l'augmentation più importante.

L'esperimento di permutation invariance (Sezione 1.7) dimostra che le Fourier features rendono il Perceiver robusto a permutazioni *casuali* dei pixel; la data augmentation complementa questa proprietà agendo su trasformazioni geometriche *strutturate* (crop, flip, scaling).

Collegamento al Perceiver

Il Perceiver usa la stessa pipeline di augmentation di ViT e ResNet su ImageNet: random resized crop + horizontal flip + normalization RGB. **Non viene usato MixUp o CutMix** nel paper originale. La pipeline è applicata solo al training: a test time si usa centro crop 224×224 + normalization. Per modalità non-immagine (audio, point cloud) la pipeline si adatta: per l'audio si usa random time cropping; per i point cloud, random jittering delle coordinate.

U Perceiver IO — Risultati Sperimentali

Il Perceiver IO (Jaegle et al., 2022, ICML) dimostra che la stessa architettura encoder-process-decode è competitiva con modelli specializzati su task radicalmente diversi, cambiando solo le output query e le proiezioni di input/output.

U.1 Optical Flow (Sintel)



Figure 88: Flusso ottico denso predetto da Perceiver IO (codifica a colori per direzione e intensità): stato dell’arte su Sintel trattando l’output come decodifica di query strutturate. *Fonte:* Jaegle et al., *Perceiver IO* (arXiv:2107.14795).

L’optical flow stima il vettore di movimento $(\Delta x, \Delta y)$ per ogni pixel tra due frame consecutivi. Il Perceiver IO fornisce query con coordinate pixel (u, v) e feature dell’immagine al decoder; il decoder produce un vettore 2D per ogni query.

Modello	Sintel Clean (AEE ↓)	Sintel Final (AEE ↓)
Perceiver IO	1.81	2.42
RAFT (specializzato)	1.43	2.71
PWC-Net (specializzato)	2.55	3.93
FlowNet2 (specializzato)	3.96	6.02

AEE = Average Endpoint Error in pixel (minore è meglio). Il Perceiver IO supera RAFT su Sintel Final — il benchmark più difficile, con motion blur e atmosfera — pur essendo un modello generale non progettato specificamente per il flusso ottico.

U.2 Language Modeling a livello byte

Il Perceiver IO è addestrato su masked language modeling (MLM) a livello di byte (non subword token). Le output query sono le posizioni dei byte mascherati; l’output è una distribuzione sui 256 possibili valori di byte.

Modello	Granularità	BPC ↓	Parametri
Perceiver IO	byte	1.74	201M
BERT-base	subword	1.69	110M
ByT5-base	byte	1.38	582M

BPC = Bits Per Character (minore è meglio). Paragonabile a BERT-base nonostante lavori a granularità byte (vocabolario più piccolo, sequenze più lunghe), senza tokenizzazione specializzata.

U.3 Multimodal Autoencoding (Kinetics-700)

Il Perceiver IO viene addestrato a ricostruire simultaneamente video (a colori, $16 \times 56 \times 56$ frame), audio (campioni raw) e label di classe da input con dropout di modalità casuali. Le output query includono indicatori di modalità: passando query video si ottiene la ricostruzione video, passando query audio si ottiene il segnale audio, ecc.

Il modello ottiene risultati competitivi senza modifiche architetturali: la stessa rete gestisce tutte e tre le modalità controllando solo quale output query viene fornita al decoder.

U.4 Classificazione ImageNet (mantenuta)

Il Perceiver IO mantiene prestazioni su ImageNet pari al Perceiver originale (circa 78% top-1 su pixel grezzi), confermando che l'aggiunta del decoder non degrada le capacità di encoding.

U.5 Messaggio chiave: generalità dimostrata

Generalità dimostrata dai numeri

La stessa architettura Perceiver IO, cambiando solo le **output query** e le proiezioni di ingresso/uscita:

- supera modelli specializzati su **optical flow** su Sintel Final;
- è paragonabile a BERT-base su **language modeling** byte-level;
- gestisce **tre modalità simultaneamente** in autoencoding video/audio/label.

Nessun componente domain-specific è stato aggiunto: l'adattamento avviene interamente attraverso le output query, il positional encoding dell'input e le proiezioni finali. Questo è il contributo centrale del paper: *one architecture, many outputs*.

V Domande Probabili per l'Esame

V.1 Sul Perceiver (domande specifiche sul progetto)

- **Qual è il problema principale che il Perceiver risolve rispetto ai Transformer?** Vedi Sezione 1.1: il Transformer standard ha complessità $O(M^2)$ nell'attenzione, rendendo inapplicabile il modello su input ad alta risoluzione. Il Perceiver introduce un collo di bottiglia latente per ridurre questa complessità. Keyword: quadratic bottleneck, scalability.
- **Perché la complessità scende da $O(M^2)$ a $O(MN)$? Da dove viene questa riduzione?** Vedi Sezione 1.1, formula della complessità. La cross-attention sostituisce la self-attention sull'input: invece di far interagire M token di input tra loro ($O(M^2)$), si fa interagire un array di $N \ll M$ latenti con l'input ($O(MN)$). Keyword: latent bottleneck, $N \ll M$.
- **Cosa sono i latenti? Come vengono inizializzati? Perché $N = 512$?** Vedi Sezione 1.2: i latenti sono un array di N vettori apprendibili di dimensione D , inizializzati casualmente (es. da $\mathcal{N}(0, 0.02^2)$) e ottimizzati durante il training. $N = 512$ è un iperparametro: abbastanza grande da catturare struttura ricca, abbastanza piccolo da mantenere efficienza computazionale. Keyword: learned latent array, iperparametro.
- **Spiega il forward pass step by step.** Vedi Sezione 1.3: (1) encoding Fourier delle coordinate input; (2) cross-attention latenti $\leftarrow$ input; (3) self-attention tra latenti (blocco Transformer); (4) ripetizione con weight sharing per L iterazioni; (5) decoding con query specifiche del task (Perceiver IO). Keyword: iterative cross-attention, weight sharing.
- **Cosa succede nella cross-attention? Da dove vengono Q, K, V?** Vedi Sezione 1.3: le Query (Q) provengono dai latenti (dimensione $N \times D$); le Key (K) e i Value (V) provengono dall'input encodificato (dimensione $M \times C$). Il risultato è un aggiornamento dei latenti che integra informazione dall'input. Keyword: Q dai latenti, KV dall'input.
- **Perché si usa il weight sharing? Che effetto ha sulle prestazioni?** Vedi Sezione 1.4: il weight sharing (tutti i blocchi iterativi condividono gli stessi pesi) riduce drasticamente il numero di parametri pur permettendo elaborazione profonda. Comporta un leggero calo di accuratezza rispetto a pesi indipendenti, ma rende il modello più compatto e favorisce una forma di recurrenza implicita. Keyword: parameter efficiency, depth vs width.
- **Qual è la differenza tra Perceiver e Perceiver IO?** Vedi Sezione 2.1: il Perceiver originale usa i latenti finali come input a un classificatore globale (operazione di pooling/global average), limitandosi a task con output fisso. Perceiver IO aggiunge un decoder basato su cross-attention, dove output query specifiche del task estraggono informazione dai latenti per produrre output di forma arbitraria. Keyword: output query, generalità.
- **Come funziona il decoder di Perceiver IO? Cosa sono le output queries?** Vedi Sezione 2.2: il decoder esegue una cross-attention dove le Query sono output query (apprendibili o derivate da coordinate/feature del task), e Key/Value provengono dall'array latente finale. Questo permette di produrre output per ogni posizione, token, o modalità desiderata. Keyword: task-specific queries, cross-attention decoder.
- **Come gestisce il Perceiver input multimodali (audio+video)?** Vedi Sezione sull'encoding multimodale: ogni modalità viene proiettata nello stesso spazio dimensionale C tramite embedding lineari separati, poi concatenata lungo l'asse dei token. Le Fourier features delle coordinate spaziotemporali permettono al modello di distinguere posizione e modalità. Keyword: modalità-agnostic backbone, shared latent space.
- **Quali risultati ha ottenuto su ImageNet? Come si confronta con ViT e ResNet?** Vedi Sezione sui risultati sperimentali: con Fourier features, il Perceiver raggiunge 78.0% top-1 su ImageNet, competitivo con ViT-B/16 (77.9%) e ResNet-50 (77.6%) senza assumere struttura

a griglia. Non supera ResNet specializzati di fascia alta, ma il vantaggio è la generalità su input diversi. Keyword: 78.0% top-1, generalità vs specializzazione.

V.2 Sulla teoria del corso (domande generali)

- **Spiega la backpropagation con la regola della catena.** Vedi la sezione sulla backpropagation: il gradiente della loss rispetto a ogni parametro si calcola applicando la regola della catena in modo ricorsivo dal layer di output verso l'input. Ogni nodo del grafo computazionale propaga il gradiente moltiplicando il gradiente ricevuto per il Jacobiano locale. Keyword: chain rule, grafo computazionale, $\partial\mathcal{L}/\partial w$.
- **Cos'è il vanishing gradient? Come lo risolvono LSTM e residual connections?** Vedi la sezione su vanishing/exploding gradient: i gradienti si moltiplicano attraverso molti layer; se i Jacobiani hanno norma < 1 il gradiente si azzerava. LSTM usa gate che permettono flusso diretto del gradiente (addizione, non moltiplicazione). Le residual connections ($x + F(x)$) garantiscono un percorso diretto di gradiente pari a 1. Keyword: gradient flow, skip connection, cell state.
- **Differenza tra self-attention e cross-attention.** Vedi la sezione sull'attenzione: nella self-attention Q, K, V provengono tutti dalla stessa sequenza (la sequenza attende a se stessa). Nella cross-attention Q proviene da una sequenza e K, V da un'altra (tipicamente encoder $\rightarrow$ decoder). Il Perceiver usa cross-attention tra latenti (Q) e input (KV). Keyword: same-source vs cross-source.
- **Perché si divide per $\sqrt{d_k}$ nella scaled dot-product attention?** Vedi la sezione sull'attenzione: il dot product $QK^\top$ ha varianza proporzionale a d_k ; senza scaling, per grandi d_k i logits diventano molto grandi e la softmax satura verso vettori one-hot, azzerando i gradienti. Dividere per $\sqrt{d_k}$ normalizza la varianza a 1. Keyword: softmax saturation, variance scaling.
- **Cos'è la Layer Normalization? Differenza con Batch Normalization?** Vedi la sezione sulla normalizzazione: LayerNorm normalizza lungo la dimensione delle feature per ogni singolo esempio (media e varianza su D feature); BatchNorm normalizza lungo la dimensione del batch per ogni feature. LayerNorm è indipendente dalla batch size e funziona bene con sequenze di lunghezza variabile, per questo viene usata nei Transformer. Keyword: feature-wise vs batch-wise.
- **Cos'è la softmax? Proprietà e stabilità numerica.** Vedi la sezione sulla softmax: $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$. Proprietà: output in $(0, 1)$, somma a 1, differenziabile. Per stabilità numerica si sottrae il massimo: $\text{softmax}(x_i - \max_j x_j)$ è equivalente ma evita overflow/underflow. Keyword: numerically stable softmax, log-sum-exp trick.
- **Come funziona Adam? Differenza con SGD?** Vedi la sezione sugli ottimizzatori: Adam mantiene una stima del primo momento (media dei gradienti, m_t) e del secondo momento (varianza non centrata, v_t), con correzione del bias. Il learning rate effettivo è adattivo per parametro: $\alpha / (\sqrt{v_t} + \epsilon)$. SGD usa un learning rate uniforme per tutti i parametri. Adam converge più rapidamente e richiede meno tuning del learning rate. Keyword: adaptive learning rate, momentum, β_1, β_2 .
- **Cos'è il dropout? Perché il Perceiver non lo usa?** Vedi la sezione sul dropout: durante il training, ogni unità viene azzerata con probabilità p , forzando il modello a non dipendere da singoli neuroni (regularizzazione implicita). Il Perceiver tipicamente non usa dropout nei layer di attenzione perché il latent bottleneck stesso funge da regolarizzatore strutturale, e l'architettura è già robusta all'overfitting grazie al weight sharing. Keyword: regularization, latent bottleneck as regularizer.
- **Spiega le Fourier features. Perché si concatena la coordinata grezza?** Vedi la sezione sull'encoding posizionale: le Fourier features codificano la posizione $p \in [-1, 1]^d$ come $[\sin(f_k \pi p), \cos(f_k \pi p)]$ per frequenze f_k logaritmicamente spaziate, permettendo di rappresentare

strutture a scale diverse. La coordinata grezza viene concatenata per preservare informazione di posizione assoluta che le Fourier features sole possono perdere in presenza di aliasing. Keyword: multiscale encoding, Nyquist, position encoding.

- **Pre-norm vs post-norm: qual è la differenza e quale usa il Perceiver?** Vedi la sezione sulla normalizzazione nei Transformer: post-norm applica LayerNorm dopo la residual connection ($\text{LN}(x + F(x))$, schema originale di Vaswani et al., 2017); pre-norm applica LayerNorm prima del sub-layer ($x + F(\text{LN}(x))$). Il Perceiver usa pre-norm, che garantisce gradienti più stabili durante il training profondo e permette di rimuovere il warm-up del learning rate. Keyword: pre-LN, training stability, gradient scale.

V.3 Sul codice e gli esperimenti

- **Come hai implementato il Perceiver? Quali framework hai usato?** Implementazione in PyTorch, seguendo la struttura del paper originale (Jaegle et al. 2021). I moduli principali sono: *CrossAttention*, *SelfAttention* (con multi-head), *FourierEncoding*, e il loop iterativo con weight sharing. Keyword: PyTorch, modularità, weight sharing loop.
- **Come hai riprodotto i risultati? Quali iperparametri hai usato?** Training su CIFAR-10 (50k/10k), ModelNet40 e WikiText-103→GLUE. Iperparametri CIFAR-10 (Perceiver): $N = 96$ latenti, $D = 384$, 4 cross-attend stages, 4 transformer blocks, $H = 3$ teste, ottimizzatore **LAMB** con $lr = 4e-3$, 120 epoche, batch 64. Perceiver IO / ModelNet40 / MLM: $N = 128$, $D = 512$, $H = 8$. (NON Adam: sia il paper sia il progetto usano LAMB.) Keyword: ablation, hyperparameters, LAMB.
- **Quali difficoltà hai incontrato nell'implementazione?** Principali sfide: (1) gestione corretta delle dimensioni nei moduli di attenzione multi-head; (2) implementazione stabile della scaled dot-product attention con masking; (3) verifica del weight sharing (stessi moduli Python riutilizzati nel loop, non copie); (4) encoding Fourier corretto per input 2D/3D con frequenze logaritmiche. Keyword: debugging shapes, tied weights.
- **Come hai gestito il training con risorse limitate rispetto al paper (64 TPU)?** Il paper usa 64 TPU v3 per training completo su ImageNet. Con GPU singola: riduzione della batch size (gradient accumulation per simulare batch grandi), uso di `torch.cuda.amp` per mixed precision, e valutazione su CIFAR-10 o ImageNet subset per tempi di training praticabili. Il confronto con i risultati del paper è quindi qualitativo sul trend, non sui numeri assoluti. Keyword: mixed precision, gradient accumulation, resource constraints.